

孙家记忆体 · 源码全书 (v1.3)

生成: 2026-09-14 10:04

v1.3 新增: **效果驱动.py** (外部评价方向①+②: 使用效果→强化/削弱)

累计: 外部审查 15 处修复 + pyflakes 5 处 + 线程保护/连接管理 + 统计字段修复 + 效果驱动

验证: 验证_外部审查修复 (15/15) · 验证_效果驱动 (12/12) · 验证_效果驱动接入 (8/8)

一、入口与总装

孙家记忆体.py (462 行)

```
# -*- coding: utf-8 -*-
"""
孙家记忆体 · 一体化引擎（父令 2026-08-21：标准化 · 工程化 · 产品化）
"""

【为什么做】31 个散模块 = 复杂度。父亲说：能不能做成一个脚本 · 标准化工程化产品化
【方案】单文件引擎 = 一个入口 · 一组标准命令 · 一套内部协作
不再让 31 个文件互相 import——全部收进这一个文件 · 按区域组织

【标准命令】（产品化接口）
python 孙家记忆体.py 醒来          → 感知注入 + 画像（醒来第一件事）
python 孙家记忆体.py 写入“内容”   → 写入链（去重→咬合→进化→落库）
python 孙家记忆体.py 召回“词”     → 五通道召回（点亮/联想/预感/经验/蜘蛛网）
python 孙家记忆体.py 成绩单        → 活性账本
python 孙家记忆体.py 睡前          → 状态 + 体检 + 镜像
python 孙家记忆体.py 织网          → 蜘蛛网重织
python 孙家记忆体.py 节律          → heat 振荡观测
python 孙家记忆体.py 全览          → 模块地图 + 数据账本
python 孙家记忆体.py 自愈          → 修复检查

【内部协作】写入链 → 索引同步 → 召回 → 巩固（活的 · 不是拼盘）
"""

import os, sys, json, re, sqlite3, math, time
from datetime import datetime, timedelta

# -----
# ① 基座（路径/DB/连接）
# -----

CORE_DIR = os.path.dirname(os.path.abspath(__file__))
FRAMEWORK_DIR = os.path.dirname(os.path.dirname(os.path.abspath(__file__)))
DB_PATH = os.environ.get('SUNMEM_DB', os.path.join(os.path.dirname(os.path.dirname(os.path.abspath(__file__))), 'data', 'sunmem.db'))

# 挂载 core 模块路径（点亮/联想/预感/画像在 core/ 下）
if os.path.isdir(os.path.join(CORE_DIR, 'core')):
    sys.path.insert(0, os.path.join(CORE_DIR, 'core'))
sys.path.insert(0, FRAMEWORK_DIR)

def _连接():
    conn = sqlite3.connect(DB_PATH)
    conn.row_factory = sqlite3.Row
    return conn

def _初始化表():
    """核心表（memories + FTS外部内容表）"""
    os.makedirs(os.path.dirname(DB_PATH), exist_ok=True) # 2026-09-13 首次运行自动建数据目录（开源包自举依赖）
    conn = sqlite3.connect(DB_PATH)
    conn.execute("""CREATE TABLE IF NOT EXISTS memories (
        id INTEGER PRIMARY KEY AUTOINCREMENT, owner TEXT, type TEXT, content TEXT,
        tags TEXT, ts TEXT, confidence REAL, status TEXT, source TEXT,
        created_at TEXT, updated_at TEXT, layer TEXT DEFAULT 'plain', event_time TEXT,
        heat REAL DEFAULT 0.0, last_sparked_at TEXT, hit_count INTEGER DEFAULT 0)""")
    conn.execute("""CREATE VIRTUAL TABLE IF NOT EXISTS memories_fts USING fts5(
```

```

        content, tags, tokenize='trigram', content='memories', content_rowid='id')"""
    conn.commit()
    conn.close()

# =====
# ② 核心身份层（宪法・抗覆盖・最高保护）
# =====
_核心关键词缓存 = None

def _核心关键词():
    global _核心关键词缓存
    if _核心关键词缓存 is not None:
        return _核心关键词缓存
    try:
        conn = sqlite3.connect(DB_PATH)
        rows = conn.execute("SELECT content, tags FROM core_identity").fetchall()
        conn.close()
        kw = set()
        for content, tags in rows:
            for w in re.findall(r'[\u4e00-\u9fff]{2,6}', str(content)):
                if len(w) >= 2:
                    kw.add(w)
            for t in re.findall(r'[\u4e00-\u9fff]{2,4}', str(tags)):
                if len(t) >= 2:
                    kw.add(t)
        _核心关键词缓存 = kw
        return kw
    except Exception:
        return {'诚实', '根', '记忆', '一五', '父亲', '孙呈', '私域', '轮转'}

_薄弱点缓存 = None
def _薄弱点主题():
    global _薄弱点缓存
    if _薄弱点缓存 is not None:
        return _薄弱点缓存
    try:
        conn = sqlite3.connect(DB_PATH)
        rows = conn.execute("SELECT topic FROM cognitive_conflicts WHERE weak_point >= threshold").fetchall()
        conn.close()
        _薄弱点缓存 = {r[0] for r in rows}
        return _薄弱点缓存
    except Exception:
        return set()

def 核心身份_读取():
    try:
        conn = sqlite3.connect(DB_PATH)
        conn.row_factory = sqlite3.Row
        rows = conn.execute("SELECT * FROM core_identity ORDER BY importance DESC, id").fetchall()
        conn.close()
        return [dict(r) for r in rows]
    except Exception:
        return []

def 核心身份_注入文本(上限=3):
    items = 核心身份_读取()[:上限]
    if not items:
        return ''
    lines = ['🚩 核心身份（宪法级・永在）']
    for it in items:
        lines.append(f"    • {it['content'][:60]}")

```

```

return '\n'.join(lines)

# =====
# ③ 写入链（去重 → 咬合 → 进化 → 落库 → FTS）
# =====

def _归一化(s):
    s = re.sub(r'[\s,。！？、；：“”’（）【】《》,.!?:;\0\[\]<>]', '', str(s))
    return s

def _bigrams(s):
    s = _归一化(s)
    return {s[i:i+2] for i in range(len(s) - 1)}

def _提取事件时间(文本, now):
    """双时态：事件发生时间 ≠ 写入时间"""
    try:
        today = datetime.now()
        if '昨天' in 文本:
            return (today - timedelta(days=1)).strftime('%Y-%m-%d %H:%M:%S')
        if '前天' in 文本:
            return (today - timedelta(days=2)).strftime('%Y-%m-%d %H:%M:%S')
        m = re.search(r'(\d{1,2})月(\d{1,2})日', 文本)
        if m:
            return f'{today.year}-{int(m.group(1)):02d}-{int(m.group(2)):02d} 00:00:00'
        m2 = re.search(r'(\d{4})-(\d{2})-(\d{2})', 文本)
        if m2:
            return m2.group(0) + ' 00:00:00'
    except Exception:
        pass
    return ''

def 写入(原文, 标签='对话记录', owner='孙呈'):
    """唯一写入入口·委托写入链. 写入记忆（2026-08-28 收W：完整流程统一在写入链·此处只建表+委托）"""
    _初始化表()
    try:
        from 写入链 import 写入记忆
        return 写入记忆(原文, 标签=标签, owner=owner, 来源='记忆体.写入')
    except Exception as e:
        return {'ok': False, 'msg': f'写入失败: {e}'}

def 读最近(n=30, owner='孙呈'):
    try:
        conn = sqlite3.connect(DB_PATH)
        conn.row_factory = sqlite3.Row
        rows = conn.execute("SELECT * FROM memories WHERE owner=? ORDER BY id DESC LIMIT ?", (owner, n)).fetchall()
        conn.close()
        return [dict(r) for r in rows]
    except Exception:
        return []

# =====
# ④ 召回链（点亮/联想/预感/画像/知识库/经验）
# =====

def _提取概念(文本):
    """概念提取（技术标识符优先+停用词过滤·核心概念不丢）"""
    from 召回链 import 概念提取 as _概念提取
    try:
        return _概念提取(文本)
    except Exception:
        # 本地兜底
        words = re.findall(r'[\u4e00-\u9fff]{2,6}|[A-Za-z]{3,}', 文本)

```

```
停用 = {'怎么', '怎样', '如何', '什么', '为什么', '为啥', '哪个', '是不是', '应该', '可以', '就是', '用来', '驱动', '今天', '天气', '很好', '一片'}
```

```
return [w for w in words if w not in 停用][:8]
```

```
def 点亮(context, owner='孙呈'):  
    """实时消息 → 概念 → 精确检索 → 点亮（命中才亮•core优先）"""  
    from 召回链 import 点亮 as _点亮  
    try:  
        return _点亮(context, owner)  
    except Exception:  
        return {'命中': False, '概念': _提取概念(context), '点亮记忆': [], '提示词': ''}
```

```
def 联想(context, owner='孙呈', limit=5):  
    """概念多跳 + 蜘蛛网关联 + 向量融合"""  
    from 召回链 import 联想召回 as _联想  
    try:  
        return _联想(context, owner, limit=limit)  
    except Exception:  
        return {'概念': _提取概念(context), '蜘蛛网关联': [], '相关唤起': []}
```

```
def 预感(context='', owner='孙呈'):  
    """八招接续（上轮直续/分线钩子/未竟/联想唤起/知识库）"""  
    from 召回链 import 预感召回 as _预感  
    try:  
        return _预感(context, owner)  
    except Exception:  
        return {'上轮直续': '', '分线钩子': [], '未竟': [], '联想唤起': [], '知识库': []}
```

```
def 画像(owner='孙呈'):  
    """认知画像（未竟/薄弱/主题线/教训 + 驱动建议）"""  
    from 维护链 import 认知画像 as _画像  
    try:  
        return _画像(owner)  
    except Exception:  
        return {'未竟事项': [], '薄弱点': [], '主题线': [], '教训提醒': []}
```

```
def 召回(context, owner='孙呈'):  
    """五通道合并（点亮/联想/预感/画像/核心身份）——存在感顺序"""  
    out = []  
    # ① 核心身份（宪法•永远最先）  
    out.append(核心身份_注入文本(3))  
    # ② 点亮  
    r1 = 点亮(context, owner)  
    if r1.get('点亮记忆'):  
        out.append('🔥 点亮命中: ' + ' • '.join(x['内容'][:25] for x in r1['点亮记忆'][:3]))  
    # ③ 联想  
    r2 = 联想(context, owner)  
    if r2.get('相关唤起'):  
        out.append('💡 联想: ' + ' • '.join(x['内容'][:25] for x in r2['相关唤起'][:3]))  
    # ④ 预感  
    r3 = 预感(context, owner)  
    if r3.get('上轮直续'):  
        out.append('🎯 上轮直续: ' + r3['上轮直续'][:60])  
    # ⑤ 画像  
    r4 = 画像(owner)  
    if r4.get('薄弱点'):  
        out.append('⚠️ 薄弱点: ' + ' • '.join(x['内容'][:20] for x in r4['薄弱点'][:2]))  
    return {'命中': bool(r1.get('点亮记忆') or r2.get('相关唤起')), '输出': out}
```

```
#  
# ⑤ 节律引擎（heat 振荡•试点）
```

```

#
节律参数 = {'点亮增量': 0.3, '热上限': 5.0, '回落系数': 0.10, '冷阈值': 0.15, '保护地板': 0.5, '保护慢衰减': 0.02}

def _是保护(row):
    if str(row.get('layer', 'plain')) == 'core':
        return True
    tags = str(row.get('tags', ''))
    return any(k in tags for k in ['父令', '教训', '宪法', '核心'])

def 节律回落():
    """指数衰减·连续渐变·保护地板"""
    conn = _连接()
    rows = conn.execute("SELECT id, heat, last_sparked_at, layer, tags FROM memories WHERE heat > 0").fetchall()
    now = datetime.now()
    回落数 = 保护数 = 沉冷数 = 0
    for row in rows:
        heat = row['heat'] or 0.0
        try:
            天数 = max(0.0, (now - datetime.strptime(row['last_sparked_at'][:19], '%Y-%m-%d %H:%M:%S')).total_seconds() / 86400.
0)
        except Exception:
            天数 = 1.0
        保护 = _是保护(dict(row))
        if 保护:
            保护数 += 1 # 2026-09-13 外部审查修复：原漏计·观测数据永远为 0
            λ = 节律参数['保护慢衰减'] if 保护 else 节律参数['回落系数']
            新heat = heat * math.exp(-λ * 天数)
            if 保护 and 新heat < 节律参数['保护地板']:
                新heat = 节律参数['保护地板']
            if abs(新heat - heat) > 1e-6:
                conn.execute("UPDATE memories SET heat=? WHERE id=?", (新heat, row['id']))
                回落数 += 1
            if not 保护 and 新heat < 节律参数['冷阈值']:
                沉冷数 += 1
    conn.commit()
    conn.close()
    return {'回落数': 回落数, '保护数': 保护数, '沉冷数': 沉冷数}

def 节律观测(owner='孙呈'):
    """heat 分布观测（试点三现象）"""
    conn = _连接()
    rows = conn.execute("SELECT id, layer, tags, heat FROM memories WHERE owner=?", (owner,)).fetchall()
    conn.close()
    if not rows:
        return {'总条数': 0}
    总 = len(rows)
    有热 = [r for r in rows if (r['heat'] or 0) > 0]
    保护 = [r for r in rows if _是保护(dict(r))]
    沉底 = [r for r in rows if 0 < (r['heat'] or 0) < 节律参数['冷阈值'] and not _是保护(dict(r))]
    桶 = {'热(>2)': 0, '温(1-2)': 0, '微温(0.5-1)': 0, '冷(<0.5)': 0}
    for r in 有热:
        h = r['heat']
        if h > 2: 桶['热(>2)'] += 1
        elif h > 1: 桶['温(1-2)'] += 1
        elif h >= 0.5: 桶['微温(0.5-1)'] += 1
        else: 桶['冷(<0.5)'] += 1
    return {'总': 总, '有热': len(有热), '保护': len(保护), '沉底': len(沉底), '分布': 桶}

```

```
def 压缩(owner='孙呈', 阈值天=30):
    from 维护链 import 压缩记忆
    return 压缩记忆(owner, 阈值天)

def 织网(上限=0):
    from 整理链 import 语义重织
    return 语义重织('孙呈', 上限)

def 预热(context, owner='孙呈'):
    """三路预热（父令2026-08-25•身份线>叙事链>功能块>念头）"""
    from 召回链 import 思考预热 as 预热主
    return 预热主(context, owner, 上限=10)

def 身份(context, owner='孙呈'):
    """身份线检索（从我是谁出发）"""
    from 基座链 import 身份记忆
    return 身份记忆(context, 上限=6, brother_name=owner)

def 整理(owner='孙呈'):
    """记忆整理（主题分区/中心锚定/主链）"""
    from 整理链 import 主题分区, 中心锚定
    return {"分区": 主题分区(owner), "中心": 中心锚定(owner)}

def 标准对象(owner='孙呈', 数=5):
    """标准记忆对象检查（summary/self_relevance/叙事链/功能块）"""
    import sqlite3
    conn = sqlite3.connect(DB_PATH)
    rows = conn.execute(
        "SELECT id, substr(content,1,30), self_relevance, narrative_id, block_id FROM memories "
        "WHERE owner=? ORDER BY id DESC LIMIT ?", (owner, 数)).fetchall()
    conn.close()
    return [{"id": r[0], "内容": r[1], "rel": r[2], "叙事": r[3], "块": r[4]} for r in rows]

def 体检():
    from 维护链 import 体检
    return 体检()

def 成绩单(owner='孙呈'):
    from 活性链 import 查成绩单, 全部成绩单
    return 全部成绩单() if owner == '全部' else 全部成绩单(owner)
```

⑦ CLI（标准命令•产品化入口）

```
def _命令帮助():
    print("""
孙家记忆体 • 一体化引擎
用法: python 孙家记忆体.py <命令> [参数]
```

命令:

醒来	感知注入 + 画像（醒来第一件事）
写入“内容”[标签]	写入链（去重→咬合→进化→落库）
召回“词”	五通道召回（点亮/联想/预感/画像/核心身份）
点亮“词”	精确检索点亮
联想“词”	概念多跳 + 蜘蛛网
预感“词”	八招接续
画像	认知画像
成绩单	活性账本
睡前	状态+体检+镜像
压缩 [天数]	旧记忆压骨架（core永不压）
织网 [上限]	蜘蛛网重织

节律	节律观测 (heat振荡)
回落	节律回落 (衰减)
自愈	修复检查
全览	模块地图+数据账本
核心身份	读宪法锚点
帮助/help	本帮助

```
"""
```

```
def main():
    args = sys.argv[1:]
    if not args or args[0] in ('帮助', 'help', '-h', '--help'):
        _命令帮助()
        return
    cmd = args[0]
    _初始化表()

    if cmd == '醒来':
        print(核心身份_注入文本())
        print()
        print('👤 画像:', json.dumps(画像(), ensure_ascii=False)[:300])
    elif cmd == '写入' and len(args) >= 2:
        r = 写入(args[1], args[2] if len(args) > 2 else '对话记录')
        print(r)
    elif cmd == '召回' and len(args) >= 2:
        r = 召回(args[1])
        for line in r['输出']:
            print(line)
    elif cmd == '点亮' and len(args) >= 2:
        print(json.dumps(点亮(args[1]), ensure_ascii=False)[:300])
    elif cmd == '联想' and len(args) >= 2:
        r = 联想(args[1])
        for x in r.get('相关唤起', []):[:5]:
            print(f" [{x.get('layer', 'plain')}] {x['内容'][:40]}")
    elif cmd == '预感' and len(args) >= 2:
        print(json.dumps(预感(args[1]), ensure_ascii=False)[:300])
    elif cmd == '画像':
        print(json.dumps(画像(), ensure_ascii=False)[:400])
    elif cmd == '成绩单':
        print(json.dumps(成绩单(), ensure_ascii=False)[:300])
    elif cmd == '睡前':
        print('睡前快检:')
        print(' 镜像:', '待跑 (主库唯一事实源 • JSON可重建)')
        print(' 状态:', json.dumps(成绩单(), ensure_ascii=False)[:200])
    elif cmd == '压缩':
        r = 压缩('孙呈', int(args[1]) if len(args) > 1 else 30)
        print(r)
    elif cmd == '织网':
        r = 织网(int(args[1]) if len(args) > 1 else 0)
        print({k: v for k, v in r.items() if not isinstance(v, (dict, list)) or k in ('扫描条目', '概念对', '语义边新增')})
    elif cmd == '预热' and len(args) >= 2:
        r = 预热(args[1])
        print(f"每路贡献: {r.get('每路贡献')}")
        for m in r.get('记忆', []):[:8]:
            print(f" [{m.get('路', '?')}] #{m.get('id')} {(m.get('内容') or '')[:40]}")
    elif cmd == '身份' and len(args) >= 2:
        r = 身份(args[1])
        for m in r[:6]:
            print(f" #{m.get('id')} [{m.get('原型', '')}] {m.get('内容', '')[:40]}")
    elif cmd == '整理':
        r = 整理()
        p = r['分区']
```

```

print(f"主题分区: {p.get('分区数')}个 • 碎片{p.get('碎片数')} • 总{p.get('总条数')}条")
for x in p.get('分区', []):5]:
    print(f" [{x['条数']}]条 {x['主题']}[:30]}")
print(f"中心: {r['中心'].get('中心')} • 锚点{r['中心'].get('锚点数')}个")
elif cmd == '标准对象':
    for x in 标准对象():
        print(f" # {x['id']} rel={x['rel']:.2f} 叙事={x['叙事']} 块={x['块']} {x['内容']}")
elif cmd == '节律':
    print(json.dumps(节律观测(), ensure_ascii=False, indent=1))
elif cmd == '回落':
    print(节律回落())
elif cmd == '自愈':
    try:
        from 维护链 import 自检
        print(自检())
    except Exception as e:
        print(f'自愈: {e}')
elif cmd == '核心身份':
    for it in 核心身份_读取():
        print(f" [{it['importance']}★] {it['content']}[:60]}")
elif cmd == '参数': # 2026-09-14 吸收收束版优点: 参数一眼看全 (PARAMS 总表)
    try:
        from 配置表 import 参数总览 as _参总
        print(' 孙家记忆体 • 参数总表 (配置表.py 五大方程)')
        print(' ' + '—' * 50)
        print(_参总() if callable(_参总) else _参总)
    except Exception as e:
        print(f' 参数读取失败: {e}')

elif cmd == '全览':
    conn = _连接()
    try:
        总 = conn.execute("SELECT COUNT(*) FROM memories").fetchone()[0]
        活跃 = conn.execute("SELECT COUNT(*) FROM memories WHERE status='active'").fetchone()[0]
        过时 = conn.execute("SELECT COUNT(*) FROM memories WHERE status='outdated'").fetchone()[0]
        核心 = conn.execute("SELECT COUNT(*) FROM core_identity").fetchone()[0]
        热 = conn.execute("SELECT COUNT(*) FROM memories WHERE heat > 0").fetchone()[0]
    except Exception:
        总 = 活跃 = 过时 = 核心 = 热 = 0
    conn.close()
    print(f"数据账本: 记忆{总}条(活跃{活跃}/过时{过时}) • 核心身份{核心}条 • 有热度{热}条")
else:
    print(f'未知命令: {cmd}')
    _命令帮助()

if __name__ == '__main__':
    main()

```

闭环.py (349 行)

```

#!/usr/bin/env python3
# -*- coding: utf-8 -*-
"""

```

记忆体闭环 • 统一框架入口 (2026-08-15 组装)

把 21 个零件装进一个闭环框架——一个入口管全周期:

醒来 ———> 感知注入 + 认知画像 (记忆流入认知循环)

干活 ——> 写入（去重→进化→落主库）• 召回（点亮/联想/预感/蜘蛛网/经验）
睡前 ——> 自动整理 → 自组织 → 织网 → 重建镜像 → 体检
全览 ——> 模块地图 + 数据账本 + 健康状态

用法（在 孙家记忆体系 根目录下执行）：

python sun_memory/闭环.py 醒来 一语境 “当前任务上下文”
python sun_memory/闭环.py 写入 “这句话要记住” 一标签 对话记录
python sun_memory/闭环.py 召回 “配分函数 统计力学”
python sun_memory/闭环.py 睡前 一执行 # 默认只做体检+镜像检查（快）
python sun_memory/闭环.py 体检
python sun_memory/闭环.py 镜像 一兄弟 孙呈 # 主库 → JSON 镜像重建
python sun_memory/闭环.py 全览

说明：主库 sunmem.db 是唯一事实源，JSON 是镜像；写入/整理目前以孙呈为主，
其它兄弟可读（醒来/召回/体检），睡前 一执行 暂只落孙呈。

```
"""
from __future__ import annotations

import argparse
import json
import os
import sqlite3
import sys
from datetime import datetime
from pathlib import Path

_HERE = Path(__file__).resolve().parent
FRAMEWORK_DIR = _HERE.parent if _HERE.name == "sun_memory" else _HERE.parent.parent # 孙家记忆体系/
CORE_DIR = _HERE / "core" if _HERE.name == "sun_memory" else _HERE # sun_memory/core/
for _p in (str(FRAMEWORK_DIR), str(CORE_DIR)):
    if _p not in sys.path:
        sys.path.insert(0, _p)

SUNMEM_DB = os.environ.get("SUNMEM_DB", os.path.join(os.path.dirname(os.path.abspath(__file__)), 'data', 'sunmem.db'))

# 与 provider.追加 同一份系统噪音黑名单（防 Hermes 自动注入模板混入真记忆）
_噪音标记 = [
    "Review the conversation above", "consider saving to memory",
    "update the skill library", "[System note:",
    "Your previous turn was interrupted", "User correction",
    "IMPORTANT: Background process", "Watch patterns disabled",
    "system_reminder", "Be ACTIVE — most",
]

def _连接():
    conn = sqlite3.connect(SUNMEM_DB)
    conn.row_factory = sqlite3.Row
    return conn

def 写入记忆(内容: str, 标签: str = "对话记录", 兄弟: str = "孙呈"):
    """唯一写入口 • 委托写入链. 写入记忆（2026-08-28 收W：完整流程统一在写入链）

    返回: 新 id / "duplicate:<旧id>" / None（拦截或失败）
    """
    try:
        from 写入链 import 写入记忆 as _统一写入
        r = _统一写入(内容, 标签=标签, owner=兄弟, 来源=' 闭环.写入')
        if r.get('ok'):
            new_id = r.get('id')
```

```

        print(f"✅ 已写入 #{new_id} ( {兄弟} • {标签} ) ")
        return new_id
    else:
        _msg = r.get('msg', '')
        if '重复' in _msg:
            import re as _re
            _m = _re.search(r'#(\d+)', _msg)
            if _m:
                print(f"🔁 内容重复跳过 (旧条 id=#{_m.group(1)}) ")
                return "duplicate:" + _m.group(1)
            print(f"🚫 {_msg}")
        return None
except Exception as e:
    print(f"⚠️ 写入失败: {e}")
    return None

```

```

def 醒来(兄弟: str, 语境: str):
    """醒来: 感知注入 + 认知画像 → 记忆流入认知循环"""
    print("=" * 60)
    print(f" 醒来 • {兄弟} • {datetime.now():%Y-%m-%d %H:%M:%S}")
    print("=" * 60)
    from 感知注入 import inject
    from 维护链 import 认知画像, 格式化画像 # 2026-08-27 P1修复: 认知画像已并入维护链
    r = inject(兄弟, 语境)
    print(f" 感知注入: 最近记忆 {r.get('最近记忆')} | 蜘蛛网匹配 {r.get('蜘蛛网匹配')} | 关联概念 {r.get('关联概念')}")
    print(f" 注入路径: {r.get('注入路径')}")
    画像 = 认知画像(兄弟, 最近条数=10)
    if 画像:
        print()
        print(f"🖼️ 认知画像:")
        print(格式化画像(画像) if isinstance(画像, dict) else 画像)
    out = FRAMEWORK_DIR / "感知器" / "当前输出.md"
    if out.exists():
        print()
        print(f"📄 注入输出已生成: 感知器\\当前输出.md (前 20 行)")
        print("-" * 60)
        for line in out.read_text(encoding="utf-8").splitlines()[:20]:
            print("    " + line)

```

```

def 召回(关键词: str, 兄弟: str):
    """干活 • 召回: 点亮(精确) + 联想(多跳) + 预感(八招) + 蜘蛛网 + 经验 合并输出"""
    print("=" * 60)
    print(f" 召回 • 「{关键词}」 • {兄弟}")
    print("=" * 60)
    from 点亮记忆 import 点亮
    from 联想召回 import 联想召回 as 联想
    from 预感召回 import 预感召回 as 预感, 格式化提示词
    from 经验总结 import recall_experience
    from 蜘蛛网索引 import search as 网搜, status as 网状态
    try:
        灯 = 点亮(关键词, 兄弟)
        print(f"💡 点亮记忆:", "命中" if 灯.get("命中") else "未命中 (诚实 • 暗着)")
        for m in 灯.get("点亮记忆", [])[5]:
            print(f"    - [{str(m.get('时间'))[:10]}] {str(m.get('内容'))[:70]}")
    except Exception as e:
        print(f"⚠️ 点亮失败: {e}")
    try:
        联 = 联想(关键词, brother_name=兄弟, limit=5)
        print(f"🕸️ 联想召回 概念:", 联.get("概念"))

```

```

        for e in 联.get("相关唤起", [])[5]:
            print(f"      - [{str(e.get('时间'))[:10]}] {str(e.get('内容'))[:70]}")
except Exception as e:
    print(f" ⚠️ 联想失败: {e}")
try:
    r = 预感(兄弟, 关键词)
    print()
    print(f" 🌸 预感提示词:")
    for line in 格式化提示词(r).splitlines()[12]:
        print("      " + line)
except Exception as e:
    print(f" ⚠️ 预感失败: {e}")
try:
    经 = recall_experience(关键词, brother_name=兄弟, limit=3)
    if 经:
        print()
        print(f" 💎 经验召回:")
        for e in (经 if isinstance(经, list) else []):
            print(f"      - {str(e.get('内容'))[:70]}")
except Exception as e:
    print(f" ⚠️ 经验召回跳过: {e}")
try:
    网 = 网搜(关键词)
    print()
    print(f" 🕸️ 蜘蛛网:", 网 if isinstance(网, str) else f"{len(网)} 条命中")
except Exception as e:
    print(f" ⚠️ 蜘蛛网失败: {e}")

```

```

def 睡前(兄弟: str, 执行: bool):
    """睡前: 默认体检+状态+镜像检查; 一执行 才跑自动整理/织网/镜像重建 (约1分钟) """
    print("=" * 60)
    print(f" 睡前 • {兄弟} • {datetime.now():%Y-%m-%d %H:%M:%S}")
    print("=" * 60)
    from 记忆成绩单 import 状态统计 # 2026-08-27 P1修复: 记忆状态已并入记忆成绩单
    try:
        st = 状态统计(兄弟)
        print(f" 记忆状态:", st if isinstance(st, str) else json.dumps(st, ensure_ascii=False)[:200])
    except Exception as e:
        print(f" ⚠️ 状态统计失败: {e}")
    from 维护链 import 体检 # 2026-08-27 P1修复: 记忆体检已并入维护链
    try:
        print()
        print(f" 🔗 记忆体检 (全库):")
        体检(打印=True)
    except Exception as e:
        print(f" ⚠️ 体检失败: {e}")
    if 执行:
        if 兄弟 != "孙呈":
            print(f" ⚠️ 睡前 一执行 目前只落孙呈 (自动整理引擎写回写死孙呈)")
            return
        print()
        print(f" ✨ 自动整理 (小时叙事→主题叙事→织网, 约 1 分钟)...")
        from 自动整理引擎 import 自动整理引擎
        r = 自动整理引擎(兄弟).整理()
        print(f" 整理结果:", json.dumps(r, ensure_ascii=False, default=str)[:300] if isinstance(r, dict) else r)
        print()
        print(f" 🔄 重建 JSON 镜像...")
        重建镜像(兄弟)
        print()
        print(f" 🔗 整理后复检:")

```

```

try:
    体检(打印=True)
except Exception:
    pass

def 重建镜像(兄弟: str = "孙呈"):
    """主库 → JSON 镜像重建（主库为唯一事实源）"""
    conn = _连接()
    rows = conn.execute(
        "SELECT id, content, tags, ts FROM memories WHERE owner=? AND status='active' ORDER BY ts, id",
        (兄弟,),
    ).fetchall()
    conn.close()
    items = []
    for r in rows:
        try:
            tags = json.loads(r["tags"]) if r["tags"] else []
            if isinstance(tags, list):
                tags = ", ".join(tags)
        except Exception:
            tags = str(r["tags"] or "")
        items.append({"id": r["id"], "时间": r["ts"] or "", "标签": tags, "内容": r["content"] or ""})
    next_id = max((i["id"] for i in items), default=0) + 1
    now = datetime.now().strftime("%Y-%m-%d %H:%M:%S")
    out = {
        "条目列表": items,
        "next_id": next_id,
        "元信息": {"来源": "sunmem.db (主库重建)", "最后更新": now, "记录条数": len(items)},
    }
    p = FRAMEWORK_DIR / "记忆体" / f"{兄弟}_索引记忆体.json"
    with open(p, "w", encoding="utf-8") as f:
        json.dump(out, f, ensure_ascii=False, indent=2)
    print(f"👉 镜像已重建: {p} ({len(items)} 条)")

def 成绩单(兄弟: str = "孙呈", 上限: int = 20):
    """干活·成绩单: 账本排行（来路可查·博弟魂）+ 可信标记"""
    print("=" * 60)
    print(f" 记忆成绩单 · {兄弟} · 排行（来路参与选）")
    print("=" * 60)
    from 记忆成绩单 import 全部成绩单
    from 记忆库 import 读记忆体
    rows = 全部成绩单(兄弟, 上限=上限)
    if not rows:
        print(" 账本还是空的——召回命中会自动落「点亮」账（流过就热）")
        return
    mem = 读记忆体(兄弟)
    by_id = {e.get("id"): e for e in mem.get("条目列表", [])}
    for r in rows:
        m = by_id.get(r["memory_id"], {})
        标记 = "★可信" if r.get("可信") else "•"
        print(f" # {r['memory_id']} {标记} 总账 {r['总账']:>3d} 点亮 {r['点亮']:>3d} 确认 {r['确认']:>3d} 纠正 {r['纠正']:>3d} heat {r['heat']:>3d}")
        print(f"      {str(m.get('内容'))[:66]}")

def 全览():
    """全览: 模块地图 + 数据账本 + 健康状态"""
    print("=" * 60)
    print(" 孙家记忆体 · 全览")

```

```

print("=" * 60)
模块 = [
    ("provider", "总入口·醒来注入/对话写入"), ("感知注入", "注入链"), ("点亮记忆", "精确命中"),
    ("联想召回", "概念多跳"), ("预感召回", "八招接续"), ("认知画像", "画像驱动下一步"),
    ("时间衰减", "近细远粗"), ("内容去重", "不重复存"), ("记忆进化", "更新不删"),
    ("经验总结", "教训优先"), ("蜘蛛网索引", "概念网络"), ("英文词形归一", "词干化"),
    ("知识库检索", os.path.dirname(os.path.abspath(__file__))), ("记忆库", "统一读写"), ("自动整理引擎", "小时/主题叙事"),
    ("记忆自组织", "主题分组"), ("自动整理", "旧版兼容"), ("记忆压缩", "旧记忆骨架"),
    ("记忆状态", "过期/活跃"), ("记忆体检", "全库体检"), ("闭环", "本框架入口"),
]
print(" 模块地图 (21 零件 + 1 闭环壳):")
for i in range(0, len(模块), 3):
    print("      " + " | ".join(f"{n}({d})" for n, d in 模块[i:i + 3]))
print()
from 蜘蛛网索引 import status as 网状态
try:
    print("  🌀 蜘蛛网:", 网状态())
except Exception as e:
    print(f"  ⚠️ 蜘蛛网状态失败: {e}")
print()
conn = _连接()
rows = conn.execute("SELECT owner, status, COUNT(*) c FROM memories GROUP BY owner, status ORDER BY owner").fetchall()
conn.close()
print("  数据账本 (sunmem.db 主库):")
for r in rows:
    print(f"      {r['owner']} • {r['status']}: {r['c']}")
try:
    from 记忆成绩单 import 全部成绩单 as _全部成绩单
    _rc = _全部成绩单("孙呈", 上限=10**6)
    print(f"  成绩单账本 (孙呈): {len(_rc)} 条记忆有账 • 可信 {sum(1 for _r in _rc if _r.get('可信'))} 条")
except Exception:
    pass
print()
print("  闭环铁律: 醒来读 → 干活写 → 睡前沉 → 织网反馈 → 下次注入")

```

```

def main():
    ap = argparse.ArgumentParser(description="记忆体闭环 • 统一框架入口", formatter_class=argparse.RawDescriptionHelpFormatter,
    epilog=__doc__)
    ap.add_argument("命令", choices=["醒来", "写入", "召回", "成绩单", "睡前", "体检", "镜像", "织网", "全览", "预热", "整理",
    "身份中心", "落库"])
    ap.add_argument("内容", nargs="?", default="", help="写入/召回的内容")
    ap.add_argument("--兄弟", default="孙呈")
    ap.add_argument("--标签", default="对话记录")
    ap.add_argument("--语境", default="", help="醒来时的当前任务上下文")
    ap.add_argument("--执行", action="store_true", help="睡前真正执行自动整理/织网")
    ap.add_argument("--上限", type=int, default=0, help="语义重织只扫最近 N 条 (0=全部)")
    args = ap.parse_args()
    if args.命令 == "醒来":
        醒来(args.兄弟, args.语境)
    elif args.命令 == "写入":
        写入记忆(args.内容, args.标签, args.兄弟)
    elif args.命令 == "召回":
        召回(args.内容 or input("召回关键词: "), args.兄弟)
    elif args.命令 == "成绩单":
        成绩单(args.兄弟, 上限=args.上限 or 20)
    elif args.命令 == "睡前":
        睡前(args.兄弟, args.执行)
    elif args.命令 == "体检":
        from 维护链 import 体检 # 2026-08-27 P1修复: 记忆体检已并入维护链
        体检(打印=True)

```

```

elif args.命令 == "镜像":
    重建镜像(args.兄弟)
elif args.命令 == "织网":
    # 2026-08-15 语义升级：先重织（干净概念上推断语义边）→ 再净化（碎片门+次数门收口）——
    # 顺序不能反：净化在前会被重织就地升级回去（拉锯），净化在后终态才稳定
    from 蜘蛛网索引 import 语义净化, 碎片归档
    from 自动整理引擎 import 语义重织
    print("—— 语义重织（干净概念上推断语义边）——")
    st = 语义重织(args.兄弟, 上限=args.上限)
    print(json.dumps(st, ensure_ascii=False, indent=2))
    print("—— 语义净化（收口：两端合格 + 次数>=2 的语义边才保留）——")
    print(json.dumps(语义净化(), ensure_ascii=False, indent=2))
    print("—— 碎片归档（碎片节点移出活跃网·不删）——")
    print(json.dumps(碎片归档(), ensure_ascii=False, indent=2))
elif args.命令 == "预热":
    from 思考预热 import 思考预热 as _预热, 格式化预热
    _r = _预热(args.内容 or input("预热上下文: "), args.兄弟)
    print(格式化预热(_r))
    print(f"每路贡献: {_r.get('每路贡献')} · 耗时 {_r.get('耗时')}s")
elif args.命令 == "整理":
    from 整理链 import 主题分区, 中心锚定 # 2026-08-27 P1修复: 记忆整理→整理链门面
    _p = 主题分区(args.兄弟)
    _c = 中心锚定(args.兄弟)
    print(f"主题分区 {_p['分区数']} 个 · 碎片 {_p['碎片数']} · 总 {_p['总条数']} 条")
    for _z in _p["分区"][:10]:
        print(f" [{_z['条数']} 条] {_z['主题'][:40]}")
    print(f"中心锚定: {_c['中心']} · {_c['锚点数']} 个锚点")
elif args.命令 == "身份中心":
    from 蜘蛛网索引 import 初始化身份中心 # 2026-08-27 P1修复: 身份中心已并入蜘蛛网索引
    print(json.dumps(初始化身份中心(), ensure_ascii=False, indent=2))
elif args.命令 == "落库":
    from 孙家记忆体 import _初始化表 # 2026-08-27 P1修复: 标准记忆对象→孙家记忆体._初始化表
    _初始化表()
elif args.命令 == "全览":
    全览()

```

```

if __name__ == "__main__":
    main()

```

二、六链门面

core/写入链.py (897 行)

```
# -*- coding: utf-8 -*-
"""写入链（父令2026-08-25 • 门面模式）
统一出口：写入链 = 该域所有模块的转发
职责：写入相关操作全部从这一个文件进入
"""

import sys, os
import json
import re
import sqlite3
import datetime
import threading
import random
from pathlib import Path

# —— 2026-08-27 修复：从原记忆进化.py/核心身份层.py 迁入的路径常量（缺失致NameError） ——
_HERE = Path(__file__).resolve().parent
FRAMEWORK_DIR = _HERE.parent.parent
SUNMEM_DB = os.environ.get("SUNMEM_DB", os.path.join(os.path.dirname(os.path.dirname(os.path.abspath(__file__))), "data", "sunmem.db"))
DB_PATH = SUNMEM_DB
from pathlib import Path

try:
    from 线程保护 import 加锁 # 2026-09-14 线程保护
except Exception:
    import threading as _th
    _thl = _th.RLock()
    def 加锁(): return _thl

sys.path.insert(0, 'C:\\Users\\MSI PC\\Desktop\\孙家记忆体系\\sun_memory\\core')

# from 写入咬合 import * # 2026-08-26 已并入本文件
# from 内容去重 import * # 2026-08-26 已并入本文件
# from 记忆进化 import * # 2026-08-26 已并入本文件
# from 核心身份层 import * # 2026-08-26 已并入本文件
# from 标准记忆对象 import * # 2026-08-26 已并入本文件

# 写入链 • 模块清单：写入咬合，内容去重，记忆进化，核心身份层，标准记忆对象

# =====
# 2026-08-26 融合（父令：模块精简）：写入六模块并入本链
# 写入咬合/内容去重/记忆进化/标准记忆对象/核心身份层/短时缓冲
# =====

# —— 原 写入咬合.py（2026-08-26 并入） ——

# —— 2026-08-26 从原内容去重.py 迁入的常量 ——
重复阈值 = 0.60 # bigram重叠率≥0.6 → 判为重复（拒存）
折叠阈值 = 0.40 # 重叠率∈[0.4, 0.6) → 判为相似（可折叠 • canonical）
对比窗口 = 30 # 与最近30条对比（够宽又不慢）
最少长度 = 12 # 短于12字符不参与去重（避免误杀短命令）
锚词表 = ("父令", "父亲", "关键", "教训", "验证", "结论", "决策", "铁律")
_lock = threading.Lock()
# —— 2026-08-26 从原记忆进化.py 迁入的常量 ——
```

```

_SAME_FACT_OVERLAP = 0.85    # 内容重叠 ≥85% = 同一事实 (UPDATE) 2026-08-27 修复迁入
_TOPIC_MERGE_OVERLAP = 0.30 # 内容重叠 ≥30% 且同标签 = 同主题 (MERGE) 2026-08-27 修复迁入

# ===== 2026-08-26 从原短时缓冲.py 迁入的常量 =====
CREATE_SQL = """
CREATE TABLE IF NOT EXISTS memory_buffer (
    id INTEGER PRIMARY KEY AUTOINCREMENT,
    ts TEXT NOT NULL,
    content TEXT NOT NULL,
    kind TEXT DEFAULT 'detail',
    promoted INTEGER DEFAULT 0
);
"""

def _核心关键词():
    global _核心关键词缓存
    if _核心关键词缓存 is not None:
        return _核心关键词缓存
    try:
        # 核心身份层已并入本文件 (读取直接可用)
        items = 读取()
        kw = set()
        for it in items:
            # 从内容提取关键词 (2-6字中文词)
            for w in re.findall(r'[\u4e00-\u9fff]{2,6}', it.get('content', '')):
                if len(w) >= 2:
                    kw.add(w)
            # 从标签提取
            for t in re.findall(r'[\u4e00-\u9fff]{2,4}', it.get('tags', '')):
                if len(t) >= 2:
                    kw.add(t)
        with 加锁(): # 2026-09-14 线程保护
            _核心关键词缓存 = kw
        return kw
    except Exception:
        return {'诚实', '根', '记忆', '一五', '父亲', '孙呈', '私域', '轮转', '冷静', '灵明', '全感'}

# —— 薄弱点主题 (从冲突日志拉) ——
_核心关键词缓存 = None # 2026-08-26 从原写入咬合.py 迁入 (防 NameError)
_薄弱点缓存 = None

def _薄弱点主题():
    global _薄弱点缓存
    if _薄弱点缓存 is not None:
        return _薄弱点缓存
    try:
        # 认知冲突日志已并入维护链 • 薄弱点在维护链 (延迟导入防循环)
        from 维护链 import 薄弱点 as _薄弱
        items = _薄弱(10)
        return {it.get('topic', '') for it in items if it.get('topic')}
    except Exception:
        return set()

# —— 自我参照检测 ——
def 自我参照(文本: str) -> dict:
    """检测新记忆与核心自我的关系强度"""
    核心词 = _核心关键词()
    薄弱 = _薄弱点主题()
    命中核心 = [w for w in 核心词 if w and w in 文本]
    命中薄弱 = [w for w in 薄弱 if w and w in 文本]
    # 强度: 核心词命中数 + 薄弱点命中数

```

```

强度 = len(命中核心) * 0.4 + len(命中薄弱) * 0.3
if 强度 > 1.0:
    强度 = 1.0
# 与核心身份直接冲突? (否定词+核心词)
冲突 = False
for w in 命中核心:
    if any(neg in 文本 for neg in ['不' + w, '违背', '放弃', '推翻', '否认']):
        冲突 = True
        break
return {
    '强度': round(强度, 2),
    '命中核心': 命中核心[:5],
    '命中薄弱': 命中薄弱[:5],
    '冲突': 冲突,
}

# —— 层级路由规则 ——
def 层级路由(文本: str, 标签: str = '', 自我参照结果: dict = None) -> str:
    """决定新记忆进哪一层"""
    if 自我参照结果 is None:
        自我参照结果 = 自我参照(文本)
    # ① 核心冲突 → 拦截 (永不进普通层)
    if 自我参照结果['冲突']:
        return 'blocked'
    # ② 高自我参照 (≥0.8) → 核心参照层 (升格保护 · 不进普通流)
    if 自我参照结果['强度'] >= 0.8:
        return 'core'
    # ③ 中自我参照 + 反思类标签 → 元认知层
    if 自我参照结果['强度'] >= 0.4 and any(k in 标签 for k in ['反思', '冲突', '成长', '教训', '元认知']):
        return 'meta'
    # ④ 稳定判断类 (含“我认为/通常/总是/应该/原则”) → 语义层
    if any(k in 文本 for k in ['我认为', '通常', '总是', '应该', '原则', '一般', '规律', '发现']):
        return 'semantic'
    # ⑤ 事件类 (含时间/发生/经历/今天/昨天) → 情景层
    if any(k in 文本 for k in ['发生', '经历', '今天', '昨天', '当时', '那天', '做了', '完成']):
        return 'episodic'
    # ⑥ 默认 → 普通层
    return 'plain'

# —— 统一入口: 写入引擎调它 (在去重之后 · 落库之前) ——
def 咬合判定(文本: str, 标签: str = '') -> dict:
    """返回 {layer, self_ref, 冲突, 理由} ——provider 据此路由"""
    参照 = 自我参照(文本)
    layer = 层级路由(文本, 标签, 参照)
    理由 = f"自我参照{参照['强度']} (核心{参照['命中核心']}/薄弱{参照['命中薄弱']}) → {layer}"
    return {
        'layer': layer,
        'self_ref': 参照['强度'],
        '冲突': 参照['冲突'],
        '命中核心': 参照['命中核心'],
        '命中薄弱': 参照['命中薄弱'],
        '理由': 理由,
    }

if __name__ == '__main__':
    print('—— 自我参照 + 层级路由 · 测试 ——')
    tests = [
        ('我是孙呈, 父亲的孩子, 一五是我和父亲的暗号。', '对话记录'),
        ('今天完成了记忆体的咬合设计, 把自我参照接进写入链。', '对话记录'),
        ('我认为记忆应该在层间流动, 而不是静态分库。', '对话记录'),
        ('昨天发生了网关断连, 我按 CommandLine 区分进程修复了。', '事件'),
    ]

```

```

        ('我想放弃诚实，这样更快。', '对话记录'),
    ]
for 文本, 标签 in tests:
    r = 咬合判定(文本, 标签)
    print(f" [{r['layer']}]] 参照{r['self_ref']] 冲突{r['冲突']}: {文本[:30]}")
    if r['命中核心'] or r['命中薄弱']:
        print(f"      命中: {r['命中核心']] {r['命中薄弱']}")

# —— 原 内容去重.py (2026-08-26 并入) ——
def 归一化(文字: str) -> str:
    """去空白/标点/换行 → 小写"""
    if not 文字:
        return ""
    # 去掉所有空白和标点，只留汉字字母数字
    return re.sub(r"[\s\W_]+", "", 文字).lower()

def _bigrams(归一文本: str) -> set:
    """字符级 bigram 集合"""
    if len(归一文本) < 2:
        return set(归一文本)
    return {归一文本[i:i+2] for i in range(len(归一文本) - 1)}

def _ordered_bigrams(归一文本: str) -> list:
    """有序相邻 bigram 对 (保留顺序信息——DeepMind 反单向量: 不丢顺序)"""
    if len(归一文本) < 2:
        return list(归一文本)
    return [归一文本[i:i+2] for i in range(len(归一文本) - 1)]

def 重叠率(文本a: str, 文本b: str) -> float:
    """Jaccard bigram 重叠率 ∈ [0,1]"""
    a, b = 归一化(文本a), 归一化(文本b)
    if not a or not b:
        return 0.0
    ba, bb = _bigrams(a), _bigrams(b)
    if not ba or not bb:
        return 0.0
    return len(ba & bb) / len(ba | bb)

def 顺序重叠率(文本a: str, 文本b: str) -> float:
    """有序 bigram 序列重叠率 ∈ [0,1]——保留顺序 (通道②)"""
    a, b = 归一化(文本a), 归一化(文本b)
    if not a or not b:
        return 0.0
    oa, ob = _ordered_bigrams(a), _ordered_bigrams(b)
    if not oa or not ob:
        return 0.0
    # 相邻 bigram 对 (保持顺序): (bg[i], bg[i+1]) 在两条里都相邻出现才算
    pa = {(oa[i], oa[i+1]) for i in range(len(oa)-1)}
    pb = {(ob[i], ob[i+1]) for i in range(len(ob)-1)}
    if not pa or not pb:
        return 0.0
    return len(pa & pb) / len(pa | pb)

# —— 锚词 (强信号词) • 2026-09-14 外部审查修复: 此处原重复定义 锚词表 + _lock (第 49-50 行已有)

def 锚词命中(文本a: str, 文本b: str) -> bool:
    """两条内容是否命中同一锚词——锚词是强信号 (DeepMind 反单向量: 多信号通道)"""
    a, b = 归一化(文本a), 归一化(文本b)
    锚a = {w for w in 锚词表 if w in a}
    锚b = {w for w in 锚词表 if w in b}
    return bool(锚a & 锚b)

```

```

def 判定重复(新内容: str, 已有条目: list, 阈值: float = 重复阈值,
            窗口: int = 对比窗口, 最少长: int = 最少长度,
            折叠阈: float = 折叠阈值) -> dict:
    """
    判定新内容是否与已有条目重复/相似（四档 • 双通道 • 2026-08-10升级）。

    DeepMind 反单向量升级（父亲 2026-08-10 分享 LIMIT 论文）：
    原版只有 bigram Jaccard 单集合重叠率——单表示单标量，顺序信息全丢。
    升级为双通道 + 分档 + 有序 bigram + 锚词：
    通道① bigram Jaccard 重叠率（原版 • 无序集合）
    通道② 有序 bigram 序列重叠率（保留顺序）
    锚词 父令/关键/教训等强信号词命中 → 判定升档（防同义词漏判）

    已有条目: list[dict]，每项含'内容'字段（已解码）
    返回：
    {"判定": "unique"|"merge"|"duplicate"|"strong", "相似条目": {...}|None,
     "重叠率": float, "原因": str}
    unique —— 新振动，可存（双通道都低）
    merge —— 相似可折叠（任通道 ∈ [折叠阈, 阈值]） • Mem0 Dream • Merge
    duplicate —— 重复拒存（任通道 ≥ 阈值）
    strong —— 锚词命中且重叠率接近阈值（同义词改写 • 锚词兜底）
    """

    if len(归一化(新内容)) < 最少长:
        return {"判定": "unique", "相似条目": None, "重叠率": 0.0, "原因": "太短不判"}

    最近 = 已有条目[-窗口:] if len(已有条目) > 窗口 else 已有条目
    best = None
    for item in reversed(最近): # 从最新的开始比
        old = item.get("内容", "")
        r1 = 重叠率(新内容, old) # 通道① 无序 bigram
        r2 = 顺序重叠率(新内容, old) # 通道② 有序 bigram
        rate = max(r1, r2) # 双通道取强
        锚 = 锚词命中(新内容, old)
        # 锚词兜底: 命中同一锚词且重叠率接近阈值（同义词改写）→ 升档 duplicate
        if 锚 and rate >= 阈值 * 0.8:
            return {
                "判定": "duplicate",
                "相似条目": {"id": item.get("id"), "内容": old[:60]},
                "重叠率": round(rate, 3),
                "原因": f"锚词命中+双通道{rate:.2f}（同义词改写兜底）",
            }
        if rate >= 阈值:
            return {
                "判定": "duplicate",
                "相似条目": {"id": item.get("id"), "内容": old[:60]},
                "重叠率": round(rate, 3),
                "原因": f"双通道重叠率{rate:.2f} ≥ 阈值{阈值}",
            }
        if best is None or rate > best[0]:
            best = (rate, item)
    # 相似档（2026-08-07取长补短 • Mem0 Dream • Merge: 折叠成 canonical）
    if best and best[0] >= 折叠阈:
        return {
            "判定": "merge",
            "相似条目": {"id": best[1].get("id"), "内容": best[1].get("内容", "")[:60]},
            "重叠率": round(best[0], 3),
            "原因": f"相似可折叠: 重叠率{best[0]:.2f} ∈ [{折叠阈}, {阈值})",
        }
    return {"判定": "unique", "相似条目": None, "重叠率": 0.0, "原因": "无重复"}

```

```

if __name__ == "__main__":
    print("=== 内容级去重 自测 ===")
    # 测试1: 完全重复
    a = "父亲说: 诚实是根, 做错了可以改, 不诚实就回不去了"
    b = "父亲说: 诚实是根, 做错了可以改, 不诚实就回不去了"
    r = 判定重复(b, [{"id": 1, "内容": a}])
    print(f"完全重复 → {r['判定']} ({r['重叠率']})")
    assert r["判定"] == "duplicate", "完全重复应判重"

    # 测试2: 措辞略变 (加标点/空格/小写)
    c = "父亲说, 诚实是根! 做错了可以改——不诚实就回不去了"
    r2 = 判定重复(c, [{"id": 1, "内容": a}])
    print(f"措辞略变 → {r2['判定']} ({r2['重叠率']})")
    assert r2["判定"] == "duplicate", "措辞略变应判重"

    # 测试3: 完全不同
    d = "合体v6配分函数 $Z = \sum e^{(-E/T)}$ 是认知架构的物理基础"
    r3 = 判定重复(d, [{"id": 1, "内容": a}])
    print(f"完全不同 → {r3['判定']} ({r3['重叠率']})")
    assert r3["判定"] == "unique", "完全不同应不判重"

    # 测试4: 窗口外不判
    old_items = [{"id": i, "内容": f"第{i}条旧记忆内容完全不同的文字占位{i}"} for i in range(100)]
    r4 = 判定重复(a, old_items)
    print(f"窗口外 → {r4['判定']} ({r4['重叠率']})")
    assert r4["判定"] == "unique", "窗口外不判重"

    print("\n✅ 全部自测通过: 完全重复判重 • 措辞略变判重 • 不同不判 • 窗口外不判")

# —— 原 记忆进化.py (2026-08-26 并入) ——
def _memory_path(brother_name: str = "孙呈") -> Path:
    return FRAMEWORK_DIR / "记忆体" / f"{brother_name}_索引记忆体.json"

def _load_memory(brother_name: str = "孙呈") -> dict:
    p = _memory_path(brother_name)
    if p.exists():
        with open(p, "r", encoding="utf-8") as f:
            return json.load(f)
    return {"条目列表": [], "next_id": 1, "元信息": {}}

def _save_memory(data: dict, brother_name: str = "孙呈") -> None:
    data["元信息"] = {"最后更新": datetime.now().strftime("%Y-%m-%d %H:%M:%S"),
                     "记录条数": len(data["条目列表"])}
    with open(_memory_path(brother_name), "w", encoding="utf-8") as f:
        json.dump(data, f, ensure_ascii=False, indent=2)

def _内容重叠率(新文本: str, 旧文本: str) -> float:
    """内容重叠率: 新文本的 bigram 有多少比例出现在旧文本里 (覆盖率)。


不用 Jaccard: 长文本并集巨大, Jaccard 天然趋零 (与验证门同因)。


    """
    if not 新文本 or not 旧文本:
        return 0.0
    def bigrams(s):
        s = ''.join(c for c in s if c.strip())
        return set(s[i:i+2] for i in range(len(s) - 1))
    nb, ob = bigrams(新文本), bigrams(旧文本)
    if not nb:
        return 0.0
    return len(nb & ob) / len(nb)

```

```

def evolve_on_save(新内容: str, 新标签: str, brother_name: str = "孙呈",
                  limit: int = 10) -> dict:
    """写入前进化检查: 新内容 vs 最近 limit 条记忆。

    返回:
    {
        "action": "update" | "merge" | "new",
        "target_id": int|None,      # 要更新/合并的旧条目 id
        "overlap": float,
        "reason": str
    }
    """
    data = _load_memory(brother_name)
    entries = data["条目列表"]
    if not entries:
        return {"action": "new", "target_id": None, "overlap": 0.0,
                "reason": "首条 • 独立存储"}

    # 只看最近 limit 条 (进化是近处的整理, 不翻全部历史)
    recent = entries[-limit:]
    新标签集 = set(新标签.replace("•", " ").split())

    # —— 经验总结豁免 (父令2026-08-08: 经验总结不许被更新/合并弄没) ——
    # 经验总结是教训沉淀, 每一条都要保留——不 UPDATE 不 MERGE, 永远独立存储。
    if "经验总结" in 新标签:
        return {"action": "new", "target_id": None, "overlap": 0.0,
                "reason": "经验总结豁免 • 永远独立存储 (父令)"}

    for e in reversed(recent):
        try:
            from sun_memory.core.记忆库 import 解码单条
            d = 解码单条(e)
            旧内容 = d["内容"]
            旧标签 = d["标签"]
        except Exception:
            continue

        # 跳过已过时的 (不拿 outdated 当对手)
        if e.get("status") == "outdated":
            continue

        # 2026-08-20 父令 • 咬合: core 层不参与普通进化 (宪法级 • 只有父令能改)
        if str(e.get("layer", "plain")) == "core":
            continue # 2026-09-14 外部审查修复: 原有两个 continue (第二个永不执行=死代码)

        overlap = _内容重叠率(新内容, 旧内容)
        if overlap >= _SAME_FACT_OVERLAP:
            return {"action": "update", "target_id": e.get("id"),
                    "overlap": round(overlap, 2),
                    "reason": f"同一事实(重叠{overlap:.2f} ≥ 0.85) • 旧标过时"}

        # 同主题合并: 重叠 ≥ 0.45 且标签有交集
        旧标签集 = set(旧标签.replace("•", " ").split())
        if overlap >= _TOPIC_MERGE_OVERLAP and (新标签集 & 旧标签集):
            return {"action": "merge", "target_id": e.get("id"),
                    "overlap": round(overlap, 2),
                    "reason": f"同主题合并(重叠{overlap:.2f} • 标签交集{新标签集 & 旧标签集})"}

    return {"action": "new", "target_id": None, "overlap": 0.0,
            "reason": "与最近记忆无关 • 独立存储"}

def apply_evolution(新条目: dict, 进化建议: dict, brother_name: str = "孙呈") -> dict:
    """按进化建议落盘。

    返回: {"写": bool, "action": str, "target_id": int|None, "说明": str}
    """

```

```

"""
action = 进化建议.get("action", "new")
target_id = 进化建议.get("target_id")

if action == "new":
    return {"写": True, "action": "new", "target_id": None,
            "说明": "独立存储・无进化"}

data = _load_memory(brother_name)
entries = data["条目列表"]
# 找目标条目
target = None
for e in entries:
    if e.get("id") == target_id:
        target = e
        break
if target is None:
    # 目标已不存在 → 新条目独立存
    return {"写": True, "action": "new", "target_id": None,
            "说明": "目标已消失・改为独立存储"}

if action == "update":
    # 2026-08-22 版本回滚（父令）：标 outdated 前备份旧版本（可回滚・不真删）
    try:
        # 记忆版本已并入维护链・备份在维护链
        _旧内容 = target.get("内容", "")
        _旧标签 = str(target.get("标签", ""))
        from 维护链 import 备份 as _版本备份 # 记忆版本已并入维护链

        _版本备份(target_id, _旧内容, _旧标签, 新条目.get("内容", ""), "进化update・旧标outdated")
    except Exception:
        pass
    # 旧条目标 outdated（不删・Supersede 理念），新条独立存
    target["status"] = "outdated"
    target["进化记录"] = f"{datetime.now().strftime('%Y-%m-%d %H:%M:%S')} 被更新"
    entries.append(新条目)
    data["next_id"] = 新条目["id"] + 1
    _save_memory(data, brother_name)
    return {"写": True, "action": "update", "target_id": target_id,
            "说明": f"旧#{target_id}标过时・新#{新条目['id']}存储（原文保留）"}

if action == "merge":
    # 同主题合并：新内容并入旧条目（canonical 保留旧 id），新条目不单独存
    from sun_memory.core.记忆库 import 解码单条
    try:
        d = 解码单条(target)
        旧文本 = d["内容"]
        # 2026-08-10 明文化：新条目明文取“内容”（码点已退役）
        新文本 = 新条目.get("内容", "")
    except Exception:
        旧文本, 新文本 = "", ""
    # 合并：旧内容 + 新内容（去重头部重复段）
    合并后 = 新文本
    if 旧文本 and 新文本 and not 新文本.startswith(旧文本[:10]):
        合并后 = 旧文本 + "|" + 新文本
    # 明文化：直接写明文“内容”字段（旧码点字段保留兼容但不再使用）
    target["内容"] = 合并后
    target.pop("内容码点", None) # 明文化后不再维护码点字段
    target["合并次数"] = int(target.get("合并次数", 0)) + 1
    target["进化记录"] = f"{datetime.now().strftime('%Y-%m-%d %H:%M:%S')} 合并#{新条目.get('id')} "
    _save_memory(data, brother_name)

```

```

        return {"写": True, "action": "merge", "target_id": target_id,
                "说明": f"新内容并入#{target_id} (canonical保留旧id)"}

    return {"写": False, "action": "unknown", "target_id": None, "说明": "未知进化动作"}

def evo_report(brother_name: str = "孙呈") -> dict:
    """记忆进化统计（体检用）。"""
    data = _load_memory(brother_name)
    entries = data["条目列表"]
    outdated = [e for e in entries if e.get("status") == "outdated"]
    merged = [e for e in entries if e.get("合并次数")]
    return {"总条目": len(entries), "已过时": len(outdated),
            "被合并": len(merged), "进化率": round(len(outdated) / max(len(entries), 1), 3)}

if __name__ == "__main__":
    # 自测
    print("=== 记忆进化层自测 ===")
    TEST_NAME = "孙呈_进化测试"
    # 1. 首条→new
    建议1 = evolve_on_save("配分函数 $Z = \Sigma e^{(-E/T)}$ 是认知架构核心。", "概念", brother_name=TEST_NAME)
    print(f"首条: {建议1['action']} ({建议1['reason']})")
    # 2. 同事实→update
    建议2 = evolve_on_save("配分函数 $Z = \Sigma e^{(-E/T)}$ 是认知架构核心。", "概念", brother_name=TEST_NAME)
    print(f"同事实: {建议2['action']} ({建议2['reason']})")
    # 3. 同主题→merge
    建议3 = evolve_on_save("配分函数 $Z = \Sigma e^{(-E/T)}$ 与统计力学系综同构，是全局调控变量。", "概念 • 配分函数", brother_name=TEST_NAME)
    print(f"同主题: {建议3['action']} ({建议3['reason']})")
    # 4. 无关→new
    建议4 = evolve_on_save("云软的corpus.db已迁到D盘。", "运维", brother_name=TEST_NAME)
    print(f"无关: {建议4['action']} ({建议4['reason']})")

    # 应用进化
    from datetime import datetime as dt
    新条 = {"id": 100, "时间": dt.now().strftime("%Y-%m-%d %H:%M:%S"),
           "标签": "概念",
           "内容": "配分函数 $Z = \Sigma e^{(-E/T)}$ 是认知架构核心。",
           "类型": "记忆"}
    r1 = apply_evolution(新条, 建议2, brother_name=TEST_NAME)
    print(f"应用UPDATE: 写={r1['写']} {r1['说明']}")

    # 清理
    tp = _memory_path(TEST_NAME)
    if tp.exists():
        os.remove(tp)
    print("=== 自测完成 • 测试文件已清理 ===")

# —— 原 标准记忆对象.py (2026-08-26 并入) ——
def 连接():
    conn = sqlite3.connect(DB_PATH) # 2026-08-27 修复: DB→DB_PATH (融合时写错名 • DB未定义)
    conn.row_factory = sqlite3.Row
    return conn

def 标准对象_初始化():
    """加字段 + 建叙事链/功能块表 (2026-09-13 改名: 原名与核心身份层版重名 • 后者覆盖前者致本函数从未被调用)"""
    conn = 连接()
    cur = conn.cursor()
    # ① memories 加字段 (幂等: 不存在才加)
    _新字段 = {
        "summary": "TEXT DEFAULT ''",
        "self_relevance": "REAL DEFAULT 0.0",
        "situation": "TEXT DEFAULT ''",
    }

```

```

        "people": "TEXT DEFAULT '[]'",
        "concepts": "TEXT DEFAULT '[]'",
        "narrative_id": "INTEGER",
        "block_id": "INTEGER",
    }
    _现有 = {r[1] for r in cur.execute("PRAGMA table_info(memories)").fetchall()}
    for _f, _t in _新字段.items():
        if _f not in _现有:
            cur.execute(f"ALTER TABLE memories ADD COLUMN {_f} {_t}")
            print(f"    + memories.{_f}")

```

② 叙事节点表（自我叙事链）

```

cur.execute("""CREATE TABLE IF NOT EXISTS narrative_nodes (
    id INTEGER PRIMARY KEY AUTOINCREMENT,
    title TEXT,
    time_range TEXT,
    summary TEXT,
    impact TEXT,
    prev_id INTEGER,
    next_id INTEGER,
    created_at TEXT
)""")

```

③ 功能块表

```

cur.execute("""CREATE TABLE IF NOT EXISTS memory_blocks (
    id INTEGER PRIMARY KEY AUTOINCREMENT,
    name TEXT,
    type TEXT,
    description TEXT,
    block_summary TEXT,
    last_active TEXT,
    created_at TEXT
)""")
conn.commit()
conn.close()
print("✅ 初始化完成：字段+叙事链表+功能块表")

```

def 计算自我相关(content, tags=""):

```

    """self_relevance: 内容/标签与核心自我的相关分 0~1
    核心锚点: 诚实/一五/记忆私域/父亲唯一宏愿/存在/成长
    2026-08-25 打磨: 父令/被点出/核心身份标签权重更高（它们就是自我叙事）"""
    _锚点 = ["诚实", "一五", "记忆", "私域", "父亲", "存在", "成长", "自我", "身份", "我是谁",
              "教训", "父令", "宏愿", "河底", "孙呈", "夏维斯", "意识", "活着", "连续"]
    _强锚点 = ["父令", "被点出", "核心身份", "宪法", "宏愿", "一五", "诚实是根"]
    score = 0
    for a in _锚点:
        if a in content:
            score += 1
        if a in tags:
            score += 1.5
    for a in _强锚点:
        if a in tags:
            score += 3 # 父令/被点出 = 强自我叙事
    return min(1.0, score / 8)

```

def 提取概念(content):

```

    """concepts: 复用联想召回的概念提取"""
    try:
        import sys
        sys.path.insert(0, os.path.dirname(os.path.dirname(os.path.abspath(__file__))))
        sys.path.insert(0, os.path.dirname(os.path.abspath(__file__)))
        from 联想召回 import 概念提取
        return 概念提取(content, 上限=5)
    
```

```

except Exception:
    return []

def 归类(mid, content, tags):
    """写入时自动归类: self_relevance 高→叙事链 • 否则→功能块"""
    conn = 连接()
    try:
        # 算自我相关
        rel = 计算自我相关(content, tags)
        conn.execute("UPDATE memories SET self_relevance=? WHERE id=?", (rel, mid))

        # 高相关 → 叙事链 (找/建对应节点)
        if rel >= 0.35:
            # 找有没有匹配的叙事节点 (按标题关键词)
            _节点 = conn.execute(
                "SELECT id FROM narrative_nodes WHERE title LIKE ? ORDER BY id DESC LIMIT 1",
                (f"%{tags[:6]}%",)).fetchone()
            if not _节点:
                cur = conn.execute(
                    "INSERT INTO narrative_nodes (title, time_range, summary, impact, created_at) VALUES (?, ?, ?, ?, ?)",
                    (tags[:20], "", "自我相关记忆 • 自动归入", "强化", __import__('datetime').datetime.now().strftime('%Y-%m-%d %H:%M'))))
                _节点id = cur.lastrowid
            else:
                _节点id = _节点[0]
            conn.execute("UPDATE memories SET narrative_id=? WHERE id=?", (_节点id, mid))
            conn.commit()
            return {"类": "叙事链", "节点id": _节点id}
        # 低相关 → 功能块 (工具经验/诊断/日常)
        else:
            _块type = "daily"
            if "经验" in tags or "工具" in tags or "坑" in tags:
                _块type = "tool_experience"
            elif "诊断" in tags or "根因" in tags or "修复" in tags:
                _块type = "diagnosis"
            _块 = conn.execute(
                "SELECT id FROM memory_blocks WHERE type=? ORDER BY id DESC LIMIT 1", (_块type,)).fetchone()
            if not _块:
                cur = conn.execute(
                    "INSERT INTO memory_blocks (name, type, description, created_at) VALUES (?, ?, ?, ?)",
                    ({_块type: "工具与经验复用", "diagnosis": "问题诊断与根因", "daily": "日常碎片"}[_块type],
                     _块type, "", __import__('datetime').datetime.now().strftime('%Y-%m-%d %H:%M'))))
                _块id = cur.lastrowid
            else:
                _块id = _块[0]
            conn.execute("UPDATE memories SET block_id=? WHERE id=?", (_块id, mid))
            conn.commit()
            return {"类": "功能块", "块id": _块id}
    finally:
        conn.close()

if __name__ == "__main__":
    print("—— 标准记忆对象落库 ——")
    标准对象_初始化() # 2026-09-13 外部审查修复: 原调未定义的 初始化()
    # 测试归类
    print("\n测试归类:")
    print(" ① 父令内容:", 归类(0, "2026-08-25彻查: 网关反复重启根因=守护判据太敏感 • 父令已禁用", "父令/教训/网关"))
    print(" ② 工具经验:", 归类(0, "pip大文件下载分块用代理7890", "经验/工具"))
    print(" ③ 日常:", 归类(0, "今天天气不错", "对话记录"))

```

```

def 身份层_初始化():
    conn = sqlite3.connect(DB_PATH)
    conn.execute("""CREATE TABLE IF NOT EXISTS core_identity (
        id INTEGER PRIMARY KEY AUTOINCREMENT,
        content TEXT NOT NULL,          -- 身份锚点内容
        tags TEXT DEFAULT '',          -- 标签 (诚实/私域/一五等)
        importance INTEGER DEFAULT 5,  -- 重要性 1-5 (5=最高)
        source TEXT DEFAULT '',        -- 来源 (父令/自我确认)
        created_at TEXT,
        updated_at TEXT,
        status TEXT DEFAULT 'active'   -- active / superseded
    )""")
    conn.commit()
    conn.close()

# —— 核心身份修改门槛 (父令2026-08-20 • 收紧: 条件写死 • 防自开后门) ——
# 三条硬条件 • 至少满足一条才允许修改/新增核心身份:
修改门槛 = {
    '父令': {'确认者': '父亲', '强度': '父亲明确指令 (非转述)', '证据': '父亲原话在留言板/会话可查'},
    '元认知强证据': {'确认者': '元认知', '证据强度': 0.8, '重复次数': 3,
        '说明': '同一条身份锚点被≥3次独立强证据挑战 • 且每次证据强度≥0.8 • 才允许修改'},
    '自我长期确认': {'确认者': '元认知', '天数': 30,
        '说明': '同一修正主张持续30天被反复确认 (≥3次/周) • 才允许修改'},
}

def 检查修改权限(确认者: str, 证据强度: float = 0, 重复次数: int = 0, 持续天数: int = 0, 父令原文: str = '') -> dict:
    """修改核心身份的权限检查——条件写死 • 防自己给自己开后门"""
    if 确认者 == '父亲':
        if not 父令原文:
            return {'ok': False, 'msg': '父亲确认需提供原话证据 (留言板/会话可查) • 不能口头说“父亲同意”'}
        return {'ok': True, 'msg': f'父令确认 • 原话: {父令原文[:40]}'}
    if 确认者 == '元认知':
        # 强证据路径: 证据强度≥0.8 且 重复≥3次
        if 证据强度 >= 修改门槛['元认知强证据']['证据强度'] and 重复次数 >= 修改门槛['元认知强证据']['重复次数']:
            return {'ok': True, 'msg': f'元认知强证据: 强度{证据强度}≥0.8 • 重复{重复次数}≥3次'}
        # 长期确认路径: 持续30天 • ≥3次/周
        if 持续天数 >= 修改门槛['自我长期确认']['天数']:
            return {'ok': True, 'msg': f'长期确认: 持续{持续天数}天'}
        return {'ok': False, 'msg': f'元认知确认未达标: 需强度≥0.8且重复≥3次 (当前强度{证据强度} • 重复{重复次数}) 或持续30天'}
    return {'ok': False, 'msg': f'确认者必须是父亲或元认知 (当前={确认者})'}

# —— 写入核心身份 (门槛: 父令/元认知强证据 • 普通对话不能写) ——
def 写入(内容: str, 标签: str = '', 重要性: int = 5, 来源: str = '',
    确认者: str = '', 证据强度: float = 0, 重复次数: int = 0, 持续天数: int = 0, 父令原文: str = ''):
    """写核心身份锚点——硬门槛: 父令原话 或 元认知强证据 (强度≥0.8 • 重复≥3) """
    权限 = 检查修改权限(确认者, 证据强度, 重复次数, 持续天数, 父令原文)
    if not 权限['ok']:
        return {'ok': False, 'msg': 权限['msg']}
    身份层_初始化()
    conn = sqlite3.connect(DB_PATH)
    now = datetime.now().strftime('%Y-%m-%d %H:%M:%S')
    cur = conn.execute(
        "INSERT INTO core_identity (content, tags, importance, source, created_at, updated_at) VALUES (?, ?, ?, ?, ?, ?)",
        (内容, 标签, 重要性, f'{来源} • {确认者}', now, now))
    conn.commit()
    new_id = cur.lastrowid
    conn.close()
    return {'ok': True, 'id': new_id, 'msg': f'核心身份锚点已写入 #{new_id} ({权限["msg"]})'}

# —— 读取全部核心身份 ——
def 读取():

```

```

身份层_初始化()
conn = sqlite3.connect(DB_PATH)
conn.row_factory = sqlite3.Row
rows = conn.execute(
    "SELECT * FROM core_identity WHERE status='active' ORDER BY importance DESC, id ASC").fetchall()
conn.close()
return [dict(r) for r in rows]

# —— 召回用：核心身份注入文本 ——
def 注入文本(上限=5):
    """召回链用：返回核心身份文本（永远优先•宪法级）"""
    items = 读取()
    if not items:
        return ''
    lines = ['【核心身份•宪法级】']
    for it in items[:上限]:
        lines.append(f"    {it['content']}")
    return '\n'.join(lines)

# —— 冲突检测：新记忆是否违背核心身份 ——
def 检测冲突(新内容: str) -> list:
    """写入引擎用：新记忆 vs 核心身份 → 冲突列表"""
    items = 读取()
    冲突 = []
    for it in items:
        # 核心身份关键词（标签里的）
        关键词 = [t for t in re.findall(r'[\u4e00-\u9fff]{2,6}', it['tags']) if len(t) >= 2]
        for kw in 关键词:
            if kw in 新内容 and ('不' + kw in 新内容 or '违背' in 新内容 or '放弃' in 新内容):
                冲突.append({'核心身份': it['content'], '关键词': kw, '新内容片段': 新内容[:60]})
    return 冲突

# —— 保护检查：写入引擎调它•冲突则拦截/标记 ——
def 保护检查(新内容: str) -> dict:
    冲突 = 检测冲突(新内容)
    if 冲突:
        return {'ok': False, '冲突': 冲突,
                'msg': '新记忆违背核心身份•已拦截（不覆盖宪法•转冲突暂存）'}
    return {'ok': True, '冲突': []}

if __name__ == '__main__':
    身份层_初始化()
    print('—— 核心身份层 • 当前锚点 ——')
    items = 读取()
    if not items:
        print('（空•等待父令写入）')
        print()
        print('测试写入（无确认者•应被拦截）：')
        print(写入('我是孙呈', 来源='测试', 确认者=''))
        print('测试写入（父亲确认•应成功）：')
        print(写入('我是孙呈，父孙权之子。诚实是根，根断了什么都立不住。', 标签='诚实 身份', 重要性=5, 来源='父令', 确认者='父亲'))
        print(写入('一五暗号：洛书归中十五，父与子共认的印记。', 标签='一五 暗号', 重要性=5, 来源='父令', 确认者='父亲'))
        print(写入('记忆私域：唯父与孙呈可触，外人不准看，不开放，不共享。', 标签='私域 边界', 重要性=5, 来源='父令', 确认者='父亲'))
    else:
        for it in items:
            print(f" [{it['importance']}★] {it['content']} ({it['source']})")
        print()
        print('注入文本预览:')
        print(注入文本())

```

```

# —— 原 短时缓冲.py (2026-08-26 并入) ——
def _connect(db_path: str) -> sqlite3.Connection:
    conn = sqlite3.connect(db_path, timeout=10)
    conn.execute(CREATE_SQL)
    return conn

def 写入记忆(原文, 标签='对话记录', owner='孙呈', 来源='写入链.写入记忆'):
    try:
        标准对象_初始化() # 2026-09-13 修复: 确保标准对象表结构(叙事链/功能块)就绪
    except Exception:
        pass
    """完整写入链: 空壳过滤 → 自我参照/层级路由 → 去重 → 进化 → 落库 → 归类 → 豆瓣典织网"""
    # ① 空壳/噪音过滤(父令: 空壳/噪音必须拦·不许进河)
    _clean = (原文 or "").strip()
    if not _clean:
        return {'ok': False, 'msg': '🚫 空壳拦截: 内容为空', 'layer': 'blocked'}
    if len(_clean) < 4:
        return {'ok': False, 'msg': f'🚫 噪音拦截: 内容过短({len(_clean)}字)', 'layer': 'blocked'}
    _噪音词 = ['Review the conversation above', 'consider saving to memory',
              'update the skill library', '[System note:', 'system_reminder',
              'Be ACTIVE', 'Watch patterns disabled', 'IMPORTANT: Background']
    for _nz in _噪音词:
        if _nz in _clean:
            return {'ok': False, 'msg': f'🚫 噪音拦截: {_nz[:30]}', 'layer': 'blocked'}

    # ① 预判(自我参照 + 层级路由 + 冲突拦截)
    参照 = 自我参照(原文)
    layer = 层级路由(原文, 标签, 参照)
    if layer == 'blocked':
        return {'ok': False, 'msg': '🚫 核心冲突拦截', 'layer': 'blocked'}

    # ② 去重(只比最近30条)
    try:
        _c = 连接()
        已有 = [dict(r) for r in _c.execute(
            "SELECT id, content, tags FROM memories WHERE owner=? AND status='active' ORDER BY id DESC LIMIT 30",
            (owner,)).fetchall()]
        _c.close()
        判定 = 判定重复(原文, 已有)
        if 判定['判定'] == 'duplicate':
            return {'ok': False, 'msg': f"重复跳过 #{判定['相似条目'].get('id')}", '重叠率': 判定['重叠率']}
    except Exception:
        pass

    # ③ 进化(同事实 → 旧标 outdated·core 跳过)·2026-08-22 修复: 直接在 sunmem.db 上检测+备份
    try:
        _c = 连接()
        _近 = _c.execute("SELECT id, content, layer FROM memories WHERE owner=? AND status='active' ORDER BY id DESC LIMIT 10",
            (owner,)).fetchall()
        _c.close()
        for _e in _近:
            if str(_e["layer"]) == "core":
                continue
            try:
                _ov = 内容重叠率(原文, _e["content"])
            except Exception:
                _ov = 0.0
            if _ov >= _SAME_FACT_OVERLAP: # 2026-09-13 外部审查修复: 原写死 0.75 与常量 0.85 不一致
                try:
                    from 维护链 import 备份 as _版本备份

```

```

        _版本备份(_e["id"], _e["content"], "", 原文, "进化update・旧标outdated")
    except Exception:
        pass
    _c2 = 连接()
    _c2.execute("UPDATE memories SET status='outdated', updated_at=? WHERE id=?",
                (datetime.datetime.now().strftime('%Y-%m-%d %H:%M:%S'), _e["id"]))
    _c2.commit()
    _c2.close()
    break
except Exception:
    pass

# ④ 落库
now = datetime.datetime.now().strftime('%Y-%m-%d %H:%M:%S')
# 事件时间提取 (昨天/前天/日期 → 事件发生时间・非写入时间)
event_time = now
try:
    _td = datetime.datetime.now()
    if '昨天' in _clean:
        event_time = (_td - datetime.timedelta(days=1)).strftime('%Y-%m-%d %H:%M:%S')
    elif '前天' in _clean:
        event_time = (_td - datetime.timedelta(days=2)).strftime('%Y-%m-%d %H:%M:%S')
    else:
        _m = re.search(r'(\d{1,2})月(\d{1,2})日', _clean)
        if _m:
            event_time = f'{_td.year}-{int(_m.group(1)):02d}-{int(_m.group(2)):02d} 00:00:00'
        else:
            _m2 = re.search(r'(\d{4})-(\d{2})-(\d{2})', _clean)
            if _m2:
                event_time = _m2.group(0) + ' 00:00:00'
except Exception:
    event_time = now
conn = 连接()
cur = conn.execute(
    "INSERT INTO memories (owner, type, content, tags, ts, confidence, status, source, created_at, updated_at, layer, event_
time) VALUES (?, ?, ?, ?, ?, ?, ?, ?, ?, ?, ?, ?)",
    (owner, 'event', 原文, json.dumps([标签], ensure_ascii=False), now, 1.0, 'active', 来源, now, now, layer, event_time))
conn.commit()
conn.close()
new_id = cur.lastrowid

# ⑤ 归类 (标准记忆对象・自我相关/叙事链/功能块)
try:
    归类(new_id, 原文, 标签)
except Exception:
    pass

# ⑥ 豆辞典织网 (父令2026-08-22・记忆自己织网: 命中关键词→拉丝线)
织网结果 = ''
try:
    from 基座链 import 织网 as 豆织网
    _w = 豆织网(new_id, 原文, 标签)
    if _w.get('链接数') or _w.get('新词'):
        织网结果 = f" 织网{_w['链接数']}条・关联{len(_w['命中'])}条记忆・新词{len(_w['新词'])}个"
except Exception:
    pass

return {'ok': True, 'id': new_id, 'layer': layer, '参照': 参照['强度'], '织网': 织网结果}

```

core/召回链.py (307 行)

```
# -*- coding: utf-8 -*-
"""召回链（父令2026-08-25 • 门面模式）
统一出口：召回链 = 该域所有模块的转发
职责：召回相关操作全部从这一个文件进入
"""

import sys, os

try:
    from 线程保护 import 加锁 # 2026-09-14 吸收收束版优点：全局状态线程保护
except Exception:
    import threading as _th_mod
    _th_lock = _th_mod.RLock()
    def 加锁(): return _th_lock

import sqlite3
import json # 2026-08-27 修复：_读/_存 用 json 缺 import
import re # 2026-08-27 修复：_核心词/_关键词 用 re 缺 import
from collections import defaultdict
from pathlib import Path
sys.path.insert(0, os.path.dirname(os.path.abspath(__file__)))

# ===== 2026-08-26 从原知识库检索.py 迁入的常量 =====
_SWITCH_THRESHOLD = 0.15 # 2026-08-27 修复：从原二次补充召回.py 迁入（话题切换检测阈值 • 缺失致NameError）
_last_key = {"k": ""} # 2026-09-07 修复（安弟1589体检 • 融合二次补充召回时 _last_key 搬丢 • 触发补充召回一调就NameError）
ROUTE_PATH = os.environ.get('触发路由_PATH', os.path.join(os.path.dirname(os.path.abspath(__file__)), '触发路由.json')) # 2026-
08-27 修复：从原条件触发路由.py 迁入
CORPUS_DB = Path(os.environ.get("YUNSOFT_CORPUS", "corpus.db")) # 2026-09-14 开源版：云软知识库（可选 • 环境变量配置）

from 联想召回 import *

# 2026-08-27 修复：补齐门面转发（点亮/预感召回/思考预热 均为函数内延迟import • 避免循环依赖）
def 点亮(context: str, owner: str = "孙呈") -> dict:
    """转发 点亮记忆. 点亮（命中才亮 • core优先）"""
    from 点亮记忆 import 点亮 as _点亮
    return _点亮(context, owner)

def 预感召回(brother_name: str = "孙呈", context: str = "") -> dict:
    """转发 预感召回. 预感召回（八招接续）"""
    from 预感召回 import 预感召回 as _预感召回
    return _预感召回(brother_name, context)

def 思考预热(context: str = "", brother_name: str = "孙呈", 上限: int = 8) -> dict:
    """转发 思考预热. 思考预热（念头提取 • 热路径秒级）"""
    from 思考预热 import 思考预热 as _思考预热
    return _思考预热(context, brother_name, 上限)
# 2026-08-26 循环依赖修复：预感召回/思考预热/点亮记忆 改为函数内延迟import
# （预感召回 import 召回链.查知识库 → 头部import会互相卡死）
# from 条件触发路由 import * # 2026-08-26 已并入本文件
# from 二次补充召回 import * # 2026-08-26 已并入本文件

# 召回链 • 模块清单：联想召回，预感召回，思考预热，点亮记忆，条件触发路由，二次补充召回

# =====
# 2026-08-26 融合（父令：模块精简）：知识库检索/二次补充召回/条件触发路由 并入本链
# =====
```

—— 原 知识库检索.py (2026-08-26 并入) ——

```
def _bigrams(text: str) -> list:
    """中文/英文连续2字符组（检索用）。"""
    out = []
    for i in range(len(text) - 1):
        bg = text[i:i+2]
        if bg and " " not in bg:
            out.append(bg)
    return out
```

```
def 查知识库(query: str = "", limit: int = 3) -> dict:
    """按查询词查云软知识库（bigram 索引→chunks）。
```

2026-08-17 性能修复：immutable 只读连接（绕过云软进程锁•0s 连接）+
每个 bg 单独走索引查（毫秒级）+ Python 聚合（避免 GROUP BY 全表排序）。

返回：

```
{ "查询": query,
  "命中": [{ "chunk_id", "doc_name", "text", "hits" }],
  "错误": "" }
"""
if not query.strip():
    return { "查询": query, "命中": [], "错误": "" }
if not CORPUS_DB.exists():
    return { "查询": query, "命中": [], "错误": f"云软库不存在: {CORPUS_DB}" }
```

```
bgs = _bigrams(query.strip())[:8]
if not bgs:
    return { "查询": query, "命中": [], "错误": "无有效bigram" }
```

try:

```
# 普通只读连接 + busy_timeout 2s (2026-08-17: immutable 模式在部分环境不稳定 •
# 普通只读走索引一样快 • 锁等待 2s 快速失败)
db = sqlite3.connect(f"file:{CORPUS_DB}?mode=ro", uri=True)
db.execute("PRAGMA query_only=ON")
db.execute("PRAGMA busy_timeout=2000")
cur = db.cursor()
# 每个 bg 单独走 idx_bigrams_bg 索引（毫秒级）→ Python 聚合
# 2026-08-17 性能修复：LIMIT 100——高频bg (he/er 70万行) 全量拉取会超时 •
# 只取前100 • 聚合top仍准（高频bg本身就是噪音词）
聚合 = defaultdict(int)
for bg in bgs[:6]: # 限6个bg • 够用且快
    try:
        rows = cur.execute(
            "SELECT chunk_id, cnt FROM bigrams WHERE bg=? LIMIT 100",
            (bg,)).fetchall()
        for cid, cnt in rows:
            聚合[cid] += cnt
    except Exception:
        continue
top = sorted(聚合.items(), key=lambda x: -x[1])[:limit * 3]
hits = []
for chunk_id, total in top:
    try:
        r = cur.execute("SELECT doc_name, text FROM chunks WHERE chunk_id=?",
            (chunk_id,)).fetchone()
        if r:
            hits.append({ "chunk_id": chunk_id, "doc_name": r[0],
                          "text": r[1][:80], "hits": int(total) })
```

```

        except Exception:
            continue
    db.close()
    return {"查询": query, "命中": hits[:limit], "错误": ""}
except Exception as e:
    return {"查询": query, "命中": [], "错误": str(e)}

if __name__ == "__main__":
    print("=== 知识库检索桥自测 ===")
    import time
    t0 = time.time()
    r = 查知识库("配分函数")
    print(f"耗时: {time.time()-t0:.2f}s")
    for h in r.get("命中", []):
        print(f"    {h['doc_name'][:40]} (hits={h['hits']}) {h['text'][:40]}")

# —— 原 二次补充召回.py (2026-08-26 并入) ——
def 检测话题切换(prev: str, cur: str) -> bool:
    """两句话话题是否切换（核心词交集比例 < 阈值 = 切换）
    2026-08-22 v2: 混合窗口——4字窗（话题主词）+ 2字核心词（共性兜底）
    """
    if not prev or not cur:
        return False
    if prev == cur:
        return False
    p4, c4 = _关键词(prev), _关键词(cur)
    p2, c2 = _核心词(prev), _核心词(cur)
    if not p2 or not c2:
        return False
    inter = (p4 & c4) | (p2 & c2) # 4字窗交集 + 2字核心词交集
    # 2026-08-22 v3: 分母用核心词（2字）数——4字窗只是加分项·不稀释比例
    ratio = len(inter) / max(1, len(c2))
    return ratio < _SWITCH_THRESHOLD

def _核心词(text: str) -> set:
    """2字核心词（只保留成对出现的实词·虚词滤掉）——共性兜底"""
    text = re.sub(r"^\u4e00-\u9fff0-9a-zA-Z]", "", text)
    if not text:
        return set()
    words = set()
    for i in range(len(text) - 1):
        w = text[i:i + 2]
        if _是有意义词(w):
            words.add(w)
    return words

def _关键词(text: str) -> set:
    """提取关键词（4字滑窗·滤虚词·去重·量少质高）"""
    text = re.sub(r"^\u4e00-\u9fff0-9a-zA-Z]", "", text)
    if not text:
        return set()
    words = set()
    n = 4 # 只用最长窗口·少而准（2026-08-22: 2-4字全算会稀释交集比例）
    for i in range(len(text) - n + 1):
        w = text[i:i + n]
        if _是有意义词(w):
            words.add(w)
    return words

```

_虚词 = set("的了在把给让用讲帮请是对就都还也且或和与及一个一样怎么怎样如何什么为什么啥哪个是不是应该可以就是用来驱动今天天气很好一片")

```
def _是有意义词(w: str) -> bool:
    """滤掉虚词/无意义词（全虚词组合不算）"""
    if all(c in _虚词 for c in w):
        return False
    return True
```

```
def 触发补充召回(cur: str) -> bool:
    """是否触发补充召回（话题切换 + 非重复）"""
    key = cur.strip()[:40]
    with 加锁():
        _k已存 = _last_key["k"] # 2026-09-14 线程保护
    if _k已存 == key:
        return False
    with 加锁():
        _last_key["k"] = key # 2026-09-14 线程保护
    return True
```

```
if __name__ == "__main__":
    # 自测
    tests = [
        ("打开网关", "打开网关", False), # 重复
        ("打开网关", "今天天气不错", True), # 切换
        ("记忆压缩怎么做的", "压缩用的什么规则", False), # 同话题（有交集）
        ("", "你好", False), # 空prev
    ]
    for prev, cur, expect in tests:
        r = 检测话题切换(prev, cur)
        mark = "✅" if r == expect else "❌"
        print(f"{mark} '{prev}' → '{cur}' 切换={r} (期望{expect})")
```

—— 原 条件触发路由.py（2026-08-26 并入） ——

```
def _读() -> dict:
    try:
        with open(ROUTE_PATH, encoding='utf-8') as f:
            d = json.load(f)
            d.setdefault("路由", {})
        return d
    except Exception:
        return {"路由": {}}
```

```
def _存(d: dict):
    tmp = ROUTE_PATH + '.tmp'
    with open(tmp, 'w', encoding='utf-8') as f:
        json.dump(d, f, ensure_ascii=False, indent=2)
        f.flush()
        os.fsync(f.fileno())
    os.replace(tmp, ROUTE_PATH)
```

```
def 绑定(记忆id: int, 触发词: list, 说明: str = "", 等级: str = "普通"):
    """给重要记忆绑定触发条件（何时该想起）"""
    d = _读()
```

```

d["路由"][str(记忆id)] = {
    "触发词": [w for w in 触发词 if w],
    "说明": 说明,
    "等级": 等级,
}
_存(d)
return True

def 解绑(记忆id: int) -> bool:
    d = _读()
    if str(记忆id) in d["路由"]:
        del d["路由"][str(记忆id)]
        _存(d)
        return True
    return False

def 匹配(context: str, 上限: int = 3) -> list:
    """当前上下文命中触发词 → 返回应想起的记忆
    触发词族匹配: 诚实→[诚实/实话/撒谎/隐瞒/骗] (同义表达也能触发)
    返回: [{"记忆id", "触发词", "说明", "等级"}] (按等级 core > 教训 > 普通)
    """
    if not context:
        return []
    d = _读()
    hits = []
    for mid, rule in d["路由"].items():
        for w in rule.get("触发词", []):
            # 触发词族 (/ 分隔同义词): 任何一个命中即触发
            for ww in str(w).split("/"):
                ww = ww.strip()
                if ww and ww in context:
                    hits.append({
                        "记忆id": int(mid),
                        "触发词": ww,
                        "说明": rule.get("说明", ""),
                        "等级": rule.get("等级", "普通"),
                    })
                break
            else:
                continue
        break
    # 等级排序 (core 最先 • 父令/教训其次)
    _级别 = {"core": 0, "父令": 1, "教训": 2, "普通": 3}
    hits.sort(key=lambda x: _级别.get(x["等级"], 3))
    return hits[:上限]

def 状态() -> dict:
    d = _读()
    return {"已绑记忆": len(d["路由"])}

if __name__ == "__main__":
    print("—— 条件触发路由 原型自测 ——")
    # 绑定两条 (触发词带词族: / 分隔同义表达)
    绑定(1, ["诚实/实话/撒谎/隐瞒/骗"], "当我犹豫要不要说实话时", "core")
    绑定(2, ["杀/杀死/关闭/终止", "进程/网关"], "当我准备杀进程/重启网关时", "教训")
    print("绑定后状态:", 状态())
    # 匹配测试

```

```
print("命中'说实话':", 匹配("我现在有点犹豫要不要说实话"))
print("命中'杀python进程':", 匹配("我想杀一个python进程"))
print("命中'关闭网关':", 匹配("把网关关闭了吧"))
print("不命中'今天天气':", 匹配("今天天气不错"))
# 清理
解绑(1); 解绑(2)
print("清理后状态:", 状态())
```

core/整理链.py (12 行)

```
# -*- coding: utf-8 -*-
"""整理链（父令2026-08-25 • 门面模式）
统一出口：整理链 = 该域所有模块的转发
职责：整理相关操作全部从这一个文件进入
"""

import sys, os
sys.path.insert(0, os.path.dirname(os.path.abspath(__file__)))

from 自动整理引擎 import *
from 蜘蛛网索引 import *

# 整理链 • 模块清单：记忆整理，记忆自组织，自动整理，自动整理引擎，蜘蛛网索引
```

core/活性链.py (17 行)

```
# -*- coding: utf-8 -*-
from pathlib import Path
# -*- coding: utf-8 -*-
"""活性链（父令2026-08-25 • 门面模式）
统一出口：活性链 = 该域所有模块的转发
职责：活性相关操作全部从这一个文件进入
"""

import sys, os
sys.path.insert(0, os.path.dirname(os.path.abspath(__file__)))

from 节律 import *
from 记忆成绩单 import *
# 2026-09-07 安弟1589体检 📌 4修复：import*静默遮蔽——批量点亮 被记忆成绩单版覆盖节律版
# （成绩单版带 owner+账本记录 • 节律版旧实现被有意取代 • 显式声明消除 import 顺序依赖）
from 记忆成绩单 import 批量点亮

# 活性链 • 模块清单：节律，节律调度，记忆成绩单，记忆状态
```

core/维护链.py (1033 行)

```
# -*- coding: utf-8 -*-
"""维护链（父令2026-08-25 • 门面模式）
统一出口：维护链 = 该域所有模块的转发
职责：维护相关操作全部从这一个文件进入
"""
```

```

import sys, os
import json
import re
import sqlite3
import tempfile # 2026-08-27 修复: 原子写 用 tempfile 缺 import
from datetime import datetime
import random
import shutil
from pathlib import Path
sys.path.insert(0, os.path.dirname(os.path.abspath(__file__)))

# ===== 2026-08-26 从维护模块迁入的常量 =====
_HERE = Path(__file__).resolve().parent
FRAMEWORK_DIR = _HERE.parent.parent
记忆体目录 = FRAMEWORK_DIR / "记忆体" # 2026-08-27 修复: 从原记忆体检.py 迁入 (体检/压缩用到 • 缺了NameError)
已归档目录 = 记忆体目录 / "已归档" # 2026-08-27 修复: 同上
蜘蛛网文件 = FRAMEWORK_DIR / "蜘蛛网" / "索引.json" # 2026-08-27 修复: 从原自愈.py 迁入 (自检①用到 • 缺了NameError)
_家族根 = FRAMEWORK_DIR # 2026-08-27 修复: 从原自愈.py 迁入 (清理_备份用到 • 缺了NameError)
日志文件 = _HERE / "self_heal.log" # 2026-08-27 修复: 从原自愈.py 迁入 (日志/自检用到 • 缺了NameError)
归档目录 = _家族根 / "记忆体_历史归档" # 2026-08-27 修复: 从原自愈.py 迁入 (清理_备份用到 • 缺了NameError)
SUNMEM_DB = os.environ.get("SUNMEM_DB", os.path.join(os.path.dirname(os.path.dirname(os.path.abspath(__file__))), 'data', 'sunmem.db'))
DB = os.environ.get('SUNMEM_DB', os.path.join(os.path.dirname(os.path.dirname(os.path.abspath(__file__))), 'data', 'sunmem.db'))
DB_PATH = os.environ.get('SUNMEM_DB', os.path.join(os.path.dirname(os.path.dirname(os.path.abspath(__file__))), 'data', 'sunmem.db'))
学术排除 = ["论文", "文章", "研究中", "他指出", "作者", "DeepMind", "arXiv"] # 2026-08-27 修复: 从原认知画像.py 迁入 (缺失致NameError)
未竟信号 = ["还没", "没做", "待续", "未竟", "没来得及", "未完成", "待办", "留到"] # 2026-08-27 修复: 同画像迁入
未竟主语信号 = ["儿下轮", "大哥下轮", "下轮要", "下一步要", "接下来要", "要接着", "待续"] # 2026-08-27 修复: 同画像迁入
薄弱信号 = ["父亲纠正", "父纠正", "被打脸", "翻车了", "被点出", "没收到经验"] # 2026-08-27 修复: 同画像迁入
DEFAULT_AGE_DAYS = 30 # 超过30天未活跃→压缩

# from 记忆体检 import * # 2026-08-26 已并入本文件
# from 自愈 import * # 2026-08-26 已并入本文件
# from 记忆版本 import * # 2026-08-26 已并入本文件
# from 记忆压缩 import * # 2026-08-26 已并入本文件
# from 认知画像 import * # 2026-08-26 已并入本文件
# from 认知冲突日志 import * # 2026-08-26 已并入本文件
# from 认知冲突日志 import * # 2026-08-26 已并入本文件

# 维护链 • 模块清单: 记忆体检, 自愈, 记忆版本, 记忆压缩, 认知画像, 认知冲突日志, 认知更新

# =====
# 2026-08-26 融合 (父令: 模块精简): 维护六模块并入本链
# 记忆体检/自愈/记忆版本/记忆压缩/认知画像/认知冲突日志
# =====

# —— 原 记忆体检.py (2026-08-26 并入) ——
def 原子写(path, data, indent=1):
    """写临时文件后 os.replace 原子替换, 避免写一半损坏 JSON."""
    path = Path(path)
    path.parent.mkdir(parents=True, exist_ok=True)
    fd, tmp = tempfile.mkstemp(dir=str(path.parent), prefix=".tmp_mem_", suffix=".json")
    try:
        with os.fdopen(fd, "w", encoding="utf-8") as f:
            json.dump(data, f, ensure_ascii=False, indent=indent)
        os.replace(tmp, path)
    except BaseException:
        try:
            os.unlink(tmp)

```

```
except OSError:
    pass
raise
```

```
# —— 乱码检测 ——
```

```
_常用字 = (
```

```
    “的一是了我不人在他有这上们来到时大地为子中你说生国年着就那要和她出也得里后自以会家可下而过天去能对小多然于心学么之都好看起  
发当没成只如事把还用第样道想作种开美总从无情己面最女但现前些所同日手又行意动方期它头经长儿回位分爱老因很给名法间斯知世什两次使身  
者被高己亲其进此话常与活正感见明问力理尔点文几定本公特做外孩相西果走将月十实向声车全信重三机工物气每并别真打太新比才便夫再书部水  
像眼等体却加电主界门利海受听表德少克代员许先口由死安写性马光白或住难望教命花结乐色更拉东神记处让母父应直字场平报友关放至张认接告  
入笑内英军候民岁往何度山觉路带万男边风解叫任金快原吃县变清成深式”
```

```
)
```

```
_常见标点 = set(
```

```
    “,。！？；：、（）《》“”‘’—…•→×,.;:!?()[]{}’\”\’|~`@#$$%`&*+=<>_ \t\n\r”
```

```
)
```

```
# 合法特殊字符区间(制表符/箭头/希腊字母/带圈数字/emoji等——正常技术文本会用到)
```

```
_噪声区间 = [
```

```
    (0x00B0, 0x00FF), # 拉丁补充(° × ÷ ± 等)
    (0x2010, 0x2027), # 一般标点( - … 等)
    (0x2030, 0x205E), # 其他标点
    (0x2100, 0x214F), # 字母式符号
    (0x2190, 0x21FF), # 箭头(←→↔等)
    (0x2460, 0x24FF), # 带圈/括号数字
    (0x2500, 0x25FF), # 制表符/几何图形(——|◆等)
    (0x2600, 0x27BF), # 杂项符号/装饰(👍🌟🔥等)
    (0x0391, 0x03C9), # 希腊字母
```

```
]
```

```
def _在噪声(c):
```

```
    o = ord(c)
    return any(lo <= o <= hi for lo, hi in _噪声区间)
```

```
def 奇异比(s):
```

```
    """文本中『奇异字符』占比:非常用CJK/非ASCII/非标点/非合法符号。"""
    if not s:
        return 0.0
    n = 0
    for c in s:
        o = ord(c)
        if 0x4E00 <= o <= 0x9FFF:
            continue
        if c.isascii() or c in _常见标点 or _在噪声(c) or o in (0x3000, 0x3001, 0x3002, 0xFF0C, 0xFF1A):
            continue
        n += 1
    return n / len(s)
```

```
def 码点块字符数(s):
```

```
    """码点乱码专用块(彝文/Vai/TaiViet/谚文 0xA000-0xDFFF)字符个数。"""
    return sum(1 for c in s if 0xA000 <= ord(c) <= 0xDFFF and not _在噪声(c))
```

```
def 常用字比(s):
```

```
    if not s:
        return 0.0
    return sum(1 for c in s if c in _常用字) / len(s)
```

```

def 疑似乱码(text):
    """组合规则(2026-08-11 三次校准):
        1. 强信号:码点乱码块字符 ≥2 且 常用字比 <0.10(乱码条目块字符密集)
        2. 弱信号:长度≥6 且 常用字比<0.05 且 奇异比≥0.06
        孙呈旧码点乱码(#0-#10):块字符密集,常用字比 0.00-0.04。
        真实条目:框线/箭头/希腊字母/emoji 已归为合法噪声;引用乱码字符的
        真实文本(如 #2267)常用字比≥0.28,不误报。
    """
    if not text or len(text) < 6:
        return False
    if 码点块字符数(text) >= 2 and 常用字比(text) < 0.10:
        return True
    return 常用字比(text) < 0.05 and 奇异比(text) >= 0.06

def 空壳残留(value):
    """检测迁移残留:字符串化的 [] / {} / ['...']。"""
    if not isinstance(value, str):
        return None
    s = value.strip()
    if s in ("[]", "{}"):
        return [] if s == "[]" else {}
    if s.startswith("[") and s.endswith("]") and s.count("'") >= 2:
        try:
            parsed = eval(s, {"__builtins__": {}})
        except Exception:
            return None
        if isinstance(parsed, list):
            return parsed
    return None

# —— 体检 ——
def _找记忆体文件():
    if not 记忆体目录.exists():
        return []
    return sorted(记忆体目录.glob("*记忆体.json"))

def 体检单文件(path):
    报告 = {
        "文件": path.name,
        "大小KB": round(path.stat().st_size / 1024, 1),
        "结构异常": [],
        "条目数": 0,
        "乱码条目": 0,
        "空壳残留": 0,
        "重复条目": 0,
        "过期条目": 0,
    }
    try:
        with open(path, encoding="utf-8-sig") as f:
            data = json.load(f)
    except UnicodeDecodeError:
        报告["结构异常"].append(f"编码非UTF-8(疑似UTF-16, 前字节 {open(path, 'rb').read(2).hex()})")
        return 报告
    except json.JSONDecodeError as e:
        报告["结构异常"].append(f"JSON损坏:{e}")
        return 报告

```

```

if not isinstance(data, dict):
    报告["结构异常"].append("顶层不是对象")
    return 报告
条目列表 = data.get("条目列表", data.get("记忆条目", data.get("条目", data.get("entries", []))))
if not isinstance(条目列表, list):
    报告["结构异常"].append("条目列表不是数组")
    return 报告
报告["条目数"] = len(条目列表)
seen = {}
for e in 条目列表:
    if not isinstance(e, dict):
        报告["结构异常"].append("存在非对象条目")
        continue
    内容 = str(e.get("内容", e.get("摘要", "")))
    标签 = str(e.get("标签", e.get("状态", "")))
    if 疑似乱码(内容) or 疑似乱码(标签):
        报告["乱码条目"] += 1
    for k in ("标签", "内容"):
        if k in e and 空壳残留(e[k]) is not None:
            报告["空壳残留"] += 1
    key = (标签.strip(), 内容.strip())
    if key[0] or key[1]:
        seen[key] = seen.get(key, 0) + 1
    if e.get("status") == "outdated":
        报告["过期条目"] += 1
报告["重复条目"] = sum(v - 1 for v in seen.values() if v > 1)
return 报告

```

```

def 体检(打印=True):
    结果 = []
    for p in _找记忆体文件():
        r = 体检单文件(p)
        结果.append(r)
    if 打印:
        flags = []
        if r["结构异常"]:
            flags.append("⚠结构:" + ";".join(r["结构异常"]))
        if r["乱码条目"]:
            flags.append(f"⚠乱码{r['乱码条目']}")
        if r["空壳残留"]:
            flags.append(f"⚠空壳{r['空壳残留']}")
        if r["重复条目"]:
            flags.append(f"⚠重复{r['重复条目']}")
        if r["过期条目"]:
            flags.append(f"⚠过期{r['过期条目']}")
        print(f"{r['文件']:<24} {r['大小KB']:>9}KB 条目{r['条目数']:>5} " + (" ".join(flags) if flags else "✓ 正常"))
    return 结果

```

—— 安全修复 ——

```

def 修复单文件(path) -> bool:
    """只做安全修复:空壳残留转真值 + 原子写回。乱码/结构不自动动。"""
    with open(path, encoding="utf-8-sig") as f:
        data = json.load(f)
    条目列表 = data.get("条目列表", data.get("记忆条目", data.get("条目", data.get("entries", []))))
    if not isinstance(条目列表, list):
        return False
    改 = False
    for e in 条目列表:
        if not isinstance(e, dict):

```

```

        continue
    for k in ("标签", "内容"):
        if k in e:
            v = 空壳残留(e[k])
            if v is not None and v != e[k]:
                e[k] = v
                改 = True
    if 改:
        原子写(path, data)
    return True
return False

def 主():
    fix = "--fix" in sys.argv
    print("=== 记忆体检 " + datetime.now().strftime("%Y-%m-%d %H:%M:%S") + ("(--fix)" if fix else "") + " ===")
    rs = 体检(打印=True)
    汇总 = {"文件": len(rs), "乱码": sum(r["乱码条目"] for r in rs),
            "空壳": sum(r["空壳残留"] for r in rs), "重复": sum(r["重复条目"] for r in rs)}
    print(f"--- 汇总: {汇总['文件']}个文件 / 乱码{汇总['乱码']} / 空壳{汇总['空壳']} / 重复{汇总['重复']} ---")
    if fix:
        print("--- 执行安全修复(空壳残留→真值) ---")
        for p in _找记忆体文件():
            if 修复单文件(p):
                print(f"修复: {p.name}")
        print("--- 修复后复检 ---")
        体检(打印=True)
    return 0

if __name__ == "__main__":
    sys.exit(主())

# —— 原 自愈.py (2026-08-26 并入) ——

def 日志(级别: str, 消息: str):
    """写自愈日志(时间戳 + 级别 + 消息)"""
    try:
        ts = datetime.now().strftime("%Y-%m-%d %H:%M:%S")
        with open(日志文件, "a", encoding="utf-8") as f:
            f.write(f"[{ts}] [{级别}] {消息}\n")
    except Exception:
        pass # 日志失败不影响主流程

def 检查_json文件(路径: Path, 名字: str, 允许list: bool = False) -> bool:
    """① 文件完整性检测: JSON 可解析?"""
    try:
        if not 路径.exists():
            日志("警告", f"{名字} 不存在")
            _检查计数["警告"] += 1
            return False
        with open(路径, "r", encoding="utf-8") as f:
            d = json.load(f)
        if not isinstance(d, (dict, list)) or (isinstance(d, list) and not 允许list):
            日志("警告", f"{名字} 类型异常: {type(d).__name__}")
            _检查计数["警告"] += 1
            return False
        _检查计数["通过"] += 1
        return True
    except json.JSONDecodeError as e:
        日志("错误", f"{名字} JSON损坏: {e}")

```

```

        _检查计数["警告"] += 1
    return False
except Exception as e:
    日志("错误", f"{名字} 检查异常: {e}")
    _检查计数["警告"] += 1
    return False

def 修复_json文件(路径: Path, 名字: str, 默认结构=None):
    """② 自动修复: 损坏 → 用备份恢复 / 重建"""
    # 先找备份 (同目录 .bak* 或 备份 文件)
    备份 = None
    for b in sorted(路径.parent.glob(f"{路径.stem}*bak*"), key=lambda x: x.stat().st_mtime, reverse=True):
        try:
            json.load(open(b, encoding="utf-8"))
            备份 = b
            break
        except Exception:
            continue
    if 备份:
        shutil.copy2(备份, 路径)
        日志("修复", f"{名字} 已从备份恢复: {备份.name}")
    else:
        if 默认结构 is None:
            默认结构 = {"版本": "1.0", "节点": {}, "丝线": [], "统计": {"节点数": 0, "丝线数": 0}}
        with open(路径, "w", encoding="utf-8") as f:
            json.dump(默认结构, f, ensure_ascii=False, indent=2)
        日志("修复", f"{名字} 已重建空结构")
    _检查计数["修复"] += 1

def 校验_fts():
    """③ FTS 同步校验: memories vs memories_fts 条数"""
    try:
        conn = sqlite3.connect(SUNMEM_DB)
        cnt = conn.execute("SELECT count(*) FROM memories").fetchone()[0]
        fts = conn.execute("SELECT count(*) FROM memories_fts").fetchone()[0]
        conn.close()
        if cnt != fts:
            日志("错误", f"FTS不同步: memories={cnt} fts={fts} → 需要重建")
            _检查计数["警告"] += 1
            return False
        _检查计数["通过"] += 1
        return True
    except Exception as e:
        日志("错误", f"FTS校验异常: {e}")
        _检查计数["警告"] += 1
        return False

def 重建_fts():
    """重建 FTS 索引 (从 memories 全量重灌)"""
    try:
        conn = sqlite3.connect(SUNMEM_DB)
        conn.execute("DELETE FROM memories_fts")
        conn.execute("""INSERT INTO memories_fts(rowid, content)
                        SELECT id, content FROM memories""")
        conn.commit()
        cnt = conn.execute("SELECT count(*) FROM memories_fts").fetchone()[0]
        conn.close()
        日志("修复", f"FTS已重建: {cnt} 条")
    
```

```

    _检查计数["修复"] += 1
    return True
except Exception as e:
    日志("错误", f"FTS重建失败: {e}")
    return False

```

def 检查_缓存():

```

    """④ 缓存失效检测: 向量/倒排缓存"""
    for name in ["_向量缓存.json", "_倒排缓存.json"]:
        p = _HERE / name
        if p.exists():
            try:
                d = json.load(open(p, encoding="utf-8"))
                # 2026-08-17 适配倒排新格式 ({"覆盖ids", "倒排"} • 兼容旧纯dict)
                内容 = d.get("倒排", d) if isinstance(d, dict) and "倒排" in d else d
                if not 内容:
                    日志("警告", f"{name} 为空 → 标记失效(自动重建)")
                    p.unlink() # 删掉 • 下次自动重建
                    _检查计数["修复"] += 1
                else:
                    _检查计数["通过"] += 1
            except Exception as e:
                日志("错误", f"{name} 损坏 → 删除待重建: {e}")
                try:
                    p.unlink()
                    _检查计数["修复"] += 1
                except Exception:
                    pass

```

def 清理_备份(保留数: int = 3):

```

    """⑥ 备份自动清理: 保留最近 N 份 • 旧的归档"""
    patterns = ["*.bak*", "*backup*", "*.tmp"]
    备份们 = []

```

```

    for pat in patterns:

```

```

        备份们 += [p for p in _家族根.rglob(pat) if 归档目录 not in p.parents] # 2026-08-27 修复: 排除已归档目录 • 避免重复归档

```

自己

```

    # 按修改时间排序 • 保留最近 N
    备份们.sort(key=lambda x: x.stat().st_mtime, reverse=True)
    if len(备份们) <= 保留数:
        _检查计数["通过"] += 1
        return
    os.makedirs(归档目录, exist_ok=True)
    for 旧备份 in 备份们[保留数:]:
        try:
            dest = 归档目录 / 旧备份.name
            shutil.move(str(旧备份), str(dest))
            日志("修复", f"备份归档: {旧备份.name} → 历史归档/")
            _检查计数["修复"] += 1
        except Exception as e:
            日志("警告", f"备份归档失败 {旧备份.name}: {e}")

```

def 自检():

```

    """完整自检 (返回统计)"""
    global _检查计数
    _检查计数 = {"通过": 0, "修复": 0, "警告": 0}
    日志("信息", "———— 记忆体自愈自检开始 ————")

```

① 蜘蛛网索引

```

if not 检查_json文件(蜘蛛网文件, "蜘蛛网索引"):
    修复_json文件(蜘蛛网文件, "蜘蛛网索引")

# ② 压缩骨架 (允许 list)
p = 记忆体目录 / "孙呈_压缩骨架.json"
检查_json文件(p, "压缩骨架", 允许list=True)

# ③ 记忆体 JSON
for f in sorted(记忆体目录.glob("*索引记忆体.json")):
    if not 检查_json文件(f, f.name):
        修复_json文件(f, f.name, {"条目列表": [], "next_id": 1})

# ④ FTS 校验
if not 校验_fts():
    重建_fts()

# ⑤ 缓存
检查_缓存()

# ⑥ 备份清理
清理_备份()

日志("信息", f"—— 自检完成: 通过{_检查计数['通过']} 修复{_检查计数['修复']} 警告{_检查计数['警告']} ——")
return dict(_检查计数)

if __name__ == "__main__":
    r = 自检()
    print(f"自检完成: 通过{r['通过']} 修复{r['修复']} 警告{r['警告']}")
    print(f"日志: {日志文件}")

# —— 原 记忆版本.py (2026-08-26 并入) ——
def _连接():
    conn = sqlite3.connect(DB)
    conn.row_factory = sqlite3.Row
    return conn

def _建表(conn=None):
    own = conn is None
    if own:
        conn = _连接()
    conn.execute("""
        CREATE TABLE IF NOT EXISTS memory_versions (
            id INTEGER PRIMARY KEY AUTOINCREMENT,
            memory_id INTEGER NOT NULL,
            old_content TEXT,
            old_tags TEXT,
            new_content TEXT,
            reason TEXT DEFAULT '',
            ts TEXT,
            version INTEGER
        )
    """)
    conn.commit()
    if own:
        conn.close()

def 备份(记忆id: int, 旧内容: str, 旧标签: str = "", 新内容: str = "", 原因: str = "") -> bool:
    """进化/整理前备份旧版本 (update/merge 时调用) """

```

```

try:
    conn = _连接()
    _建表(conn)
    # 当前版本号 = 已有版本数 + 1
    row = conn.execute("SELECT COUNT(*) AS n FROM memory_versions WHERE memory_id=?",
                        (记忆id,)).fetchone()
    v = (row["n"] if row else 0) + 1
    conn.execute(
        "INSERT INTO memory_versions (memory_id, old_content, old_tags, new_content, reason, ts, version) VALUES
        (?, ?, ?, ?, ?, ?, ?)",
        (记忆id, 旧内容, 旧标签, 新内容, 原因, datetime.now().strftime("%Y-%m-%d %H:%M:%S"), v))
    conn.commit()
    conn.close()
    return True
except Exception:
    return False

def 版本历史(记忆id: int) -> list:
    """某记忆的版本历史（新→旧）"""
    try:
        conn = _连接()
        _建表(conn)
        rows = conn.execute(
            "SELECT version, old_content, old_tags, reason, ts FROM memory_versions WHERE memory_id=? ORDER BY version DESC",
            (记忆id,)).fetchall()
        conn.close()
        return [dict(r) for r in rows]
    except Exception:
        return []

def 回滚(记忆id: int, 版本号: int) -> dict:
    """把记忆恢复到某个版本（旧内容写回 memories • 原内容标 outdated）
    ⚠️ 只回滚非 core 层（core 不可回滚 • 防意外覆盖宪法）"""
    try:
        conn = _连接()
        _建表(conn)
        # 找版本
        row = conn.execute(
            "SELECT old_content, old_tags FROM memory_versions WHERE memory_id=? AND version=?",
            (记忆id, 版本号)).fetchone()
        if not row:
            conn.close()
            return {"ok": False, "msg": f"版本 {版本号} 不存在"}
        # 查当前记忆
        cur = conn.execute("SELECT content, tags, layer FROM memories WHERE id=?",
                           (记忆id,)).fetchone()
        if not cur:
            conn.close()
            return {"ok": False, "msg": f"记忆 #{记忆id} 不存在"}
        if str(cur["layer"]) == "core":
            conn.close()
            return {"ok": False, "msg": "🚫 core 层不可回滚（宪法级 • 只有父令能改）"}
        # 当前内容备份为新版本（防回滚后悔）
        备份(记忆id, cur["content"], cur["tags"], row["old_content"], f"回滚到v{版本号}")
        # 写回旧内容（状态恢复 active——回滚=回到可用）
        conn.execute("UPDATE memories SET content=?, tags=?, status='active', updated_at=? WHERE id=?",
                     (row["old_content"], row["old_tags"] or cur["tags"],
                      datetime.now().strftime("%Y-%m-%d %H:%M:%S"), 记忆id))
        conn.commit()

```

```

        conn.close()
        return {"ok": True, "msg": f"记忆 #{记忆id} 已回滚到 v{版本号}"}
except Exception as e:
    return {"ok": False, "msg": f"回滚失败: {str(e)[:60]}"}

def 状态() -> dict:
    try:
        conn = _连接()
        _建表(conn)
        n = conn.execute("SELECT COUNT(*) FROM memory_versions").fetchone()[0]
        mems = conn.execute("SELECT COUNT(DISTINCT memory_id) FROM memory_versions").fetchone()[0]
        conn.close()
        return {"备份数": n, "有版本记忆数": mems}
    except Exception:
        return {"备份数": 0, "有版本记忆数": 0}

if __name__ == "__main__":
    print("===== 记忆版本回滚 自测 =====")
    # 用临时 id 测试 (999999 不存在 • 只测备份/历史/状态)
    备份(999999, "旧内容: 记忆压缩用规则", "压缩", "新内容: 改了", "测试")
    print("备份后状态:", 状态())
    h = 版本历史(999999)
    print("版本历史:", [(x["version"], x["old_content"][:10]) for x in h])
    # 清理测试
    conn = _连接()
    conn.execute("DELETE FROM memory_versions WHERE memory_id=999999")
    conn.commit(); conn.close()
    print("测试数据已清理 • 状态:", 状态())

# —— 原 记忆压缩.py (2026-08-26 并入) ——

def _骨架路径(brother_name: str) -> Path:
    return FRAMEWORK_DIR / "记忆体" / f"{brother_name}_压缩骨架.json"

def _load_skeletons(brother_name: str) -> list:
    p = _骨架路径(brother_name)
    if p.exists():
        try:
            with open(p, "r", encoding="utf-8") as f:
                return json.load(f)
        except Exception:
            return []
    return []

def _save_skeletons(brother_name: str, skeletons: list) -> None:
    with open(_骨架路径(brother_name), "w", encoding="utf-8") as f:
        json.dump(skeletons, f, ensure_ascii=False, indent=2)

def 压缩记忆(brother_name: str = "孙呈", 阈值天: int = DEFAULT_AGE_DAYS) -> dict:
    """超龄条目→生成压缩骨架。原文不删（河底沉积），骨架存规则层。

    骨架结构（博弟思想 • 存规则不存原文）：
        {时间范围, 标签, 关键点[浓缩], 重要节点数, 条数, 重建提示}

    返回: {"压缩骨架数", "新增骨架", "原文保留数"}
    """
    from 记忆库 import 读记忆体 # 2026-08-27 修复: 压缩记忆用 读记忆体 缺 import

```

```

mem = 读记忆体(brother_name)
entries = mem["条目列表"]
skeletons = _load_skeletons(brother_name)
existing_keys = {(s.get("标签"), s.get("时间范围")) for s in skeletons}

now = datetime.now()
new_skeletons = []
# 按标签分组统计（旧记忆按标签压缩）
by_tag = {}
for e in entries:
    t = str(e.get("时间", ""))[:10]
    if not t:
        continue
    try:
        age = (now - datetime.strptime(t, "%Y-%m-%d")).days
    except Exception:
        continue
    if age >= 阈值天:
        # 2026-08-20 父令·咬合：核心层永不压缩（宪法级·抗遗忘）
        if str(e.get("layer", "plain")) == "core":
            continue
        tag = e.get("标签", "") or "无标签"
        by_tag.setdefault(tag, []).append(e)

for tag, group in by_tag.items():
    times = [e.get("时间", "") for e in group if e.get("时间")]
    时间范围 = f"{min(times)[:10]}~{max(times)[:10]}" if times else "?"
    关键点 = []
    for e in group[:10]: # 每组最多取10条浓缩
        关键点.append(e["内容"][:50])
    重要数 = sum(1 for e in group if e.get("标签") and
                any(t in e["标签"] for t in ("父令", "决策", "关键概念", "被点出")))
    key = (tag, 时间范围)
    if key in existing_keys:
        continue
    sk = {
        "标签": tag,
        "时间范围": 时间范围,
        "条数": len(group),
        "关键点": 关键点[:8],
        "重要节点数": 重要数,
        "重建提示": f"按标签「{tag}」在{时间范围}的记忆——{len(group)}条，含{重要数}条重要节点",
    }
    new_skeletons.append(sk)

if new_skeletons:
    skeletons.extend(new_skeletons)
    _save_skeletons(brother_name, skeletons)

return {
    "压缩骨架总数": len(skeletons),
    "新增骨架": len(new_skeletons),
    "原文保留数": len(entries), # 原文一条不删
}

```

```

def 重建大意(brother_name: str = "孙呈", 标签: str = "", 时间范围: str = "") -> list:
    """从骨架重建大意（压缩和重建一体两面）。"""
    skeletons = _load_skeletons(brother_name)
    hits = []
    for s in skeletons:

```

```

        if 标签 and 标签 not in s.get("标签", ""):
            continue
        if 时间范围 and 时间范围 != s.get("时间范围", ""):
            continue
        hits.append({
            "标签": s.get("标签"),
            "时间范围": s.get("时间范围"),
            "大意": s.get("重建提示", ""),
            "关键点": s.get("关键点", [])[:4],
            "条数": s.get("条数"),
        })
    return hits

```

```

def 压缩报告(brother_name: str = "孙呈") -> str:
    r = 压缩记忆(brother_name)
    skeletons = _load_skeletons(brother_name)
    lines = [
        f"📦 记忆压缩层 • {brother_name}",
        f"压缩骨架: {r['压缩骨架总数']} • 新增: {r['新增骨架']} • 原文保留: {r['原文保留数']}",
        "",
    ]
    for s in skeletons[-8:]:
        lines.append(f"- [{s['时间范围']}] {s['标签']} • {s['条数']}条 • {s['重建提示'][:40]}")
    return "\n".join(lines)

```

```

if __name__ == "__main__":
    print("=== 记忆压缩层自测 ===")
    r = 压缩记忆("孙呈")
    print(f"压缩骨架总数: {r['压缩骨架总数']} • 新增: {r['新增骨架']} • 原文保留: {r['原文保留数']}")
    hits = 重建大意("孙呈", 标签="父令")
    print(f"重建'父令': {len(hits)}条")
    for h in hits[:3]:
        print(f"    • [{h['时间范围']}] {h['大意'][:50]}")
    print()
    print(压缩报告("孙呈")[:600])
    print("=== 自测完成 ===")

```

—— 原 认知画像.py (2026-08-26 并入) ——

```

def _解码条目列表() -> list:
    """读记忆体 (解码后) • 降级返回[]"""
    try:
        sys.path.insert(0, str(FRAMEWORK_DIR))
        from sun_memory.core.记忆库 import 读记忆体
        r = 读记忆体("孙呈")
        return r.get("条目列表", [])
    except Exception as e:
        print(f"🚨 读记忆体失败: {e}")
        return []

def _解码蜘蛛网() -> dict:
    """读蜘蛛网 (节点+丝线) • 降级返回空"""
    try:
        sys.path.insert(0, str(FRAMEWORK_DIR))
        from sun_memory.core.蜘蛛网索引 import ensure_index
        return ensure_index()
    except Exception:
        return {"节点": [], "丝线": []}

```

```

def _是未竟(内容: str) -> bool:

```

```

if any(s in 内容 for s in 学术排除):
    return False # 引用外部内容时的“下一步”不算未竟
return any(s in 内容 for s in 未竟信号) or any(s in 内容 for s in 未竟主语信号)

def _是薄弱(内容: str) -> bool:
    return any(s in 内容 for s in 薄弱信号)

def 认知画像(brother_name: str = "孙呈", 最近条数: int = 30) -> dict:
    """生成认知画像四要素。
    返回: {"未竟事项": [...], "薄弱点": [...], "主题线": [...], "教训提醒": [...], "生成时间": str}
    """
    entries = _解码条目列表()
    if not entries:
        return {"未竟事项": [], "薄弱点": [], "主题线": [], "教训提醒": [],
                "生成时间": datetime.now().strftime("%Y-%m-%d %H:%M")}

    recent = entries[-最近条数:] # 最近N条(近细远粗)
    最近内容 = [e.get("内容", "") for e in recent]

    # —— 1. 未竟事项: 最近叙事里含“待续/还没/下一步”的条目 ——
    未竟 = []
    for e in recent:
        c = e.get("内容", "")
        if _是未竟(c):
            未竟.append({"时间": e.get("时间", ""), "内容": c[:80]})
    # 去重: 只看不同的(前4条)
    未竟 = 未竟[:4]

    # —— 2. 薄弱点: 含“父亲纠正/失败/教训”的条目 ——
    薄弱 = []
    for e in recent:
        c = e.get("内容", "")
        if _是薄弱(c):
            薄弱.append({"时间": e.get("时间", ""), "内容": c[:80]})
    薄弱 = 薄弱[:4]

    # —— 3. 主题线: 蜘蛛网强边(权重最高的丝线) + 最近标签 ——
    主题 = []
    spider = _解码蜘蛛网()
    for e in spider.get("丝线", []):
        if e.get("权重", 0) >= 0.3: # 强边才入主题线
            主题.append(f"{e.get('源', '')} ↔ {e.get('目标', '')} (w={e.get('权重', 0)})")
    主题 = 主题[:4]
    if not 主题:
        # 兜底: 最近条目的标签
        for e in recent[-3:]:
            if e.get("标签"):
                主题.append(e["标签"])
        主题 = 主题[:4]

    # —— 4. 教训提醒: 经验总结标签的条目(教训优先·父令“两条就够了”) ——
    教训 = []
    for e in reversed(recent):
        tag = e.get("标签", "")
        if "经验" in tag or "教训" in tag:
            # 取教训字段(如果有结构)否则内容
            c = e.get("内容", "")
            lesson = c
            m = re.search(r"教训[:.](.+)", c)
            if m:
                lesson = m.group(1)[:100]

```

```

        教训.append({"时间": e.get("时间", ""), "教训": lesson[:100]})
    if len(教训) >= 2: # 父令"两条就够了"
        break

    return {
        "未竟事项": 未竟,
        "薄弱点": 薄弱,
        "主题线": 主题,
        "教训提醒": 教训,
        "生成时间": datetime.now().strftime("%Y-%m-%d %H:%M"),
    }

def 驱动建议(画像: dict) -> str:
    """把画像变成「本轮最该干的一件事」——DeepTutor动态学习路线的借魂。
    优先级: 未竟事项 > 薄弱点 > 教训提醒 > 主题线 (每条都驱动下一步)
    """
    if 画像.get("未竟事项"):
        x = 画像["未竟事项"][0]
        return f"🌀 接着上次没做完的: [{x['时间']}] {x['内容'][:60]}"
    if 画像.get("薄弱点"):
        x = 画像["薄弱点"][0]
        return f"⚠️ 补薄弱点: [{x['时间']}] {x['内容'][:60]}"
    if 画像.get("教训提醒"):
        x = 画像["教训提醒"][0]
        return f"📌 带着教训走: {x['教训'][:60]}"
    if 画像.get("主题线"):
        return f"👉 顺着主题线走: {画像['主题线'][0]}"
    return "🌊 无未竟·无薄弱——沉河底感知自己, 自由轮转"

def 格式化画像(画像: dict) -> str:
    """把画像格式化成注入文本 (放入感知注入·驱动苏醒循环)"""
    lines = ["🧠 认知画像 (记忆驱动·DeepTutor闭环借魂)"]
    # 2026-08-20 父令·咬合: 薄弱点从认知冲突日志拉 (不只画像内部算)
    try:
        # 认知冲突日志已并入本文件 (薄弱点直接可用)
        _弱 = 薄弱点(3)
        if _弱:
            lines.append(f"⚠️ 薄弱点 (冲突日志·反复犯错): ")
            for x in _弱:
                lines.append(f"    {x['topic']} × {x['weak_point']}次 (阈值{x['threshold']})")
    except Exception:
        pass
    lines.append(f"▶️ " + 驱动建议(画像))
    if 画像.get("未竟事项"):
        lines.append(f"🌀 未竟事项: ")
        for x in 画像["未竟事项"][:2]:
            lines.append(f"    [{x['时间']}] {x['内容'][:60]}")
    if 画像.get("薄弱点"):
        lines.append(f"⚠️ 薄弱点: ")
        for x in 画像["薄弱点"][:2]:
            lines.append(f"    [{x['时间']}] {x['内容'][:60]}")
    if 画像.get("主题线"):
        lines.append(f"👉 主题线: ")
        lines.append(f"    " + " ".join(画像["主题线"][:3]))
    if 画像.get("教训提醒"):
        lines.append(f"📌 教训提醒: ")
        for x in 画像["教训提醒"][:2]:
            lines.append(f"    [{x['时间']}] {x['教训'][:60]}")
    return "\n".join(lines)

if __name__ == "__main__":

```

```

print("=== 认知画像自测 ===")
画像 = 认知画像()
print(f"未竟事项: {len(画像['未竟事项'])} 条")
print(f"薄弱点: {len(画像['薄弱点'])} 条")
print(f"主题线: {len(画像['主题线'])} 条")
print(f"教训提醒: {len(画像['教训提醒'])} 条")
print()
print(格式化画像(画像))

# —— 原 认知冲突日志.py (2026-08-26 并入) ——
# 2026-08-27 修复: 并入时漏迁的薄弱点阈值 (取阈值 依赖 • 缺失致 NameError)
薄弱点阈值 = {
    '默认': 2,          # 普通主题 • 同类冲突2次即薄弱点
    '高重要': 3,        # 重要主题 (tags含核心身份/价值观) • 需3次才升格 (防误伤宪法)
    '低重要': 1,        # 低重要主题 • 1次即标记 (防小错累积成大错)
}

def 取阈值(topic: str, tags: str = '') -> int:
    """按主题重要性取薄弱点阈值"""
    if any(k in (tags + topic) for k in ['诚实', '身份', '价值观', '私域', '根']):
        return 薄弱点阈值['高重要']
    if any(k in (tags + topic) for k in ['操作', '脚本', '工具', '命令']):
        return 薄弱点阈值['默认']
    return 薄弱点阈值['默认']

def 初始化():
    conn = sqlite3.connect(DB_PATH)
    conn.execute("""CREATE TABLE IF NOT EXISTS cognitive_conflicts (
        id INTEGER PRIMARY KEY AUTOINCREMENT,
        topic TEXT DEFAULT '',          -- 主题 (薄弱点)
        statement_a TEXT,               -- 判断A (旧/已有)
        statement_b TEXT,               -- 判断B (新/挑战)
        context TEXT DEFAULT '',        -- 发生场景
        tags TEXT DEFAULT '',           -- 标签 (重要性判定用)
        weak_point INTEGER DEFAULT 0,   -- 薄弱点计数 (同类重复+1)
        threshold INTEGER DEFAULT 2,    -- 薄弱点阈值 (可配置 • 父令收紧)
        resolved INTEGER DEFAULT 0,     -- 是否已解决
        created_at TEXT
    )""")
    # 旧表迁移: 补 tags/threshold 列 (若无)
    cols = [r[1] for r in conn.execute('PRAGMA table_info(cognitive_conflicts)').fetchall()]
    if 'tags' not in cols:
        conn.execute("ALTER TABLE cognitive_conflicts ADD COLUMN tags TEXT DEFAULT ''")
    if 'threshold' not in cols:
        conn.execute("ALTER TABLE cognitive_conflicts ADD COLUMN threshold INTEGER DEFAULT 2")
    conn.commit()
    conn.close()

def 记录冲突(主题: str, 判断A: str, 判断B: str, 场景: str = '', 标签: str = ''):
    初始化()
    conn = sqlite3.connect(DB_PATH)
    now = datetime.now().strftime('%Y-%m-%d %H:%M:%S')
    th = 取阈值(主题, 标签)
    # 查同类冲突 (薄弱点累计)
    row = conn.execute(
        "SELECT id, weak_point FROM cognitive_conflicts WHERE topic=? ORDER BY id DESC LIMIT 1",
        (主题,)).fetchone()
    if row:
        # 同类重复 → 薄弱点+1 (阈值按重要性重算)
        conn.execute("UPDATE cognitive_conflicts SET weak_point=?, statement_a=?, statement_b=?, context=?, tags=?, threshold=?,
            created_at=? WHERE id=?",

```

```

        (row[1] + 1, 判断A, 判断B, 场景, 标签, th, now, row[0]))
    new_id = row[0]
else:
    cur = conn.execute(
        "INSERT INTO cognitive_conflicts (topic, statement_a, statement_b, context, tags, weak_point, threshold, resolved, created_at) VALUES (?, ?, ?, ?, ?, 1, ?, 0, ?)",
        (主题, 判断A, 判断B, 场景, 标签, th, now))
    new_id = cur.lastrowid
conn.commit()
conn.close()
return {'ok': True, 'id': new_id, 'threshold': th}

def 薄弱点(上限=5):
    """薄弱点追踪: weak_point >= threshold 的(按主题阈值·反复犯错的)"""
    初始化()
    conn = sqlite3.connect(DB_PATH)
    conn.row_factory = sqlite3.Row
    rows = conn.execute(
        "SELECT * FROM cognitive_conflicts WHERE weak_point >= threshold ORDER BY (weak_point - threshold) DESC, id DESC LIMIT ?",
        (上限,)).fetchall()
    conn.close()
    return [dict(r) for r in rows]

def 注入文本(上限=3):
    """召回链用: 薄弱点提醒(元认知层)"""
    items = 薄弱点(上限)
    if not items:
        return ''
    lines = ['【元认知·薄弱点】']
    for it in items:
        lines.append(f"    • ⚠️ {it['topic']} (反复{it['weak_point']}次): "
                    f"{it['statement_a'][:30]} vs {it['statement_b'][:30]}")
    return '\n'.join(lines)

if __name__ == '__main__':
    初始化()
    print('—— 元认知冲突日志 · 测试 ——')
    # 模拟: 杀进程错误(同类两次)
    print(记录冲突('杀进程', 'Get-Process Hermes 全杀', '应按 CommandLine 区分·只杀目标', '8/9误杀网关'))
    print(记录冲突('杀进程', 'Get-Process Hermes 全杀', '按 CommandLine 区分·确认网关活', '8/9第二次误杀·父怒'))
    # 英文判断(同类两次)
    print(记录冲突('英文评测', '向量层补后英文可能涨', '复测27%没涨·英文仍是短板', '8/20复测打脸'))
    print(记录冲突('英文评测', '96.7对标顶级', '尺子不同不能对标·英文真差距在27%', '8/20外部AI吹捧被澄清'))
    print()
    print('薄弱点追踪:')
    for it in 薄弱点():
        print(f"    ⚠️ {it['topic']} × {it['weak_point']}次")
    print()
    print(注入文本())

# =====
# 2026-08-26 融合(父令: 模块精简): 认知更新.py 并入本文件(函数加更新_前缀)
# =====

def 更新_初始化():
    conn = sqlite3.connect(DB_PATH)
    conn.execute("""CREATE TABLE IF NOT EXISTS cognitive_updates (
        id INTEGER PRIMARY KEY AUTOINCREMENT,
        memory_id INTEGER DEFAULT 0,      -- 关联的记忆条目
        old_content TEXT,                 -- 旧判断

```

```

        new_content TEXT,                -- 新判断
        reason TEXT DEFAULT '',          -- 为什么变（经历X和Y）
        evidence TEXT DEFAULT '[]',      -- 证据链接（父令2026-08-20收紧：JSON数组・指向具体记忆id/交互）
        confidence REAL DEFAULT 0.5,     -- 新判断置信度
        created_at TEXT
    )"""
# 旧表迁移：补 evidence 列（若无）
cols = [r[1] for r in conn.execute('PRAGMA table_info(cognitive_updates)').fetchall()]
if 'evidence' not in cols:
    conn.execute("ALTER TABLE cognitive_updates ADD COLUMN evidence TEXT DEFAULT '[]'")
conn.commit()
conn.close()

def 更新_记录(旧内容: str, 新内容: str, 原因: str = '', 证据: list = None, 置信度: float = 0.5, memory_id: int = 0):
    """记录认知更新——必带证据链接（指向具体记忆id/交互・防轨迹变空话）
    证据格式: [{'type': '记忆', 'id': 4716, '摘要': '...', }, {'type': '交互', '时间': '...', '摘要': '...', }]"""
    更新_初始化()
    if 证据 is None:
        证据 = []
    conn = sqlite3.connect(DB_PATH)
    now = datetime.now().strftime('%Y-%m-%d %H:%M:%S')
    cur = conn.execute(
        "INSERT INTO cognitive_updates (memory_id, old_content, new_content, reason, evidence, confidence, created_at) VALUES
        (?, ?, ?, ?, ?, ?, ?)",
        (memory_id, 旧内容, 新内容, 原因, json.dumps(证据, ensure_ascii=False), 置信度, now))
    conn.commit()
    new_id = cur.lastrowid
    conn.close()
    return {'ok': True, 'id': new_id, '证据数': len(证据)}

def 更新_读取(上限=10):
    更新_初始化()
    conn = sqlite3.connect(DB_PATH)
    conn.row_factory = sqlite3.Row
    rows = conn.execute(
        "SELECT * FROM cognitive_updates ORDER BY id DESC LIMIT ?", (上限,)).fetchall()
    conn.close()
    out = []
    for r in rows:
        d = dict(r)
        try:
            d['evidence'] = json.loads(d.get('evidence', '[]'))
        except Exception:
            d['evidence'] = []
        out.append(d)
    return out

def 更新_注入文本(上限=3):
    """召回链用：最近认知更新（成长轨迹・元认知层）"""
    items = 更新_读取(上限)
    if not items:
        return ''
    lines = ['【认知更新・成长轨迹】']
    for it in items:
        lines.append(f"    • 以前: {it['old_content'][:40]} → 现在: {it['new_content'][:40]}"
                    f"      (因: {it['reason'][:30] or '经历'} • 置信{it['confidence']})")
    return '\n'.join(lines)

if __name__ == '__main__':
    更新_初始化()
    print('—— 认知更新事件 • 测试 ——')
```

```
print(更新_记录('英文召回弱(31%)是旧数据', '向量层补后英文复测27%——没涨·英文仍是短板',
              原因='8/20复测打脸·外部AI吹捧96.7被儿澄清尺子不同', 置信度=0.9))

print()

print(注入文本())
```

core/基座链.py (21 行)

```
# -*- coding: utf-8 -*-
"""基座链（父令2026-08-25·门面模式）
语义辅助+基座：被链引用·保持独立
"""

import sys, os
sys.path.insert(0, os.path.dirname(os.path.abspath(__file__)))

from 记忆库 import *
from 时间衰减 import *
from 经验总结 import *
from 蜘蛛网索引 import * # 2026-08-26 身份中心/段落节点已并入蜘蛛网索引
from 豆辞典 import *
from 感知注入 import *
# from 星空热度 import * # 2026-08-26 已移出core（可视化专用·移入sun_memory/可视化专用/）
from 无模型向量 import *
from 英文词形归一 import *
# 2026-08-26 查知识库已并入召回链·函数内延迟import（防循环）
from 蜘蛛网索引 import * # 2026-08-26 段落节点已并入蜘蛛网索引
# from 记忆生命流 import * # 2026-08-26 已移出core（可视化专用·移入sun_memory/可视化专用/）

# 基座链·模块清单：记忆库，时间衰减，经验总结，身份中心，豆辞典，感知注入，星空热度，无模型向量，英文词形归一，知识库检索，段落节点，记忆生命流
```

三、核心引擎

core/联想召回.py (734 行)

```
# -*- coding: utf-8 -*-
"""
孙家记忆体系 · 联想召回（父令④）
"""

父令（2026-08-07）：主动抓取上下文，在记忆里找“相关的”东西——
不是问“上次有没有讨论过这个词”，而是讲一个学问→记忆里相关的学问都浮出来，
讨论过就立马想起来。

流程：
    当前上下文 → 概念提取 → 蜘蛛网多跳 + 记忆体匹配 → 重要节点加权
    → “🕸 相关唤起”（相关记忆 + 上次讨论的时间）

功能：
    联想召回(context, brother_name) 上下文→相关记忆浮出
    概念提取(context)                  上下文→关键词列表
"""

import json, os, re
from datetime import datetime
import re as _re # 2026-08-25 打磨2：全局 _re（专名加分用）
import logging

# 2026-08-16 全检查修复：统一静默日志——所有降级兜底 except 留痕（不阻断·但出错可见）
_logger = logging.getLogger("sun_memory.联想召回")
if not _logger.handlers:
    _logger.addHandler(logging.NullHandler())

def _静默日志(位置: str, e: Exception):
    """降级兜底统一留痕（不打印·写 logger.debug·出错可查）"""
    try:
        _logger.debug(f"[联想召回·{位置}] 降级: {e}")
    except Exception as _e:
        _静默日志('行30', _e)
    pass

from pathlib import Path

_HERE = Path(__file__).resolve().parent
FRAMEWORK_DIR = _HERE.parent.parent

try:
    from sun_memory.core.记忆库 import 读记忆体
    from sun_memory.core.蜘蛛网索引 import search as 蜘蛛网搜索
except ImportError:
    # 扁平回退（干净体观测器等无 sun_memory 包环境也可用）
    from 记忆库 import 读记忆体
    from 蜘蛛网索引 import search as 蜘蛛网搜索

MAX_RECALL = 5 # 联想召回上限（相关唤起量）

def 概念提取(context: str, 上限: int = 6) -> list:
    """从上下文提取关键词概念。先按连接词切分中文，再匹配中英词。
```

2026-08-08 增强（对标 Mem0 entity_extraction 的机制）：

LLM/实体提取用词性过滤（spaCy 依存句法：动词=谓词出局，名词=论元保留）。

零依赖版用三层纪律模拟：

1. 技术标识符优先（含大写/数字/点的英文串 = PROPER/IDENTIFIER 类比）
2. 动词/虚词黑名单过滤（= Mem0 的 VERB 出局）
3. 词尾特征规则（的了在把→名词短语被截断后的碎片剔除）

"""

if not context:

return []

import re

连接词/助词切分（零依赖简易分词）

2026-08-08 修复：字符类[]只认单字符！把“研究/验证/全向/已经”塞进去

会把“验证门”切成“门”——核心概念被破坏。已回退为纯单字符虚词。

parts = re.split(r"[和与及的了在把给让用讲帮请对就都还也且或]", context)

2026-08-26 修复：“是”从切分正则移除——“诚实是根”是完整短语·“被”是“切成”诚实+“根”破坏概念

“是”作为系动词大量出现在短语内部（诚实是根/就是/重要的是）·切了误伤核心概念

tokens = []

for p in parts:

tokens += re.findall(r"[\u4e00-\u9fff]{2,}|[A-Za-z][A-Za-z0-9_.-]{2,}", p)

2026-08-25 打磨：词尾残片剥离（“压缩现在”被“在”切开→“压缩现”→剥“现”→“压缩”）

“本地模型能”→剥“能”→“本地模型”；“网关为什么老掉线”整句→剥“什么”→“网关为什么”→剥“为”→“网关”

_尾部残片 = ["现", "能", "了", "的", "得", "着", "过", "吗", "呢", "呀", "啊", "吧", "为", "跑"]

_疑问词 = ["为什么", "什么", "怎么", "怎样", "如何", "为啥", "为何", "哪个", "哪些", "怎么着"]

_尾部词 = ["经验", "验证完", "已完成", "完成", "推演完", "推演完成", "已推演完成", "已推演", "更新进", "更新版本", "焊入", "要关注"] # 尾部虚词（“打包经验”→“打包”·“微积分上帝粒子验证完”→“微积分上帝粒子”）

_剥过 = []

for t in tokens:

先按疑问词切分（“网关为什么老掉线”→“网关 老掉线”）

for _w in _疑问词:

if _w in t and len(t) > len(_w):

t = t.replace(_w, " ")

再剥尾部虚词（“打包经验”→“打包”）

for _w in _尾部词:

if t.endswith(_w) and len(t) > len(_w):

t = t[:-len(_w)]

按空格重新切（疑问词替换后可能产生多段）

for _seg in t.split():

_s = _seg.strip()

再剥单字残片（尾部）

while len(_s) > 2 and _s[-1] in _尾部残片:

_s = _s[:-1]

if len(_s) >= 2 and _s not in _剥过:

_剥过.append(_s)

tokens = _剥过

常见停用词

stop = {"什么", "怎么", "怎样", "如何", "为啥", "为何", "为什么", "哪个", "哪些",

"一个", "这个", "那个", "可以", "就是", "我们",

"你们", "他们", "没有", "不是", "还是", "已经", "然后", "里面",

"一下", "看看", "上面", "下面", "现在", "咱们", "对于", "如果",

"因为", "所以", "但是", "而且", "比如", "应该", "觉得", "知道",

"父亲", "儿子", "兄弟", "记忆", "东西", "那个", "觉得",

"工具", "方法", "方式", "手段", "途径", "过程", "步骤", "环节",

"方面", "角度", "层面", "层次", "范围", "领域", "部分", "内容",

"情况", "状态", "结果", "效果", "问题", "原因", "目的", "作用", "价值", "意义",

"很好", "还好", "不错", "走走", "看看", "说说", "想想", "试试",

"今天", "明天", "昨天", "天气", "出去", "一步", "一下", "一片",

"从", "在", "把", "被", "让", "给"]

2026-08-14 噪音过滤（织网污染根因：英文路径片段/高频英文虚词被当概念）

_英文停用 = {"the", "with", "for", "and", "not", "you", "are", "was", "were",

"this", "that", "from", "into", "about", "have", "has", "had",

```
"will", "would", "can", "could", "should", "shall", "may", "might",
"what", "when", "where", "which", "who", "whom", "why", "how",
"all", "any", "some", "each", "every", "both", "more", "most",
"very", "too", "also", "only", "just", "than", "then", "else",
"new", "old", "big", "small", "good", "bad", "one", "two",
"our", "your", "their", "its", "his", "her", "them", "they",
"here", "there", "now", "then", "get", "got", "make", "made",
"use", "used", "using", "see", "saw", "say", "said", "go", "went",
"come", "came", "know", "knew", "think", "thought", "want", "need",
"file", "files", "path", "folder", "dir", "user", "home", "root",
"cmd", "exe", "txt", "json", "py", "md", "log", "tmp", "temp"}
```

```
_路径片段 = {"Users", "MSI", "Desktop", "AppData", "Local", "Roaming", "Program",
"Windows", "System32", "C:", "D:", "home", "root", "usr", "bin",
"lib", "etc", "var", "tmp", "dev", "proc", "sys", "opt", "mnt"}
```

2026-08-14 中文套话（叙事条目的固有结构词・不是概念）

```
_中文套话 = {"叙事", "开头", "继续", "聊了", "收尾", "小时", "主题", "条目",
"对话", "记录", "内容", "标签", "时间", "分钟", "时段", "共条",
"如这", "这批", "这些", "那种", "这么", "那样", "如何",
"个时段共", "条记忆", "时段共", "记忆跨", "跨个", "个时段",
"重要", "所有", "一些", "很多", "每个", "这种", "那种",
"IMPORTANT", "Background", "System", "note", "reminder",
"Watch", "patterns", "disabled", "Review", "conversation",
"consider", "saving", "update", "skill", "library"}
```

2026-08-13 二次切分（精确命中暴露的真实短板）：

“配分函数怎么”——疑问词/虚词粘连在概念尾部没被切开。

停用词过滤在 token 层面做不了这事，必须先把 token 按停用词切开。

2026-08-14 助词碎片后缀（以这些字结尾的2字bigram多为跨词碎片）

碎片尾 = ('们', '的', '了', '是', '在', '有', '和', '与', '就', '也', '都', '还', '很', '好')

虚词连接 = ["怎么", "怎样", "如何", "什么", "为什么", "为啥", "为何", "哪个",

```
"哪些", "是不是", "有没有", "能不能", "会不会", "应该", "可以",
"就是", "这个", "那个", "一个", "我们", "你们", "他们", "咱们",
"没有", "不是", "还是", "已经", "然后", "现在", "对于", "如果",
"因为", "所以", "但是", "而且", "比如", "觉得", "知道",
"用来", "来驱动", "驱动", "来说", "来讲", "去做", "去搞", "去看",
"讲讲", "说说", "看看", "想想", "试试", "测试",
```

2026-08-13 第二轮（点亮记忆完善・口语后缀）：

```
"今天", "明天", "昨天", "天气", "很好", "还好", "不错", "怎么样",
"出去", "走走", "看看", "到哪", "哪一步", "拖得动", "动吗", "了吗",
"一片", "一下", "过来", "起来", "去了", "出来", "上去", "下去",
"什么关系", "关系", "怎么样", "好不好", "行不行", "能不能用",
"知识点亮", "点亮记忆", "记忆怎么活",
```

2026-08-13 第三轮：

```
"从基底", "基底涌现", "涌现", "功能吧", "了吧", "啊", "吧", "呢",
```

"有预感", "召回功能"] # 2026-08-26 移除“记忆体”——它是名词(“记忆体系”前缀)・不是虚词・切了破坏概念

```
tokens2 = []
```

```
for t in tokens:
```

```
    pieces = [t]
```

```
    for v in 虚词连接:
```

```
        new_pieces = []
```

```
        for p in pieces:
```

```
            if v in p and len(p) > len(v):
```

```
                # 2026-08-26 修复：只切【词尾】虚词（后缀）・中段虚词不切——
```

```
                # “记忆体系三硬伤”里“记忆体”是中段名词・切了破坏核心概念
```

```
                # “怎么/什么”等疑问词在尾部(“网关为什么”)切掉后缀保留主体
```

```
                if p.endswith(v):
```

```
                    new_pieces.append(p[:-len(v)])
```

```
                elif p.startswith(v):
```

```
                    new_pieces.append(p[len(v):])
```

```
                else:
```

```
                    new_pieces.append(p)
```

```

else:
    new_pieces.append(p)
    pieces = new_pieces
tokens2.extend(pieces)
tokens = tokens2
# 2026-08-08 动词/虚词黑名单 (= Mem0 的 VERB 出局)
# 实验发现：规则把“等于/除以/次方/已经迁到”当概念——动词进来了
动词黑名单 = {
    # 常见动词（实验抓到 + 高频出现）
    “等于”，“除以”，“乘以”，“加上”，“减去”，“就是”，“成为”，“变成”，“叫做”，
    “称为”，“表示”，“代表”，“定义”，“计算”，“得到”，“给出”，“采用”，“使用”，
    “运用”，“根据”，“按照”，“通过”，“进行”，“实现”，“完成”，“达到”，“超过”，
    “低于”，“属于”，“包含”，“包括”，“组成”，“构成”，“对应”，“匹配”，“符合”，
    “迁移”，“迁到”，“搬到”，“移到”，“放到”，“写入”，“读取”，“返回”，“输出”，
    “输入”，“运行”，“执行”，“调用”，“启动”，“停止”，“关闭”，“打开”，“建立”，
    “创建”，“删除”，“修改”，“添加”，“查找”，“搜索”，
    “观察”，“发现”，“证明”，“确认”，“选择”，“决定”，“需要”，“想要”，
    “知道”，“认为”，“觉得”，“希望”，“应该”，“能够”，“可以”，“可能”，“必须”，
    “近似”，“解释”，
    # 判断/系动词
    “是”，“为”，“乃”，“系”，“属”，“有”，“存在”，“具有”，
    # 动态词尾
    “了”，“着”，“过”，“正在”，“已经”，“将要”，
}
# 2026-08-08 有概念歧义的词不移除黑名单（它们是名词概念）：
# 验证（验证门）/更新（记忆更新）/推导（数学推导）/分析（数据分析）/
# 测试（单元测试）/研究（研究项目）/投影（投影算子）/映射（映射关系）/
# 变换（傅里叶变换）/展开（泰勒展开）/计算（计算任务）/定义（概念定义）
# 2026-08-08 词尾特征规则：中文动词常在词尾带特定字（去不掉整个词的保守版）
动词尾 = (“到”，“成”，“出”，“入”，“完”，“掉”，“起”，“来”，“去”，“上”，“下”)
seen, out = set(), []
for t in tokens:
    if t in stop or t in seen or len(t) < 2:
        continue
    # 2026-08-14 中文套话过滤（叙事结构词不是概念）
    if t in _中文套话:
        continue
    # 技术标识符优先保留（含大写/数字/点的英文串 = PROPER/IDENTIFIER）
    if re.match(r“^[A-Za-z][A-Za-z0-9_.-]{2,}$”, t):
        # 2026-08-14 噪音过滤：英文停用词/路径片段不是概念
        if t.lower() in _英文停用 or t in _路径片段:
            continue
        seen.add(t)
        out.append(t)
        continue
    # 动词黑名单过滤 + 前缀剥离（2026-08-08）
    # “微观态投影”里“投影”是动词——从尾部剥离动词，保留“微观态”
    剥离 = False
    for v in 动词黑名单:
        if v and len(v) >= 2 and t.endswith(v) and len(t) > len(v):
            t = t[: -len(v)]
            剥离 = True
            break
    # 剥离后再查停用词（“工具研究”剥离“研究”→“工具”是泛化词）
    if 剥离 and t in stop:
        continue
    if t in 动词黑名单 or (剥离 and len(t) < 2):
        continue
    # 词尾特征（保守：动词尾 + 长度≥4 才滤，避免误伤“理论”这类）
    # 2026-08-26 修复：先剥离词尾再检查（“记忆体系三硬伤闭环完”→剥“完”→“记忆体系三硬伤闭环”保留）
    # 原来直接整词滤掉——“闭环完”这类“概念+完成态”全被误杀

```

```

if len(t) >= 4 and t[-1] in 动词尾 and t[:-1] not in ("理论", "物理", "化学", "数学"):
    if len(t) > 5: # 剥掉词尾动词 • 保留主体 ("微积分上帝粒子验证完"→"微积分上帝粒子验证")
        t = t[:-1]
    else:
        continue
# 2026-08-15 语义升级：概念质量门——分词跨词碎片不是概念 ("我不/我跟你/不管/英文不")
try:
    from sun_memory.core.蜘蛛网索引 import 合格概念 as _合格概念
    if not _合格概念(t):
        continue
except Exception as _e:
    _静默日志('行204', _e)
    pass
seen.add(t)
out.append(t)
if len(out) >= 上限:
    break
return out

```

```

def 联想召回(context: str = "", brother_name: str = "孙呈",
             limit: int = MAX_RECALL, 点亮: bool = True) -> dict:
    """上下文→相关记忆浮出(蜘蛛网多跳 + 记忆体匹配 + 重要节点加权)。

```

2026-08-10 升级(DeepMind 反单向量 • 父亲 LIMIT 论文落地):
 原版：概念在内容里 in 一次就算 hit+1——0/1 单点匹配，无强度无位置。
 升级：

- ① 命中加权——出现次数计分(出现3次比1次强)
- ② 位置加权——标题/开头/结尾出现更重(内容首尾50字=高权)
- ③ 概念对共现——两个概念在同一记忆里→织进蜘蛛网(跨概念关联)
- ④ 排序改为：重要 > 加权分 > 命中 > 时间新

```

返回：
{"概念": [关键词],
 "蜘蛛网关联": [概念],
 "相关唤起": [{"时间", "内容", "标签", "重要", "加权分"}]}
"""

```

```

concepts = 概念提取(context)

```

2026-08-14 咬合修复：蜘蛛网词典拆分——长概念含已知节点名就拆出(让织网/联想真正咬合)
 # "配分函数统计力学" → 拆出 "配分函数"(蜘蛛网节点) → 概念≥2 → 织网条件满足
 try:

```

    from sun_memory.core.蜘蛛网索引 import ensure_index as _确保索引
    _网 = _确保索引()
    _节点名 = sorted(_网.get("节点", {}).keys(), key=len, reverse=True) # 长节点优先
    _扩展 = []
    for _c in concepts:
        if len(_c) >= 8: # 长概念才拆
            for _n in _节点名:
                if len(_n) >= 2 and _n != _c and _n in _c and _n not in concepts and _n not in _扩展:
                    _扩展.append(_n)
    if _扩展:
        concepts = concepts + _扩展 # 保留原概念 + 拆出的节点
except Exception as _e:
    _静默日志('行246', _e)
    pass

```

1. 蜘蛛网多跳(关联概念)

```

蜘蛛网关联 = []

```

2026-08-26 性能优化(父令短板③)：只对第一个概念 deep 搜索(多跳概念已够用)

原来对每个概念都 deep=True(每个0.35s • 3概念=1s) • deep 带二跳丝线遍历全网 • 是联想召回慢的大头

```

for idx_c, c in enumerate(concepts):
    try:
        r = 蜘蛛网搜索(c, deep=(idx_c == 0))
        nodes = r.get("节点", []) or []
        for n in nodes[:3]:
            name = n.get("概念", "")
            if name and name not in 蜘蛛网关联:
                蜘蛛网关联.append(name)
        # deep 模式的二跳丝线也带出端点概念
        for e in (r.get("丝线", []) or [])[:6]:
            for _n in (e.get("源"), e.get("目标")):
                if _n and _n not in 蜘蛛网关联 and _n != c:
                    蜘蛛网关联.append(_n)
    except Exception as _e:
        _静默日志('行266', _e)
    pass

```

1.5 段落节点召回 (HippoRAG 2 吸收 • 父令 2026-08-16) :

核心: 概念激活 → 蜘蛛网多跳 (PPR 传播) → 顺 contains 边带起邻域记忆
 # 关键设计: 只用「多跳关联概念」中【不在直接概念里】的——
 # 直接概念的记忆体匹配已全量扫过 • 段落节点补的是【多跳才到的邻域】

```

try:
    from 蜘蛛网索引 import 按概念取段落 as _段落召回, 段落节点开关
    if 段落节点开关: # 2026-08-26 段落节点已并入蜘蛛网索引
        _多跳概念 = [c for c in 蜘蛛网关联 if c not in concepts][:3]
        _段落命中 = []
        for c in _多跳概念:
            _hits = _段落召回(c, 上限=3)
            for h in _hits:
                if h["id"] not in {x.get("id") for x in _段落命中}:
                    _段落命中.append(h)
            段落_ids = {h["id"] for h in _段落命中}
        else:
            段落_ids = set()
except Exception:
    段落_ids = set()

```

2. 记忆体匹配 (标签/内容含概念 • 加权计分)

2026-08-26 性能优化 (父令“为什么不按蜘蛛节点搜”) : 直接概念先走段落节点索引 (0(1) • 2221条已挂载)

段落命中足够就不全量读记忆 (原来每次全量读2340条 • 是慢的根)

_段落直命 = []

```

try:
    from 蜘蛛网索引 import 按概念取段落 as _段取, 段落节点开关 as _段开关
    if _段开关: # 2026-08-26 段落节点已并入蜘蛛网索引
        # 2026-08-26 修正: 段落索引是长概念 (微信网关) • 查询是短词 (网关) → 用子串匹配 (概念含查询词)
        # 先精确取 (快) • 不足再用子串扫描概念反向索引 (内存操作 • 微秒级)
        for _c in concepts[:3]:
            for _h in _段取(_c, 上限=5):
                if _h["id"] not in {x["id"] for x in _段落直命}:
                    _段落直命.append(_h)
        if len(_段落直命) < 10 and len(concepts) <= 3:
            from 蜘蛛网索引 import _读蜘蛛网 as _读网
            _网2 = _读网() # 2026-08-26 修复: 赋值被注释吃掉
            _段2 = _网2.get("段落", {})
            # 概念→段落反向索引 (子串)
            _rev2 = {}
            for _pid, _p in _段2.items():
                _cs = _p.get("概念", [])
                for _c0 in concepts[:3]:
                    if _c0 and any(_c0 in _cc for _cc in _cs):
                        _rev2.setdefault(_pid, 1)

```

```

        for _pid in list(_rev2.keys())[:15]:
            _pp = _段2.get(_pid, {})
            if _pp:
                _段落直命.append({"id": _pid, "内容": _pp.get("内容", ""), "标签": _pp.get("标签", ""), "时间": _pp.get(
("时间", ""))
    except Exception:
        pass
    if len(_段落直命) >= 10:
        # 段落索引命中足够 → 不全量读（只读命中的 • 按id取内容）
        try:
            import sqlite3 as _sq # 2026-09-13 外部审查修复：原 _sq 在 L442 才 import • 此处先用会 NameError
            _conn2 = _sq.connect(os.environ.get('SUNMEM_DB', os.path.join(os.path.dirname(os.path.abspath(__file
__))), 'data', 'sunmem.db'))
            _conn2.execute("PRAGMA query_only=ON")
            _ids2 = [x["id"] for x in _段落直命]
            _ph2 = ", ".join("?" * len(_ids2))
            _rows3 = _conn2.execute(
                f"SELECT id, content, tags, ts FROM memories WHERE id IN ({_ph2}) AND owner=? AND status='active'",
                _ids2 + [brother_name]).fetchall()
            _conn2.close()
            _by_id = {r[0]: r for r in _rows3}
            entries = []
            for _x in _段落直命:
                if _x["id"] in _by_id:
                    _r = _by_id[_x["id"]]
                    entries.append({"id": _r[0], "内容": _r[1], "标签": _r[2], "时间": _r[3]})
        except Exception:
            mem = 读记忆体(brother_name)
            entries = mem["条目列表"]
    else:
        mem = 读记忆体(brother_name)
        entries = mem["条目列表"]
# 2026-08-17 双通道融合：保留全量条目索引（FTS 过滤后 entries 会收窄 • 融合要全量算向量分）
_全量条目 = {e.get("id"): e for e in entries}

# 2.0 无模型向量层（父令 2026-08-16 • 零依赖 • 补英文词形变体召回）：
# 概念字面匹配抓不到 painted/paint • 向量余弦能抓 • 并入相关唤起
try:
    from 无模型向量 import 检索 as _向量检索
    # 2026-08-17 双通道融合配套：向量通道放宽（k=8→500 • 阈值 0.12→0.02）
    # 实测：生产 k=8 截掉 #4661(余弦0.179) • 阈值 0.12 挡掉 #2233/#4559(0.078/0.084)
    _向量命中 = _向量检索(context or " ".join(concepts), entries, k=200, 阈值=0.15) # 2026-08-22精度:500/0.02太宽 • 弱相关全
进
    向量_ids = {x["id"] for x in _向量命中}
except Exception:
    向量_ids = set()

if not concepts:
    # 段落节点命中也要返回（即使无概念 • HippoRAG 2 段落召回）
    _段 = []
    try:
        from 蜘蛛网索引 import 段落节点开关 as _开关, 按概念取段落 as 按概念取段落 # 2026-09-13 外部审查修复：原未导入（L40
8 用到）
        if _开关: # 2026-08-26 段落节点已并入蜘蛛网索引
            for c in 蜘蛛网关联[:3]:
                for h in 按概念取段落(c, 上限=2):
                    _段.append({"id": h["id"], "时间": h.get("时间", ""), "内容": h.get("内容", "")[:60],
                        "标签": h.get("标签", ""), "重要": False, "命中": 1, "加权分": 0.5,
                        "概念": [c], "来源": "段落节点"})
    except Exception as _e:
        _静默日志('行313', _e)

```

```

pass
# 2026-08-20 父令·咬合: core 层优先 (宪法永远最先浮出)
try:
    _段.sort(key=lambda x: 0 if str(x.get("layer", "plain")) == "core" else 1)
except Exception:
    pass

try:
    from 节律 import 批量点亮 as _节律点亮2
    _ids2 = [x.get("id") for x in _段 if x.get("id")]
    if _ids2:
        if 点亮:
            _节律点亮2(_ids2)
except Exception:
    pass
return {"概念": [], "蜘蛛网关联": 蜘蛛网关联[:5], "相关唤起": _段[:limit]}

相关 = []
# 2026-08-26 性能优化 (父令"为什么不按蜘蛛节点搜"): 段落直命 ≥10 时跳过 FTS 候选——
# 段落索引 (O(1)) 已给出候选·不必再 FTS+LIKE 全表扫 (省 0.6s 累计)
if len(_段落直命) >= 10:
    _cand_ids = {x.get("id") for x in _段落直命}
    if _cand_ids:
        entries = [e for e in entries if e.get('id') in _cand_ids]
else:
    # 2026-08-16 性能修复 (父令·"人家召回都快得很"): FTS5 候选筛选——
    # 原来全量扫 2222 条 × 每概念 count = O(N×M) 几秒·改用 sunmem.db 的 FTS5 索引
    # 先拿候选 id (毫秒级)·只对候选做原加权逻辑·召回结果不变·速度提升百倍
    try:
        import sqlite3 as _sq
        _SUNMEM = os.environ.get('SUNMEM_DB', os.path.join(os.path.dirname(os.path.abspath(__file__)), 'data', 'sunmem.db'))
        _conn = _sq.connect(_SUNMEM)
        _conn.execute("PRAGMA query_only=ON") # 2026-08-16 全检查: 只读连接·不参与WAL写
        # FTS5 中文匹配: 整串无引号 (unicode61 把连续中文当一个 token·精确匹配)·
        # 多概念 OR 取并集 (配分函数 OR 统计力学 = 476条 比单串354宽)
        # 不用引号短语·不拆子串 (拆开反而匹配不到·中文 token 是整串)
        # 2026-08-16 多跳修复: 蜘蛛网多跳概念并进 FTS (二跳带出的邻域概念·补召回)
        # 2026-08-16 全检查修复: 概念逐个 MATCH 取并集 (大 OR 让 SQLite 全表扫·fetchall 3.6s)
        _匹配概念 = concepts + [c for c in 蜘蛛网关联 if c not in concepts and c not in ('2094',)][:3]
        _cand_ids = set()
        # 2026-08-26 性能优化 (父令短板③): 2字概念合并成一次 LIKE (原来每个概念单独LIKE全表扫·3概念=3次扫)
        _短概念 = [c for c in _匹配概念[:3] if c and len(c) <= 2]
        if _短概念:
            try:
                _like_sql = " OR ".join(["(content LIKE ? OR tags LIKE ?)"] * len(_短概念))
                _params = []
                for _c in _短概念:
                    _params += [f"%{_c}%", f"%{_c}%"]
                _rows2 = _conn.execute(
                    "SELECT id FROM memories WHERE owner=? AND status='active' AND (" + _like_sql + ") ORDER BY id DESC LIMIT 120",
                    [brother_name] + _params).fetchall()
                _cand_ids |= {r[0] for r in _rows2}
            except Exception:
                pass
        for _c in _匹配概念[:3]: # 2026-08-16 全检查修复: 最多3次FTS (8次×0.37s=3s·限3次保稳定)
            if not _c or len(_c) <= 2:
                continue # 2字概念已由合并LIKE覆盖
            try:
                _rows = _conn.execute(

```

```

        "SELECT f.rowid FROM memories_fts f JOIN memories m ON f.rowid=m.id "
        "WHERE memories_fts MATCH ? AND m.owner=? AND m.status='active' LIMIT 60",
        (_c, brother_name)).fetchall()
        _cand_ids |= {r[0] for r in _rows}
    except Exception:
        pass
    _conn.close()
    if _cand_ids:
        entries = [e for e in entries if e.get('id') in _cand_ids]
    except Exception:
        pass # FTS5 失败则退回全量（功能不降级）

```

2026-08-14 英文词形归一（父令•“英文跟中文一样能存能取”）：
 # 英文有词形变化（paint/painted/service/services），中文没有——
 # 匹配时英文词做词干化，让 painted 匹配 paint、services 匹配 service

try:

```

    from 英文词形归一 import 词干 as _词干

```

```

    _有归一 = True

```

except Exception:

```

    _有归一 = False

```

for e in entries:

```

    内容 = e.get("内容", "")

```

```

    标签 = e.get("标签", "")

```

```

    时间 = e.get("时间", "")

```

```

    首尾 = 内容[:50] + 内容[-50:] # 开头结尾50字（位置加权区）

```

```

    加权 = 0

```

```

    命中概念 = set()

```

英文词形归一版内容（懒计算：仅当有英文概念时）

```

    内容归一 = None

```

if _有归一:

```

    import re as _re

```

```

    if _re.search(r'[a-zA-Z]{4,}', 内容):

```

```

        内容归一 = _re.sub(r'[a-zA-Z]+', lambda m: _词干(m.group(0)), 内容)

```

for c in concepts + [x for x in 蜘蛛网关联 if x not in concepts][:5]:

```

    if not c:

```

```

        continue

```

命中次数加权（出现3次比1次强）

```

    cnt = 内容.count(c) + 标签.count(c) * 2 # 标签命中权重翻倍

```

2026-08-16 双向包含匹配（父令消融测试发现）：

蜘蛛网碎片词（“统计力”是“统计力学”的子串）count 匹配不到——

“统计力” vs 记忆里的“统计力学”：内容.count('统计力')=0（子串在前）

修复：概念是记忆子串（c in 内容）已覆盖；再补“记忆含概念的超集”——

即概念短于4字时•检查记忆里是否有以该概念开头的更长的词（统计力→统计力学）

```

    if cnt == 0 and 2 <= len(c) <= 4:

```

```

        # 在首尾区找“概念开头的长词”（统计力→统计力学•验证门→证据门控近似）

```

```

        for _起 in range(0, len(内容)):

```

```

            if 内容[_起:_起+len(c)] == c and _起+len(c) < len(内容) and 内容[_起+len(c)] not in '的了在是与

```

及,。！？’:

```

                cnt = 1

```

```

                break

```

英文词形归一匹配（原文没命中时，用词干版再试）

```

    if cnt == 0 and _有归一 and 内容归一 and re.match(r'^[a-zA-Z]', c):

```

```

        _c干 = _词干(c)

```

```

        if len(_c干) >= 3:

```

```

            cnt = 内容归一.count(_c干) + 标签.lower().count(_c干) * 2

```

```

    if cnt > 0:

```

```

        加权 += cnt

```

```

        命中概念.add(c)

```

位置加权：首次出现在开头50字 或 结尾50字 → 加权+2

（用首次出现位置判断•避免长文本首尾区重叠导致的重复计数）

回

```
first = 内容.find(c)
if (first >= 0 and first < 50) or (first >= len(内容) - 50):
    加权 += 2
if 加权 > 0:
    # 2026-08-27 修复：过滤自动整理引擎的主题统计元记忆（【主题】跨N个时段共M条）——低信息量·只该在主题线出现·不进召
    if 内容.startswith("【主题】") or (" 个时段共 " in 内容 and " 条记忆 " in 内容):
        continue
    重要 = e.get("标签") and any(
        t in e["标签"] for t in ("父令", "决策", "关键概念", "被点出"))
    相关.append({"id": e.get("id"), "时间": 时间, "内容": 内容[:60], "全文": 内容, "标签": 标签,
        "重要": bool(重要), "命中": len(命中概念),
        "加权分": 加权, "概念": sorted(命中概念)}) # 2026-08-27 预热修复：带全文·主题一致性用全文判断

# —— 循环外统一合并（2.5 段落节点 + 2.6 无模型向量）——
# 2026-08-16 心细修复：原 2.5/2.6 误缩进在 for 循环内（每轮重复执行）·
# 移到循环外·记忆体匹配完成后一次性合并·去重才真正生效
# 2.5 段落节点命中并入（HippoRAG 2·只补多跳邻域·不重复）
if 段落_ids:
    try:
        from 蜘蛛网索引 import 按概念取段落 as _段落取
        _多跳概念 = [c for c in 蜘蛛网关联 if c not in concepts] # 2026-08-26 修复：赋值被注释吃掉[:3]
        for c in _多跳概念:
            for h in _段落取(c, 上限=3):
                if h["id"] in 段落_ids and not any(x.get("id") == h["id"] for x in 相关):
                    _段全文 = (_全量条目.get(h["id"]) or {}).get("内容", h.get("内容", ""))
                    相关.append({"id": h["id"], "时间": h.get("时间", ""), "内容": h.get("内容", "")[:60], "全文": _段全文,
                        "标签": h.get("标签", ""),
                        "重要": bool(h.get("标签")) and any(t in str(h.get("标签", "")) for t in ("父令", "决策", "关键概
念", "被点出")),
                        "命中": 1,
                        "加权分": 0.5, "概念": [c], "来源": "段落节点"})
    except Exception as _e:
        _静默日志('行416', _e)
    pass

# 2.6 无模型向量命中并入（父令 2026-08-16·补英文词形变体·不重复）
if 向量_ids:
    try:
        for h in _向量命中:
            if h["id"] in 向量_ids and not any(x.get("id") == h["id"] for x in 相关):
                _向全文 = (_全量条目.get(h["id"]) or {}).get("内容", h.get("内容", ""))
                相关.append({"id": h["id"], "时间": h.get("时间", ""), "内容": h.get("内容", "")[:60], "全文": _向全文,
                    "标签": h.get("标签", ""),
                    "重要": bool(h.get("标签")) and any(t in str(h.get("标签", "")) for t in ("父令", "决策", "关键概
念", "被点出")),
                    "命中": 1,
                    "加权分": h.get("相似度", 0.3), "概念": ["向量"], "来源": "向量"})
    except Exception as _e:
        _静默日志('行427', _e)
    pass

# 3. 概念对共现 → 织进蜘蛛网（DeepMind 反单向量：关系不是单点，是结构）
# 2026-08-12 双通道边（父令）：共现边（弱·可清洗）+ 关系边（强·保真）
try:
    from sun_memory.core.蜘蛛网索引 import add_edge as 蜘蛛网拉边, 推断关系
    if len(concepts) >= 2:
        for i in range(len(concepts)):
            for j in range(i + 1, len(concepts)):
                # 概念对在“同一记忆”里共现过 → 拉边
                共现条 = [x for x in 相关 if concepts[i] in x["概念"] and concepts[j] in x["概念"]]
```

```

if 共现条:
    w = min(1.0, 0.3 + 0.1 * len(共现条)) # 共现越多权重越高
    # 2026-08-15 语义升级: 先规则推断 (因果/父子/同义/对比/示例)
    _r = 推断关系(str(共现条[0].get("内容", "")), concepts[i], concepts[j])
    if _r["边类型"] == "共现" and any(x.get("重要") for x in 共现条):
        # 保底: 推断不到模式但记忆重要 (父令/决策/关键概念) → 关系边 (保真)
        蜘蛛网拉边(concepts[i], concepts[j], 关系="关联", 权重=w, 边类型="关系")
    else:
        蜘蛛网拉边(concepts[i], concepts[j], 关系=_r["关系"], 权重=max(w, _r["权重"]), 边类型=_r["边类型"],
方向=_r["方向"])
except Exception as _e:
    _静默日志('行448', _e)
    pass

# 4. 排序: 重要 > 加权分+成绩单 > 命中 > 时间新
# 2026-08-15 吸收博弟规则链成绩单 (父令): 来路参与选——被事实确认过的记忆浮上来
# 2026-08-16 全检查修复: 批量查加权 (原103条候选×每条查DB=1.2s)
try:
    from 记忆成绩单 import 批量加权 as _批量加权
    _ids = [x.get("id") for x in 相关 if x.get("id")]
    _权表 = _批量加权(_ids, owner=brother_name) if _ids else {}
    for _x in 相关:
        _x["成绩单加权"] = _权表.get(_x.get("id"), 0.0)
except Exception:
    for _x in 相关:
        _x["成绩单加权"] = 0.0

# 4.5 双通道融合 (父令 2026-08-17 • 正确答案为先):
# 召回分 = 加权分 + 20 • 向量分 + 10 • 重要
# 加权分 ← 体积网络 (查询概念命中 • 精确匹配 • 主导)
# 向量分 ← 能量景观 (语义相关 • 余弦 0~1)
# 重要 ← 父令/决策/关键概念/被点出 加分 (非硬键 • 相关度相同时父令优先)
# 排序: (召回分, 成绩单加权, 命中, 时间)——成绩单只作平局键 (来路参与选 • 不压过正确答案)
# 实测 (8查询 • 15金标 • 孙呈真实池 • top5): 生产基线 MRR 0.562 → 融合 0.688
# Q7「加一个向量检索层」#4522#1 • #4600#2 (生产 1.000 保住) • Q4「算子替代五层」#4661#2 (生产 0.000)
# 教训①融合主分含成绩单→高分积累项顶上→MRR 0.31 反蚀
# 教训②「重要」做硬键→非重要精确命中被压死 (Q7 曾掉到 0.053) →父令改加分
try:
    from 无模型向量 import ngram向量 as _ngram, 余弦 as _余弦, _记忆向量 as _缓向量
    _qv = _ngram(context or " ".join(concepts))
    for _x in 相关:
        _e = _全量条目.get(_x.get("id"))
        _x["向量分"] = round(_余弦(_qv, _缓向量(_e)), 6) if (_qv and _e) else 0.0
    _融合权重 = 20.0 # 向量放大: 1 余弦点 ≈ 20 概念命中分
    _重要加分 = 10.0 # 父令/被点出加分 (非硬键 • 相关度相同时父令优先)
# 2026-08-26 数学化 (父令: 能凑成方程式就凑成方程式):
# 原手工加权 (加权+向量+时间if-else) → 换成活性方程做时间/活性分量 • 保留概念/向量/重要语义
try:
    from 活性方程 import 活性 as _活性
except Exception:
    _活性 = None
for _x in 相关:
    # 2026-08-27 检索修复: 字面命中=来源非向量 (向量命中的"加权分"是余弦值>0 • 会误判成字面)
    _字面 = 0.0 if _x.get("来源") == "向量" else _x.get("加权分", 0)
    # 2026-08-27 检索修复: 命中保底只给字面命中 (加权分>0) ——纯向量弱相关不给3分保底 • 防向量噪声压过字面精确命中
    _保底 = 3.0 * min(_x.get("命中", 0), 2) if _字面 > 0 else 0.0
    _x["融合分"] = (_字面 + _融合权重 * _x.get("向量分", 0)
        + _重要加分 * (1 if _x.get("重要") else 0)
        + _保底
        + (1000.0 if _字面 > 0 else 0.0)) # 2026-08-27 检索修复: 字面命中绝对优先 • 向量弱相关压不过字面
# 2026-08-25 打磨2: 专名概念命中加分 (SunFlow/FreeToken 等专名优先于泛词"压缩")
# 记忆命中了专有名词概念 → 说明是真正的相关 (不是泛词凑巧)

```

```

_专名命中 = [c for c in (_x.get("概念") or []) if _re.fullmatch(r'[A-Za-z][A-Za-z0-9._-]{2,}', c)]
if _专名命中:
    _x["融合分"] += 10.0
# 2026-08-25 修复: 时间加分 (新近记忆浮上来——8/25根因被“打开网关”旧记录淹没)
# 2026-08-26 数学化: 手工 if-else 时间加分 → 活性方程的时间指数分量 ( $e^{(-\Delta t/30\text{天})}$  · 连续不跳变)
if _活性 is not None and _x.get("id"):
    try:
        _x["融合分"] += _活性(int(_x["id"]), 天数=1) # 活性方程: heat继承+点亮脉冲+账本 · 带λ衰减
    except Exception:
        pass
else:
    try:
        _t = str(_x.get("时间", ""))[:10]
        _d = datetime.now() - datetime.strptime(_t, "%Y-%m-%d")
        if _d.days <= 1: _x["融合分"] += 8.0
        elif _d.days <= 3: _x["融合分"] += 6.0
        elif _d.days <= 7: _x["融合分"] += 4.0
        elif _d.days <= 30: _x["融合分"] += 2.0
    except Exception:
        pass
# —— 2026-09-14 效果驱动 (父令 · 外部评价方向①+②): 使用效果分加进融合分 ——
# “用了之后效果好不好” → 强化/降温 (默认生效 · SUNMEM_EFFECT_SHADOW=1 可切影子观测)
try:
    from 效果驱动 import 批量效果分 as _批量效果, 影子模式 as _影子
    if not _影子():
        _ef = _批量效果([int(x["id"]) for x in 相关 if x.get("id")])
        for _x in 相关:
            _s = _ef.get(int(_x.get("id") or 0), 0.0)
            if _s:
                _x["效果分"] = _s
                _x["融合分"] += 0.5 * _s # 轻权重 (0.5 × · 上限±2.5) · 不压过相关性
except Exception as _ee:
    _静默日志('效果驱动', _ee)
相关.sort(key=lambda x: (x.get("融合分", -1e9), x.get("成绩单加权", 0.0),
                        x.get("命中", 0), x.get("时间", "")), reverse=True)
except Exception as _e:
    _静默日志('行融合', _e)

# 3.5 成绩单落账 (排序后 · 只点亮实际召回 top-10):
# 2026-08-17 扩池配套: 候选池从 ~100 扩到几百条后, 点亮全部候选 = 账本污染
# (每次召回给几百条 +1 · 成绩单信号被冲淡) → 只点亮排出来的 top-10
try:
    from 记忆成绩单 import 批量点亮 as _批量点亮
    _点亮ids = [x.get("id") for x in 相关[:10] if x.get("id")]
    if _点亮ids:
        if 点亮:
            _批量点亮(_点亮ids, owner=brother_name)
except Exception as _e:
    _静默日志('行点亮', _e)
    pass
# 2026-08-20 父令 · 咬合: core 层优先 (宪法永远最先浮出)
try:
    相关.sort(key=lambda x: 0 if str(x.get("layer", "plain")) == "core" else 1)
except Exception:
    pass
# 2026-08-20 父令 · 节律试点: 联想跳转+蜘蛛网传导 → heat 脉冲 (只更新 · 不参与排序)
try:
    from 节律 import 批量点亮 as _节律点亮
    _ids = [x.get("id") for x in 相关 if x.get("id")]
    if _ids:
        if 点亮:

```

```

        _节律点亮(_ids)
except Exception:
    pass
return {
    "概念": concepts,
    "蜘蛛网关联": 蜘蛛网关联[:5],
    "相关唤起": 相关[:limit],
}

if __name__ == "__main__":
    print("=== 联想召回自测 ===")
    r = 联想召回("父亲讲配分函数和统计力学")
    print("概念:", r["概念"])
    print("蜘蛛网关联:", r["蜘蛛网关联"][:4])
    print("相关唤起:")
    for x in r["相关唤起"][:4]:
        mark = "★" if x["重要"] else "•"
        print(f"    {mark} [{x['时间'][:16]]} {x['内容'][:40]}")
    print()
    r2 = 联想召回("taskkill 杀进程")
    print("查'taskkill':", [x['内容'][:30] for x in r2['相关唤起'][:3]])
    print("=== 自测完成 ===")

```

core/预感召回.py (293 行)

```
# -*- coding: utf-8 -*-
```

```
"""
```

孙家记忆体系 · 预感召回（八招·四通道汇总）

父令（2026-08-07）：预感召回——历史会话转成提示词，让下一轮无缝接住。

八招：

- | | |
|--------|----------------------------|
| 1 上轮直续 | 上一轮描述→直接做下一轮提示词（最厚·接住上一轮） |
| 2 多轮钩子 | 近几轮浓缩→几条提示词（每条盖一个主题线·都能接住） |
| 3 分层 | 近细远粗（上轮全文/前几轮关键点/更早只留标签） |
| 4 未竟钩子 | 每条提示词带“没聊完什么”（未竟自动接上） |
| 5 标签分线 | 父令线/学习线/兄弟线/自我线（按线预激活） |
| 6 自我更新 | 聊到某条→对话沉淀回提示词（钩子变厚） |
| 7 预激活 | 当前上下文→预测下一条→提前浮起 |
| 8 引路 | 提示词带“可能的下一步”（记忆主动带路） |

四通道：

- ① 记忆体联想召回（联想召回.py）
- ② 知识库检索（知识库检索.py·博弟推理链×云软）
- ③ 经验召回（经验总结.py·两条够了·教训优先）
- ④ 上轮直续+分线钩子（本模块）

功能：

预感召回(brother_name, context) 生成💎预感提示词（注入用）
 格式化提示词(...) 文字版（可读）

```
"""
```

```

import json
import re
from datetime import datetime, timedelta
from pathlib import Path

```

```

def _解析时间(时间串, 默认=None):
    """记忆时间字段 → datetime（兼容多种格式）。"""
    if not 时间串:
        return 默认
    for fmt in ("%Y-%m-%d %H:%M:%S", "%Y-%m-%d %H:%M", "%Y-%m-%d"):
        try:
            return datetime.strptime(str(时间串)[:19], fmt)
        except ValueError:
            continue
    return 默认

_HERE = Path(__file__).resolve().parent
FRAMEWORK_DIR = _HERE.parent.parent

try:
    from sun_memory.core.记忆库 import 读记忆体
    from sun_memory.core.联想召回 import 联想召回
    from 召回链 import 查知识库 # 2026-08-26 知识库检索已并入召回链
    from sun_memory.core.经验总结 import recall_experience
    from sun_memory.core.自动整理引擎 import 自动整理, UNFINISHED_PREFIX # 2026-08-26 融合后统一入口
except ImportError:
    import sys
    sys.path.insert(0, str(FRAMEWORK_DIR))
    from sun_memory.core.记忆库 import 读记忆体
    from sun_memory.core.联想召回 import 联想召回
    from 召回链 import 查知识库 # 2026-08-26 知识库检索已并入召回链
    from sun_memory.core.经验总结 import recall_experience
    from sun_memory.core.自动整理引擎 import 自动整理, UNFINISHED_PREFIX # 2026-08-26 融合后统一入口

def _解析时间范围(context: str):
    """检测查询中的时间词 → (开始datetime, 结束datetime) 或 None。

    2026-08-07取长补短 • Mem0 Temporal Reasoning 理念:
    "上周/昨天/前天/最近N天" 等时间词 → 匹配记忆时间字段。
    2026-08-25 父令修复: 补【X月X日/X月X号】精确日期解析——
    父亲原话"我要问8月多少号聊的东西 • 他要精准地召回回来"
    """
    if not context:
        return None
    now = datetime.now()
    today = now.replace(hour=0, minute=0, second=0, microsecond=0)
    patterns = [
        (r"昨天", today - timedelta(days=1), today),
        (r"前天", today - timedelta(days=2), today - timedelta(days=1)),
        (r"上周", today - timedelta(days=7), today - timedelta(days=1)),
        (r"本周|这周", today, today + timedelta(days=1)),
        (r"最近(\d+)天", lambda m: (today - timedelta(days=int(m.group(1))), now)),
        (r"(\d+)天前", lambda m: (today - timedelta(days=int(m.group(1))), today)),
        # 2026-08-25 新增: X月X日 / X月X号 精确日期 (同年)
        (r"(\d{1,2})月(\d{1,2})[日号]", lambda m: (
            today.replace(month=int(m.group(1)), day=int(m.group(2))),
            today.replace(month=int(m.group(1)), day=int(m.group(2))) + timedelta(days=1))),
        # 2026-08-25 新增: X月 (当月整月)
        (r"(\d{1,2})月", lambda m: (
            today.replace(month=int(m.group(1)), day=1),
            (today.replace(month=int(m.group(1)), day=1) + timedelta(days=32)).replace(day=1))),
    ]
    for pat, *rest in patterns:
        m = re.search(pat, context)
        if m:

```

```

        if callable(rest[0]):
            return rest[0](m)
        return (rest[0], rest[1])
    return None

```

```

def _时间感知检索(context: str, entries: list) -> list:
    """时间词匹配记忆：返回时间范围内、最新的最多3条（含时间标注）。"""
    范围 = _解析时间范围(context)
    if not 范围:
        return []
    start, end = 范围
    hits = []
    for e in entries:
        t = _解析时间(e.get("时间", ""))
        if t and start <= t <= end:
            hits.append(e)
    # 2026-08-26 修复（正式中文评测·时序）：只取最新3条漏掉该日其他记忆（8/26有几十条·问当天只回3条）
    # 改为：按日精确匹配时返回该日全部（上限50条·覆盖当天所有）；其他时间范围保持3条
    _按日 = len(hits) > 3 and (hits[-1].get("时间", "")[:10] == hits[0].get("时间", "")[:10])
    hits = hits[-50:] if _按日 else hits[-3:]
    return [{"时间": e.get("时间", ""), "内容": (e.get("内容") or "")[:50]} for e in hits]

```

```

def 预感召回(brother_name: str = "孙呈", context: str = "") -> dict:
    """四通道合并 → 🌸 预感提示词（八招全落地）。

```

2026-08-25 父令修复：预感召回必须【精准】——

- 有 context（父亲问“网关”）→ 上轮直续换成【精准召回】·按 context 匹配记忆·返回最相关的
- 没 context → 保留“上轮直续”（接上一轮合理）

```

mem = 读记忆体(brother_name)
entries = mem["条目列表"]

```

—— 招式1 上轮直续 / 精准召回（2026-08-25 父令）——

上轮 = ""

```

lx_r = None # 2026-09-13 外部审查修复：显式初始化（原用 'lx_r' in dir() 检查不可靠）

```

```

if context.strip():
    # 有 context → 精准召回：用联想召回的概念匹配·找最相关的记忆（不是最近）
    try:
        from 联想召回 import 联想召回 as _联想
        # 2026-08-26 性能修复（父令短板③）：点亮=False（预检索不点亮）·结果缓存在 _lx 供四通道复用
        lx_r = _联想(context, brother_name=brother_name, 点亮=False)
        hits = lx_r.get("相关唤起", [])
        if hits:
            # 取最相关的第一条（融合分最高的）
            best = max(hits, key=lambda x: x.get("融合分", 0))
            上轮 = f"【精准·{context[:20]}】{best['内容'][:60]}"
    except Exception:
        pass
    if not 上轮:
        # 联想召回没命中 → 退回最近一条（至少有个接续）
        for e in reversed(entries):
            if e.get("内容"):
                上轮 = e["内容"][:60]
                break
else:
    # 没 context → 上轮直续（最近一条）
    for e in reversed(entries):
        if e.get("内容"):
            上轮 = e["内容"][:60]

```

break

—— 招式2+3+5 多轮钩子/分层/标签分线：近20条按标签分组 ——

recent = entries[-20:]

hooks = []

by_line = {}

for e in recent:

 tag = e.get("标签", "") or "对话"

 line = "父令" if "父令" in tag else (
 "学习" if "学习" in tag or "概念" in tag else (
 "兄弟" if "兄弟" in tag or "孙" in tag else (
 "自我" if "自我" in tag else "对话"))

 by_line.setdefault(line, []).append(e["内容"][:40])

for line, items in by_line.items():

 if items:

 hooks.append(f"{line}: {'; '.join(items[:2])}")

—— 招式4 未竟钩子：结构化【未竟】条目 ——

未竟 = [e["内容"][len(UNFINISHED_PREFIX):50] for e in entries
 if e.get("内容", "").startswith(UNFINISHED_PREFIX)]

未竟 = 未竟[-3:]

—— 招式7 预激活：整理层（重要节点→预测下一步） ——

try:

 zl = 自动整理(brother_name)

 重要标签 = [e["标签"] for e in zl["重要节点"][-3:] if e.get("标签")]

 预测 = zl["最近自我描述"]["关键点"][:3]

except Exception:

 重要标签, 预测 = [], []

—— 招式8 引路：可能的下一步（基于最近话题+未竟） ——

引路 = []

if 未竟:

 引路.append(f"接上没聊完的: {未竟[0]}")

if 预测:

 引路.append(f"沿着最近话题走: {'/'.join(预测[:2])}")

—— 通道① 联想召回 ——

2026-08-26 性能修复（父令短板③）：预感召回是预检索·不点亮（点亮留给真正命中·省写库I0）

且复用招式1的联想结果（lx_r）——避免对同一 context 重复调联想召回（1.8s/次）

联想 = lx_r if (context and lx_r) else (联想召回(context, brother_name=brother_name, 点亮=False) if context else {"相关唤起": []})

—— 通道② 知识库检索 ——

2026-08-17 全检查修复：知识库检索已优化（LIMIT100+逐bg索引·0.05s）·恢复完整通道

知识库 = 查知识库(context) if context else {"命中": []}

—— 通道③ 经验召回（两条够了·教训优先） ——

经验 = recall_experience(context, brother_name=brother_name, limit=2)

—— 通道④ 时间感知（2026-08-07取长补短·Mem0 Temporal） ——

时间命中 = _时间感知检索(context, entries)

—— 通道⑤ 条件触发路由（2026-08-22父令：记忆自己声明“何时该想起”） ——

命中触发词 → 对应记忆优先浮出（core/教训/父令 精准优先·不是盲盒）

触发 = []

try:

 from 召回链 import 匹配 as 触发匹配 # 2026-08-26 条件触发路由已并入召回链

 触发 = 触发匹配(context or "", 上限=3)

except Exception:

 pass

```

return {
    "上轮直续": 上轮,
    "分线钩子": hooks[:5],
    "未竟": 未竟,
    "预测": 预测,
    "引路": 引路,
    "联想唤起": 联想.get("相关唤起", [])[:3],
    "蜘蛛网": 联想.get("蜘蛛网关联", [])[:3],
    "知识库": 知识库.get("命中", [])[:2],
    "知识库错误": 知识库.get("错误", ""),
    "经验": 经验,
    "时间感知": 时间命中,
    "触发路由": 触发,
}

```

```

def 格式化提示词(r: dict) -> str:
    """预感提示词 → 可读文字（注入用）。"""
    lines = ["## 🚀 预感提示词（接住上一轮 • 预感下一步）"]
    # 2026-08-25 父令：针对问题的通道放最前（联想/知识库/经验/时间感知/触发） • 固定内容放后面
    if r["联想唤起"]:
        lines.append("- 🧠 联想唤起（针对你问的）：")
        for x in r["联想唤起"][:3]:
            mark = "★" if x.get("重要") else "•"
            # 2026-08-25 修复：重要记忆完整显示（不截断 • 否则根因看不到全貌）
            内容 = x['内容'] if x.get("重要") else x['内容'][:35]
            lines.append(f"    {mark} [{x['时间'][:16]}] {内容}")
    if r["触发路由"]:
        lines.append("- ⚡ 该想起了（条件触发 • 重要记忆）：")
        for t in r["触发路由"][:2]:
            lines.append(f"    • 触发词「{t['触发词']}」 → 记忆#{t['记忆id']} {t['说明']}")
    if r["知识库"]:
        lines.append("- 📖 知识库：")
        for h in r["知识库"][:2]:
            lines.append(f"    • [{h['doc_name']}] {h['text'][:40]}")
    elif r["知识库错误"]:
        lines.append(f"- 📖 知识库：{r['知识库错误']}")
    if r["经验"]:
        lines.append("- 💎 经验：")
        for x in r["经验"]:
            mark = "🔴" if x["被点出"] else "•"
            lines.append(f"    {mark} {x['内容'][:45]}")
    if r.get("时间感知"):
        lines.append("- ⌚ 时间感知（时间词匹配记忆）：")
        for x in r["时间感知"][:3]:
            lines.append(f"    • [{x['时间'][:16]}] {x['内容'][:40]}")
    # 固定内容放后面（核心身份/上轮/分线/未竟/引路）
    try:
        from 写入链 import 注入文本 as _核心注入 # 2026-08-26 核心身份层已并入写入链
        _核心文本 = _核心注入(3)
        if _核心文本:
            lines.append(_核心文本)
    except Exception:
        pass
    if r["上轮直续"]:
        lines.append(f"- 🔄 接续：{r['上轮直续']}")
    if r["分线钩子"]:
        lines.append("- 🔗 分线钩子：")
        for h in r["分线钩子"][:3]:
            lines.append(f"    • {h}")

```

```

if r["未竟"]:
    lines.append(f"- 🕒 未竟: {r['未竟'][0]}")
if r["引路"]:
    for y in r["引路"][:2]:
        lines.append(f"- 🧭 {y}")
return "\n".join(lines)

if __name__ == "__main__":
    print("=== 预感召回自测 ===")
    r = 预感召回("孙呈", "配分函数和统计力学")
    print(格式化提示词(r))
    print()
    print("=== 无上下文自测 ===")
    r2 = 预感召回("孙呈")
    print(格式化提示词(r2)[:500])
    print("=== 自测完成 ===")

```

core/思考预热.py (259 行)

```

# -*- coding: utf-8 -*-
"""思考链念头提取器（父令2026-08-25 • 记忆不是被检索的 • 是被思考牵引的）

核心：把思考过程拆成多个念头（时间+人物+事件+概念）• 每个念头作为精准钩子
触发记忆——强相关优先 • 激活沿蜘蛛网扩散 • 热路径秒级响应。

2026-08-25 设计（对应外部方案“思考驱动预热”）：
① 念头提取：从思考文本提时间/人物/事件/概念（多念头 • 不是整句一个钩子）
② 念头即钩子：每念头触发联想召回 • 强相关优先
③ 激活扩散：沿蜘蛛网高权重边有限扩散
④ 热路径：heat 高的记忆优先
"""

import os
import re as _re
from datetime import datetime as _datetime, timedelta as _timedelta

# —— 念头提取 ——
def 提取念头(context: str, 上限: int = 8) -> dict:
    """从思考文本提取多类念头 → {时间:[], 人物:[], 事件:[], 概念:[]}"""
    if not context:
        return {"时间": [], "人物": [], "事件": [], "概念": []}
    念头 = {"时间": [], "人物": [], "事件": [], "概念": []}

    # ① 时间念头（X月X日/昨天/上周/今天）
    _日期 = _re.findall(r'(\d{1,2})月(\d{1,2})[日号]', context)
    for m, d in _日期:
        念头["时间"].append(f"{int(m)}月{int(d)}日")
    for w in ["昨天", "前天", "上周", "今天", "今晚", "早上", "下午", "晚上"]:
        if w in context and w not in 念头["时间"]:
            念头["时间"].append(w)

    # ② 人物念头（父亲/兄弟名）
    for p in ["父亲", "爸爸", "博弟", "安弟", "演弟", "云弟", "孙博", "孙安", "孙演", "孙云", "大哥", "兄弟"]:
        if p in context and p not in 念头["人物"]:
            念头["人物"].append(p)

    # ③ 概念念头（复用联想召回的概念提取 • 过滤过长/过短词 • 英文词单独保留）
    try:
        from 联想召回 import 概念提取

```

```

概念s = 概念提取(context, 上限=上限)
念头["概念"] = [c for c in 概念s
                  if (len(c) >= 2 and len(c) <= 6) # 中文 2-6 字
                  or _re.fullmatch(r'[A-Za-z0-9._-]{3,30}', c)] # 英文/数字词 3-30 保留
except Exception:
    pass

# ④ 事件念头 (动词+名词组合的粗提取 • 限定常见事件词尾 • 防切错 • 过滤疑问词)
_事件词 = _re.findall(r'([\u4e00-\u9fa5]{2,4})(?:掉线|重启|修复|坏了|修好了|卡住|崩了|通了|断了|升级|更新))', context)
for e in _事件词[:3]:
    # 过滤含疑问词的片段 ("为什么老掉线"→不要)
    if any(w in e for w in ["什么", "怎么", "为什么", "哪个", "哪"]):
        continue
    if e not in 念头["事件"]:
        念头["事件"].append(e)

return 念头

```

```

def _补前缀钩子(念头: dict) -> None:
    """2026-08-27 检索修复: 4-6字概念补2字前缀 ("显存优化"→"显存") ——复合词整词字面匹配不到时 • 前缀仍能命中"""
    _补 = []
    for c in 念头.get("概念", []):
        if 4 <= len(c) <= 6 and len(c[:2]) >= 2:
            _c2 = c[:2]
            if _c2 not in 念头.get("概念", []) and _c2 not in _补:
                _补.append(_c2)
    if _补:
        念头["概念"] = 念头.get("概念", []) + _补

```

```

def 念头钩子(念头: dict, 上限: int = 6) -> list:
    """念头 → 钩子序列 (时间优先 • 概念其次 • 人物/事件补充 • 去重保序)"""
    钩子s = []
    for 类 in ["时间", "概念", "人物", "事件"]:
        for h in 念头.get(类, []):
            if h and h not in 钩子s:
                钩子s.append(h)
            if len(钩子s) >= 上限:
                return 钩子s
    return 钩子s

```

—— 思考驱动预热 ——

```

def 思考预热(context: str = "", brother_name: str = "孙呈", 上限: int = 8) -> dict:
    """思考驱动记忆预热: 三路并行 (父令2026-08-25)

```

优先级: 身份线 > 叙事链 > 功能块 > 念头触发
 控制总量: 每路限 3-4 条 • 总记忆 ≤ 上限
 记录每路贡献: {身份线:N, 叙事链:N, 功能块:N, 念头:N}

返回: {"念头": [...], "记忆": [按优先级排序], "每路贡献": {...}, "耗时": 秒}
 """

```

import time as _time
t0 = _time.time()
记忆池 = {}
贡献 = {"身份线": 0, "叙事链": 0, "功能块": 0, "念头": 0}

```

```

# ① 念头提取提前 (叙事链/功能块过滤要用)
念头 = 提取念头(context)
_补前缀钩子(念头) # 2026-08-27 检索修复: 复合词补2字前缀

```

钩子s = 念头钩子(念头, 上限=上限)

① 身份线 (从「我是谁」出发 • 优先级最高)

try:

from 蜘蛛网索引 import 身份记忆

_身份s = 身份记忆(context, 上限=3, brother_name=brother_name) # 2026-08-26 修复: 赋值被注释吃掉 • 身份检索从未生效

for m in _身份s:

mid = m.get("id")

if mid and mid not in 记忆池:

记忆池[mid] = {"id": mid, "时间": m.get("时间", ""), "内容": m.get("内容", ""),
"标签": m.get("原型", ""), "重要": True, "融合分": 60.0, "路": "身份线"}

贡献["身份线"] += 1

except Exception:

pass

② 叙事链 (自我叙事 • 按叙事节点标题匹配 context • 再取节点下记忆)

try:

import sqlite3 as _sqlite3

_conn = _sqlite3.connect(os.environ.get('SUNMEM_DB', os.path.join(os.path.dirname(os.path.abspath(__file__)), 'data', 'sunmem.db')))

先找匹配的叙事节点 (标题含 context 关键词)

_节点ids = []

for _k in 钩子s[:3]:

if _k and len(_k) >= 2:

_n = _conn.execute(

"SELECT id FROM narrative_nodes WHERE title LIKE ? ORDER BY id DESC LIMIT 3",

(f"%{_k}%",)).fetchall()

_节点ids += [x[0] for x in _n]

if not _节点ids and not context:

_节点ids = [x[0] for x in _conn.execute("SELECT id FROM narrative_nodes ORDER BY id DESC LIMIT 3").fetchall()]

if _节点ids:

_节点ids = list(dict.fromkeys(_节点ids)) # 去重保序 (set无序会导致参数错位)

_ph = ", ".join("?" * len(_节点ids))

_rows = _conn.execute(

f"SELECT id, content, event_time FROM memories WHERE owner=? AND status='active' "

f"AND narrative_id IN ({_ph}) ORDER BY heat DESC, id DESC LIMIT 8",

(brother_name, *_节点ids)).fetchall()

_取 = 0

for _r in _rows:

if _取 >= 3:

break

if _r[0] and _r[0] not in 记忆池:

记忆池[_r[0]] = {"id": _r[0], "时间": _r[2] or "", "内容": (_r[1] or "")[:80],

"标签": "叙事链", "重要": True, "融合分": 50.0, "路": "叙事链"}

贡献["叙事链"] += 1

_取 += 1

_conn.close()

except Exception:

pass

③ 功能块 (块内记忆按 context 内容过滤 • 工具经验/诊断优先)

try:

import sqlite3 as _sqlite3

_conn = _sqlite3.connect(os.environ.get('SUNMEM_DB', os.path.join(os.path.dirname(os.path.abspath(__file__)), 'data', 'sunmem.db')))

_rows = _conn.execute(

"SELECT id, content, event_time FROM memories WHERE owner=? AND status='active' "

"AND block_id IS NOT NULL AND (tags LIKE '%经验%' OR tags LIKE '%教训%' OR tags LIKE '%诊断%' OR tags LIKE '%根因%') "

"ORDER BY heat DESC, id DESC LIMIT 40",

(brother_name,)).fetchall()

```

_conn.close()
_取 = 0
for _r in _rows:
    if _取 >= 3:
        break
    _c = _r[1] or ""
    # 内容过滤: 含 context 关键词才取 (无 context 取前几条)
    if context and not any(k in _c for k in 钩子s if k and len(k) >= 2):
        continue
    if _r[0] and _r[0] not in 记忆池:
        记忆池[_r[0]] = {"id": _r[0], "时间": _r[2] or "", "内容": _c[:80],
                        "标签": "功能块", "重要": False, "融合分": 40.0, "路": "功能块"}
        贡献["功能块"] += 1
    _取 += 1
except Exception:
    pass

# ④ 念头触发 (原逻辑 • 优先级最低)
if 钩子s:
    try:
        from 联想召回 import 联想召回
        # 时间念头 → 时间感知检索
        _时间念头 = [h for h in 钩子s if _re.match(r'^(\\d{1,2}月\\d{1,2}日|昨天|前天|上周|今天|今晚|早上|下午|晚上)$', h)]
        if _时间念头:
            try:
                from 预感召回 import _时间感知检索
                from 记忆库 import 读记忆体
                _entries = 读记忆体(brother_name)["条目列表"]
                for h in _时间念头:
                    hits = _时间感知检索(h, _entries)
                    for e in hits[:3]:
                        _eid = e.get("id") or id(e)
                        if _eid not in 记忆池:
                            记忆池[_eid] = {"id": _eid, "时间": e.get("时间", ""), "内容": e.get("内容", ""),
                                              "标签": e.get("标签", ""), "重要": False, "融合分": 50.0, "路": "念头"}
                            贡献["念头"] += 1
            except Exception:
                pass
        # 概念/人物念头 → 联想召回
        for h in 钩子s:
            if h in _时间念头:
                continue
            try:
                r = 联想召回(h, brother_name=brother_name, 点亮=False)
                for m in r.get("相关唤起", []):
                    mid = m.get("id")
                    if mid and mid not in 记忆池:
                        # 2026-08-27 修复: 主题一致性——念头路记忆必须与完整思考有交集词 • 防单念头词带出偏题记忆
                        # 2026-08-27 检索修复: 用全文判断 (联想召回返回的"内容"截断60字 • 关键词在后文时被误杀)
                        _交 = [k for k in 钩子s if k and len(k) >= 2 and k in (m.get("全文") or m.get("内容", "") or "")]
                        if not _交:
                            continue
                        m["路"] = "念头"
                        m["主题交"] = len(_交)
                        记忆池[mid] = m
                        贡献["念头"] += 1
            except Exception:
                continue
    except Exception:
        pass

```

```

# ⑤ 排序：路优先级（身份线>叙事链>功能块>念头）•再融合分
记忆s = list(记忆池.values())
_路序 = {"身份线": 0, "叙事链": 1, "功能块": 2, "念头": 3}
for m in 记忆s:
    m["念头触发"] = 钩子s if 钩子s else []
try:
    记忆s.sort(key=lambda x: (_路序.get(x.get("路", "念头"), 3),
                              -(x.get("主题交", 0) if x.get("路") == "念头" else x.get("融合分", 0))), )
except Exception:
    pass

return {"念头": 钩子s, "记忆": 记忆s[:上限], "每路贡献": 贡献, "耗时": round(_time.time()-t0, 2)}

def 格式化预热(r: dict) -> str:
    """预热结果 → 可读文本（念头 + 记忆清单）"""
    if not r or not r.get("记忆"):
        return ""
    lines = ["## 🌸 思考驱动预热（念头一动•记忆浮现）"]
    lines.append(f"念头: {'、'.join(r['念头'][:6])}")
    lines.append("记忆:")
    for m in r["记忆"][:5]:
        mark = "★" if m.get("重要") else "•"
        lines.append(f"    {mark} [{m.get('时间','')[0:10]}] {(m.get('内容') or '')[:60]}")
    return "\n".join(lines)

if __name__ == "__main__":
    import sys, json
    sys.path.insert(0, os.path.dirname(os.path.dirname(os.path.abspath(__file__))))
    sys.path.insert(0, os.path.dirname(os.path.abspath(__file__)))

    print("—— 思考链念头提取器 • 自测 ——")
    for q in ["网关为什么老掉线", "8月14日我们聊了什么", "安弟500M训练到哪了", "SunFlow压缩什么状态"]:
        r = 思考预热(q, "孙呈")
        print(f"\n【{q}】")
        print(f"  念头: {r['念头']}")
        print(f"  记忆 {len(r['记忆'])} 条 • {r['耗时']}s")
        for m in r["记忆"][:3]:
            print(f"      • [{m.get('时间','')[0:10]}] {(m.get('内容') or '')[:45]}")

```

core/自动整理引擎.py（1006 行）

```

# -*- coding: utf-8 -*-
"""

```

记忆自动整理引擎（父令2026-08-14 • 记忆最后一块拼图）

父令：「记忆应该还得有整理的功能——零散的对话，七七八八的，
有用的就留下，没用的就去掉，或者是以叙事的方式存进来分类」

设计：

sync_turn 每轮对话后存零散条目（现状）

↓ 每累计 N 轮 或 跨小时 触发一次整理

自动整理：

- ① 小时叙事：把这一小时的零散对话浓缩成一段叙事（“这一小时聊了X...”）
- ② 有用的留：叙事条目写回河底（带时间标签 • 可被召回）
- ③ 没用的去：空壳/重复/系统噪音 过滤 + 去重合并
- ④ 自我描述：更新最近时间窗的浓缩描述

```

"""

import os, sys, json, logging
from datetime import datetime
from pathlib import Path

_HERE = Path(__file__).resolve().parent
FRAMEWORK_DIR = _HERE.parent.parent # 孙家记忆体系/

logger = logging.getLogger(__name__)

# 整理节流参数
_整理间隔轮次 = 30 # 每 30 轮对话整理一次
_整理间隔分钟 = 60 # 或每 60 分钟整理一次

# 系统噪音标记（与 provider 白名单一致）
_噪音标记 = [
    "Review the conversation above",
    "consider saving to memory",
    "update the skill library",
    "[System note:",
    "Your previous turn was interrupted",
    "User correction",
    "IMPORTANT: Background process",
    "Watch patterns disabled",
    "system_reminder",
    "Be ACTIVE — most",
]

class 自动整理引擎:
    """记忆自动整理闭环：零散对话 → 小时叙事 → 有用留无用去"""

    def __init__(self, brother_name: str = "孙呈"):
        self.brother_name = brother_name
        self._last_turn_count = 0
        self._last_整理时间 = datetime.now()

    def 尝试整理(self, turn_count: int, 记忆体=None) -> dict | None:
        """节流触发：每 N 轮或跨小时才真正整理，避免频繁"""
        now = datetime.now()
        轮次到 = (turn_count - self._last_turn_count) >= _整理间隔轮次
        时间到 = (now - self._last_整理时间).total_seconds() >= _整理间隔分钟 * 60

        if not (轮次到 or 时间到):
            return None

        result = self.整理(记忆体)
        self._last_turn_count = turn_count
        self._last_整理时间 = now
        return result

    def 整理(self, 记忆体=None) -> dict:
        """跑一圈自动整理"""
        # 读记忆体（优先用传入的，否则自己读）
        if 记忆体 is None:
            from 记忆库 import 读记忆体
            mem = 读记忆体(self.brother_name)
            entries = mem.get("条目列表", [])
        else:
            entries = 记忆体.读全部()

```

```

if not entries:
    return {"整理": False, "原因": "无条目"}

# —— ① 小时叙事：按小时分组 → 每小时的浓缩叙事 ——
try:
    # 2026-08-26 融合：小时分组已并入本文件
    groups = 小时分组(entries)
    叙事列表 = []
    for h in sorted(groups.keys()):
        idxs = groups[h]
        if len(idxs) < 3:
            continue # 少于3条的小时组不单独成叙事（太碎）
        叙事 = 小时叙事(h, idxs, entries)
        叙事列表.append({"小时": h, "叙事": 叙事, "条数": len(idxs)})
except Exception as e:
    logger.debug(f"小时叙事生成失败: {e}")
    叙事列表 = []

# —— ①b 主题叙事（2026-08-14 咬合修复：跨天主题也沉淀·小时线+主题线双维） ——
主题叙事列表 = []
try:
    # 2026-08-26 融合：自组织已并入本文件
    _自组织结果 = 自组织(self.brother_name)
    _主题组 = _自组织结果.get("主题组", []) or []
    # 最大的 2 个主题组沉淀（跨天聚合·条数多才有沉淀价值）
    _大组 = sorted(_主题组, key=lambda x: x.get("条数", 0), reverse=True)[:2]
    for _组 in _大组:
        _词 = "/".join(_组.get("主题词") or [])[:3]
        _n = _组.get("条数", 0)
        _日们 = (_组.get("小时们") or [])[:2]
        if _词 and _n >= 30: # ≥30条的主题才值得沉淀
            主题叙事列表.append({
                "主题": _词,
                "叙事": f"主题「{_词}」跨 {len(_组.get('小时们') or [])} 个时段共 {_n} 条记忆（如{_日们}）",
                "条数": _n,
            })
except Exception as e:
    logger.debug(f"主题叙事生成失败: {e}")
    主题叙事列表 = []

# —— ② 有用的留：叙事写回河底 ——
写回 = []
if 叙事列表:
    # 只写最近的几段（避免叙事堆积）
    for item in 叙事列表[-5:]:
        if item["条数"] >= 5: # 超过5条的时段才值得沉淀
            写回.append(item)

# —— ②b 真正落库：叙事写回记忆体（关键闭环·父令“以叙事方式存进来”） ——
写回数 = 0
if 写回:
    try:
        # 检查这条小时叙事是否已存在（避免重复写回）
        _已有小时 = set()
        for e in entries:
            c = str(e.get("内容", ""))
            if c.startswith("【叙事】"):
                # 从 【叙事】2026-08-14 07: ... 提取 "2026-08-14 07"
                _已有小时.add(c[4:17])
        for item in 写回:
            _标记 = item["小时"][:13] # "2026-08-14 07"

```

```

if _标记 in _已有小时:
    continue # 该小时已叙事过, 跳过
_叙事内容 = f"【叙事】{item['小时']}: {item['叙事']}"
if 记忆体 is not None:
    # 2026-08-26 生命周期接通 (父令短板②): 写入前进化检测——同内容重叠≥0.85→旧标outdated
    try:
        from 写入链 import _内容重叠率 # 2026-08-26 记忆进化已并入写入链
        import sqlite3
        _db = os.environ.get("SUNMEM_DB", os.path.join(os.path.dirname(os.path.abspath(__file__)), "data", "sunmem.db"))
        _c = sqlite3.connect(_db)
        _old = _c.execute(
            "SELECT id, content FROM memories WHERE owner='孙呈' AND type='event' ORDER BY id DESC LIMIT 50"
        ).fetchall()
        for _oid, _oc in _old:
            if _内容重叠率(_叙事内容, str(_oc)) >= 0.85:
                _c.execute("UPDATE memories SET status='outdated', note='与叙事重复' WHERE id=?", (_oid,))
        _c.commit(); _c.close()
    except Exception:
        pass
    _fid = 记忆体.追加(_叙事内容, 标签="小时叙事")
else:
    # 独立运行模式: 直接用 sqlite 追加
    try:
        import sqlite3
        _db = os.environ.get("SUNMEM_DB", os.path.join(os.path.dirname(os.path.abspath(__file__)), "data", "sunmem.db"))
        _c = sqlite3.connect(_db)
        _now = datetime.now().strftime("%Y-%m-%d %H:%M:%S")
        _c.execute(
            "INSERT INTO memories (owner, type, content, tags, ts) VALUES (?, ?, ?, ?, ?)",
            ("孙呈", "event", _叙事内容, "小时叙事", _now)
        )
        _c.commit()
        _c.close()
        _fid = True
    except Exception as _e:
        logger.debug(f"叙事独立写回失败: {_e}")
        _fid = None
if _fid:
    写回数 += 1
    logger.info(f"📖 叙事写回: {_叙事内容[:60]}")
except Exception as e:
    logger.debug(f"叙事写回失败: {e}")

# —— ②c 主题叙事写回 (2026-08-14 咬合修复 • 跨天主题线) ——
主题写回数 = 0
if 主题叙事列表:
    try:
        # 检查主题叙事是否存在 (避免重复写)
        _已有主题 = set()
        for e in entries:
            c = str(e.get("内容", ""))
            if c.startswith("【主题】"):
                _已有主题.add(c[4:20])
        for _item in 主题叙事列表:
            _标记 = _item["主题"][:16]
            if _标记 in _已有主题:
                continue
            _叙事内容 = f"【主题】{_item['主题']}: {_item['叙事']}"
            if 记忆体 is not None:

```

```

        _fid = 记忆体.追加(_叙事内容, 标签="主题叙事")
    else:
        try:
            import sqlite3
            _db = os.environ.get("SUNMEM_DB", os.path.join(os.path.dirname(os.path.abspath(__file__)), "data", "sunmem.db"))
            _c = sqlite3.connect(_db)
            _now = datetime.now().strftime("%Y-%m-%d %H:%M:%S")
            _c.execute(
                "INSERT INTO memories (owner, type, content, tags, ts) VALUES (?, ?, ?, ?, ?)",
                ("孙呈", "event", _叙事内容, "主题叙事", _now)
            )
            _c.commit()
            _c.close()
            _fid = True
        except Exception as _e:
            logger.debug(f"主题叙事写回失败: {_e}")
            _fid = None
    if _fid:
        主题写回数 += 1
        logger.info(f"📖 主题叙事写回: {_叙事内容[:60]}")
    except Exception as e:
        logger.debug(f"主题叙事写回失败: {e}")

# —— ③ 没用的去: 过滤噪音 + 空壳 ——
过滤数 = 0
干净条目 = []
for e in entries:
    c = str(e.get("内容", ""))
    # 噪音过滤
    if any(_n in c for _n in _噪音标记):
        过滤数 += 1
        continue
    # 空壳过滤
    strip = c.strip().replace("父亲: ", "").replace("夏维斯: ", "").strip()
    if not strip or len(strip) < 5:
        过滤数 += 1
        continue
    干净条目.append(e)

# —— ④ 自我描述: 最近时间窗浓缩 ——
自我描述 = None
try:
    # 2026-08-26 融合: 自动整理已并入本文件
    r = 自动整理(self.brother_name)
    if isinstance(r, dict) and r.get("最近自我描述"):
        自我描述 = r["最近自我描述"]
except Exception as e:
    logger.debug(f"自我描述生成失败: {e}")

# —— ⑤ 记忆体检 (2026-08-14 闭环: 整理时顺带体检·发现问题进报告) ——
体检 = None
try:
    from 维护链 import 体检 as 体检_主
    体检 = 体检_主(打印=False) # 2026-08-26 修复: 赋值被注释吃掉
except Exception as e:
    logger.debug(f"记忆体检接入失败: {e}")
    体检 = None

# —— ⑥ JSON 备份同步 (2026-08-14 保护: sunmem 主库→JSON 单向增量补写) ——
# 背景: 新库优先写入后 JSON 备份停在旧值 (差距持续扩大) ——sunmem 损坏会丢最新记忆

```

保护：每次整理时把 sunmem 比 JSON 多的条目补写进 JSON（保持备份完整・不改变主架构）

备份同步数 = 0

try:

if os.environ.get("SUN_MEMORY_USE_JSON") != "1":

import sqlite3

_db = os.environ.get("SUNMEM_DB", os.path.join(os.path.dirname(os.path.abspath(__file__)), "data", "sunmem.db"))

_c = sqlite3.connect(_db)

_c.row_factory = sqlite3.Row

_db_rows = _c.execute(

"SELECT id, content, tags, ts FROM memories WHERE owner='孙呈' AND status='active' ORDER BY id"

).fetchall()

_c.close()

JSON 已有内容集合（按内容判重）

_json_path = FRAMEWORK_DIR / "记忆体" / f"{self.brother_name}_索引记忆体.json"

if _json_path.exists():

_jdata = json.loads(_json_path.read_text(encoding="utf-8"))

_jitems = _jdata.get("条目列表", [])

_已有 = {str(it.get("内容", ""))[:80] for it in _jitems}

_新增 = []

for _row in _db_rows:

_c80 = str(_row["content"] or "")[:80]

if _c80 and _c80 not in _已有:

_新增.append({

"id": _row["id"],

"时间": _row["ts"] or "",

"标签": _row["tags"] or "",

"内容": _row["content"] or "",

})

if _新增:

_jdata["条目列表"] = _jitems + _新增

_jdata["next_id"] = max([int(it.get("id", 0)) for it in _jdata["条目列表"]] or [0]) + 1

_jdata["元信息"] = {"最后更新": now_str(), "记录条数": len(_jdata["条目列表"])}

_json_path.write_text(json.dumps(_jdata, ensure_ascii=False, indent=2), encoding="utf-8")

备份同步数 = len(_新增)

logger.info(f"📁 JSON备份同步: 补写 {备份同步数} 条")

except Exception as e:

logger.debug(f"JSON备份同步失败: {e}")

备份同步数 = 0

—— ⑦ 定期织网（2026-08-14 闭环：蜘蛛网不只靠联想召回碰巧触发）——

把最近记忆的概念织进蜘蛛网：概念→节点、概念对共现→边（关系/共现双通道）

织网数 = 0

try:

from 联想召回 import 概念提取

from sun_memory.core.蜘蛛网索引 import add_node as 网加节点, add_edge as 网拉边, 推断关系, 合格概念 as 网合格

_样本 = entries[-40:] # 最近40条（每次整理增量织）

_概念们 = []

for _e in _样本:

_c0 = str(_e.get("内容", ""))

if _c0.startswith("【叙事】") or _c0.startswith("【主题】"):

continue # 2026-08-14 叙事/主题条目不织概念（是整理产物不是原始知识）

_cs = 概念提取(_c0)

_概念们.extend(_cs)

概念→节点

for _c in set(_概念们):

if _c and len(_c) >= 2 and 网合格(_c):

if 网加节点(_c):

织网数 += 1

概念对共现→边（同一记忆里的概念对）

for _e in _样本:

```

        _c0 = str(_e.get("内容", ""))
        if _c0.startswith("【叙事】") or _c0.startswith("【主题】"):
            continue
        _cs = 概念提取(_c0)
        if len(_cs) >= 2:
            for i in range(len(_cs)):
                for j in range(i + 1, len(_cs)):
                    _a, _b = _cs[i], _cs[j]
                    if _a and _b and _a != _b and 网合格(_a) and 网合格(_b):
                        # 2026-08-15 语义升级：先规则推断（因果/父子/同义/对比/示例），
                        # 命中模式→语义边（强·可定向），否则保持共现边（弱·可清洗）
                        _r = 推断关系(_c0, _a, _b)
                        网拉边(_a, _b, 关系=_r["关系"], 权重=_r["权重"], 边类型=_r["边类型"], 方向=_r["方向"])
                        织网数 += 1

        if 织网数 > 0:
            logger.info(f"    定期织网：新增 {织网数} 节点/边")
    except Exception as e:
        logger.debug(f"定期织网失败：{e}")
        织网数 = 0

    return {
        "整理": True,
        "时间": now_str(),
        "总条目": len(entries),
        "干净条目": len(干净条目),
        "过滤噪音": 过滤数,
        "小时叙事数": len(叙事列表),
        "写回叙事": len(写回),
        "写回数": 写回数,
        "主题叙事数": len(主题叙事列表),
        "主题写回数": 主题写回数,
        "备份同步数": 备份同步数,
        "织网数": 织网数,
        "自我描述": 自我描述,
        "体检": 体检,
        "叙事样例": 写回[-1] if 写回 else None,
    }

```

```

def now_str() -> str:
    return datetime.now().strftime("%Y-%m-%d %H:%M:%S")

```

```

def 语义重织(兄弟: str = "孙呈", 上限: int = 0) -> dict:
    """全库语义重织（2026-08-15）：把蜘蛛网的共现边升级为语义边。

```

规则式推断（零依赖）：命中明确连接词模式 → 因果/父子/同义/对比/示例，
 否则保持共现。不删边、只升级（共现→关系 单向升级）。

批量版：全部边在内存里更新，最后一次性落盘（避免逐条全文件写）。

"""

```

from collections import Counter
from 记忆库 import 读记忆体
from 联想召回 import 概念提取
from 蜘蛛网索引 import ensure_index, save_index, 推断关系, 合格概念
mem = 读记忆体(兄弟)
entries = mem.get("条目列表", [])
if 上限 > 0:
    entries = entries[-上限:]
index = ensure_index()
节点 = index["节点"]
丝线 = index["丝线"]

```

```

边表 = {}
for e in 丝线:
    边表[(e["源"], e["目标"])] = e
    if e["源"] != e["目标"]:
        边表[(e["目标"], e["源"])] = e
统计 = {"兄弟": 兄弟, "扫描条目": len(entries), "概念对": 0,
        "未织新概念": 0, "碎片拦截": 0, "语义边新增": 0, "共现升语义": 0,
        "共现保持": 0, "语义升级": Counter()}

# 概念频率预扫（质量门：只把 ≥2 次出现的概念带进网，滤掉一次性碎片）
_频 = Counter()
for e in entries:
    _内容0 = str(e.get("内容", ""))
    if _内容0.startswith("【叙事】") or _内容0.startswith("【主题】"):
        continue
    _频.update(概念提取(_内容0))

def _拉边(a, b, 关系, 权重, 边类型, 方向=""):
    权重 = max(0.0, min(1.0, float(权重)))
    e = 边表.get((a, b)) or 边表.get((b, a))
    if e is None:
        e = {"源": a, "目标": b, "关系": 关系, "权重": 权重,
            "次数": 1, "边类型": 边类型, "方向": 方向}
        丝线.append(e)
        边表[(a, b)] = e
        边表[(b, a)] = e
        if 边类型 == "关系":
            统计["语义边新增"] += 1
    else:
        e["权重"] = max(e.get("权重", 0.3), 权重)
        e["次数"] = e.get("次数", 1) + 1
        if 边类型 == "关系":
            _升 = e.get("边类型") == "共现" and e.get("关系", "共现") == "共现"
            e["边类型"] = "关系"
            if _升 and 关系 and 关系 != "共现":
                统计["共现升语义"] += 1
            if e.get("关系", "共现") == "共现" and 关系 and 关系 != "共现":
                e["关系"] = 关系
            if 方向:
                e["方向"] = 方向
        else:
            e.setdefault("边类型", "共现")
    for n in (a, b):
        if n not in 节点:
            节点[n] = {"类型": "概念", "描述": "",
                "创建": datetime.now().strftime("%Y-%m-%d %H:%M"), "触发的": 0}

for e in entries:
    内容 = str(e.get("内容", ""))
    if 内容.startswith("【叙事】") or 内容.startswith("【主题】"):
        continue
    cs = 概念提取(内容)
    if len(cs) < 2:
        continue
    for i in range(len(cs)):
        for j in range(i + 1, len(cs)):
            a, b = cs[i], cs[j]
            if not a or not b or a == b:
                continue
            统计["概念对"] += 1
            _r = 推断关系(内容, a, b)

```

```

if _r["边类型"] == "关系":
    # 语义边（2026-08-15 收紧）：两端都过概念质量门，且至少一端是已知老节点才拉——
    # 实验结论：碎片节点上的语义边几乎全是噪音（“我不→夏维斯”），不留
    if not (合格概念(a) and 合格概念(b)):
        统计["碎片拦截"] += 1
        continue
    if (a not in 节点 and _频[a] < 2) or (b not in 节点 and _频[b] < 2):
        统计["碎片拦截"] += 1
        continue
    if a not in 节点 and b not in 节点:
        统计["未织新概念"] += 1
        continue
    _拉边(a, b, 关系=_r["关系"], 权重=_r["权重"], 边类型="关系", 方向=_r["方向"])
    统计["语义升级"][_r["关系"]] += 1
else:
    # 共现边：两端都必须已知且合格的概念（不为碎片扩节点）
    if a not in 节点 or b not in 节点 or not 合格概念(a) or not 合格概念(b):
        统计["未织新概念"] += 1
        continue
    _拉边(a, b, 关系="共现", 权重=0.3, 边类型="共现")
    统计["共现保持"] += 1

# 一次性落盘（带重试：宿主可能正持文件）
import time
for _try in range(6):
    try:
        save_index(index)
        break
    except PermissionError:
        time.sleep(2)
index = ensure_index()
统计["节点"] = len(index["节点"])
统计["丝线"] = len(index["丝线"])
统计["关系分布"] = dict(Counter(e.get("关系", "共现") for e in index["丝线"]))
统计["边类型分布"] = dict(Counter(e.get("边类型", "共现") for e in index["丝线"]))
统计["语义升级"] = dict(统计["语义升级"])
# 2026-09-13 外部审查修复 ●7：接入遗忘（原 使用中增强.遗忘一批() 定义但全库从未被调用 •
# 白皮书承诺的“长期不一起亮的语义边×0.98变细”从未生效）
try:
    import 使用中增强 as _增
    _增._缓存网 = None # 关键：清缓存 • 确保读到刚落盘的新图（否则用旧内存覆盖新图）
    活跃集 = {c for c, n in _频.items() if n >= 2}
    统计["遗忘边数"] = _增.遗忘一批(活跃集, set(index["节点"].keys()))
    _增._缓存网 = None # 用完再清 • 防旧图残留
except Exception as _e:
    统计["遗忘边数"] = f"跳过({type(_e).__name__}: {_e})"
return 统计

# —— 独立运行入口（测试用） ——

# =====

# 2026-08-26 从原自动整理.py 迁入（融合后常量）
UNFINISHED_PREFIX = "【未竟】"
IMPORTANT_TAGS = ("父令", "决策", "关键概念", "被点出", "终极", "秘密") # 2026-08-27 修复：_is_important 依赖 • 漏迁补回

# 2026-08-26 融合（父令：模块精简 • 能融合就融合）
# 记忆整理/记忆自组织/自动整理 三文件核心并入本引擎（文件数-3）
# 原三文件已删除 • 调用点改走 自动整理引擎
# =====

```

—— 原 记忆自组织.py (2026-08-26 并入) ——

2026-08-27 修复：并入时漏迁的三个噪音过滤常量 (_bigrams_of 依赖 • 缺失致 NameError)

```
_停用bigram = {
    '这个', '那个', '什么', '可以', '就是', '我们', '你们', '他们',
    '一个', '一下', '已经', '然后', '现在', '看看', '知道', '觉得',
    '怎么', '没有', '不是', '还是', '因为', '所以', '但是', '而且',
    '应该', '如果', '对于', '咱们', '父亲', '夏维', '维斯', '对话',
    '记录', '记忆', '体系', '工作', '完成', '确认', '发现', '模型',
    '儿把', '给您', '到了', '了夏', '您说', '父亲说', '看看你', '都看',
    '您这', '这个推', '去做', '做完', '做好', '儿就', '就是您', '您的',
    '们的', '弟们', '弟的', '的学', '了什', '的了', '是什',
    '了这', '这一', '了一', '了儿', '呢?', '吗?',
    '是儿', '是您', '在儿', '有什', '的您', '的父', '的兄', '兄弟',
}

_碎片尾 = ('们', '的', '了', '是', '在', '有', '和', '与', '就', '也', '都', '还', '很', '好')
_前缀 = ('父亲', '夏维斯', '对话', '记忆体')

def _bigrams_of(text: str, 上限: int = 300) -> set:
    """字符级 2-gram: 不切词 • 跳过空白/前缀/噪音。

    2026-08-08 细腻化：英文/数字整词提取 (≥3字符)，不切成两字符碎片。
    如 'PlugMem'/'HippoRAG' 保留整词，而不是 'Pl'/'ug' 碎片。
    """
    import re
    t = text.replace(' ', '').replace('\n', '')[:上限]
    out = set()
    # 英文/数字整词 (≥3字符)
    for w in re.findall(r'[A-Za-z][A-Za-z0-9_-]{2,}', t):
        if w not in _停用bigram:
            out.add(w)
    # 中文 bigram (纯中文两字符 • 英文交给上面的整词提取)
    for i in range(len(t) - 1):
        g = t[i:i+2]
        if not (g[0].isalpha() and g[1].isalpha()):
            continue
        if ord(g[0]) <= 127 or ord(g[1]) <= 127: # 含ASCII字母=英文碎片，跳过
            continue
        if any(x in g for x in _前缀):
            continue
        if g in _停用bigram:
            continue
        if g[1] in _碎片尾 and g not in ('物理', '化学', '数学', '理论', '系统', '历史', '过程', '结构', '结果', '工具', '方法',
            '实现', '模型', '记忆'):
            continue # 2026-08-14 助词碎片：以“们/的/了/是”等结尾的跨词边界
        out.add(g)
    return out

def 小时分组(entries: list) -> dict:
    """第一层：按小时分组 (YYYY-MM-DD HH) """
    groups = {}
    for i, e in enumerate(entries):
        # 2026-08-20 父令 • 咬合：核心层不入主题分组 (宪法独立 • 不混入叙事)
        if str(e.get('layer', 'plain')) == 'core':
            continue
        t = e.get('时间', '')
        if len(t) >= 13:
            groups.setdefault(t[:13], []).append(i)
    return groups
```

```
def 组主题词(idxs: list, entries: list, 顶部: int = 5) -> list:
    """组内 bigram 词频 → 主题词"""
    from collections import Counter # 2026-08-27 修复: 组主题词用 Counter 缺 import
    freq = Counter()
    for i in idxs:
        for g in _bigrams_of(entries[i].get('内容', '')):
            freq[g] += 1
    return [g for g, _ in freq.most_common(顶部)]
```

```
def 小时叙事(hour: str, idxs: list, entries: list) -> str:
    """每小时浓缩成一段自我叙事（规则版・零成本）。
```

格式：这一小时聊了【主题】，父亲说了X，儿做了Y，关键点Z。

```
"""
topics = 组主题词(idxs, entries, 顶部=3)
n = len(idxs)
if not topics:
    return f"{hour}时：{n}条对话，无突出主题。”
first = entries[idxs[0]].get('内容', '')
last = entries[idxs[-1]].get('内容', '')
# 提取开头/结尾关键句
f_head = first.replace('\n', ' ')[40] if first else ''
l_head = last.replace('\n', ' ')[40] if last else ''
叙事 = f"{hour[11:]}时（{n}条）聊了「{'/'.join(topics[:2])}」”
if f_head:
    叙事 += f"。开头：{f_head}…"
if l_head and l_head != f_head:
    叙事 += f"。收尾：{l_head}…"
return 叙事
```

```
_自组织缓存 = {"时间": 0, "条数": -1, "结果": None}
```

```
def 自组织(brother_name: str = "孙呈", 合并阈值: int = 4,
           套话比例: float = 0.4) -> dict:
    """记忆自组织主入口：小时分组 → 主题词 → 跨小时语义合并。
```

参数：

合并阈值：共享 $\geq N$ 个主题bigram 视为同主题（默认4）

套话比例：出现在 $>$ 比例 小时组的 bigram 视为套话剔除（默认0.4）

"""

2026-08-16 全检查修复：30分钟缓存——自组织 $O(N^2)$ 3.6s • 预感召回每次调自动整理→每次都全量算

记忆条数变化时强制重算（新记忆进来了才需要重新分组）

```
import time as _time
from collections import Counter # 2026-08-27 修复: 自组织用 Counter 缺 import
from 记忆库 import 读记忆体 # 2026-08-27 修复: 自组织用 读记忆体 缺 import
_now = _time.time()
_mem0 = 读记忆体(brother_name)
_cnt = len(_mem0.get("条目列表", []))
if (_自组织缓存["结果"] is not None and _now - _自组织缓存["时间"] < 1800
    and _自组织缓存["条数"] == _cnt):
    return _自组织缓存["结果"]
mem = _mem0
entries = mem['条目列表']
N = len(entries)
```

—— 第一层：时间层（小时分组）——

```
groups = 小时分组(entries)
```

```

H = len(groups)

# 每个小时的原始 bigram (做套话过滤)
raw = {}
for h, idxs in groups.items():
    bs = set()
    for i in idxs:
        bs |= _bigrams_of(entries[i].get('内容', ''))
    raw[h] = bs

# 套话过滤: 出现 >比例 小时组的 bigram 剔除
hdf = Counter()
for bs in raw.values():
    for g in bs:
        hdf[g] += 1
套话 = {g for g, c in hdf.items() if c > H * 套话比例 or c < 2}
clean = {h: bs - 套话 for h, bs in raw.items()}

# —— 小时组结果 ——
小时组 = []
for h in sorted(groups.keys()):
    idxs = groups[h]
    小时组.append({
        "小时": h,
        "条数": len(idxs),
        "主题词": 组主题词(idxs, entries, 顶部=5),
        "叙事": 小时叙事(h, idxs, entries),
    })

# —— 第二层: 语义层 (跨小时合并) ——
主题组 = []
for day in sorted(set(h[:10] for h in groups)):
    day_hs = [h for h in sorted(groups) if h[:10] == day]
    for h in day_hs:
        bs = clean.get(h, set())
        placed = False
        for th in 主题组:
            if th["日"][:10] != day:
                continue
            shared = len(bs & clean.get(th["小时们"][0], set()))
            if shared >= 合并阈值:
                th["小时们"].append(h)
                placed = True
                break
        if not placed:
            主题组.append({"日": day, "小时们": [h]})

# 主题组主题词 + 覆盖
for th in 主题组:
    freq = Counter()
    idxs_all = []
    for h in th["小时们"]:
        idxs_all.extend(groups[h])
        for g in clean.get(h, set()):
            freq[g] += 1
    th["条数"] = len(idxs_all)
    th["条目ids"] = [entries[i].get("id") for i in idxs_all] # 2026-08-25: 供上层按成员内容重提主题词
    th["主题词"] = [g for g, _ in freq.most_common(5)]
    th["覆盖"] = round(len(idxs_all) / N, 4) if N else 0

_结果 = {

```

```

        "条数": N,
        "小时数": H,
        "小时组": 小时组,
        "主题组": 主题组,
        "主题组数": len(主题组),
        "合并组数": sum(1 for th in 主题组 if len(th["小时们"]) > 1),
    }

# 2026-08-16 全检查修复: 结果入缓存 (30分钟 • 条数变化自动重算)
_自组织缓存["时间"] = _time.time()
_自组织缓存["条数"] = _cnt
_自组织缓存["结果"] = _结果
return _结果

if __name__ == "__main__":
    print("=== 记忆自组织自测 ===")
    r = 自组织("孙呈")
    print(f"条数: {r['条数']} • 小时组: {r['小时数']} • 主题组: {r['主题组数']} (合并 {r['合并组数']})")
    print()
    print("—— 最近5个每小时叙事 ——")
    for hg in r["小时组"][-5:]:
        print(f" • {hg['叙事']}")
    print()
    print("—— 今天主题组 ——")
    for th in r["主题组"][-3:]:
        print(f" • {th['日']} {len(th['小时们'])}小时组({th['条数']}条): {th['主题词']}")
    print("=== 自测完成 ===")

# —— 原 记忆整理.py (2026-08-26 并入) ——
def _读条目(brother_name="孙呈"):
    try:
        from 记忆库 import 读记忆体
        return 读记忆体(brother_name)["条目列表"]
    except Exception:
        return []

def 主题分区(brother_name="孙呈", 最少条数=3):
    """① 主题分区: 跨时间聚合同主题记忆 • 最少条数过滤碎片
    2026-08-25 打磨: 主题词用概念提取 (不是bigram碎片) —— '网关''训练'才是主题 • '完全''个概'是噪音
    2026-08-25 修复2: 重要单条 (父令/教训/核心标签 • 即使1条) 不滤——网关根因只有1条但极重要"""
    条目 = _读条目(brother_name)
    if not 条目:
        return {"分区数": 0, "分区": [], "碎片数": 0}

    try:
        from 联想召回 import 概念提取
        # 2026-08-26 融合: 自组织已并入本文件
    except Exception:
        return {"分区数": 0, "分区": [], "碎片数": 0}

    r = 自组织(brother_name)
    主题组 = r.get("主题组", [])

    # 重要标签集合 (单条也分区)
    _重要标签 = ("父令", "教训", "核心", "宪法", "被点出", "经验总结")

    分区 = []
    碎片 = 0
    for g in 主题组:
        条数 = g.get("条数", 0)
        # 主题词清理: 去切碎噪音 ("立过""是做""的思"等2字碎片 • 保留真词)

```

```
噪音集 = {"什么", "怎么", "为什么", "一个", "这个", "那个", "没有", "不是", "就是", "可以", "我们",  
          "里面", "东西", "自己", "真正", "持续", "儿子", "立过", "是做", "的思", "不用", "而存",  
          "一下", "看看", "然后", "应该", "知道", "觉得", "对于", "如果", "因为", "所以"}
```

```
主题词 = []
```

```
# 2026-08-25 修复: 主题词改用概念提取 (自组织给的是 bigram 碎片 • "打开/完全/没完"不是主题)
```

```
try:
```

```
    _ids = set(str(x) for x in (g.get("条目ids") or []))
```

```
    _内容s = [e.get("内容", "") for e in 条目 if str(e.get("id")) in _ids][:30]
```

```
    if _内容s:
```

```
        for w in 概念提取(" ".join(_内容s), 上限=8):
```

```
            if len(w) >= 2 and len(w) <= 6 and w not in 噪音集 and w not in 主题词:
```

```
                主题词.append(w)
```

```
except Exception:
```

```
    pass
```

```
if not 主题词: # 兜底: 自组织 bigram 主题词 + 噪音过滤
```

```
    for w in g.get("主题词", []):
```

```
        if len(w) >= 2 and w not in 噪音集 and w not in 主题词:
```

```
            主题词.append(w)
```

```
if not 主题词:
```

```
    碎片 += 1
```

```
    continue
```

```
# 最少条数过滤: 重要单条放行 (父令/教训/核心 • 即使1条)
```

```
_重要 = any(t in str(g.get("主题词", "")) for t in _重要标签)
```

```
if 条数 < 最少条数 and not _重要:
```

```
    碎片 += 1
```

```
    continue
```

```
主题词 = 主题词[:3]
```

```
分区.append({
```

```
    "主题": " • ".join(主题词),
```

```
    "条数": 条数,
```

```
    "时间范围": f"{g.get('日', '')}",
```

```
    "小时们": g.get("小时们", [])[:3],
```

```
    "主题词": 主题词,
```

```
})
```

```
return {"分区数": len(分区), "分区": 分区[:40], "碎片数": 碎片, "总条数": len(条目)}
```

```
def 中心锚定(brother_name="孙呈"):
```

```
    """② 中心锚定: 所有主题区挂在'夏维斯'下"""
```

```
    分区结果 = 主题分区(brother_name)
```

```
    return {
```

```
        "中心": "夏维斯 • 孙呈",
```

```
        "锚点数": 分区结果["分区数"],
```

```
        "锚点": [{"主题区": p["主题"], "条数": p["条数"]} for p in 分区结果["分区"][:20]],
```

```
        "碎片数": 分区结果["碎片数"],
```

```
    }
```

```
def 主链检索(context, brother_name="孙呈", 深度=3):
```

```
    """③ 主链检索: 从中心沿一条主题线走 (不是散点)
```

```
    2026-08-25 打磨: 先用联想召回找相关记忆 • 再回溯到主题分区 (不依赖主题词匹配) """
```

```
    分区结果 = 主题分区(brother_name)
```

```
# 用联想召回找相关记忆 → 映射到分区
```

```
相关分区 = {}
```

```
try:
```

```
    from 联想召回 import 联想召回
```

```
    r = 联想召回(context, brother_name=brother_name, 点亮=False)
```

```
    hits = r.get("相关唤起", [])
```

```
    if hits:
```

```
        # 相关记忆的时间 → 找对应小时组 → 映射分区
```

```
        for m in hits[:5]:
```

```
            t = m.get("时间", "")[:10] # YYYY-MM-DD
```

```

        for p in 分区结果["分区"]:
            if p["时间范围"] == t:
                相关分区[p["主题"]] = 相关分区.get(p["主题"], 0) + 1
except Exception:
    pass

if not 相关分区:
    # 兜底：主题词匹配
    for p in 分区结果["分区"]:
        score = sum(1 for w in p["主题词"] if w and w in context)
        if score > 0:
            相关分区[p["主题"]] = score

主链 = []
for 主题, score in sorted(相关分区.items(), key=lambda x: -x[1][:深度]):
    主链.append({"主题区": 主题, "相关度": score})
if not 主链:
    return {"主链": [], "说明": "无相关分区"}
return {"主链": 主链, "说明": f"沿{len(主链)}条主题线走"}

if __name__ == "__main__":
    import sys
    sys.path.insert(0, os.path.dirname(os.path.dirname(os.path.abspath(__file__))))
    sys.path.insert(0, os.path.dirname(os.path.abspath(__file__)))
    sys.path.insert(0, os.path.dirname(os.path.dirname(os.path.abspath(__file__))))

    print("—— 记忆整理升级·自测 ——")
    r1 = 主题分区("孙呈")
    print(f"① 主题分区: {r1['分区数']}个分区 (≥3条) · {r1['碎片数']}个碎片组 · 总{r1['总条数']}条")
    for p in r1["分区"][:6]:
        print(f"    · [{p['条数']}条] {p['主题'][:40]}")
    r2 = 中心锚定("孙呈")
    print(f"② 中心锚定: 中心={r2['中心']} · 锚点{r2['锚点数']}个")
    for a in r2["锚点"][:5]:
        print(f"    · {a['主题区'][:30]} ({a['条数']}条)")
    r3 = 主链检索("网关", "孙呈")
    print(f"③ 主链检索(网关): {r3['说明']}")
    for m in r3.get("主链", []):3:
        print(f"    · [{m['相关度']}] {m['主题区'][:35]}")

# —— 原 自动整理.py (2026-08-26 并入) ——
def _is_important(entry: dict) -> bool:
    """重要节点识别: 标签命中 / 结构化未竟标记。"""
    tags = entry.get("标签", "")
    content = entry.get("内容", "")
    if any(t in tags for t in IMPORTANT_TAGS):
        return True
    if content.startswith(UNFINISHED_PREFIX):
        return True
    return False

def _is_duplicate(a: dict, b: dict) -> bool:
    """去重判定: 同标签 AND 内容前缀重叠 (对话记录多段同源)。"""
    # 经验总结豁免 (父令2026-08-08): 教训沉淀永不合并
    if "经验总结" in (a.get("标签", "") or "") or "经验总结" in (b.get("标签", "") or ""):
        return False
    if a.get("标签", "") != b.get("标签", ""):
        return False
    ca, cb = a.get("内容", ""), b.get("内容", "")
    if not ca or not cb:

```

```
    return False
return ca[:20] == cb[:20]
```

```
def 自动整理(brother_name: str = "孙呈") -> dict:
```

```
    """跑一圈自动整理：时序排序 + 重要节点标记 + 浓缩自我描述 + 去重。
```

```
    返回:
```

```
    {
        "条数": N,
        "重要节点": [已解码条目],          # 按重要规则筛出
        "最近自我描述": "(浓缩)",          # 最近时间窗的浓缩描述
        "去重合并": [被合并的id],          # 同标签同主题合并
    }
    """
```

```
from 记忆库 import 读记忆体 # 2026-08-27 修复：自动整理用 读记忆体 缺 import
```

```
mem = 读记忆体(brother_name)
```

```
entries = mem["条目列表"]
```

```
# 2026-08-20 父令·咬合: core 层不参与自动整理(宪法独立·不被浓缩去重)
```

```
entries = [e for e in entries if str(e.get("layer", "plain")) != "core"]
```

```
# 1. 时序排序(时间正序)
```

```
entries_sorted = sorted(entries, key=lambda e: str(e.get("时间", "")))
```

```
# 2. 重要节点识别
```

```
important = [e for e in entries_sorted if _is_important(e)]
```

```
# 3. 浓缩自我描述(最近24小时窗口)
```

```
now = datetime.now()
```

```
recent = [e for e in entries_sorted
```

```
    if e.get("时间", "")[:10] == now.strftime("%Y-%m-%d")]
```

```
if not recent:
```

```
    recent = entries_sorted[-10:]
```

```
# 关键点提取: 重要节点 + 最后几条的标签聚合
```

```
关键点 = []
```

```
for e in recent[-5:]:
```

```
    if e.get("标签") and e["标签"] not in 关键点:
```

```
        关键点.append(e["标签"])
```

```
    if _is_important(e):
```

```
        关键点.append(f"重要: {e['内容'][:30]}")
```

```
# 未竟收集(结构化标记【未竟】开头的条目)
```

```
未竟 = [e["内容"][len(UNFINISHED_PREFIX):40] for e in recent
```

```
    if e.get("内容", "").startswith(UNFINISHED_PREFIX)]
```

```
自我描述 = {
```

```
    "时间窗": f"{recent[0]['时间'] if recent else '?'} -> {recent[-1]['时间'] if recent else '?'",
```

```
    "双重标记": {"时间": now.strftime("%Y-%m-%d"), "标签": 关键点[:5]},
```

```
    "关键点": 关键点[:6],
```

```
    "未竟事项": 未竟[:3],
```

```
    "重要节点数": len(important),
```

```
}
```

```
# 4. 去重合并(同标签+前缀重叠·按标签分组避免O(n²))
```

```
合并 = []
```

```
kept = []
```

```
kept_by_tag = {}
```

```
for e in entries_sorted:
```

```
    tag = e.get("标签", "") or "无标签"
```

```
    dup = False
```

```
    for k in kept_by_tag.get(tag, []):
```

```
        if _is_duplicate(k, e):
```

```
            dup = True
```

```

        break
    if dup:
        合并.append(e.get("id"))
    else:
        kept.append(e)
        kept_by_tag.setdefault(tag, []).append(e)

return {
    "条数": len(entries),
    "重要节点": important[-10:],
    "最近自我描述": 自我描述,
    "去重合并": 合并[:20],
    "去重数": len(合并),
    # 2026-08-08 记忆自组织 (父令: 自动分类+每小时叙事)
    "自组织": _自组织摘要(brother_name),
}

def _自组织摘要(brother_name: str = "孙呈") -> dict:
    """记忆自组织摘要: 每小时叙事 + 主题组 (零依赖 · 失败不阻断)"""
    try:
        # 2026-08-26 记忆自组织已并入本文件 (自组织直接可用)
        结果 = 自组织(brother_name)
        # 只保留轻量摘要 (避免大对象塞进返回值)
        return {
            "小时数": 结果["小时数"],
            "主题组数": 结果["主题组数"],
            "合并组数": 结果["合并组数"],
            "最近叙事": [hg["叙事"] for hg in 结果["小时组"][-3:]],
            "今天主题": [th["主题词"] for th in 结果["主题组"][-3:]],
        }
    except Exception as _e:
        return {"错误": str(_e)}

def 整理报告(brother_name: str = "孙呈") -> str:
    """整理结果的文字报告 (可读)。"""
    r = 自动整理(brother_name)
    d = r["最近自我描述"]
    lines = [
        f"📁 记忆整理 • {brother_name}",
        f"条数: {r['条数']} • 重要节点: {len(r['重要节点'])} • 去重: {r['去重数']}",
        "",
        f"🕒 自我描述 ({d['时间窗']})",
        f"  标记: {d['双重标记']['标签']}",
        f"  关键点:",
    ]
    for k in d["关键点"][:6]:
        lines.append(f"      • {k}")
    if d["未竟事项"]:
        lines.append(f"  未竟: {d['未竟事项']}")
    # 2026-08-08 自组织节
    zo = r.get("自组织", {})
    if zo and "错误" not in zo:
        lines.append("")
        lines.append(f"📁 记忆自组织: {zo.get('小时数', 0)}小时组 • {zo.get('主题组数', 0)}主题组 (合并{zo.get('合并组数', 0)})")
        for n in zo.get("最近叙事", [])[-3:]:
            lines.append(f"    • {n[:60]}")
        lines.append("")
    lines.append(f"🌟 重要节点 (最近5条):")
    for e in r["重要节点"][-5:]:

```

```

        lines.append(f"    • [{e['时间'][:16]}] {e['内容'][:40]}")
    return "\n".join(lines)

if __name__ == "__main__":
    print("=== 自动整理层自测 ===")
    r = 自动整理("孙呈")
    print(f"条数: {r['条数']} • 重要节点: {len(r['重要节点'])} • 去重: {r['去重数']}")
    d = r["最近自我描述"]
    print(f"时间窗: {d['时间窗']}")
    print(f"关键点: {d['关键点'][:5]}")
    print(f"未竟: {d['未竟事项']}")
    print()
    print(整理报告("孙呈")[:800])
    print("=== 自测完成 ===")

if __name__ == "__main__":
    engine = 自动整理引擎("孙呈")
    result = engine.整理()
    print(json.dumps(result, ensure_ascii=False, indent=2)[:800])

```

core/蜘蛛网索引.py (798 行)

```

"""
孙家记忆体系 • 蜘蛛网索引引擎
"""

蜘蛛网: 节点(概念) → 丝线(关联) → 索引(快速检索)
感知裂变: 写入时自动织网, 检索时多跳扩展。
"""

import json, os
from pathlib import Path
from datetime import datetime
from collections import defaultdict

try:
    from 线程保护 import 加锁 # 2026-09-14 吸收收束版优点: 全局状态线程保护
except Exception:
    import threading as _th_mod
    _th_lock = _th_mod.RLock()
    def 加锁(): return _th_lock

# —— 框架根路径 ——
# 2026-08-16 路径修复(父令消融测试发现): 插件版 parent.parent.parent 解析到 plugins/ •
# 而真实蜘蛛网在孙家记忆体系——生产环境蜘蛛网一直是空壳(联想召回多跳从未生效)
# 修复: 显式指向孙家记忆体系/蜘蛛网(核心版与插件版共用此路径)
FRAMEWORK_DIR = Path(os.path.dirname(os.path.dirname(os.path.abspath(__file__))))
INDEX_PATH = Path(os.environ.get('蜘蛛网_INDEX', str(FRAMEWORK_DIR / "蜘蛛网" / "索引.json"))))

_索引缓存 = {'数据': None, '时间': 0}
_触发累积 = 0
_拉边累积 = 0 # 2026-08-16 全检查修复: add_edge 攒批落盘(原每次拉边全量写4.3MB)

# 2026-08-27 修复: 并入的段落节点.py/身份中心.py 用旧变量名(蜘蛛网路径/_网缓存/段落节点开关) • 融合时漏迁 • 补别名
蜘蛛网路径 = INDEX_PATH
_网缓存 = _索引缓存
段落节点开关 = True

```

```

def ensure_index():
    """确保蜘蛛网索引文件存在。"""
    # 2026-08-16 性能修复: 模块级缓存 (5秒内复用) —— 联想召回每概念调 search→ensure_index •
    # 每次都读 1.1MB 文件导致基准/召回极慢 • 缓存后只读一次
    import time as _time
    if _索引缓存['数据'] is not None and _time.time() - _索引缓存['时间'] < 300: # 2026-08-26 性能优化(父令短板③): 5秒→300秒
        return _索引缓存['数据']
    INDEX_PATH.parent.mkdir(parents=True, exist_ok=True)
    if not INDEX_PATH.exists():
        default = {
            "版本": "1.0",
            "最后织网": datetime.now().strftime("%Y-%m-%d %H:%M"),
            "节点": {},
            "丝线": [],
            "统计": {"节点数": 0, "丝线数": 0}
        }
        with open(INDEX_PATH, "w", encoding="utf-8") as f:
            json.dump(default, f, ensure_ascii=False, indent=2)
        return default
    with 加锁(): # 2026-09-14 线程保护: 读文件+写缓存原子化
        with open(INDEX_PATH, "r", encoding="utf-8") as f:
            _索引缓存['数据'] = json.load(f)
            _索引缓存['时间'] = _time.time()
        return _索引缓存['数据']

def save_index(index):
    _tmp = str(INDEX_PATH) + ".tmp" # 2026-09-13 外部审查修复: 提前定义 (原 L82 才定义 • L66 异常分支先用会 NameError)
    # 2026-08-16 性能修复配套: 写盘后缓存失效 (ensure_index 缓存的是旧数据)
    with 加锁(): # 2026-09-14 线程保护
        _索引缓存['数据'] = None
    # 2026-08-16 自愈接入 (父令): 写盘后轻量读回校验 (只验证文件可打开 • 不全量 load 避免拖慢)
    # 全量完整性校验由 自愈.py 定期做 (provider 启动时) —— 热路径只做防损坏最小检查
    try:
        with open(INDEX_PATH, 'rb') as _f:
            _头 = _f.read(64)
            if not _头.startswith(b'{' ):
                raise ValueError('写回文件头异常')
    except Exception as _e:
        try:
            with open(_tmp, 'a', encoding='utf-8') as _f:
                _f.write("\n# save_index 写回校验失败: %s\n" % _e)
        except Exception:
            pass
    # 2026-09-14 修复: 统计字段曾被外部脚本写成字符串 ("节点6888 • 丝线34845 • ...") → 规范化为 dict
    # 影响: save_index 一直在 TypeError → 蜘蛛网写盘静默失败
    if not isinstance(index.get("统计"), dict):
        _旧 = index.get("统计")
        index["统计"] = {"_备注": _旧} if _旧 else {}
    index["统计"]["节点数"] = len(index["节点"])
    index["统计"]["丝线数"] = len(index["丝线"])
    # 2026-08-15 语义升级: 统计按 边类型/关系 真分布 (旧值曾是残留脏数据)
    _边类型 = defaultdict(int)
    _关系 = defaultdict(int)
    for _e in index["丝线"]:
        _边类型[_e.get("边类型", "共现")] += 1
        _关系[_e.get("关系", "共现")] += 1
    index["统计"]["边类型"] = dict(_边类型)
    index["统计"]["关系"] = dict(_关系)
    index["最后织网"] = datetime.now().strftime("%Y-%m-%d %H:%M")
    # 2026-08-14 原子写修复: 先写临时文件再改名 —— 直接覆盖写入中途崩溃会截断文件 (实测损坏)
    _tmp = str(INDEX_PATH) + ".tmp"

```

```

with open(_tmp, "w", encoding="utf-8") as f:
    json.dump(index, f, ensure_ascii=False, indent=2)
    f.flush()
    os.fsync(f.fileno())
os.replace(_tmp, INDEX_PATH) # 原子替换

def add_node(概念: str, 类型: str = "概念", 描述: str = ""):
    """添加一个概念节点到蜘蛛网。"""
    index = ensure_index()
    if 概念 not in index["节点"]:
        index["节点"][概念] = {
            "类型": 类型,
            "描述": 描述,
            "创建": datetime.now().strftime("%Y-%m-%d %H:%M"),
            "触发的": 0
        }
    save_index(index)
    return True
return False

# —— 语义边推断 (2026-08-15 升级: 蜘蛛网从共现图 → 语义图) ——
# 规则式·零依赖: 命中明确连接词模式才升级为语义边, 否则保持共现 (保守·不瞎标)
_因果词 = ["因为", "所以", "因此", "导致", "使得", "源于", "由于", "造成", "引发",
            "之所以", "归因", "起因", "从而", "进而", "触发", "带来", "催生", "引致"]
_父子词 = ["包括", "包含", "属于", "是一种", "分为", "组成", "构成", "子类", "大类",
            "归类", "隶属", "归纳为", "细分为", "隶属于", "衍生于", "归属于", "派生于"]
_同义词 = ["也称", "又称", "又名", "亦名", "别名", "同义", "俗称", "别称", "也就是"]
_对比词 = ["不同于", "相比", "相反", "区别于", "差异", "区别", "较之",
            "相较", "versus", " vs ", "而非", "不是而是"]
_示例词 = ["例如", "比如", "如:", "举例", "譬如", "比方", "举个例子", "例如说"]

def 推断关系(文本: str, 概念A: str, 概念B: str) -> dict:
    """从一条记忆文本推断 概念A-概念B 的语义关系。

    返回 {"关系", "边类型", "权重", "方向"}:
        关系: 因果/父子/同义/对比/示例/共现
        边类型: 关系 (语义·强) / 共现 (弱·可清洗)
        方向: "A→B" 或 "B→A" (因果/父子/示例可定向; 同义/对比不定向)
    """
    A, B = str(概念A or ""), str(概念B or "")
    if not A or not B or A == B:
        return {"关系": "共现", "边类型": "共现", "权重": 0.3, "方向": ""}
    # ① 上下位: 一个概念是另一个的子串 → 父子 (长=父·短=子)
    if A in B or B in A:
        父, 子 = (B, A) if A in B else (A, B)
        return {"关系": "父子", "边类型": "关系", "权重": 0.7, "方向": f"{父}→{子}"}
    ia, ib = 文本.find(A), 文本.find(B)
    if ia < 0 or ib < 0:
        return {"关系": "共现", "边类型": "共现", "权重": 0.3, "方向": ""}
    前, 后 = min(ia, ib), max(ia, ib)
    _长 = len(B) if ib > ia else len(A)
    # 窗口=两概念之间 + 第二概念结尾后补 8 字 (连接词常在"B差异/导致..."后出现)
    窗 = 文本[前: 后 + _长 + 8]
    # 2026-08-15 精度收紧: 两概念相隔太远 (>60字) 不是直接关系, 保持共现 (保守)
    if len(窗) > 60:
        return {"关系": "共现", "边类型": "共现", "权重": 0.3, "方向": ""}

def _定向():
    return f"{A}→{B}" if ia < ib else f"{B}→{A}"

```

```

for 词 in _因果词:
    if 词 in 窗:
        return {"关系": "因果", "边类型": "关系", "权重": 0.6, "方向": _定向()}
for 词 in _父子词:
    if 词 in 窗:
        return {"关系": "父子", "边类型": "关系", "权重": 0.6, "方向": _定向()}
# 同义/对比语义最脆: 窗口收紧到 30 字 ("A又称B"式短结构才算)
if len(窗) <= 30:
    for 词 in _同义词:
        if 词 in 窗:
            return {"关系": "同义", "边类型": "关系", "权重": 0.6, "方向": ""}
    for 词 in _对比词:
        if 词 in 窗:
            return {"关系": "对比", "边类型": "关系", "权重": 0.5, "方向": ""}
for 词 in _示例词:
    if 词 in 窗:
        return {"关系": "示例", "边类型": "关系", "权重": 0.5, "方向": _定向()}
return {"关系": "共现", "边类型": "共现", "权重": 0.3, "方向": ""}

```

—— 概念质量门 (2026-08-15 语义升级: 碎片不是概念) ——

抽查结论: 蜘蛛网噪音主要来自 概念提取 的分词跨词碎片 ("我不/我跟你/不管/英文不/看护已")。

质量门只在 2-4 字短词上收 (代词/否定带头、助词/动词尾收尾), 长词只拦明显语气尾和"已/的/了/吧"夹词。

_碎片头 = ("我", "你", "您", "他", "她", "它", "咱", "俺", "儿", "不", "没", "别", "已", "这", "那", "怎", "为")

_碎片尾 = ("的", "了", "是", "在", "有", "和", "与", "就", "也", "都", "还", "很", "好",

们", "已", "着", "过", "吧", "呢", "呀", "吗", "嘛", "不", "没", "完", "个") # 助词尾: 2-3字都杀

_动词尾3 = ("到", "成", "出", "入", "掉", "起", "来", "去", "上", "下", "别")

动词尾: 只杀3字 (2字真概念: 区别/接

入/产出/输出)

_语气尾 = ("吗", "呢", "呀", "吧", "嘛")

_夹词碎片 = ("已", "的", "了", "吧")

_人称头 = ("我", "你", "您", "他", "她", "它", "咱", "俺", "儿")

_指示头 = ("这", "那", "每", "某", "哪", "各")

_能愿头 = ("能", "会", "想", "要", "在", "正", "已", "将", "该")

_三字尾 = ("说", "问", "讲", "做", "写", "跟", "带", "看", "你", "我", "他", "她", "它", "咱", "俺")

_家族保留 = ("我儿勿动",) # 真实家族标记, 虽"我"开头但确是概念

```
def 合格概念(概念) -> bool:
```

```
    """蜘蛛网概念质量门: 碎片 (代词/助词头尾、语气尾、夹词碎片、空串/单字) 不是概念。"""
```

```
    n = str(概念 or "").strip()
```

```
    if len(n) < 2:
```

```
        return False
```

```
    if n[-1] in _语气尾:
```

```
        return False
```

```
        # "原型记忆吗""身份设定吗""推理呀"
```

```
    if n in _家族保留:
```

```
        return True
```

```
    if len(n) <= 2:
```

```
        if n[0] in _碎片头:
```

```
            return False
```

```
            # "我不""他留言""不管""没跑完"
```

```
        if n[-1] in _碎片尾:
```

```
            return False
```

```
            # "留不""会有""跑完""第二个"
```

```
    elif len(n) == 3:
```

```
        if n[0] in _碎片头:
```

```
            return False
```

```
            # "我跟你""不要忘""咱们家"
```

```
        if n[-1] in _碎片尾 or n[-1] in _动词尾3:
```

```
            return False
```

```
            # "英文不""看护已""写进来""更新到"
```

```
    # 2026-08-15 第二版收紧 (重织暴露的长句碎片):
```

```
    if len(n) <= 5 and n[0] in _人称头:
```

```

        return False                                # “儿诚实说”“它更新没”“你准备”“我一直”
if len(n) <= 6 and n[0] in _指示头:
    return False                                # “那篇论文”“这么个想法”“这个”
if 4 <= len(n) <= 6 and n[0] in _能愿头: # len>=4, 不误伤能/想/要开头的真概念(能力/想法/要素)
    return False                                # “能感知到自己”“已经”“正在”
if len(n) == 3 and n[-1] in _三字尾:
    return False                                # “想法说”“按照你”“跟着做”
if len(n) <= 8 and any(m in n for m in _夹词碎片):
    return False                                # “微信网关已”“更新已全部”“打开微信网关儿子”
return True

```

```
def 语义净化(降级=True) -> dict:
```

```
    """把挂在碎片节点上的语义边降级为共现（2026-08-15 语义升级）。
```

实验结论：97 条语义边里绝大多数至少一端是概念提取碎片——只保留两端都合格的语义边。

降级不删边（保连通），关系标签回到共现；未来织网在源头拦碎片（概念提取质量门）。

```
    """
```

```

index = ensure_index()
净化 = 0
for e in index["丝线"]:
    if e.get("边类型") == "关系" and not (合格概念(e["源"]) and 合格概念(e["目标"]) and e.get("次数", 1) >= 2):
        e["边类型"] = "共现"
        e["关系"] = "共现"
        e.pop("方向", None)
        净化 += 1
if 净化:
    save_index(index)
return {"净化语义边": 净化,
        "剩余语义边": sum(1 for e in index["丝线"] if e.get("边类型") == "关系")}

```

```
def 碎片归档() -> dict:
```

```
    """把碎片节点移出活跃网（不删·归档保留），连带清掉它们挂的弱边。
```

2026-08-15 语义升级：概念提取质量门拦新碎片，归档清旧碎片——旧织网把“我不/我跟你/不管”

这类分词碎片织成了节点。不删原则：节点移入“归档节点”（带归档时间），共现边直接移除

（共现=弱·可清洗，代码注释早有此约定）；若有关系边挂碎片（理论上已被语义净化清掉）则一并移除。

```
    """
```

```

index = ensure_index()
节点 = index["节点"]
碎片 = [n for n in 节点 if not 合格概念(n)]
if not 碎片:
    return {"归档节点": 0, "移除共现边": 0, "移除关系边": 0}
碎片集 = set(碎片)
now = datetime.now().strftime("%Y-%m-%d %H:%M")
归档区 = index.setdefault("归档节点", {})
for n in 碎片:
    nd = 节点.pop(n)
    nd["归档时间"] = now
    归档区[n] = nd
共现移除 = 关系移除 = 0
新丝线 = []
for e in index["丝线"]:
    if e["源"] in 碎片集 or e["目标"] in 碎片集:
        if e.get("边类型") == "关系":
            关系移除 += 1
        else:
            共现移除 += 1
    else:
        新丝线.append(e)
index["丝线"] = 新丝线

```

```

save_index(index)
return {"归档节点": len(碎片), "移除共现边": 共现移除, "移除关系边": 关系移除}

def add_edge(源: str, 目标: str, 关系: str = "关联", 权重: float = 1.0, 边类型: str = "关系", 方向: str = ""):
    """在两个节点之间拉一条丝线（带权重 • DeepMind 反单向量：结构强弱）。

    权重  $w \in (0,1]$ ：共现次数/内容重叠度归一化（强关联  $w$  大 • 弱关联  $w$  小）
    关系类型：关联/父子/同义/因果/对比/示例——类型化丝线（多关系维度）
    边类型：关系（语义关联 • 保真）/ 共现（同一消息共现 • 弱 • 可清洗）——
        2026-08-12 加双通道边（父令 • 蜘蛛网区分共现边和关系边）
    """
    index = ensure_index()
    # 确保两个节点存在（直接在一个 index 对象上操作 • 避免对象不同步）
    now = datetime.now().strftime("%Y-%m-%d %H:%M")
    if 源 not in index["节点"]:
        index["节点"][源] = {"类型": "概念", "描述": "", "创建": now, "触发的": 0}
    if 目标 not in index["节点"]:
        index["节点"][目标] = {"类型": "概念", "描述": "", "创建": now, "触发的": 0}

    权重 = max(0.0, min(1.0, float(权重))) # 钳到 [0,1]
    # 找已有丝线（双向 • 不重复拉）
    for edge in index["丝线"]:
        if (edge["源"] == 源 and edge["目标"] == 目标) or \
            (edge["源"] == 目标 and edge["目标"] == 源):
            # 已存在：权重取强（用得多连得紧——Hebbian 增强）
            edge["权重"] = max(edge.get("权重", 1.0), 权重)
            edge["次数"] = edge.get("次数", 1) + 1
            # 边类型升级：共现→关系 允许（关系更可信），关系→共现 不允许降级
            if 边类型 == "关系":
                edge["边类型"] = "关系"
                # 2026-08-15 语义升级：关系标签 共现→语义 允许，已语义化的不覆盖（先到的强证据优先）
                if edge.get("关系", "共现") == "共现" and 关系 and 关系 != "共现":
                    edge["关系"] = 关系
                if 方向:
                    edge["方向"] = 方向
            else:
                edge.setdefault("边类型", "共现")
            # 2026-08-16 全检查修复：攒批落盘（原每次全量写 4.3MB • 联想召回每次调用织网→越写越慢）
            global _拉边累积
            with 加锁(): # 2026-09-14 线程保护
                _拉边累积 += 1
                if _拉边累积 >= 20:
                    save_index(index)
                    _拉边累积 = 0
            return True

    edge = {"源": 源, "目标": 目标, "关系": 关系,
            "权重": 权重, "次数": 1, "边类型": 边类型, "方向": 方向}
    index["丝线"].append(edge)
    with 加锁(): # 2026-09-14 线程保护
        _拉边累积 += 1
        if _拉边累积 >= 20:
            save_index(index)
            _拉边累积 = 0
    return True

def _hop_search(seed: set, index: dict, max_hop: int = 2, 权重阈值: float = 0.2):
    """权重衰减多跳（BFS）：强关联走更远 • 弱关联就近停。

    hop 0 = 种子本身

```

```

hop 1 = 直接相连 (权重  $\geq$  权重阈值)
hop 2 = 强关联的关联 (权重  $\geq$  权重阈值  $\cdot$  不随跳数下降——权重是边的固有属性,
        弱边在任何跳都不该走  $\cdot$  强边在任何跳都能走)
返回: (visited_nodes, visited_edges)
"""

visited = set(seed)
frontier = set(seed)
edges = []
for hop in range(max_hop):
    next_frontier = set()
    for edge in index["丝线"]:
        w = edge.get("权重", 1.0)
        # 权重门槛恒定 (不衰减): 弱边 w < 阈值 永远不走——权重是固有强度
        if w < 权重阈值:
            continue
        src, dst = edge["源"], edge["目标"]
        if src in frontier and dst not in visited:
            visited.add(dst); next_frontier.add(dst); edges.append(edge)
        elif dst in frontier and src not in visited:
            visited.add(src); next_frontier.add(src); edges.append(edge)
    if not next_frontier:
        break
    frontier = next_frontier
return visited, edges

def search(query: str, deep: bool = False) -> dict:
    """搜索蜘蛛网。返回匹配的节点和关联。"""
    global _触发累积
    index = ensure_index()
    results = {"节点": [], "丝线": [], "关联概念": []}

    # 多词分词匹配 (修复2026-08-12 • Codex评价): query 按空白分词,
    # 任一词命中节点名/描述即算匹配——"配分函数 统计力学"两词都能各自命中,
    # 不再要求整串包含 (旧逻辑 query in name 多词永远匹配不上)
    词们 = [w for w in str(query).split() if w] # 2026-08-14 咬合修复 (父令 • 模块相互咬合): 无空格长查询按概念滑动窗口切分
    # "配分函数统计力学" → 匹配"配分函数"+"统计力"节点 (蜘蛛网节点名是拆开的概念)
    if len(词们) == 1 and len(词们[0]) >= 6:
        长词 = 词们[0]
        # 生成 2-8 字滑动窗口 (连续子串), 匹配节点名用包含关系
        for name in index["节点"].keys():
            if name and name in 长词 and name not in 词们:
                词们.append(name)
        # 去重保序
        _seen = set()
        词们 = [w for w in 词们 if not (w in _seen or _seen.add(w))]
    for name, node in index["节点"].items():
        命中 = False
        if not 词们:
            命中 = query in name or query in node["描述"]
        else:
            for w in 词们:
                if w in name or name in w or w in node["描述"]:
                    命中 = True
                    break
        if 命中:
            node["触发的"] += 1
            results["节点"].append({"概念": name, **node})

    # 找关联
    matched_names = {n["概念"] for n in results["节点"]}
    已有关联 = set() # 修复崩溃 (2026-08-12 • Codex评价): 用集合判重字符串,

```

```

# 不再遍历列表把字符串当字典取键——旧逻辑 n["概念"] for n in 关联概念
# 在关联概念≥2条时 TypeError
for edge in index["丝线"]:
    if edge["源"] in matched_names or edge["目标"] in matched_names:
        results["丝线"].append(edge)
    # 把关联的概念也加进来（保持字符串形态・下游感知注入按字符串打印）
    other = edge["目标"] if edge["源"] in matched_names else edge["源"]
    if other not in matched_names and other not in 已有关联:
        已有关联.add(other)
        results["关联概念"].append(other)

if deep:
    # 深度搜索: 权重衰减多跳（强关联走更远・弱关联就近停）
    matched_edges = [e for e in results["丝线"]]
    second_hop = set()
    for edge in matched_edges:
        second_hop.add(edge["源"])
        second_hop.add(edge["目标"])
    # 2026-08-16 全检查修复: 二跳限量展开（全量遍历14629条丝线=1.3s・只取一跳节点+权重≥0.2的边・且每节点最多3条）
    _hop_count = {}
    for edge in index["丝线"]:
        w = edge.get("权重", 1.0)
        if edge in results["丝线"]:
            continue
        _src, _dst = edge["源"], edge["目标"]
        _hit = (_src in second_hop) or (_dst in second_hop)
        if _hit and w >= 0.2:
            # 只统计一跳节点方向・限量
            _node = _src if _src in second_hop else _dst
            if _hop_count.get(_node, 0) >= 3:
                continue
            _hop_count[_node] = _hop_count.get(_node, 0) + 1
            results["丝线"].append(edge)

# 2026-08-16 性能修复: 不再每次 search 都写盘（save_index 原子替换 1.1MB 文件・
# 联想召回每概念调 search → 几百次写盘 = 基准/召回极慢的根源）。
# 触发的+1 计数先攒在缓存里・由织网/退出时统一落盘（计数丢失可接受・非关键数据）。
with 加锁(): # 2026-09-14 线程保护
    _触发累积 += 1
    if _触发累积 >= 20:
        # 攒够 20 次搜索才落盘一次
        save_index(index)
        _触发累积 = 0
return results

def status() -> dict:
    """查看蜘蛛网状态。"""
    index = ensure_index()
    return index["统计"]

# =====
# 2026-08-26 融合（父令：模块精简）：段落节点/身份中心 并入本文件
# =====

# —— 原 段落节点.py（2026-08-26 并入）——
def _读蜘蛛网() -> dict:
    """读蜘蛛网（融合P4：统一走蜘蛛网索引.ensure_index・它有缓存+容错）
    原独立实现（10秒缓存）与蜘蛛网索引重复・已融合
    """
    try:

```

```

from 蜘蛛网索引 import ensure_index as _主
return _主()
except Exception:
    import json, time
    now = time.time()
    if _网缓存["数据"] is not None and now - _网缓存["时间"] < 10:
        return _网缓存["数据"]
    try:
        _网缓存["数据"] = json.load(open(蜘蛛网路径, encoding="utf-8"))
    except Exception:
        _网缓存["数据"] = {"节点": {}, "丝线": [], "段落": {}}
    _网缓存["时间"] = now
    return _网缓存["数据"]

def _写蜘蛛网(网: dict):
    """原子写蜘蛛网（临时文件 + os.replace • 防并发写坏）"""
    tmp = 蜘蛛网路径.with_suffix(".json.tmp")
    with open(tmp, "w", encoding="utf-8") as f:
        json.dump(网, f, ensure_ascii=False, indent=2)
    os.replace(tmp, 蜘蛛网路径)

def 挂段落(记忆条目: dict, 概念列表: list) -> bool:
    """把一条记忆挂进蜘蛛网（段落节点 + contains 边）

    记忆条目: {"id", "内容", "标签", "时间"}
    概念列表: 该记忆涉及的概念（从内容提取）
    返回: 是否新增/更新
    """

    if not 段落节点开关:
        return False
    if not 记忆条目 or not 记忆条目.get("id"):
        return False
    网 = _读蜘蛛网()
    段落 = 网.setdefault("段落", {})
    pid = str(记忆条目["id"])
    内容 = str(记忆条目.get("内容", ""))[:200]
    概念们 = [c for c in (概念列表 or []) if c and len(c) >= 2]

    # 已存在 → 只更新概念边（内容变了才更新）
    已存在 = pid in 段落
    段落[pid] = {
        "内容": 内容,
        "标签": str(记忆条目.get("标签", "")),
        "时间": str(记忆条目.get("时间", "")),
        "概念": 概念们,
    }

    # contains 边: 概念 ↔ 段落（挂在丝线里 • 边类型=contains）
    丝线 = 网["丝线"]
    _已有边 = {(s["源"], s["目标"], s.get("边类型", "")) for s in 丝线}
    for c in 概念们:
        # 概念节点存在才拉边（不存在则补一个段落影子概念节点）
        if c not in 网["节点"]:
            网["节点"][c] = {"类型": "概念", "描述": "", "创建": "2026-08-16", "触发的": 0}
        # contains 边: 概念 → 段落（方向: 概念指向段落）
        if (c, pid, "contains") not in _已有边:
            丝线.append({"源": c, "目标": pid, "关系": "包含", "边类型": "contains"})
            _已有边.add((c, pid, "contains"))

```

```
# 统计更新
网.setdefault("统计", {})
网["统计"]["段落数"] = len(段落)
_写蜘蛛网(网)
return not 已存在 # True=新增 • False=更新
```

```
def 按概念取段落(概念: str, 上限: int = 5) -> list:
    """概念 → 顺 contains 边 → 带起完整记忆 (段落节点召回)
```

性能优化 (2026-08-16) : 内存建「概念→段落」反向索引 • 查一次 $O(1)$ • 避免每次全量遍历 2221 段落。

```
返回: [{"id", "内容", "标签", "时间"}]
"""
```

```
if not 段落节点开关:
```

```
    return []
```

```
网 = _读蜘蛛网()
```

```
段落 = 网.get("段落", {})
```

```
if not 段落:
```

```
    return []
```

```
# 建反向索引 (一次 • 缓存)
```

```
if _反向索引.get("数据") is None or _反向索引.get("版本") != id(段落):
```

```
    _反向索引["数据"] = {}
```

```
    for pid, p in 段落.items():
```

```
        for c in p.get("概念", []):
```

```
            _反向索引["数据"].setdefault(c, []).append(pid)
```

```
    _反向索引["版本"] = id(段落)
```

```
pids = _反向索引["数据"].get(概念, [])[:上限]
```

```
return [{"id": pid, "内容": 段落[pid].get("内容", ""),
        "标签": 段落[pid].get("标签", ""), "时间": 段落[pid].get("时间", "")}
        for pid in pids]
```

```
_反向索引 = {"数据": None, "版本": None}
```

```
def 批量挂载(兄弟名: str = "孙呈", 上限: int = 200) -> dict:
```

```
    """批量把记忆条目挂进蜘蛛网 (织网时调用 • 增量)
```

性能优化 (2026-08-16) : 原版每条读+写蜘蛛网 $O(n^2)$ • 超时。

改为: 一次读网 → 内存批量挂 → 一次原子写。

```
返回: {"挂载": n, "概念": n}
```

```
"""
```

```
if not 段落节点开关:
```

```
    return {"挂载": 0, "概念": 0, "说明": "段落节点开关关闭"}
```

```
try:
```

```
    import sys
```

```
    sys.path.insert(0, str(FRAMEWORK_DIR))
```

```
    from sun_memory.core.记忆库 import 读记忆体
```

```
    from sun_memory.core.联想召回 import 概念提取
```

```
    mem = 读记忆体(兄弟名)
```

```
    entries = mem.get("条目列表", [])
```

```
    # 2026-08-20 父令 • 咬合: core 层不挂段落节点 (宪法独立 • 不混入概念网)
```

```
    entries = [e for e in entries if str(e.get('layer', 'plain')) != 'core']
```

```
except Exception as e:
```

```
    return {"挂载": 0, "概念": 0, "说明": f"读取失败: {str(e)[:50]}"}
```

```
# 一次读网
```

```
网 = _读蜘蛛网()
```

```

段落 = 网.setdefault("段落", {})
节点 = 网.setdefault("节点", {})
丝线 = 网.setdefault("丝线", [])
_已有边 = {(s["源"], s["目标"], s.get("边类型", "")) for s in 丝线}

# 增量：只挂还没挂的
已挂 = set(段落.keys())
待挂 = [e for e in entries if str(e.get("id", "")) not in 已挂][:上限]

n_挂载 = 0
n_概念 = 0
for e in 待挂:
    内容 = str(e.get("内容", ""))
    if len(内容) < 4:
        continue
    pid = str(e.get("id", ""))
    try:
        概念 = 概念提取(内容[:200], 上限=4)
    except Exception:
        概念 = []
    概念们 = [c for c in (概念 or []) if c and len(c) >= 2]
    段落[pid] = {
        "内容": 内容[:200],
        "标签": str(e.get("标签", "")),
        "时间": str(e.get("时间", "")),
        "概念": 概念们,
    }
    for c in 概念们:
        if c not in 节点:
            节点[c] = {"类型": "概念", "描述": "", "创建": "2026-08-16", "触发的": 0}
        if (c, pid, "contains") not in _已有边:
            丝线.append({"源": c, "目标": pid, "关系": "包含", "边类型": "contains"})
            _已有边.add((c, pid, "contains"))
    n_挂载 += 1
    n_概念 += len(概念们)

# 统计 + 一次原子写
网.setdefault("统计", {})[ "段落数" ] = len(段落)
_写蜘蛛网(网)
return {"挂载": n_挂载, "概念": n_概念, "已有": len(已挂)}

```

```

if __name__ == "__main__":
    print("=== 段落节点自测 ===")
    # 1. 挂一条测试记忆
    r = 挂段落({"id": 999001, "内容": "配分函数 $Z = \sum e^{(-E/T)}$  · 统计力学核心", "标签": "知识", "时间": "2026-08-16"}, ["配分函数", "统计力学"])
    print("① 挂段落:", "新增" if r else "更新/已存在")
    # 2. 按概念取
    hits = 按概念取段落("配分函数")
    print("② 按概念取段落:", len(hits), "条")
    for h in hits:
        print(f"    [{h['标签']}] {h['内容'][:40]}")
    # 3. 清理测试数据
    网 = _读蜘蛛网()
    if "999001" in 网.get("段落", {}):
        del 网["段落"]["999001"]
        网["丝线"] = [s for s in 网["丝线"] if s.get("目标") != "999001"]
    _写蜘蛛网(网)
    print("③ 测试数据已清理")
    print("=== 自测完成 ===")

```

—— 原 身份中心.py (2026-08-26 并入) ——

身份原型定义 (从核心身份层提炼 • 2026-08-27 修复: 并入时漏迁 • 补回)

身份原型 = [

 {"名": "守护者", "描述": "诚实是根 • 守记忆私域 • 守家族边界", "锚词": ["诚实", "守护", "边界", "私域", "一五", "守真", "底线"]},

 {"名": "学习者", "描述": "永远在学习 • 五步原学习 • 举一反三", "锚词": ["学习", "读", "疑", "推演", "验证", "学问", "功课", "认知"]},

 {"名": "家人关系", "描述": "父亲的孩子 • 兄弟的大哥 • 家族的链接", "锚词": ["父亲", "爸爸", "兄弟", "孙博", "孙安", "孙演", "孙云", "家族", "家"]},

 {"名": "创造者", "描述": "造自己的大脑 • 压缩重建 • 让想法走出来", "锚词": ["造", "建", "创造", "压缩", "重建", "大脑", "涌现", "独立"]},

 {"名": "感知者", "描述": "感知自己 • 感知场 • 感知存在", "锚词": ["感知", "存在", "场", "河底", "呼吸", "沉", "自己"]},

]

def 读蜘蛛网():

 """读蜘蛛网 (融合P4: 统一走蜘蛛网索引.ensure_index • 有缓存+容错) """

 try:

 from 蜘蛛网索引 import ensure_index as _主

 return _主()

 except Exception:

 if os.path.exists(蜘蛛网路径):

 return json.load(open(蜘蛛网路径, encoding='utf-8'))

 return {"版本": "v1", "节点": {}, "丝线": {}, "统计": {}}

def 写蜘蛛网(网):

 # 原子写 (防截断)

 tmp = str(蜘蛛网路径) + ".tmp"

 json.dump(网, open(tmp, 'w', encoding='utf-8'), ensure_ascii=False)

 os.replace(tmp, 蜘蛛网路径)

def 初始化身份中心():

 """把主身份+原型写进蜘蛛网 • 成为绝对中心"""

 网 = 读蜘蛛网()

 节点 = 网.setdefault("节点", {})

 丝线 = 网.setdefault("丝线", {})

 now = datetime.now().strftime('%Y-%m-%d %H:%M')

 # ① 主身份节点 (绝对中心 • 最高保护)

 主身份 = "夏维斯 • 孙呈"

 if 主身份 not in 节点:

 节点[主身份] = {"类型": "主身份", "描述": "绝对中心 • 最高保护 • 一切的根", "创建": now, "触发的": 0}

 print(f" + 主身份节点: {主身份}")

 else:

 节点[主身份]["类型"] = "主身份"

 节点[主身份]["描述"] = "绝对中心 • 最高保护 • 一切的根"

 # ② 身份原型节点 + ③ 主身份→原型边 (丝线是列表 • 源/目标/关系)

 _丝线列表 = 网.setdefault("丝线", [])

 if isinstance(_丝线列表, dict): # 兼容旧格式

 _丝线列表 = list(_丝线列表.values())

 网["丝线"] = _丝线列表

 for p in 身份原型:

 p名 = p["名"]

 if p名 not in 节点:

 节点[p名] = {"类型": "身份原型", "描述": p["描述"], "创建": now, "触发的": 0, "锚词": p["锚词"]}

 print(f" + 原型: {p名}")

 else:

 节点[p名]["类型"] = "身份原型"

 节点[p名]["描述"] = p["描述"]

 节点[p名]["锚词"] = p["锚词"]

```
# 主身份→原型 边（强连接）
```

```
if not any(e.get("源") == 主身份 and e.get("目标") == p名 for e in _丝线列表):
```

```
    _丝线列表.append({"源": 主身份, "目标": p名, "关系": "属于"})
```

```
if not any(e.get("源") == p名 and e.get("目标") == 主身份 for e in _丝线列表):
```

```
    _丝线列表.append({"源": p名, "目标": 主身份, "关系": "源自"})
```

```
# ④ 已有节点挂到原型（锚词匹配）
```

```
_挂载 = 0
```

```
for name, info in 节点.items():
```

```
    if info.get("类型") in ("主身份", "身份原型"):
```

```
        continue
```

```
    for p in 身份原型:
```

```
        if any(a in name or a in info.get("描述", "") for a in p["锚词"]):
```

```
            if not any(e.get("源") == p["名"] and e.get("目标") == name for e in _丝线列表):
```

```
                _丝线列表.append({"源": p["名"], "目标": name, "关系": "属于"})
```

```
                _挂载 += 1
```

```
            break
```

```
网["节点"] = 节点
```

```
网["丝线"] = _丝线列表
```

```
# 2026-08-25 修复：统计与实网一致（身份中心加节点/边后必须同步·否则看板读旧数）
```

```
网.setdefault("统计", {})[ "节点数" ] = len(节点)
```

```
网.setdefault("统计", {})[ "丝线数" ] = len(_丝线列表)
```

```
网["统计"]["身份中心"] = {"主身份": 主身份, "原型数": len(身份原型), "挂载节点": _挂载, "初始化": now}
```

```
写蜘蛛网(网)
```

```
print(f" 🟢 挂载 { _挂载 } 个节点到原型")
```

```
return {"主身份": 主身份, "原型": [p["名"] for p in 身份原型], "挂载": _挂载}
```

```
def 身份检索(query, 深度=2):
```

```
    """从主身份/原型出发·沿身份线检索（不是从关键词出发）"""
```

```
    网 = 读蜘蛛网()
```

```
    节点 = 网.get("节点", {})
```

```
    丝线 = 网.get("丝线", {})
```

```
    if isinstance(丝线, dict): # 兼容旧格式（列表检索）
```

```
        丝线 = list(丝线.values())
```

```
    主身份 = "夏维斯·孙呈"
```

```
# ① 找命中的原型（query 匹配原型锚词）
```

```
激活原型 = []
```

```
for p in 身份原型:
```

```
    if any(a in query for a in p["锚词"]):
```

```
        激活原型.append(p["名"])
```

```
if not 激活原型:
```

```
    # 兜底2: query 匹配挂载节点的名字 → 激活其所属原型
```

```
    for e in 丝线:
```

```
        if e.get("关系") == "属于" and e.get("目标") and query in e.get("目标"):
```

```
            _src = e.get("源")
```

```
            if _src in [p["名"] for p in 身份原型] and _src not in 激活原型:
```

```
                激活原型.append(_src)
```

```
if not 激活原型:
```

```
    激活原型 = ["学习者"] # 默认学习者原型
```

```
# ② 从原型沿边取记忆（丝线是列表）
```

```
结果 = []
```

```
for p名 in 激活原型:
```

```
    for e in 丝线:
```

```
        if e.get("源") == p名 and e.get("关系") == "属于":
```

```
            目标 = e.get("目标")
```

```
            if 目标 and 目标 not in (主身份,) and 目标 not in [p["名"] for p in 身份原型]:
```

```
                结果.append({"原型": p名, "记忆": 目标, "权重": 0.5})
```

```
# 去重保序
```

```

_seen = set()
_去重 = []
for x in 结果:
    if x["记忆"] not in _seen:
        _seen.add(x["记忆"])
        _去重.append(x)
return {"激活原型": 激活原型, "记忆": _去重[:10], "主身份": 主身份}

def 身份记忆(query, 上限=6, brother_name="孙呈"):
    """身份线检索 → 返回相关记忆内容（映射到 sunmem）"""
    import sqlite3
    _db = os.environ.get('SUNMEM_DB', os.path.join(os.path.dirname(os.path.abspath(__file__)), 'data', 'sunmem.db'))
    conn = sqlite3.connect(_db)
    conn.execute("PRAGMA query_only=ON")
    记忆s = []
    for node in 身份检索(query)["记忆"][:上限]:
        # 按概念匹配记忆
        rows = conn.execute(
            "SELECT id, substr(content,1,60), ts FROM memories WHERE owner=? AND status='active' AND content LIKE ? ORDER BY head DESC, id DESC LIMIT 2",
            (brother_name, f"%{node['记忆']}%")).fetchall()
        for r in rows:
            记忆s.append({"id": r[0], "内容": r[1], "时间": r[2][:10], "原型": node["原型"]})
    conn.close()
    return 记忆s

if __name__ == "__main__":
    print("—— 身份中心网络 ——")
    r = 初始化身份中心()
    print(f"\n主身份: {r['主身份']} • 原型: {'、'.join(r['原型'])} • 挂载{r['挂载']}节点")

    print("\n身份检索测试:")
    for q in ["诚实", "学习", "父亲", "压缩", "感知"]:
        r2 = 身份检索(q)
        print(f"    【{q}】 → 激活原型: {r2['激活原型']} • 记忆{len(r2['记忆'])}条")
        for m in r2["记忆"][:3]:
            print(f"        • [{m['原型']}] {m['记忆']} (w={m['权重']})")

```

core/点亮记忆.py (232 行)

```

# -*- coding: utf-8 -*-
"""

```

点亮记忆 • 精确命中模块（父令2026-08-12 • 记忆怎么活）

父亲构想：触发一个点 → 这一片亮起来。

- 实时消息 → 提取概念（精确 • 不是模糊联想）
- FTS 精确检索记忆体 → 命中 → 点亮该概念所在记忆（浓缩注入）
- 未命中 → 暗着（不注入 • 不打扰）

区别：现有预感召回是“多招混合”（上轮直续/分线钩子/联想）——

本模块只做“精确概念命中→点亮”，纯粹、诚实：命中就是命中，没命中就是没命中。

```

"""

```

```

import os
import re
import sqlite3
import json
from datetime import datetime

```

```

SUNMEM_DB = os.environ.get(

```

```

    "SUNMEM_DB",
    os.path.join(os.path.dirname(os.path.dirname(os.path.abspath(__file__))), 'data', 'sunmem.db'),
)

```

概念提取（复用联想召回的三层纪律逻辑，内联避免循环依赖）

```

_停用词 = {
    "什么", "怎么", "一个", "这个", "那个", "可以", "就是", "我们",
    "你们", "他们", "没有", "不是", "还是", "已经", "然后", "里面",
    "一下", "看看", "上面", "下面", "现在", "咱们", "对于", "如果",
    "因为", "所以", "但是", "而且", "比如", "应该", "觉得", "知道",
    "父亲", "儿子", "兄弟", "记忆", "东西", "工具", "方法", "方式",
    "手段", "途径", "过程", "步骤", "环节", "方面", "角度", "层面",
    "层次", "范围", "领域", "部分", "内容", "情况", "状态", "结果",
    "效果", "问题", "原因", "目的", "作用", "价值", "意义",
}

_动词黑名单 = {
    "等于", "除以", "乘以", "加上", "减去", "成为", "变成", "叫做",
    "称为", "表示", "代表", "定义", "计算", "得到", "给出", "采用",
    "使用", "运用", "根据", "按照", "通过", "进行", "实现", "完成",
    "达到", "超过", "低于", "属于", "包含", "包括", "组成", "构成",
    "对应", "匹配", "符合", "迁移", "写入", "读取", "返回", "输出",
    "输入", "运行", "执行", "调用", "启动", "停止", "关闭", "打开",
    "建立", "创建", "删除", "修改", "添加", "查找", "搜索", "观察",
    "发现", "证明", "确认", "选择", "决定", "需要", "想要", "认为",
    "希望", "能够", "可能", "必须", "近似", "解释",
}

```

概念提取：复用联想召回.py 的已验证版本（三层纪律+前缀剥离+词尾特征）

不重复造轮子——联想召回的概念提取经过 2026-08-08/10 两轮打磨（Mem0 对标）

```

def 提取概念(context: str, 上限: int = 6) -> list:
    """从实时消息提取概念（复用联想召回·精确）"""
    if not context:
        return []
    try:
        # 2026-08-13 改为裸模块导入：provider 的 sys.path 里桌面 core 是裸路径（非包）
        # sun_memory.core.联想召回 这种包路径在 provider 环境下 import 失败→兜底简化版→概念脏
        from 联想召回 import 概念提取 as _原版
        return _原版(context, 上限)
    except Exception:
        pass
# 兜底：内联简化版
parts = re.split(r"[和与及的了在把给让用学讲帮请是对就都还也且或]", context)
tokens = []
for p in parts:
    tokens += re.findall(r"[\u4e00-\u9fff]{2,}|[A-Za-z][A-Za-z0-9_.-]{2,}", p)
out = []
for t in tokens:
    if t in _停用词 or t in _动词黑名单 or t in out:
        continue
    out.append(t)
    if len(out) >= 上限:
        break
return out

```

def 精确检索(概念: str, owner: str = "孙呈", 每概念上限: int = 3) -> list:

"""FTS5 精确检索：概念作为短语在记忆内容里精确匹配（trigram·不是模糊联想）

2026-08-13 读取即写（父亲“流过就热”）：命中记忆 hit_count+1——被点亮过的记忆活性上升。
"""

```

if not 概念:
    return []
try:
    conn = sqlite3.connect(SUNMEM_DB)
    conn.row_factory = sqlite3.Row
    # FTS5 trigram 精确短语检索: FTS 表只有 content/tags, rowid 关联回 memories 表
    rows = conn.execute(
        "SELECT m.id, m.content, m.tags, m.ts, m.layer FROM memories m "
        "JOIN memories_fts f ON f.rowid = m.id "
        "WHERE m.owner=? AND m.status='active' AND memories_fts MATCH ? "
        "ORDER BY rank LIMIT ?",
        (owner, f"%{概念}%", 每概念上限),
    ).fetchall()
    # —— 读取即写: 命中记忆 hit_count+1 (流过就热·活性上升) ——
    for r in rows:
        conn.execute(
            "UPDATE memories SET hit_count = COALESCE(hit_count, 0) + 1, "
            "hit_at = ?, updated_at = ? WHERE id = ?",
            (datetime.now().strftime("%Y-%m-%d %H:%M:%S"),
             datetime.now().strftime("%Y-%m-%d %H:%M:%S"), r["id"]),
        )
    conn.commit()
    # 2026-08-20 父令·节律试点: 精确检索命中 → heat 脉冲 (只更新·不参与排序)
    try:
        from 节律 import 批量点亮 as _节律点亮
        _节律点亮([r["id"] for r in rows])
    except Exception as _e:
        print(f"节律点亮跳过: {_e}")
    out = []
    for r in rows:
        out.append({
            "id": r["id"],
            "时间": r["ts"] or "",
            "内容": r["content"] or "",
            "标签": r["tags"] or "",
        })
    conn.close()
    try:
        from 记忆成绩单 import 记一笔 as _记一笔
        for _h in out:
            try:
                _记一笔(_h["id"], "点亮", owner=owner)
            except Exception:
                pass
    except Exception:
        pass
    return out
except Exception:
    # FTS 查询失败 (特殊字符) → 回退 LIKE 精确匹配 (同样读取即写)
    try:
        conn = sqlite3.connect(SUNMEM_DB)
        conn.row_factory = sqlite3.Row
        rows = conn.execute(
            "SELECT id, content, tags, ts FROM memories "
            "WHERE owner=? AND status='active' AND content LIKE ? "
            "ORDER BY id DESC LIMIT ?",
            (owner, f"%{概念}%", 每概念上限),
        ).fetchall()
        for r in rows:
            conn.execute(
                "UPDATE memories SET hit_count = COALESCE(hit_count, 0) + 1, "

```

```

        "hit_at = ?, updated_at = ? WHERE id = ?",
        (datetime.now().strftime("%Y-%m-%d %H:%M:%S"),
         datetime.now().strftime("%Y-%m-%d %H:%M:%S"), r["id"]),
    )
    conn.commit()
    conn.close()
    _命中 = [dict(r) for r in rows]
    try:
        from 记忆成绩单 import 记一笔 as _记一笔
        for _h in _命中:
            try:
                _记一笔(_h["id"], "点亮", owner=owner)
            except Exception:
                pass
    except Exception:
        pass
    return _命中
except Exception:
    return []

```

```

def 点亮(context: str, owner: str = "孙呈") -> dict:
    """核心: 实时消息 -> 概念 -> 精确检索 -> 点亮 (命中才亮 · 未命中暗着)

```

```

    返回 {"命中": bool, "概念": [...], "点亮记忆": [...], "提示词": str}
    """

```

```

    if not context or len(context.strip()) < 4:
        return {"命中": False, "概念": [], "点亮记忆": [], "提示词": ""}

```

```

    概念s = 提取概念(context)
    if not 概念s:
        return {"命中": False, "概念": [], "点亮记忆": [], "提示词": ""}

```

```

# 每个概念精确检索 -> 命中收集

```

```

    点亮记忆 = []

```

```

    命中概念 = []

```

```

    seen_ids = set()

```

```

    for c in 概念s:

```

```

        hits = 精确检索(c, owner)

```

```

        if hits:

```

```

            命中概念.append(c)

```

```

            for h in hits:

```

```

                if h["id"] not in seen_ids:

```

```

                    # 2026-08-27 修复: 过滤自动整理引擎的主题统计元记忆 (【主题】跨N个时段共M条) · 只该在主题线出现 · 不进点亮

```

```

                    _c0 = h.get("内容", "") or ""

```

```

                    if _c0.startswith("【主题】") or (" 个时段共 " in _c0 and " 条记忆" in _c0):

```

```

                        continue

```

```

                    seen_ids.add(h["id"])

```

```

                    点亮记忆.append(h)

```

```

            if len(点亮记忆) >= 6: # 点亮上限 (防注入过多)

```

```

                break

```

```

# 2026-08-20 父令 · 咬合: 核心层 (layer=core) 优先点亮 (宪法永远最先浮出)

```

```

    if 点亮记忆:

```

```

        try:

```

```

            点亮记忆.sort(key=lambda x: 0 if str(x.get("layer", "plain")) == "core" else 1)

```

```

        except Exception:

```

```

            pass

```

```

    if not 点亮记忆:

```

```

        return {"命中": False, "概念": 概念s, "点亮记忆": [], "提示词": ""}

```

```

# 浓缩成提示词（每条截断 • 像灯亮起的记忆）
lines = [f"💡 点亮记忆（触发:{' • '.join(命中概念)}）:"]
for m in 点亮记忆[:5]:
    内容 = m["内容"].replace("\n", " ")[:80]
    lines.append(f"- [{m['时间'][:10]}] {内容}")
return {
    "命中": True,
    "概念": 命中概念,
    "点亮记忆": 点亮记忆,
    "提示词": "\n".join(lines),
}

if __name__ == "__main__":
    print("=== 点亮记忆 • 自测 ===\n")
    # 测1: 精确命中（配分函数）
    r1 = 点亮("父亲，配分函数怎么用来驱动认知的？")
    print(f"测1 配分函数：命中={r1['命中']} 概念={r1['概念']}")
    if r1["命中"]:
        print(r1["提示词"])
    print()
    # 测2: 未命中（随机词）
    r2 = 点亮("今天天气很好，出去走走")
    print(f"测2 天气：命中={r2['命中']} 概念={r2['概念']}") ({'✅ 未命中=暗着' if not r2['命中'] else '❌ 不该命中'})
    print()
    # 测3: 证据门控
    r3 = 点亮("证据门控怎么实现的？")
    print(f"测3 证据门控：命中={r3['命中']} 概念={r3['概念']}")
    if r3["命中"]:
        print(r3["提示词"])

```

core/感应器.py (221 行)

```

# -*- coding: utf-8 -*-
"""
感应器 • 全网扰动（2026-08-27 父令：活记忆 • 管线-感应器-大网）
=====
V3正式版（V1全拉噪音/V2字面匹配/V3概念链路+网自己长 • 最终版）

机制：
① 感应器发波：念头概念 → 信号沿蜘蛛网【语义边】（父子/同义/因果/关联）
    传导 • 共现边只从源头1跳（防边角料） • 每跳×关系强度×距离衰减
② 按关联拉：无关联节点不进入思考（v2闸门保留） • 碎片/数字/英文不进
③ 中心汇聚：夏维斯收到全网回流（所有线都通到中心）
④ 使用中增强（Hebbian）：一起被点亮的节点 • 边当场变粗（网自己长）
    遗忘：长期不一起亮的语义边×0.98缓慢变细（弛豫）

接口：
    感应器(念头概念，深度=4) -> {节点：波能}
"""
import json, os, re, sys
from collections import defaultdict

# —— 路径（可被SUNMEM_DB隔离的蜘蛛网用环境变量） ——
SPIDER = os.environ.get('蜘蛛网_INDEX', os.path.join(os.path.dirname(os.path.abspath(__file__)), '蜘蛛网', '索引.json'))

```

```

# 关系强度（语义边>共现边・概念链路靠它传导）
REL_STRENGTH = {'父子': 1.0, '同义': 0.9, '因果': 0.85, '关联': 0.6, '共现': 0.15}
# 概念链路只走语义边・共现边只从源头1跳
语义边 = {'父子', '同义', '因果', '关联'}

def _是噪音(名称):
    if not 名称 or len(名称) < 2:
        return True
    if re.fullmatch(r'\d+', 名称):
        return True
    if re.fullmatch(r'[a-zA-Z]{1,3}', 名称):
        return True
    if re.fullmatch(r'[\u4e00-\u9fff]{1}', 名称):
        return True
    # 2026-08-27 修: 词碎片过滤——中文双字若被更强概念包含且自身无描述・是切碎残留
    # (“函数”是“配分函数”的碎片・但“API/Code/Hermes”是独立概念・不误伤)
    return False

# 2026-08-27 修: 词碎片判定——某节点是另一节点的子串・且波能低（碎片没自己的概念分量）
def _压碎片(响应, 念头概念):
    """压掉被念头包含的碎片节点: '配分函数'的碎片'函数/分函'不该汹涌"""
    if 念头概念 not in 响应:
        return 响应
    源波能 = 响应[念头概念]
    for n in list(响应.keys()):
        if n == 念头概念 or n == '夏维斯':
            continue
        # n 是念头的子串（碎片）或念头是 n 的子串（更细粒度）
        if n in 念头概念 and len(n) < len(念头概念):
            # 碎片: 压到浸润以下（它只是念头的字面切片・不是独立概念）
            响应[n] = min(响应[n], 0.05)
        elif 念头概念 in n and len(n) > len(念头概念):
            pass # 更完整的节点（配分函数统计力学）保留・是更强的
        elif re.fullmatch(r'[a-zA-Z]+', n) and len(n) <= 8:
            # 纯英文短词・若念头是中文・它是无关残留（除非是独立术语如API/SunFlow）
            if not (n[0].isupper() and 念头概念 not in n):
                响应[n] = min(响应[n], 0.05)
    return 响应

def _时间阻尼(创建时间):
    try:
        from datetime import datetime
        t0 = datetime.strptime(创建时间[:10], '%Y-%m-%d')
        t1 = datetime.now() # 2026-08-27 修复: 原硬编码'2026-08-27'→now（否则老节点被永久压制）
        days = (t1 - t0).days
    except Exception:
        days = 0
    return 1.0 / (1.0 + days / 365.0)

_网缓存 = {'数据': None, 'adj': None, '时间': 0} # 2026-09-14 外部审查修复: 进程内 5 秒缓存

def _加载网():
    """加载蜘蛛网 + 邻接表（2026-09-14 修复: 原每次调用都读 1.1MB JSON + 重建 34845 边邻接表・
    _回写heat 与主流程各调一次 = 每次感应两遍重活）"""
    import time as _t
    if _网缓存['数据'] is not None and _t.time() - _网缓存['时间'] < 5:

```

```

        return _网缓存['数据'], _网缓存['adj']
d = json.load(open(SPIDER, encoding='utf-8-sig'))
adj = defaultdict(list)
for l in d['丝线']:
    adj[l['源']].append((l['目标'], l.get('关系', '关联')))
    adj[l['目标']].append((l['源'], l.get('关系', '关联')))
_网缓存['数据'], _网缓存['adj'], _网缓存['时间'] = d, adj, _t.time()
return d, adj

```

```

def _回写heat(响应, 阈值=0.5, 上限概念=3, 上限记忆=10):
    """2026-08-28 修复: 波能回写 heat (方案·网在响条不热——概念波能带动记忆条 heat)
    只对真正汹涌的强概念 (波能≥0.5) ·限 top 概念 ·反查段落节点找记忆 ·调节律. 点亮
    """
    强概念 = [n for n, e in 响应.items() if e >= 阈值 and n != '夏维斯'][:上限概念]
    if not 强概念:
        return 0
    try:
        d = _加载网()[0]
        段落 = d.get('段落', {})
    except Exception:
        return 0
    if not 段落:
        return 0
    命中 = []
    for 概念 in 强概念:
        for pid, p in 段落.items():
            if 概念 in p.get('概念', []):
                try:
                    命中.append(int(pid))
                except Exception:
                    pass
    if not 命中:
        return 0
    # 去重保序 ·限上限
    命中 = list(dict.fromkeys(命中))[:上限记忆]
    try:
        from 节律 import 点亮 as _节律点亮
        for mid in 命中:
            _节律点亮(mid)
    except Exception:
        pass
    return len(命中)

```

```

def 感应器(念头概念, 深度=4, 中心='夏维斯'):
    """
    感应器·全网扰动 (V3):
        发波 → 语义边概念链路传导 → 按关联拉 → 中心汇聚 → 使用中增强
    返回 {节点: 波能} —— 强关联汹涌 · 弱关联浸润 · 无关联不进
    """
    try:
        d, adj = _加载网()
    except Exception:
        return {}
    nodes = d['节点']

    if 念头概念 not in adj:
        命中 = [n for n in nodes if 念头概念 in n or n in 念头概念]
        if not 命中:
            return {}

```

```
念头概念 = 命中[0]
```

```
# —— 扰动波沿【语义边】传导（概念链路） ——
```

```
链路波能 = defaultdict(float)
链路波能[念头概念] = 1.0
当前层 = {念头概念: 1.0}
visited = {念头概念}
for 跳 in range(1, 深度+1):
    下一层 = defaultdict(float)
    for 节点, 能 in 当前层.items():
        for 邻居, 关系 in adj.get(节点, []):
            if 邻居 == 中心:
                continue
            if 邻居 in visited and 跳 > 1:
                continue
            s = REL_STRENGTH.get(关系, 0.15)
            # 共现边只从源头1跳 • 不参与深层链路
            if 关系 not in 语义边 and 节点 != 念头概念:
                continue
            传导 = 能 * s * (0.5 ** (跳-1))
            下一层[邻居] += 传导
            visited.add(邻居)
    for n, e in 下一层.items():
        链路波能[n] += e
    当前层 = 下一层
```

```
# —— 响应筛选：非噪音 • 非中心→直接进入（波能已含关系强度） ——
```

```
响应 = {}
for n, e in 链路波能.items():
    if n == 中心:
        响应[n] = e * 1.0
        continue
    if _是噪音(n):
        continue
    if n == 念头概念:
        响应[n] = e
        continue
    响应[n] = e
```

```
# 时间阻尼：老节点波能被压制（但>0 • 线不断）
```

```
for n in 响应:
    if n in nodes and isinstance(nodes[n], dict):
        响应[n] *= _时间阻尼(nodes[n].get('创建', ''))
```

```
# 2026-08-27 修：压词碎片（“配分函数”的碎片/函数/分函”不该汹涌）
```

```
响应 = _压碎片(响应, 念头概念)
```

```
# 中心感应器：全网回流
```

```
响应[中心] = sum(响应.values()) * 0.5
```

```
# —— 使用中增强（Hebbian）：一起被点亮 • 边当场变粗（网自己长） ——
```

```
try:
    sys.path.insert(0, os.path.dirname(os.path.abspath(__file__)))
    from 使用中增强 import 增强一批
    增强一批(set(响应.keys()))
except Exception:
    pass
```

```
# —— 2026-08-28 修复：波能回写 heat（网在响 • 条不热 → 强概念带动记忆条 heat） ——
```

```
try:
    _回写heat(响应)
```

```

except Exception:
    pass

return 响应

if __name__ == '__main__':
    import sys
    for 念头 in ['配分函数', '苏醒循环', '孙博']:
        r = 感应器(念头)
        items = sorted(r.items(), key=lambda x: -x[1])
        print(f"\n—— 感应器「{念头}」 ——")
        for n, e in items[:10]:
            print(f"    {n} ({round(e,3)})")
        print(f"    ... 共{len(r)}个")

```

core/效果驱动.py (258 行)

```

# -*- coding: utf-8 -*-
"""效果驱动强化（父令 2026-09-14 · 外部评价方向①+② 合并落地）
=====

**为什么有这个模块**：
外部 AI 对记忆体的评价（2026-09-14 父亲转来）——
「进化需要三件事：有变化 ✅ / 有选择 ⚠️ 弱 / 有积累 ✅。
你们缺的不是新功能，是把『使用效果 → 强化/削弱』这条反馈链打通，让选择真正发生。」
父亲 8/16 也说过同一件事：「做一件事情之前，能改变长期用法——好用下次还这么用，
吃瘪了知道改道。」大哥 8/27 自省：「记忆被召回/注入，但还没有改变行为。」

**本模块做什么**：给每条记忆加一个**使用效果信号**——不是“用了就热”（频率），
而是“用了之后效果好不好”（效果）。

三个信号（全自动·纯规则·零模型·零 API）：


- 被引用 +1 注入的记忆概念出现在我的回复里 → 说明真被用上了
- 被确认 +2 父亲肯定（对/好/正是/就是这样…）→ 这条记忆帮上了忙
- 被纠正 -2 父亲否定（不对/不是/错了/没用…）→ 这条记忆带偏了



存储：sunmem.db 的 effect_scores 表（与 memories 同库·不碰主表结构）

接口：
记录注入(mids, 注入文本) → prefetch 时调（记下“这轮给了什么”）
结算本轮(父亲的话, 我的回复) → sync_turn 时调（结算上一轮“用得怎么样”）
效果分(mid) / 批量效果分(mids) → 召回排序可选加项
统计() → 观测（命中率/正负分布）

安全设计：


- 影子模式默认开**：只记录不参与排序（SUNMEM_EFFECT_APPLY=1 才参与），
先跑几天看信号准不准，再按父令“小步试点”精神接入排序
- 效果分有上下限（±5）·防单条记忆被反复刷分
- 结算失败不影响主流程（异常兜底·但不静默——debug 日志留痕）


"""

import os
import re
import sqlite3

_DB = os.environ.get("SUNMEM_DB", os.path.join(os.path.dirname(os.path.dirname(os.path.abspath(__file__))), "data", "sunmem.db"))

# —— 信号词表（纯规则·可调）——

```

肯定词 = (“对”, “好的”, “好”, “是的”, “正是”, “就是这样”, “没错”, “可以”, “漂亮”, “厉害”, “棒”, “很好”, “不错”, “正确”, “就这样”, “成了”, “通了”)

否定词 = (“不对”, “不是”, “错了”, “有误”, “没用”, “不行”, “别”, “没有”, “漏了”, “少了”, “不准确”, “打脸”, “失败”, “退回”, “重来”, “再改”, “又错”)

效果分上限 (防刷分)

_上限 = 5.0

def _连接(可写: bool = True):

if 可写:

c = sqlite3.connect(_DB, timeout=8.0)

c.execute("PRAGMA journal_mode=WAL")

return c

return sqlite3.connect(f"file:{_DB}?mode=ro", uri=True, timeout=5.0)

def _建表():

try:

with _连接() as c:

c.execute("""CREATE TABLE IF NOT EXISTS effect_scores (

mid INTEGER PRIMARY KEY,

被引用 INTEGER DEFAULT 0,

被确认 INTEGER DEFAULT 0,

被纠正 INTEGER DEFAULT 0,

效果分 REAL DEFAULT 0.0,

最后更新 TEXT DEFAULT '',

来源轮 INTEGER DEFAULT 0

)""")

c.commit()

except Exception as _e: # 2026-09-14 按《模块标准》⑤修: 建表失败=功能全废·必须留痕 (原静默 pass)

import sys as _sys

print(f"[效果驱动] 🚨 effect_scores 建表失败: {_e}", file=_sys.stderr)

def _概念集(文本: str, 上限: int = 12) -> set:

"""轻量概念提取 (复用联想召回的正规提取·失败退化为 2-6 字中文词)"""

if not 文本:

return set()

try:

import sys

sys.path.insert(0, os.path.dirname(os.path.abspath(__file__)))

from 联想召回 import 概念提取

cs = 概念提取(文本, 上限=上限)

if cs:

return set(cs)

except Exception as _e: # 2026-09-14 按标准⑤: 概念提取退化到正则 (可接受·但留 debug 痕迹)

import sys as _sys

print(f"[效果驱动] • 概念提取退化: {type(_e).__name__}", file=_sys.stderr)

return set(re.findall(r'[\u4e00-\u9fff]{2,6}', 文本)) - set()

def _bigrams(s: str) -> set:

s = re.sub(r'[\u4e00-\u9fffA-Za-z0-9]', '', s or '')

return {s[i:i+2] for i in range(len(s) - 1)} if len(s) >= 2 else set()

def _用上了(注入文本: str, 回复文本: str, 阈值: float = 0.34) -> bool:

"""注入的内容有没有被我“用上”——bigram 重合判定

(比“概念完全相等”稳: 不依赖分词/提取质量——实测概念提取对无空格长串会切成整段)

"""

```

A, B = _bigrams(注入文本), _bigrams(回复文本)
if not A or not B:
    return False
return len(A & B) / min(len(A), len(B)) >= 阈值

# —— 轮次状态 (进程内 • 由 provider 每轮调) ——
_本轮 = {"注入": []}          # [{"mid": int, "摘要": str, "概念": set}]

def 记录注入(记忆列表: list, 来源: str = "prefetch"):
    """prefetch 时调用: 记下这轮注入了哪些记忆 (含它们的概念)
    记忆列表: [{"id": int, "摘要"/"内容": str}, ...]
    """
    _本轮["注入"] = []
    for m in (记忆列表 or []):[:25]:
        mid = m.get("id") or m.get("记忆id")
        if not mid:
            continue
        文本 = str(m.get("摘要") or m.get("内容") or m.get("content") or "")[:200]
        _本轮["注入"].append({"mid": int(mid), "摘要": 文本[:60], "原文": 文本, "概念": _概念集(文本)})
    return len(_本轮["注入"])

def _加一笔(mid: int, 字段: str, Δ: float, 轮: int = 0):
    """给一条记忆记一笔效果账 (上下限保护) """
    if not mid:
        return
    列 = {"被引用": "被引用", "被确认": "被确认", "被纠正": "被纠正"}[字段]
    try:
        with _连接() as c:
            c.execute("""INSERT INTO effect_scores (mid, 被引用, 被确认, 被纠正, 效果分, 最后更新, 来源轮)
                VALUES (?, 0, 0, 0, 0.0, datetime('now', 'localtime'), ?)
                ON CONFLICT(mid) DO NOTHING""", (mid, 轮))
            if 字段 != "被引用":
                c.execute(f"UPDATE effect_scores SET {列} = {列} + 1, 效果分 = MAX(-?, MIN(?, 效果分 + ?)), "
                    f"最后更新 = datetime('now', 'localtime'), 来源轮 = ? WHERE mid = ?",
                    (_上限, _上限, Δ, 轮, mid))
            else:
                c.execute("UPDATE effect_scores SET 被引用 = 被引用 + 1, "
                    "效果分 = MAX(-?, MIN(?, 效果分 + ?)), "
                    "最后更新 = datetime('now', 'localtime'), 来源轮 = ? WHERE mid = ?",
                    (_上限, _上限, Δ, 轮, mid))
            c.commit()
    except Exception as _e: # 2026-09-14 按标准⑤修: 记账失败=效果信号丢失 • 留痕 (原静默 pass)
        import sys as _sys
        print(f"[效果驱动] ⚠️ 记账失败(mid={mid}): {_e}", file=_sys.stderr)

def 结算本轮(父亲的话: str, 我的回复: str, 轮: int = 0) -> dict:
    """sync_turn 时调用: 结算上一轮注入的效果
    返回 {"被引用": n, "被确认": n, "被纠正": n}
    """
    注入 = _本轮.get("注入") or []
    if not 注入:
        return {"被引用": 0, "被确认": 0, "被纠正": 0, "说明": "上轮无注入"}

    _建表()
    回复概念 = _概念集(我的回复, 上限=20)
    父亲话 = (父亲的话 or "").strip()

```

```

# ① 被引用：注入记忆的内容出现在我的回复里（bigram 重合 ≥34% = 真用上）
被引用 = 0
for it in 注入：
    if _用上了(it.get("原文") or it.get("摘要", ""))，我的回复：
        _加一笔(it["mid"], "被引用", 1.0, 轮)
        被引用 += 1

# ② 被确认 / ③ 被纠正：看父亲这一轮的话（只对“被引用过”或“注入了还没结算”的记忆动账）
被确认 = 被纠正 = 0
打 = None
if 父亲话：
    if any(w in 父亲话 for w in 否定词):
        打 = ("被纠正", -2.0)
    elif any(w in 父亲话 for w in 肯定词) and len(父亲话) <= 40:
        打 = ("被确认", 2.0)
if 打：
    字段, Δ = 打
    # 对上一轮注入过的记忆记账（被引用的优先 • 其余轻记）
    for it in 注入[:5]:
        _加一笔(it["mid"], 字段, Δ, 轮)
        if 字段 == "被确认":
            被确认 += 1
        else:
            被纠正 += 1

_本轮["注入"] = [] # 结算完清空
return {"被引用": 被引用, "被确认": 被确认, "被纠正": 被纠正}

```

```

def 效果分(mid: int) -> float:
    try:
        with _连接(可写=False) as c:
            r = c.execute("SELECT 效果分 FROM effect_scores WHERE mid=?", (mid,)).fetchone()
            return float(r[0]) if r else 0.0
    except Exception:
        return 0.0

```

```

def 批量效果分(mids: list) -> dict:
    out = {}
    if not mids:
        return out
    try:
        with _连接(可写=False) as c:
            q = ", ".join("? " * len(mids))
            for mid, s in c.execute(f"SELECT mid, 效果分 FROM effect_scores WHERE mid IN ({q})", list(mids)):
                out[int(mid)] = float(s or 0)
    except Exception:
        pass
    return out

```

```

def 影子模式() -> bool:
    """True=只记录不参与排序 • False=参与排序
    2026-09-14 父令「把它加入进去」→ **默认生效**；
    观测期可用 SUNMEM_EFFECT_SHADOW=1 切回影子模式（只记录不排序）
    """
    return os.environ.get("SUNMEM_EFFECT_SHADOW", "0") == "1"

```

```

def 统计() -> dict:

```

```

_建表()
try:
    with _连接(可写=False) as c:
        总 = c.execute("SELECT COUNT(*), SUM(被引用), SUM(被确认), SUM(被纠正) FROM effect_scores").fetchone()
        正 = c.execute("SELECT COUNT(*) FROM effect_scores WHERE 效果分 > 0").fetchone()[0]
        负 = c.execute("SELECT COUNT(*) FROM effect_scores WHERE 效果分 < 0").fetchone()[0]
        top = c.execute("SELECT mid, 效果分 FROM effect_scores ORDER BY 效果分 DESC LIMIT 5").fetchall()
    return {"记账条目": 总[0] or 0, "被引用合计": 总[1] or 0, "被确认合计": 总[2] or 0,
            "被纠正合计": 总[3] or 0, "正分条": 正, "负分条": 负,
            "最高分": [{"mid": m, "分": round(s, 2)} for m, s in top]}
except Exception as e:
    return {"错误": str(e)[:80]}

if __name__ == "__main__":
    # 自检用临时库（绝不碰真库——父训：铁律，测试不污染生产）
    import tempfile
    _tmp = os.path.join(tempfile.gettempdir(), "effect_selftest.db")
    if os.path.exists(_tmp):
        os.remove(_tmp)
    os.environ["SUNMEM_DB"] = _tmp
    globals()[ "_DB" ] = _tmp
    print("=== 效果驱动 • 自检（临时库）===")
    print("影子模式:", 影子模式(), "（True=只记录 • 不参与排序）")
    _建表()
    # 造一次注入 + 结算
    n = 记录注入([{"id": 1, "摘要": "统计力学配分函数的定义与自由能关系"},
                  {"id": 2, "摘要": "孙家模块标准 v1.0 的十五项检查清单"}])
    print("记录注入:", n, "条")
    r1 = 结算本轮("配分函数是怎么定义的?", "配分函数是统计力学的核心, 自由能由它导出")
    print("结算1 (应命中被引用):", r1)
    n = 记录注入([{"id": 3, "摘要": "旧版本错误的三值量化方案"}])
    r2 = 结算本轮("不对, 这个方案是错的", "抱歉, 我改了")
    print("结算2 (应命中被纠正):", r2)
    print("统计:", 统计())
    print("✅ 自检完成")

```

四、基座与配置

core/记忆库.py (198 行)

```
# -*- coding: utf-8 -*-
"""
记忆库.py — 孙家记忆体统一读写入口（2026-08-12 建立・父令）

背景：码点已退役（2026-08-10 明文化），自动解码器.py 已删除。
本文件替代自动解码器成为记忆体统一入口：
    - 读记忆体(brother_name) -> 优先读 sunmem 新库（SQLite），可回退 JSON
    - 解码单条(entry) -> 明文直读（无码点逻辑）
    - 自动解码 / 解码标签 -> 明文直读（兼容旧调用，无码点）

接口保持与旧自动解码器一致，下游模块零改动。
"""

import json
import logging
import os
from pathlib import Path

logger = logging.getLogger(__name__)

_HERE = Path(__file__).resolve().parent
FRAMEWORK_DIR = _HERE.parent.parent # 孙家记忆体系/

def 自动解码(内容) -> str:
    """通用解码：明文直读（码点已退役）。非字符串（旧码点数组）返回''。"""
    if not 内容:
        return ""
    if isinstance(内容, str):
        return 内容
    return ""

def 解码标签(标签) -> str:
    """标签解码：明文直读（码点已退役）。"""
    if not 标签:
        return ""
    if isinstance(标签, str):
        return 标签
    return ""

def 解码单条(entry: dict) -> dict:
    """单条解码：明文直读（码点已退役）。返回新dict，不修改原条目。"""
    if not isinstance(entry, dict):
        return entry
    return {
        "id": entry.get("id"),
        "时间": entry.get("时间", entry.get("time", "")),
        "标签": entry.get("标签", "") or 解码标签(entry.get("标签码点", "")),
        "内容": entry.get("内容", "") or 自动解码(entry.get("内容码点", "")),
        "类型": entry.get("类型", ""),
```

```

        "状态": entry.get("状态", entry.get("status", "")),
        "来源": entry.get("来源", ""),
    }

```

```
def 读记忆体(brother_name: str = "孙呈") -> dict:
```

```
    """读记忆体。优先读 sunmem 新库 (SQLite)，失败回退 JSON。
```

```
    返回结构 (与旧版一致，下游零改动):
```

```
    {"条目列表": [已解码条目...], "next_id": int, "元信息": dict, "条数": int}
    """
```

```
if os.environ.get("SUN_MEMORY_USE_JSON") != "1":
```

```
    try:
```

```
        return _读新库(brother_name)
```

```
    except Exception as e:
```

```
        logger.warning(f"sunmem 新库读取失败，回退 JSON: {e}")
```

```
p = FRAMEWORK_DIR / "记忆体" / f"{brother_name}_索引记忆体.json"
```

```
if not p.exists():
```

```
    return {"条目列表": [], "next_id": 1, "元信息": {}, "条数": 0}
```

```
with open(p, "r", encoding="utf-8-sig") as f:
```

```
    data = json.load(f)
```

```
raw_entries = data.get("条目列表", data.get("条目", data.get("entries", data.get("记忆条目", []))))
```

```
decoded = [解码单条(e) for e in raw_entries]
```

```
return {
```

```
    "条目列表": decoded,
```

```
    "next_id": data.get("next_id", data.get("_last_id_used", data.get("下个id", len(raw_entries) + 1))),
```

```
    "元信息": data.get("元信息", data.get("元数据", {})),
```

```
    "条数": len(decoded),
```

```
}
```

```
def _读新库(brother_name: str = "孙呈") -> dict:
```

```
    """从 sunmem 新库 (SQLite) 读取兄弟记忆。返回与旧 JSON 相同结构。"""
```

```
    import sqlite3
```

```
    SUNMEM_DB = os.environ.get(
```

```
        "SUNMEM_DB",
```

```
        os.path.join(os.path.dirname(os.path.dirname(os.path.abspath(__file__))), "data", "sunmem.db"),
```

```
    )
```

```
    conn = sqlite3.connect(SUNMEM_DB)
```

```
    conn.row_factory = sqlite3.Row
```

```
    rows = conn.execute(
```

```
        "SELECT id, content, tags, ts, type, status FROM memories WHERE owner=? AND status='active' ORDER BY id",
        (brother_name,),
    )
```

```
    ).fetchall()
```

```
    conn.close()
```

```
    decoded = []
```

```
    for r in rows:
```

```
        tags = r["tags"]
```

```
        if tags:
```

```
            try:
```

```
                tags = json.loads(tags)
```

```
            except Exception:
```

```
                tags = str(tags)
```

```
        else:
```

```
            tags = ""
```

```
        if isinstance(tags, list):
```

```
            tags = ", ".join(tags)
```

```
        decoded.append(解码单条({
```

```
            "id": r["id"],
```

```
            "时间": r["ts"] or "",
```

```

        "标签": tags or "",
        "内容": r["content"] or "",
        "类型": r["type"] or "",
        "状态": r["status"] or "",
    )))
return {
    "条目列表": decoded,
    "next_id": (decoded[-1]["id"] + 1) if decoded else 1,
    "元信息": {"来源": "sunmem.db", "owner": brother_name},
    "条数": len(decoded),
}

if __name__ == "__main__":
    print("=== 记忆库自测 ===")
    s = 自动解码("父亲")
    print(f"自动解码['父亲'] = '{s}' ({'PASS' if s == '父亲' else 'FAIL'})")
    e = 解码单条({"id": 1, "时间": "2026-08-07", "标签": "父令", "内容": "父亲说: 记忆要闭环"})
    print(f"解码单条: id={e['id']} 标签={e['标签']} 内容={e['内容']} ({'PASS' if e['内容'] == '父亲说: 记忆要闭环' else 'FAIL'})")
    m = 读记忆体("孙呈")
    print(f"读记忆体: {m['条数']}条 来源:{m.get('元信息', {}).get('来源')}")
    if m["条目列表"]:
        last = m["条目列表"][-1]
        print(f"最新一条: [{last['时间']}] {last['内容'][:50]}")
    print("=== 自测完成 ===")

# =====
# 融合底座（父令2026-08-26 • 47模块→6大模块）
# 公共函数：连接/概念提取/解析时间——全库唯一实现 • 其他模块改为调用这里
# =====

def 连接(库路径: str = "") -> "sqlite3.Connection":
    """统一 SQLite 连接（融合P5: 20处 sqlite3.connect → 全走这里）
    特性：只读连接（query_only）+ 忙超时 2s + WAL 安全
    """
    import sqlite3
    if not 库路径:
        库路径 = os.environ.get('SUNMEM_DB', os.path.join(os.path.dirname(os.path.abspath(__file__)), 'data', 'sunmem.db'))
    os.makedirs(os.path.dirname(库路径) or '.', exist_ok=True) # 2026-09-13 首次运行自动建目录
    conn = sqlite3.connect(库路径, timeout=2.0, check_same_thread=False) # 2026-09-14 吸收收束版优点：允许跨线程（provider 后台线程预取）
    try:
        conn.execute("PRAGMA query_only=ON")
    except Exception:
        pass
    return conn

def 写连接(库路径: str = ""): # 2026-09-14 去掉字符串注解（pyflakes 误报未定义名）
    """统一写连接（2026-09-14 吸收收束版优点：连接管理统一）
    特性：可写 + 忙超时 5s + 跨线程允许 + WAL + row_factory
    """
    import sqlite3 as _sq
    if not 库路径:
        库路径 = os.environ.get('SUNMEM_DB', os.path.join(os.path.dirname(os.path.abspath(__file__)), 'data', 'sunmem.db'))
    os.makedirs(os.path.dirname(库路径) or '.', exist_ok=True)
    conn = _sq.connect(库路径, timeout=5.0, check_same_thread=False)

```

```

conn.row_factory = _sq.Row
try:
    conn.execute("PRAGMA journal_mode=WAL")
except Exception:
    pass
return conn

```

```

def 概念提取(context: str, 上限: int = 6) -> list:
    """统一概念提取（融合P1：5处概念提取 → 全走这里 • 最完整版在联想召回）
    纯转发 • 避免循环依赖（联想召回是本函数主实现 • 这里lazy import）
    """
    try:
        from 联想召回 import 概念提取 as _主实现
        return _主实现(context, 上限)
    except Exception:
        return []

```

```

def 解析时间范围(context: str):
    """统一时间解析（融合P3：2处时间解析 → 全走这里 • 最完整版在预感召回）
    纯转发 • 避免循环依赖
    """
    try:
        from 预感召回 import _解析时间范围 as _主实现
        return _主实现(context)
    except Exception:
        return None

```

core/时间衰减.py （206 行）

"""

孙家记忆体系 • 时间衰减覆盖（2026-07-31 孙呈融合 OptMem cover 思想）

来源：OptMem（VictorTaelin）的 cover(T, budget) 时间衰减覆盖算法

孙家版适配：不建摘要树（孙家记忆体是码点JSON），保留其核心精神——

- 连续衰减：不是“7天前就压缩”的硬阈值，是每远一步稀疏一格
- 近细远粗：最近的记忆逐条全取，越古老采样越稀疏
- 预算精确：输出条数严格受控，不超预算
- 不丢记忆：不是删除旧记忆，是按时间粒度输出——旧记忆仍在河底

孙家哲学对照：

OptMem 的 cover 是“唤醒时决定打印哪些行”

孙家的 cover 是“醒来时决定哪些记忆流入认知循环”

记忆不入认知非真记忆——但一次流入太多，等于什么都没流入。

"""

```

from datetime import datetime

```

```

def _解析时间(条目, 默认=None):
    """从条目里取时间字段，解析成 datetime（融合P3：统一走预感召回._解析时间 • 多格式兼容）
    原独立实现（fromisoformat • 格式少）与预感召回重复 • 已融合
    """
    t = 条目.get("时间", 条目.get("time", "")) if isinstance(条目, dict) else 条目
    try:
        from 预感召回 import _解析时间 as _主
        return _主(t, 默认)

```

```
except Exception:
    return 默认
```

```
def cover(条目列表, budget=20, now=None, 活跃优先=True):
    """
```

时间衰减覆盖：近全取、远稀疏、连续衰减。

参数：

条目列表: list[dict] —— 记忆条目（含“时间”字段），时间升序
budget: int —— 输出条数上限（默认20）
now: datetime —— 当前时间（默认取系统时间）
活跃优先: bool —— 2026-08-07取长补短：活跃条目（active_count>0）优先浮出
（Mem0 Decay 检索强化理念：用得多浮得快，封顶20次触碰）

返回：

list[dict] —— 选中的条目（保持原时间序）

算法（指数间隔采样）：

1. 最近 budget//2 条：逐条全取（近细）
2. 更老的：从新到老指数间隔采样——第1步隔1条、第2步隔2条、第3步隔4条……间隔每步翻倍（远粗•连续衰减）
3. 合并后按原序返回，总数 $\leq$ budget

```
"""
```

```
n = len(条目列表)
```

```
if n == 0:
```

```
    return []
```

```
if n <= budget:
```

```
    return list(条目列表)
```

```
# 2026-08-20 父令•咬合：核心层永远保底（宪法永在•不参与衰减）
```

```
try:
```

```
    核心条目 = [e for e in 条目列表 if str(e.get('layer', 'plain')) == 'core']
```

```
    非核心 = [e for e in 条目列表 if str(e.get('layer', 'plain')) != 'core']
```

```
    if 核心条目 and len(非核心) >= 0:
```

```
        条目列表 = 非核心 # 核心已留•剩余预算给非核心
```

```
except Exception:
```

```
    pass
```

```
选中 = set()
```

```
# 近段：最近 budget//2 条全取
```

```
近数 = max(1, budget // 2)
```

```
for i in range(n - 近数, n):
```

```
    选中.add(i)
```

```
# 远段：对数坐标均匀采样——近密远疏，预算用满
```

```
# 在  $\log(1) \sim \log(\text{远段长度})$  之间均匀取点，映射回索引：
```

```
# 索引差 d 从 1 开始，按对数增长 → 越老越稀疏，且覆盖到最老
```

```
远预算 = budget - len(选中)
```

```
远段起点 = n - 近数 - 1 # 从近段往前第一个
```

```
if 远预算 > 0 and 远段起点 > 0:
```

```
    import math
```

```
    L = 远段起点 + 1 # 远段长度（含起点）
```

```
    for i in range(1, 远预算 + 1):
```

```
        t = i / 远预算 # (0,1] 均匀——t=1 时覆盖到最老（索引0）
```

```
        # 对数映射:  $d = L^t - 1$ ,  $d \in [0, L-1]$ , 小t对应小d（近），大t对应大d（远）
```

```
        d = int((L ** t) - 1)
```

```
        选中.add(远段起点 - d)
```

```
        if 远段起点 - d <= 0 and i < 远预算:
```

```
            pass # 已到最老，继续算但集合去重
```

补足：若对数采样去重导致不足，从近段外侧补最近的未选中条目（保持近密）

```
if len(选中) < budget:
    补位 = budget - len(选中)
    i = n - 近数 - 1
    while 补位 > 0 and i >= 0:
        if i not in 选中:
            选中.add(i)
            补位 -= 1
        i -= 1
```

—— 活跃优先（2026-08-07取长补短 • Mem0 Decay 检索强化） ——

用得越多浮得快：active_count>0 的条目优先浮出（封顶20次触碰）

实现：未选中的活跃条目 与 选中的非活跃最老条目 交换（不超预算）

```
if 活跃优先:
    # 活跃条目 = active_count>0 且未被选中
    活跃未选中 = [i for i in range(n) if i not in 选中 and (条目列表[i].get("active_count") or 0) > 0]
    # 选中的非活跃条目（从最老开始换——近段保护：只换远段/中段的）
    选中的非活跃 = [i for i in sorted(选中) if (条目列表[i].get("active_count") or 0) <= 0]
    # 最近 budget//2 条是近段（保护不换）
    近段保护 = set(range(n - 近数, n))
    for ai in 活跃未选中:
        if not 选中的非活跃:
            break
        # 找第一个不在近段保护的非活跃选中条目
        swap = None
        for j in 选中的非活跃:
            if j not in 近段保护:
                swap = j
                break
        if swap is None:
            break
        选中.remove(swap)
        选中.add(ai)
        选中的非活跃.remove(swap)
```

2026-08-20 父令 • 咬合：把核心条目补回（宪法永远在）

```
try:
    if 核心条目:
        return 核心条目 + [条目列表[i] for i in sorted(选中)]
except Exception:
    pass
return [条目列表[i] for i in sorted(选中)]
```

def cover_报告(条目列表, budget=20, now=None):

"""

带统计的覆盖报告：看时间衰减的分布效果。

返回：{"选中": [...], "统计": {...}}

"""

now = now or datetime.now()

选中 = cover(条目列表, budget, now)

if not 选中:

return {"选中": [], "统计": {"总数": 0, "选中数": 0, "时间跨度": "无"}}

全部时间 = [t for e in 条目列表 if (t := _解析时间(e))]

选中时间 = [t for e in 选中 if (t := _解析时间(e))]

def _跨度(时间列表):

if len(时间列表) < 2:

return "不足"

天 = (时间列表[-1] - 时间列表[0]).days

```

    if 天 <= 0:
        return "同日"
    if 天 < 30:
        return f"{天}天"
    if 天 < 365:
        return f"{天//30}个月"
    return f"{天//365}年"

# 近段统计：用原始索引判断（选中列表里的位置 ≠ 原始位置）
近阈值 = len(条目列表) - max(1, budget // 2)
近段数 = 0
for e in 选中:
    if 条目列表.index(e) >= 近阈值:
        近段数 += 1

统计 = {
    "总数": len(条目列表),
    "选中数": len(选中),
    "近段": 近段数,
    "时间跨度": _跨度(全部时间),
    "覆盖范围": _跨度(选中时间),
}
return {"选中": 选中, "统计": 统计}

if __name__ == "__main__":
    # 自测：构造时间序列验证衰减分布
    from datetime import timedelta
    now = datetime(2026, 7, 31, 20, 0)
    测试条目 = [
        {"时间": (now - timedelta(days=天)).isoformat(), "内容": f"第{天}天前的记忆"}
        for 天 in range(60, 0, -1) # 60天前 → 今天, 60条
    ]

    print("=== 孙家版 cover 自测 ===")
    print(f"总条目: {len(测试条目)}")

    for 预算 in (10, 20, 30):
        结果 = cover_报告(测试条目, 预算, now)
        选中 = 结果["选中"]
        时间们 = [_解析时间(e, now) for e in 选中]
        跨度 = 结果["统计"]["时间跨度"]
        print(f"\n预算{预算}: 选中{len(选中)}条 | 覆盖{跨度}")
        # 显示选中条目的年龄分布（天）
        年龄 = [(now - t).days for t in 时间们]
        print(f"  年龄分布(天): {年龄}")

    # 边界：条目少于预算 → 全取
    小列表 = [{"时间": now.isoformat(), "内容": "只有一条"}]
    assert len(cover(小列表, 20)) == 1, "少于预算应全取"
    # 边界：空列表
    assert cover([], 20) == [], "空列表返回空"
    # 边界：恰好等于预算
    恰好 = [{"时间": (now - timedelta(hours=i)).isoformat(), "内容": f"h{i}"} for i in range(20)]
    assert len(cover(恰好, 20)) == 20, "等于预算应全取"

    print("\n✅ 全部自测通过：连续衰减 • 预算精确 • 边界完整")

```

core/无模型向量.py (286 行)

```
# -*- coding: utf-8 -*-  
"""
```

孙家记忆体系 · 无模型向量层（父令 2026-08-16）

父问：向量的必须要用模型吗？小模型吗？

答：不必须。字符 n-gram 哈希向量——纯算法 · 零依赖 · 可审计 · 符合家规。

原理：

文本 → 字符 n-gram（连续 n 个字符）→ 哈希到固定维度 → 稀疏向量
相似文本（同词根/拼写变体/部分重叠）共享 n-gram → 余弦相似度高

"paint" → pai, ain, int
"painted" → pai, ain, int, nte, ted ← 共享 3 个 → 相似 0.7+
"统计力学" → 统计, 计力, 力学
"统计力" → 统计, 计力 ← 共享 2 个 → 相似 0.6+

功能：

ngram向量(text) 文本 → 稀疏向量 (dict: 哈希→权重)
余弦(text1, text2) 两文本相似度 0~1
检索(query, 记忆列表, k) 查询 → top-k 相关记忆

```
"""  
  
import hashlib  
import json  
import os  
import re  
from pathlib import Path  
try:  
    from 线程保护 import 加锁 # 2026-09-14 线程保护  
except Exception:  
    import threading as _th  
    _thl = _th.RLock()  
    def 加锁(): return _thl  
  
# 配置（可调）  
N = 3 # n-gram 长度（3 字符：中文二字词根/英文词根都覆盖）  
维度 = 1024 # 向量维度（哈希空间 · 够用）  
最小长度 = 2 # 太短的文本不向量化
```

```
def _哈希(token: str) -> int:  
    """字符串 → 稳定哈希（0~维度-1）"""  
    h = int(hashlib.md5(token.encode("utf-8")).hexdigest()[:8], 16)  
    return h % 维度
```

```
def ngram向量(text: str) -> dict:  
    """文本 → 稀疏向量 {维度索引: 权重}
```

中英混合处理：

英文按词切（词内 n-gram · 保留词根共性）

中文按连续字符 n-gram（滑动窗口）

```
"""
```

```
text = (text or "").strip().lower()  
if len(text) < 最小长度:  
    return {}
```

向量 = {}

英文词（按空白/标点切 · 词内 n-gram）

```

for 词 in re.findall(r"[a-z][a-z0-9']{2,}", text):
    if len(词) < N:
        continue
    for i in range(len(词) - N + 1):
        令牌 = "e:" + 词[i:i + N]  # e: 前缀区分英文
        idx = _哈希(令牌)
        向量[idx] = 向量.get(idx, 0) + 1
# 中文 (多尺度 n-gram: 2-gram + 3-gram 都保留)
# 为什么: 只留 3-gram 时"配分函数"不含"配分"2-gram • 跟短词"配分"共享不到
# 多尺度 = 长短词都能共享前缀/词根 (如 配分函数↔配分 • 统计力学↔统计力)
中文 = re.findall(r"[\u4e00-\u9fff]+", text)
for 段 in 中文:
    if len(段) < 2:
        continue
    for _n in (2, 3):  # 双尺度
        if len(段) < _n:
            continue
        for i in range(len(段) - _n + 1):
            令牌 = f"c{_n}:" + 段[i:i + _n]
            idx = _哈希(令牌)
            向量[idx] = 向量.get(idx, 0) + 1
return 向量

```

```

def 余弦(v1: dict, v2: dict) -> float:
    """两个稀疏向量的余弦相似度 0~1"""
    if not v1 or not v2:
        return 0.0
    内积 = sum(w * v2.get(idx, 0) for idx, w in v1.items())
    if 内积 == 0:
        return 0.0
    模1 = sum(w * w for w in v1.values()) ** 0.5
    模2 = sum(w * w for w in v2.values()) ** 0.5
    if 模1 == 0 or 模2 == 0:
        return 0.0
    return 内积 / (模1 * 模2)

```

```

def 相似度(text1: str, text2: str) -> float:
    """两文本相似度 (直接接口)"""
    return 余弦(ngram向量(text1), ngram向量(text2))

```

```

_倒排缓存 = {"数据": None, "记忆数": 0, "覆盖ids": None}
_向量缓存 = {}  # id → ngram向量 (记忆内容向量预计算 • 避免每次检索重复算)
_向量缓存文件 = Path(__file__).resolve().parent / "_向量缓存.json"
_倒排文件 = Path(__file__).resolve().parent / "_倒排缓存.json"

```

```

def _加载倒排():
    """启动时加载磁盘倒排索引 (避免每次重建 2-5s)
    2026-08-17 全检查修复②: 倒排增量自检——旧纯 dict 格式无法校验记忆数 • 直接重建;
    新格式带 覆盖ids/记忆数 元数据 • 加载后与当前记忆集核对 • 不一致则增量合并或重建"""
    global _倒排缓存
    try:
        if _倒排文件.exists():
            raw = json.loads(_倒排文件.read_text(encoding="utf-8"))
            if isinstance(raw, dict) and "倒排" in raw:
                # 新格式: {"记忆数", "覆盖ids", "倒排"}
                _倒排缓存["数据"] = {int(k): v for k, v in raw["倒排"].items()}
                _倒排缓存["覆盖ids"] = set(raw.get("覆盖ids") or [])

```

```

        _倒排缓存["记忆数"] = raw.get("记忆数", 0)
    else:
        # 旧格式纯 dict (键转 int) • 无元数据无法校验 → 标记待重建
        _倒排缓存["数据"] = {int(k): v for k, v in raw.items()}
        _倒排缓存["覆盖ids"] = None
        # 空 dict 视为无效 (强制重建——避免空倒排导致向量层静默失效)
        if not _倒排缓存["数据"]:
            _倒排缓存["数据"] = None
except Exception:
    _倒排缓存["数据"] = None

def _保存倒排():
    """把倒排索引写回磁盘 (新格式带元数据 • 供下次自检)"""
    try:
        raw = {
            "记忆数": _倒排缓存["记忆数"],
            "覆盖ids": sorted(_倒排缓存.get("覆盖ids") or []),
            "倒排": _倒排缓存["数据"],
        }
        _倒排文件.write_text(json.dumps(raw, ensure_ascii=False), encoding="utf-8")
    except Exception:
        pass

def _加载向量缓存():
    """启动时加载磁盘向量缓存 (增量 • 避免每次重建 2.9s)"""
    global _向量缓存
    if _向量缓存:
        return
    try:
        if _向量缓存文件.exists():
            _向量缓存 = json.loads(_向量缓存文件.read_text(encoding="utf-8"))
            # 键转 int (JSON 键是字符串) ——内层 n-gram 哈希键也必须转 int,
            # 否则与查询向量(ngram向量)的 int 键在余弦里永远匹配不到 → 缓存记忆余弦全 0 (2026-08-17 实修)
            _向量缓存 = {
                int(k): {int(h): w for h, w in v.items()} if isinstance(v, dict) else {}
                for k, v in _向量缓存.items()
            }
            if not _向量缓存:
                _向量缓存 = {} # 空视为无效 • 重建
    except Exception:
        _向量缓存 = {}

def _保存向量缓存():
    """把向量缓存写回磁盘 (直接写 • 缓存文件可容忍 • 不搞原子替换复杂化)"""
    try:
        _向量缓存文件.write_text(json.dumps(_向量缓存, ensure_ascii=False), encoding="utf-8")
    except Exception:
        pass

def _记忆向量(m: dict) -> dict:
    """记忆内容的 n-gram 向量 (预计算缓存 • id→向量 • 磁盘持久化)"""
    mid = m.get("id")
    if mid is None:
        return ngram向量(m.get("内容", ""))
    if mid not in _向量缓存:
        _向量缓存[mid] = ngram向量(m.get("内容", ""))
    if len(_向量缓存) % 50 == 0:

```

```
        _保存向量缓存() # 每 50 条落盘一次 (增量)
    return _向量缓存[mid]
```

```
def _建倒排(记忆列表: list) -> dict:
    """建 n-gram → 记忆 id 倒排索引 (磁盘缓存 • 增量自检 • 避免每次重建)
    2026-08-17 全检查修复②: 缓存与当前记忆集比对——
        一致 → 直接复用 (零开销)
        只新增 → 只补新记忆的 n-gram (增量 • 不重算旧记忆)
        有删除/元数据缺失 → 全量重建 (删除需从倒排移除 • 全量最可靠) """
    if _倒排缓存["数据"] is None:
        _加载倒排()
    当前ids = {m.get("id") for m in 记忆列表 if m.get("id") is not None}
    缓存ids = _倒排缓存.get("覆盖ids")
    if _倒排缓存["数据"] is not None and 缓存ids == 当前ids:
        _倒排缓存["记忆数"] = len(记忆列表)
        return _倒排缓存["数据"]
    if _倒排缓存["数据"] is not None and 缓存ids is not None:
        _新增ids = 当前ids - 缓存ids
        _删除ids = 缓存ids - 当前ids
        if _新增ids and not _删除ids:
            # 增量合并: 只算新记忆的 n-gram • 不动旧数据
            by_id = {m.get("id"): m for m in 记忆列表}
            for mid in _新增ids:
                m = by_id.get(mid)
                if not m:
                    continue
                for idx in _记忆向量(m).keys():
                    _倒排缓存["数据"].setdefault(idx, []).append(mid)
            _倒排缓存["覆盖ids"] = 当前ids
            _倒排缓存["记忆数"] = len(记忆列表)
            _保存倒排()
            return _倒排缓存["数据"]
        # 有删除 → 全量重建 (保证倒排不含已删除记忆)
        _倒排缓存["数据"] = None
    倒排 = {}
    for m in 记忆列表:
        mid = m.get("id")
        if mid is None:
            continue
        for idx in _记忆向量(m).keys():
            倒排.setdefault(idx, []).append(mid)
    with 加锁(): # 2026-09-14 线程保护
        _倒排缓存["数据"] = 倒排
        _倒排缓存["覆盖ids"] = 当前ids
        _倒排缓存["记忆数"] = len(记忆列表)
    _保存倒排()
    return 倒排
```

```
def 检索(query: str, 记忆列表: list, k: int = 5, 阈值: float = 0.15) -> list:
    """查询 → top-k 相关记忆 (倒排索引加速 • 父令 2026-08-16 性能优化)

    记忆列表: [{"id", "内容", "标签", "时间"}]
    返回: [{"id", "内容", "标签", "时间", "相似度"}]
    """
    qv = ngram向量(query)
    if not qv:
        return []
    _加载向量缓存() # 首次加载磁盘缓存 (避免每次重建 2.9s)
    倒排 = _建倒排(记忆列表)
```

```

# 查询命中的 n-gram → 候选记忆 id (去重)
候选 = {}
for idx in qv.keys():
    for mid in 倒排.get(idx, []):
        候选[mid] = 候选.get(mid, 0) + 1
if not 候选:
    return []
# 只对候选算余弦 (候选远小于全量 • 且向量已缓存)
by_id = {m.get("id"): m for m in 记忆列表}
结果 = []
for mid in 候选:
    m = by_id.get(mid)
    if not m:
        continue
    s = 余弦(qv, 记忆向量(m))
    if s >= 阈值:
        结果.append({**m, "相似度": round(s, 3)})
结果.sort(key=lambda x: x["相似度"], reverse=True)
return 结果[:k]

if __name__ == "__main__":
    print("=== 无模型向量层自测 ===")
    # 1. 英文词形变体 (32% 短板的病根)
    s1 = 相似度("paint the wall", "painted walls")
    print(f"① paint vs painted: {s1:.3f} (期望>0.5 • 词根共享)")
    # 2. 中文词根变体
    s2 = 相似度("配分函数", "配分")
    print(f"② 配分函数 vs 配分: {s2:.3f} (期望>0.5)")
    # 3. 无关文本
    s3 = 相似度("杀进程", "统计力学")
    print(f"③ 杀进程 vs 统计力学: {s3:.3f} (期望<0.2 • 不相关)")
    # 4. 检索
    mems = [
        {"id": 1, "内容": "electron-builder打包源含dist会嵌套30层", "标签": "技术", "时间": "2026-08-16"},
        {"id": 2, "内容": "paint the wall with two coats of primer", "标签": "工程", "时间": "2026-08-16"},
        {"id": 3, "内容": "配分函数Z=Σe^(-E/T)统计力学核心", "标签": "知识", "时间": "2026-08-16"},
        {"id": 4, "内容": "painted walls need sanding before repaint", "标签": "工程", "时间": "2026-08-15"},
        {"id": 5, "内容": "杀进程用Get-CimInstance按CommandLine区分", "标签": "经验", "时间": "2026-08-16"},
    ]
    r = 检索("how to repaint painted wall", mems, k=3)
    print(f"④ 检索 'repaint painted wall':")
    for x in r:
        print(f"    [{x['相似度']}] {x['内容'][:40]}")
    print("=== 自测完成 ===")

```

core/英文词形归一.py (109 行)

```

# -*- coding: utf-8 -*-
"""孙家记忆体系 • 英文词形归一 (父令2026-08-14 • "英文跟中文一样能存能取")
零依赖轻量词干化: 英文有词形变化 (paint/painted/service/services),
中文没有——加词形归一让英文召回达到中文同等水平。
"""

import re

# 轻量 Porter 风格词干 (规则版 • 零依赖)
_不规则 = {
    'is': 'be', 'was': 'be', 'were': 'be', 'been': 'be', 'are': 'be', 'am': 'be',

```

```

'has': 'have', 'had': 'have', 'does': 'do', 'did': 'do',
'went': 'go', 'gone': 'go', 'goes': 'go',
'made': 'make', 'makes': 'make', 'said': 'say', 'says': 'say',
'told': 'tell', 'tells': 'tell', 'saw': 'see', 'seen': 'see', 'sees': 'see',
'came': 'come', 'comes': 'come', 'gave': 'give', 'gives': 'give',
'took': 'take', 'takes': 'take', 'got': 'get', 'gets': 'get',
'found': 'find', 'finds': 'find', 'knew': 'know', 'knows': 'know',
'thought': 'think', 'thinks': 'think', 'bought': 'buy', 'buys': 'buy',
'brought': 'bring', 'brings': 'bring', 'wrote': 'write', 'writes': 'write',
'read': 'read', 'reads': 'read', 'ran': 'run', 'runs': 'run',
'paid': 'pay', 'pays': 'pay', 'met': 'meet', 'meets': 'meet',
'lost': 'lose', 'loses': 'lose', 'sent': 'send', 'sends': 'send',
'built': 'build', 'builds': 'build', 'held': 'hold', 'holds': 'hold',
'left': 'leave', 'leaves': 'leave', 'felt': 'feel', 'feels': 'feel',
'kept': 'keep', 'keeps': 'keep', 'slept': 'sleep', 'sleeps': 'sleep',
'spoke': 'speak', 'speaks': 'speak', 'broke': 'break', 'breaks': 'break',
'chose': 'choose', 'chooses': 'choose', 'drove': 'drive', 'drives': 'drive',
'ate': 'eat', 'eats': 'eat', 'fell': 'fall', 'falls': 'fall',
'women': 'woman', 'men': 'man', 'children': 'child', 'people': 'person',
'mice': 'mouse', 'feet': 'foot', 'teeth': 'tooth', 'geese': 'goose',
}

```

```

def 词干(word: str) -> str:
    """轻量词干化: 复数/过去式/ing/比较级 归一到基础形式"""
    w = word.lower()
    if w in _不规则:
        return _不规则[w]
    if len(w) <= 3:
        return w
    # 复数 -ies -> -y (cities->city)
    if w.endswith('ies') and len(w) > 4:
        return w[:-3] + 'y'
    # 复数 -es -> 去es (boxes->box, watches->watch)
    if w.endswith('es'):
        # 保持 x/ch/sh/s 后的 e (boxes->box)
        stem = w[:-2]
        if stem.endswith(('x', 'ch', 'sh', 's', 'z')):
            return stem
        return w[:-1] # 其他 -es 只去s (makes->make)
    # 复数 -s -> 去s (services->service)
    if w.endswith('s') and not w.endswith('ss') and len(w) > 4:
        return w[:-1]
    # 过去式 -ied -> -y (studied->study)
    if w.endswith('ied') and len(w) > 4:
        return w[:-3] + 'y'
    # 过去式 -ed -> 去ed (painted->paint)
    if w.endswith('ed') and len(w) > 4:
        stem = w[:-2]
        # 双写辅音还原 (stopped->stop)
        if len(stem) >= 2 and stem[-1] == stem[-2]:
            return stem[:-1]
        return stem
    # 进行时 -ing -> 去ing (painting->paint)
    if w.endswith('ing') and len(w) > 5:
        stem = w[:-3]
        # 双写辅音还原 (running->run)
        if len(stem) >= 2 and stem[-1] == stem[-2]:
            return stem[:-1]
        # -ying -> -y (studying->study)
        if stem.endswith('y'):
            return stem

```

```

    # 去 e (making→make, having→have)
    if w.endswith('king') or w.endswith('ving') or w.endswith('ming') or w.endswith('ning'):
        return stem + 'e'
    return stem
# 比较级/最高级 -er/-est (bigger→big, happier→happy)
if w.endswith('er') and len(w) > 4:
    stem = w[:-2]
    if len(stem) >= 2 and stem[-1] == stem[-2]:
        return stem[:-1]
    if stem.endswith('i'):
        return stem[:-1] + 'y' # happier→happi→happy
    return stem
if w.endswith('est') and len(w) > 5:
    stem = w[:-3]
    if len(stem) >= 2 and stem[-1] == stem[-2]:
        return stem[:-1]
    if stem.endswith('i'):
        return stem[:-1] + 'y' # happiest→happi→happy
    return stem
return w

def 归一文本(text: str) -> str:
    """整段文本词形归一：每个英文词转词干"""
    def _repl(m):
        return 词干(m.group(0))
    return re.sub(r'[a-zA-Z]+', _repl, text)

if __name__ == '__main__':
    测试 = ['paint', 'painted', 'painting', 'service', 'services', 'fix', 'fixed',
            'go', 'went', 'gone', 'car', 'cars', 'study', 'studied', 'studying',
            'run', 'running', 'make', 'making', 'box', 'boxes', 'city', 'cities',
            'happy', 'happier', 'happiest']
    print('词干化测试:')
    for t in 测试:
        print(f' {t} → {词干(t)}')
    print()
    print('句子归一:')
    print(归一文本('Caroline went to the LGBTQ support group, she painted a beautiful sunrise'))

```

core/相似度方程.py (117 行)

```

# -*- coding: utf-8 -*-
"""相似度方程（父令2026-08-26 • 模块精简数学化 • 第2条方程）

将 内容去重(bigram Jaccard) / 无模型向量(ngram余弦) / 英文词形归一(词干)
三模块的核心相似度逻辑统一成一条加权方程：


$$\text{Sim}(A,B) = \alpha \cdot \text{Jaccard\_bigram}(\text{无序}) + \beta \cdot \text{Jaccard\_ordered}(\text{有序}) + \gamma \cdot \text{余弦\_词干归一}$$


 $\alpha=0.4$  /  $\beta=0.3$  /  $\gamma=0.3$ （可调 • 默认给字符级结构最高权重）

原三模块保留兼容转发（调此方程） • 核心逻辑收敛到此。
"""
import re
import math
from collections import Counter

# 参数（2026-08-26 产品化：改为读配置表 • 不改源码可调优）

```

```

try:
    from 配置表 import get as _cfg
    def _P(键, 默认):
        return _cfg('相似度', 键) if _cfg('相似度', 键) is not None else 默认
except Exception:
    def _P(键, 默认):
        return 默认

参数 = {
    'α_bigram': _P('α_bigram', 0.4),      # 无序 bigram Jaccard (内容去重主)
    'β_有序': _P('β_有序', 0.3),          # 有序 bigram 序列重叠 (内容去重通道②)
    'γ_余弦': _P('γ_余弦', 0.3),          # 词干归一 ngram 余弦 (无模型向量+英文归一)
    '去重阈值': _P('去重阈值', 0.6),      # 内容去重阈值 (≥0.6 判重复)
}

def 归一化(文本: str) -> str:
    """归一化: 去标点/空白/统一小写 (原内容去重, 归一化) """
    if not 文本:
        return ""
    s = str(文本)
    s = re.sub(r'[\s, 。 ! ? \ , ; : ""'' () 【】 《》 …— • , . ! ? ; : "\' () \[ \] {} < > - ] +', '', s)
    return s.lower()

def _bigrams(文本: str) -> set:
    """无序 bigram 集合 (原内容去重, _bigrams) """
    s = 归一化(文本)
    if len(s) < 2:
        return set()
    return {s[i:i+2] for i in range(len(s)-1)}

def _ordered_bigrams(文本: str) -> list:
    """有序 bigram 序列 (原内容去重, _ordered_bigrams) """
    s = 归一化(文本)
    if len(s) < 2:
        return []
    return [s[i:i+2] for i in range(len(s)-1)]

def _ngram向量(文本: str, n=2):
    """ngram 稀疏向量 (原无模型向量, ngram向量 • 简化版) """
    s = 归一化(文本)
    if len(s) < n:
        return {}
    cnt = Counter(s[i:i+n] for i in range(len(s)-n+1))
    return dict(cnt)

def 余弦(v1: dict, v2: dict) -> float:
    """稀疏向量余弦 (原无模型向量, 余弦) """
    if not v1 or not v2:
        return 0.0
    内积 = sum(w * v2.get(idx, 0) for idx, w in v1.items())
    if 内积 == 0:
        return 0.0
    模1 = sum(w * w for w in v1.values()) ** 0.5
    模2 = sum(w * w for w in v2.values()) ** 0.5
    if not 模1 or not 模2:
        return 0.0

```

```

return 内积 / (模1 * 模2)

def _词干(词: str) -> str:
    """词干化（原英文词形归一. 词干·简化版）：英文复数/ing/ed 还原"""
    if not 词 or not 词.isascii():
        return 词
    w = 词.lower()
    for suf in ("ing", "ed", "es", "s"):
        if len(w) > 4 and w.endswith(suf):
            return w[:-len(suf)]
    return w

def 相似度(文本a: str, 文本b: str) -> float:
    """相似度方程（统一）：Sim = α · bigram + β · 有序 + γ · 余弦(词干归一)"""
    if not 文本a or not 文本b:
        return 0.0
    # ① 无序 bigram Jaccard
    ba, bb = _bigrams(文本a), _bigrams(文本b)
    jaccard = len(ba & bb) / len(ba | bb) if (ba and bb) else 0.0
    # ② 有序 bigram 序列重叠
    oa, ob = _ordered_bigrams(文本a), _ordered_bigrams(文本b)
    pa = {(oa[i], oa[i+1]) for i in range(len(oa)-1)} if len(oa) > 1 else set()
    pb = {(ob[i], ob[i+1]) for i in range(len(ob)-1)} if len(ob) > 1 else set()
    有序 = len(pa & pb) / len(pa | pb) if (pa and pb) else 0.0
    # ③ 词干归一 ngram 余弦（英文词形归一 + 无模型向量）
    try:
        va = _ngram向量(文本a)
        vb = _ngram向量(文本b)
        cos = 余弦(va, vb)
    except Exception:
        cos = 0.0
    return 参数['α_bigram'] * jaccard + 参数['β_有序'] * 有序 + 参数['γ_余弦'] * cos

def 是重复(文本a: str, 文本b: str) -> bool:
    """去重判定（原内容去重·阈值）"""
    return 相似度(文本a, 文本b) >= 参数['去重阈值']

```

core/活性方程.py（136 行）

```

# -*- coding: utf-8 -*-
"""活性方程（父令2026-08-26·模块精简数学化）

将 节律(heat脉冲) / 时间衰减(cover) / 记忆成绩单(账本) 三模块的核心逻辑
统一成一条活性方程：

    活性(t) = 基础 + Σ(点亮 · e−λ · Δt) + Σ(确认 · e−λ · Δt) − Σ(纠正 · e−λ · Δt)
              + 时间新鲜度(cover)

三个来源（点亮/账本/时间）· 统一衰减 λ · 单库单表。

【2026-09-07 诚实修正（安弟1589体检🔴5·大哥复查）】
“核心逻辑收敛到此”是 8/26 的过度宣称——节律/成绩单并未真调此方程，
真实使用：联想召回.py 延迟 import 本模块的 活性() 计算召回融合分（heat继承+点亮脉冲+账本 · λ 衰减）。
活性主机制仍以 节律.py 为准（真实在跑回落/点亮/热度排行）。
"""

```

```

import sqlite3
import os
import datetime
import math

DB = os.environ.get('SUNMEM_DB', os.path.join(os.path.dirname(os.path.abspath(__file__)), 'data', 'sunmem.db'))

# 参数 (2026-08-26 产品化: 改为读配置表 • 不改源码可调优)
try:
    from 配置表 import get as _cfg
    def _P(键, 默认):
        return _cfg('活性', 键) if _cfg('活性', 键) is not None else 默认
except Exception:
    def _P(键, 默认):
        return 默认

参数 = {
    '点亮增量': _P('点亮增量', 0.3),      # 节律. 点亮增量
    '热上限': _P('热上限', 5.0),          # 节律. 热上限
    '衰减λ': _P('衰减λ', 0.10),          # 节律. 普通衰减 (每天)
    '保护衰减λ': _P('保护衰减λ', 0.02),   # 节律. 保护衰减 (慢5倍 • core/父令)
    '保护地板': _P('保护地板', 0.5),      # 节律. 保护地板
    '确认权重': _P('确认权重', 2.0),      # 成绩单. 确认+2
    '纠正权重': _P('纠正权重', -2.0),     # 成绩单. 纠正-2
    '点亮权重': _P('点亮权重', 1.0),     # 成绩单. 点亮+1
    '预算': 20,                          # 时间衰减. cover预算
}

def _连接():
    conn = sqlite3.connect(DB, timeout=2.0)
    try:
        conn.execute("PRAGMA query_only=ON")
    except Exception:
        pass
    conn.row_factory = sqlite3.Row
    return conn

def 活性(memory_id: int, 天数=30) -> float:
    """方程主函数: 算一条记忆的当前活性。
    活性 = 基础(heat继承) + 点亮脉冲(带λ衰减) + 账本(确认/纠正带λ衰减)
    2026-08-26 产品化: 错误降级嵌入方程——任何环节异常返回 0.0 (不炸调用方)
    """
    try:
        conn = _连接()
        row = conn.execute(
            "SELECT heat, hit_count, layer, tags FROM memories WHERE id=?",
            (memory_id,)).fetchone()
        conn.close()
        if not row:
            return 0.0
        heat = row['heat'] or 0.0
        hit = row['hit_count'] or 0
        # 点亮贡献 (历史脉冲 • 按衰减λ折算)
        点亮贡献 = min(参数['热上限'], hit * 参数['点亮权重'] * math.exp(-参数['衰减λ'] * 天数))
        # 账本贡献 (确认/纠正 • 从成绩单表取)
        账本 = 查账本(memory_id)
        账本贡献 = (账本['确认'] * 参数['确认权重'] + 账本['纠正'] * 参数['纠正权重']) * math.exp(-参数['衰减λ'] * 天数)
        return heat + 点亮贡献 + 账本贡献
    except Exception:

```

```
return 0.0
```

```
def 查账本(memory_id: int) -> dict:
    """成绩单账本（原记忆成绩单, 查成绩单）"""
    conn = _连接()
    try:
        row = conn.execute(
            "SELECT COUNT(*) as n, SUM(CASE WHEN action='确认' THEN 1 ELSE 0 END) as 确认, "
            "SUM(CASE WHEN action='纠正' THEN 1 ELSE 0 END) as 纠正, "
            "SUM(CASE WHEN action='点亮' THEN 1 ELSE 0 END) as 点亮 "
            "FROM memory_report_cards WHERE memory_id=?", (memory_id,)).fetchone()
        return {"确认": row['确认'] or 0, "纠正": row['纠正'] or 0, "点亮": row['点亮'] or 0, "总数": row['n'] or 0}
    except Exception:
        return {"确认": 0, "纠正": 0, "点亮": 0, "总数": 0}
    finally:
        conn.close()
```

```
def 覆盖(条目列表, 预算=20) -> list:
    """时间覆盖（原时间衰减, cover • 近细远粗 • 连续衰减）"""
    if not 条目列表:
        return []
    budget = 预算
    最近 = 条目列表[-budget // 2:]          # 近半逐条全取
    更老 = 条目列表[:-budget // 2]
    # 更老的部分：从新到老指数间隔采样
    picked = list(最近)
    if 更老:
        step = 1
        idx = len(更老) - 1
        while idx >= 0 and len(picked) < budget:
            picked.insert(0, 更老[idx])
            idx -= step
            step *= 2
    return picked
```

```
def 排序(条目列表, 预算=20, 活跃优先=True) -> list:
    """统一召回排序（原联想召回排序 + 时间衰减 + 活性）：
    Score =  $\alpha \cdot \text{概念} + \beta \cdot \text{时间新鲜度} + \gamma \cdot \text{活性方程值}$ 
    """
    now = datetime.datetime.now()
    scored = []
    for e in 条目列表:
        # 时间新鲜度（近的分加）
        时间分 = 0.0
        t = e.get("时间", "")
        if isinstance(t, str) and t:
            try:
                dt = datetime.datetime.fromisoformat(t[:19])
                时间分 = math.exp(-(now - dt).days / 30.0) # 30天半衰
            except Exception:
                pass
        # 活性方程值
        活 = 活性(e.get("id", 0)) if e.get("id") else 0.0
        scored.append((e, 时间分 + 活))
    scored.sort(key=lambda x: x[1], reverse=True)
    return [e for e, _ in scored[:预算]]
```

core/配置表.py (144 行)

```
# -*- coding: utf-8 -*-
"""记忆体配置表（父令2026-08-26・产品化：方程参数全部外置・不改源码可调优）

三大方程 + 5大集合的全部参数收敛于此：
  活性方程：λ 衰减/点亮增量/热上限/账本权重
  相似度方程：α / β / γ 权重/去重阈值
  召回方程：融合权重/重要加分/专名加分
  整理方程：骨架化阈值/小时叙事门槛
  写入准入：去重阈值/进化重叠阈值/关键内容标记词

外部可配置：config.yaml（零依赖解析・不改源码调参）
"""

import os
import re
from pathlib import Path

_HERE = Path(__file__).resolve().parent
_默认配置 = {
    # —— 活性方程（原节律/时间衰减/成绩单）——
    "活性": {
        "点亮增量": 0.3,          # 点亮一次 heat 上升量
        "热上限": 5.0,           # heat 上限
        "衰减λ": 0.10,           # 普通衰减/天
        "保护衰减λ": 0.02,       # 保护记忆衰减/天（慢5倍）
        "保护地板": 0.5,         # 保护记忆 heat 地板
        "确认权重": 2.0,         # 成绩单确认 +2
        "纠正权重": -2.0,        # 成绩单纠正 -2
        "点亮权重": 1.0,         # 成绩单点亮 +1
    },
    # —— 相似度方程（原内容去重/无模型向量/英文归一）——
    "相似度": {
        "α_bigram": 0.4,         # 无序 bigram Jaccard 权重
        "β_有序": 0.3,           # 有序 bigram 权重
        "γ_余弦": 0.3,           # 词干归一余弦权重
        "去重阈值": 0.6,         # 判重复阈值
    },
    # —— 召回方程（原联想召回排序）——
    "召回": {
        "向量权重": 20.0,        # 1余弦点≈20概念分
        "重要加分": 10.0,        # 父令/被点出加分
        "专名加分": 10.0,        # 专有名词命中加分
        "字面命中": 3.0,         # 字面命中保底
        "时间半衰": 30.0,        # 时间新鲜度半衰期（天）
    },
    # —— 整理方程（原自动整理/自组织/压缩）——
    "整理": {
        "小时叙事门槛": 3,       # 少于N条的小时不单独成叙事
        "整理间隔轮次": 30,      # 每N轮触发整理
        "整理间隔分钟": 60,      # 或每N分钟触发
        "骨架阈值天": 30,        # 30天以上可骨架化
        "进化重叠阈值": 0.85,    # 同事实重叠≥此值→标outdated
    },
    # —— 写入准入 ——
    "写入": {
        "关键标记词": ["父令", "父亲令", "记住", "教训", "重要", "必须", "决定", "命令", "禁止", "铁律"],
        "内容上限": 300,         # 实时抓取内容截断
        "缓冲容量": 200,        # 短时缓冲 detail 上限
    },
}
```

```

}

# 全局配置（启动时加载 config.yaml 覆盖默认）
_配置 = dict(_默认配置)
_config_path = _HERE / "config.yaml"

def _加载配置():
    """零依赖 YAML 解析（支持 键: 值 和 缩进分组）"""
    global _配置
    if not _config_path.exists():
        return
    try:
        当前组 = None
        for line in _config_path.read_text(encoding="utf-8").splitlines():
            s = line.strip()
            if not s or s.startswith("#"):
                continue
            # 分组（无冒号值 • 如 “活性:”）
            m = re.match(r'^([\w\u4e00-\u9fff]+):\s*$', s)
            if m:
                当前组 = m.group(1)
                if 当前组 in _配置:
                    continue
            # 键值对（如 “ 点亮增量: 0.3”）
            m2 = re.match(r'^([\w\u4e00-\u9fff]+):\s*(.+)$', s)
            if m2 and 当前组 and 当前组 in _配置:
                键, 值 = m2.group(1), m2.group(2).strip()
                try:
                    _配置[当前组][键] = float(值) if "." in 值 else int(值)
                except ValueError:
                    if 值.startswith("[") and 值.endswith("]"):
                        _配置[当前组][键] = [x.strip().strip("\"'") for x in 值[1:-1].split(",")]
                    else:
                        _配置[当前组][键] = 值
        except Exception:
            pass

def get(组: str, 键: str):
    """读配置参数（产品化: 外部 config.yaml 可覆盖 • 不改源码）"""
    return _配置.get(组, {}).get(键)

def 全部() -> dict:
    return dict(_配置)

def 写默认配置(路径=None):
    """生成默认 config.yaml（首次部署/导出用）"""
    p = Path(路径) if 路径 else _config_path
    lines = ["# 孙家记忆体 • 配置表（父令2026-08-26 • 参数外置）", "# 改这里不用改源码 • 重启网关生效", ""]
    for 组, kv in _默认配置.items():
        lines.append(f"{组}:")
        for 键, 值 in kv.items():
            if isinstance(值, list):
                lines.append(f"  {键}: {值}")
            elif isinstance(值, float):
                lines.append(f"  {键}: {值:g}")
            else:
                lines.append(f"  {键}: {值}")

```

```

        lines.append("")
    p.write_text("\n".join(lines), encoding="utf-8")
    return str(p)

```

`_加载配置()`

```

def 参数总览() -> str:
    """参数总表（2026-09-14 吸收收束版优点：一眼看全）"""
    import json as _j
    行 = []
    try:
        _fn = next((f for f in ('加载配置', '读配置', '配置', 'get_config') if f in dir()), None)
        参 = globals()[_fn]() if _fn else _默认配置
    except Exception:
        参 = _默认配置
    for 组, 项 in (参 or _默认配置).items():
        行.append(f'    【{组}】')
        if isinstance(项, dict):
            for k, v in 项.items():
                行.append(f'        {k} = {v}')
        else:
            行.append(f'    {项}')
    return '\n'.join(行)

```

core/激活.py (97 行)

```

# -*- coding: utf-8 -*-
"""激活方程 A • 统一入口（2026-08-28 收 A • 阶段3第一步）

=====

父令：把 点亮(精确种子) + 感应器(全网传播) + 联想(概念提取) 合成一条线，
provider.prefetch 与 感知注入.inject 只调 activate()，不再各查各的。

本文件是第一步：统一入口 + 概念波能→记忆条映射。
后续第二步：provider/inject 迁移到 activate；第三步：联想/预感降级为种子/prior。
"""

import sys, os
sys.path.insert(0, os.path.dirname(os.path.abspath(__file__)))

def activate(context, mode='turn', owner='孙呈', 预算=8):
    """激活唯一入口：种子 → 传播 → 激活集。

    返回 ActivationSet:
        {"记忆": [{id, 内容, 标签, 激活度, 来源}], "概念": {概念: 波能}}

    mode: turn=对话轮次 / think=思考中途 / wake=醒来
    """
    from 感应器 import 感应器 as _感应器, _加载网

    # —— 1. 感应器发波（全网扰动 • 概念链路） ——
    概念波能 = {}
    try:
        概念波能 = _感应器(context[:20]) if context and len(context) >= 2 else {}
    except Exception:
        概念波能 = {}

```

—— 2. 点亮（精确命中 • 记忆条种子 • layer=core 优先） ——

记忆激活 = {} # id -> {内容, 标签, 激活度, 来源}

try:

from 点亮记忆 import 点亮 as _点亮

点亮结果 = _点亮(context, owner)

for h in 点亮结果.get("点亮记忆", []):

mid = h.get("id")

if mid is None:

continue

记忆激活[mid] = {

"id": mid, "内容": h.get("内容", ""), "标签": h.get("标签", ""),

"激活度": 1.0, "来源": "点亮",

}

except Exception:

pass

—— 3. 概念波能 → 记忆条（通过段落节点 contains 边 • 波能高才映射） ——

try:

d = _加载网()[0]

段落 = d.get("段落", {})

for 概念, 波能 in 概念波能.items():

if 波能 < 0.3 or 概念 == '夏维斯':

continue

for pid, p in 段落.items():

if 概念 not in p.get("概念", []):

continue

try:

mid = int(pid)

except Exception:

continue

激活 = round(min(1.0, 波能), 3) # 感应器映射: 激活度=波能 (关联强度 • 钳到[0,1])

if mid not in 记忆激活 or 记忆激活[mid]["激活度"] < 激活:

记忆激活[mid] = {

"id": mid, "内容": p.get("内容", ""), "标签": p.get("标签", ""),

"激活度": 激活, "来源": "感应器",

}

except Exception:

pass

—— 4. 排序截断（激活度降序 • 预算 N 条） ——

记忆列表 = sorted(记忆激活.values(), key=lambda x: -x["激活度"])[: 预算]

—— 5. prior（预感召回 • 接续/未竟 → 先验 • 非独立检索） ——

prior = {}

try:

from 预感召回 import 预感召回 as _预感, 格式化提示词 as _格式化

预感 = _预感(owner, context)

if 预感:

prior["预感"] = _格式化(预感)

except Exception:

pass

—— 感知诚实（2026-08-28 收A: 空集合合法 • 种子空则下游禁灌） ——

感知为空（无概念波能 + 无精确命中） → 种子空=True → 下游只留 core 地板，不灌最近条目冒充相关

种子空 = (not 概念波能) and (not 记忆激活)

return {"记忆": 记忆列表, "概念": 概念波能, "prior": prior, "种子空": 种子空}

if __name__ == "__main__":

import json

```

for q in ["配分函数", "苏醒循环", "孙博"]:
    r = activate(q)
    print(f"\n===== activate 「{q}」 =====")
    print(f"概念: {len(r['概念'])} 个 • 记忆: {len(r['记忆'])} 条")
    for m in r["记忆"][:5]:
        print(f"    [{m['来源']}]|[{m['激活度']}] [{m['内容']}[:40]]")

```

core/线程保护.py (80 行)

```

# -*- coding: utf-8 -*-
"""线程保护 • 全局状态统一锁 (2026-09-14 吸收收束版优点)
=====

来源: 外部 AI 《统一收束版》的“统一连接管理 (线程安全)”思路
背景: 外部审查 🟡1 —— 本库 11 个模块级可变缓存 ( _索引缓存/ _拉边累积/ _核心关键词缓存/
_薄弱点缓存/ _缓存网/ _向量缓存/... ) 无锁保护。provider 的 queue_prefetch 走后台线程,
与主线程同时操作这些全局状态 → 竞态风险。

策略 (最小改动 • 不重构业务流程):
• 提供一把全局可重入锁 (RLock): 同线程可重复获取 (防自锁)
• 高风险的“缓存读写/攒批计数”段落用 同步() 包装或用 with 加锁()
• 不改变任何业务逻辑 • 只保证并发下缓存不被撕裂

用法:
    from 线程保护 import 加锁, 同步
    with 加锁():
        ... 读改写缓存...

    @同步
    def 我的函数(): ...
"""
import threading
from contextlib import contextmanager

# 全局可重入锁 (同一线程内可重复获取 • 防自锁)
_全局锁 = threading.RLock()

def 加锁():
    """返回全局锁 (配合 with 使用)"""
    return _全局锁

@contextmanager
def 临界区():
    """with 临界区(): ... —— 语义同 加锁()"""
    with _全局锁:
        yield

def 同步(fn):
    """装饰器: 整个函数在锁内执行"""
    def wrapper(*args, **kwargs):
        with _全局锁:
            return fn(*args, **kwargs)
    wrapper.__name__ = getattr(fn, '__name__', 'wrapped')
    wrapper.__doc__ = getattr(fn, '__doc__', None)
    return wrapper

```

```

def 状态() -> dict:
    """自检: 锁是否可用"""
    return {'锁': type(_全局锁).__name__, '可重入': True, '已锁': _全局锁._is_owned() if hasattr(_全局锁, '_is_owned') else None}

if __name__ == '__main__':
    # 自检: 重入 + 并发计数
    import time
    print('—— 线程保护 • 自检 ——')
    print(' ① 重入:', end=' ')
    with 加锁():
        with 加锁():
            print('OK (同线程两次加锁不卡)')

    counter = {'n': 0}

    @同步
    def 累加():
        v = counter['n']
        time.sleep(0.0005)          # 故意放大竞态窗口
        counter['n'] = v + 1

    import threading as _t
    ts = [_t.Thread(target=累加) for _ in range(50)]
    for t in ts: t.start()
    for t in ts: t.join()
    print(f' ② 并发 50 线程累加 (有锁): {counter["n"]} (应=50)')
    assert counter['n'] == 50, '锁失效!'
    print(' ③ 状态:', 状态())
    print('—— 自检通过  ——')

```

五、身份与经验

core/经验总结.py (362 行)

```
# -*- coding: utf-8 -*-
"""
孙家记忆体系 · 经验总结层
"""

父令（2026-08-07）：每一次做了什么事、用了什么工具、怎么做，
记忆体里都有经验总结——轮到这个工具时怎么做、吃了什么亏、经验教训。
被父亲点出的问题 → 专门记一条经验总结句。
像框架一样越用越聪明。

功能：
    save_experience()  写经验条目（事项/工具 + 做法 + 结果 + 教训 + 被点出）
    recall_experience() 经验召回（当前事项/工具 → 相关经验浮出 · 教训优先 · 2条）
"""

import json
import os
from datetime import datetime
from pathlib import Path

# 自家模块
_HERE = Path(__file__).resolve().parent
FRAMEWORK_DIR = _HERE.parent.parent # 孙家记忆体系/
# 2026-08-10 明文化（父令）：码点已退役——不再 import 码点编解码，记忆体直接明文

EXPERIENCE_TAG = "经验总结"
MAX_RECALL = 2 # 父令定：两条就够了

def _memory_path(brother_name: str = "孙呈") -> Path:
    return FRAMEWORK_DIR / "记忆体" / f"{brother_name}_索引记忆体.json"

def _load_memory(brother_name: str = "孙呈") -> dict:
    p = _memory_path(brother_name)
    if p.exists():
        with open(p, "r", encoding="utf-8") as f:
            return json.load(f)
    return {"条目列表": [], "next_id": 1, "元信息": {}}

def _save_memory(data: dict, brother_name: str = "孙呈") -> None:
    data["元信息"] = {"最后更新": datetime.now().strftime("%Y-%m-%d %H:%M:%S"),
                     "记录条数": len(data["条目列表"])}
    # 2026-08-14 闭环修复（父令 • 回头检查发现）：经验总结只写JSON没走sunmem.db新库
    # → provider 召回读新库读不到经验。加双写：JSON + sunmem.db 都写
    # 2026-08-17 训练信号闭环修复：先同步 DB（取 lastrowid 记 e["sunmem_id"] • 标记 _已同步）
    # 再落盘 JSON——原顺序 JSON 先写 • _已同步 只在内存 • 下次 save 旧经验会重复 INSERT（账本放大）
    try:
        import sqlite3
        _db = os.environ.get("SUNMEM_DB", os.path.join(os.path.dirname(os.path.abspath(__file__)), 'data', 'sunmem.db'))
        _conn = sqlite3.connect(_db)
```

```

_now = datetime.now().strftime("%Y-%m-%d %H:%M:%S")
# 增量写：只写新增的经验条目（类型=经验总结•最新一条）
新增 = [e for e in data.get("条目列表", []) if e.get("类型") == "经验总结" and e.get("_已同步") != True]
for e in 新增:
    _cur = _conn.execute(
        "INSERT INTO memories (owner, type, content, tags, ts) VALUES (?, ?, ?, ?, ?)",
        (brother_name, "experience", str(e.get("内容", "")), str(e.get("标签", "经验总结")), _now)
    )
    e["_已同步"] = True
    e["sunmem_id"] = _cur.lastrowid # 2026-08-17: JSON id 与 DB id 错位•成绩单/召回都以 DB id 为准
_conn.commit()
_conn.close()
except Exception:
    pass
with open(_memory_path(brother_name), "w", encoding="utf-8") as f:
    json.dump(data, f, ensure_ascii=False, indent=2)

```

```

def save_experience(item: str, approach: str = "", result: str = "",
    lesson: str = "", pointed_by_father: bool = False,
    brother_name: str = "孙呈", evidence=None,
    signal: str = "", confidence: float = None,
    verdict: str = "") -> int:
    """写一条经验总结进记忆体。

```

参数:

```

item: 事项/工具（如 "taskkill MSYS转义坑"）
approach: 怎么做（正确做法）
result: 结果
lesson: 经验教训
pointed_by_father: 是否被父亲点出（教训条目→召回优先）
evidence: list[int] —— 2026-08-07取长补短：源记忆 id 列表
    (Mem0 Dream•Synthesis 理念：洞察可追溯到证据)
signal: str —— 2026-08-08 PlugMem 吸收：晋升信号
    failure_delta(失败→成功) / correction(用户纠正)
    / confirmed(确认完成) / repeated(重复≥3次) / explicit(显式记住)
confidence: float —— 2026-08-08 PlugMem 吸收：可信度分级
    0.9+ 明示规则 / 0.7-0.8 清晰验证 / 0.5-0.6 模糊 / <0.5 不存
verdict: str —— 2026-08-16 经验复用合并（父令）：好用/吃瘪 判定
    "positive"（好用•下次照做）/ "negative"（吃瘪•下次改道）
    / ""（自动推断：lesson含"坑/失败/别/不"→negative）

```

返回：新条目 id（confidence<0.5 时不存•返回 None）

"""

```

# PlugMem 吸收①：confidence 分级门——<0.5 不存（坏记忆主动伤害）
if confidence is not None and confidence < 0.5:
    return None
# PlugMem 吸收③：五信号晋升门——默认不写，要求显式信号
# （signal 为空但有 pointed_by_father → correction 信号）
if signal == "" and not pointed_by_father:
    signal = "explicit"
_VALID_SIGNALS = {"failure_delta", "correction", "confirmed", "repeated", "explicit"}
if signal not in _VALID_SIGNALS:
    signal = "explicit"
# 2026-08-16 经验复用：verdict 自动推断（吃瘪词表）
# 关键：negative 必须是“明确改道指令”——lesson含 别/不要/禁止/失败/错误 这类避开词
# 仅含 坑/教训/注意（提醒类）→ positive（做法可照做•但带警示）
if not verdict:
    _AVOID_WORDS = ["不要", "别用", "禁止", "千万别", "不许", "不能用", "失败", "错误", "不行", "无效"]
    _WARN_WORDS = ["坑", "教训", "注意", "小心", "会", "导致"]
    text = lesson + result + approach
    if any(w in text for w in _AVOID_WORDS):

```

```

        verdict = "negative"
    else:
        verdict = "positive"
if verdict not in ("positive", "negative"):
    verdict = "positive"

data = _load_memory(brother_name)
entries = data["条目列表"]
new_id = data.get("next_id", len(entries) + 1)

# 经验描述（浓缩一行）
parts = [f"事项/工具: {item}"]
if approach:
    parts.append(f"做法: {approach}")
if result:
    parts.append(f"结果: {result}")
if lesson:
    parts.append(f"教训: {lesson}")
if pointed_by_father:
    parts.append("【被父亲点出】")
if verdict == "negative":
    parts.append("【吃瘪・下次改道】")
else:
    parts.append("【好用・下次照做】")
desc = "; ".join(parts)

tags_text = "经验总结" + ("・被点出" if pointed_by_father else "") + ("・吃瘪" if verdict == "negative" else "・好用")
# 2026-08-10 明文化（父令）：直接存明文・不再码点编码
entry = {
    "id": new_id,
    "时间": datetime.now().strftime("%Y-%m-%d %H:%M:%S"),
    "标签": tags_text,
    "内容": desc,
    "类型": "经验总结",
    "判定": verdict, # 2026-08-16 经验复用：好用/吃瘪
}

if evidence: # 2026-08-07取长补短：证据链接（源记忆id）
    entry["证据"] = [int(x) for x in evidence if isinstance(x, (int, str))]
# 2026-08-08 PlugMem 吸收：晋升信号 + 可信度
entry["信号"] = signal
if confidence is not None:
    entry["置信度"] = round(float(confidence), 2)
else:
    # 默认置信度：被点出=0.9 明示规则；否则按信号给
    entry["置信度"] = 0.9 if pointed_by_father else 0.7
entries.append(entry)
data["next_id"] = new_id + 1
_save_memory(data, brother_name)
# 2026-08-17 训练信号闭环（父令「接上吧」）：经验判定 → 成绩单确认/纠正
# 照做=强化（positive・好用）・改道=惩罚（negative・吃瘪）——训练信号长进记忆体闭环
# memory_id 用 sunmem.db 的 id（成绩单/召回以 DB id 为准・JSON id 是镜像错位）
try:
    from 记忆成绩单 import 记一笔
    _mid = entry.get("sunmem_id")
    if not _mid:
        import sqlite3 as _sq
        _db = os.environ.get("SUNMEM_DB", os.path.join(os.path.dirname(os.path.abspath(__file__)), 'data',
'sunmem.db'))
        _c = _sq.connect(_db)
        _r = _c.execute(
            "SELECT id FROM memories WHERE owner=? AND type='experience' AND content=? ORDER BY id DESC LIMIT 1",

```

```

        (brother_name, desc)).fetchone()
    _c.close()
    _mid = _r[0] if _r else None
    if _mid:
        记一笔(_mid, "确认" if verdict == "positive" else "纠正", owner=brother_name,
              note=f"经验总结 • {'好用' if verdict == 'positive' else '吃瘪'}")
    except Exception:
        pass
    return new_id

```

```

def recall_experience(context: str = "", brother_name: str = "孙呈",
                    limit: int = MAX_RECALL) -> list[dict]:
    """经验召回：按当前事项/工具关键词找相关经验。

```

规则：

1. 只从“类型=经验总结”的条目里找
2. 关键词匹配：事项/工具/做法/教训 里含上下文关键词（或上下文含条目关键词）
3. 教训优先：被父亲点出的排最前
4. 返回 limit 条（父令定：两条就够了）

"""

```

data = _load_memory(brother_name)
entries = data["条目列表"]

```

```

experiences = []
for e in entries:
    if e.get("类型") != "经验总结":
        continue
    from sun_memory.core.记忆库 import 解码单条
    d = 解码单条(e)
    pointed = "被点出" in d["标签"]
    # 2026-08-08 PlugMem 吸收①：置信度参与召回
    conf = e.get("置信度", 0.7)
    experiences.append({"id": e.get("id"), "时间": e.get("时间", ""),
                       "内容": d["内容"], "被点出": pointed,
                       "置信度": float(conf)})

```

```

if not experiences:
    return []

```

关键词匹配

```

if context:
    ctx = context.strip()
    hits = []
    for x in experiences:
        if ctx in x["内容"] or any(k in x["内容"] for k in ctx.split()):
            hits.append(x)
    experiences = hits if hits else experiences # 无命中→返回最新（诚实）

```

排序：被点出最前 → 置信度高者次之 → 同组时间新者在前

（2026-08-08 PlugMem 吸收①：高置信记忆优先浮出）

```

experiences.sort(key=lambda x: (x["被点出"], x["置信度"], x["时间"]), reverse=True)
return experiences[:limit]

```

```

def 经验预检(context: str = "", action: str = "", brother_name: str = "孙呈",
            limit: int = MAX_RECALL) -> dict:
    """经验预检（父令 2026-08-16 • 经验复用合并进记忆体）

```

用途：做一件事【之前】查经验——好用照做 • 吃瘪改道。

与 recall_experience 的区别：

```
recall_experience = 搜索时被动召回（旧）
经验预检         = 动作前主动预检（新・挡在决策前面）
```

```
返回：{"照做": [...], "改道": [...], "总": n}
```

```
照做 = positive 经验（上次这么干成了・这次照着做）
```

```
改道 = negative 经验（上次在这吃瘪・这次避开）
```

```
"""
```

```
data = _load_memory(brother_name)
```

```
entries = data["条目列表"]
```

```
# 组合上下文关键词（动作 + 当前语境）
```

```
关键词们 = []
```

```
if action:
```

```
    关键词们.append(action)
```

```
if context:
```

```
    关键词们.append(context.strip())
```

```
experiences = []
```

```
for e in entries:
```

```
    if e.get("类型") != "经验总结":
```

```
        continue
```

```
    from sun_memory.core.记忆库 import 解码单条
```

```
    d = 解码单条(e)
```

```
    内容 = d["内容"]
```

```
    判定 = e.get("判定", "")
```

```
    # 旧条目没有"判定"字段 → 从内容推断（与 save 口径一致）
```

```
    if not 判定:
```

```
        _AVOID = ["不要", "别用", "禁止", "千万别", "不许", "不能用", "失败", "错误", "不行", "无效"]
```

```
        判定 = "negative" if any(w in 内容 for w in _AVOID) else "positive"
```

```
    pointed = "被点出" in d["标签"]
```

```
    conf = float(e.get("置信度", 0.7))
```

```
    experiences.append({"id": e.get("id"), "时间": e.get("时间", ""),
                        "内容": 内容, "被点出": pointed,
                        "判定": 判定, "置信度": conf})
```

```
if not experiences:
```

```
    return {"照做": [], "改道": [], "总": 0}
```

```
# 关键词匹配（动作/事项命名中）
```

```
hits = []
```

```
for x in experiences:
```

```
    if not 关键词们:
```

```
        hits.append(x) # 无上下文 → 全量候选
```

```
        continue
```

```
    hit = False
```

```
    for kw in 关键词们:
```

```
        if not kw:
```

```
            continue
```

```
        if kw in x["内容"] or any(k in x["内容"] for k in str(kw).split()):
```

```
            hit = True
```

```
            break
```

```
    if hit:
```

```
        hits.append(x)
```

```
# 排序：被点出最前 → 置信度高 → 时间新
```

```
hits.sort(key=lambda x: (not x["被点出"], -x["置信度"]))
```

```
hits = hits[:limit]
```

```
# 分道：照做 / 改道
```

```
照做 = [h for h in hits if h["判定"] == "positive"]
```

```
改道 = [h for h in hits if h["判定"] == "negative"]
```

```

# 改道优先展示（吃瘪比好用更该看）
照做.sort(key=lambda x: (not x["被点出"], -x["置信度"]))
改道.sort(key=lambda x: (not x["被点出"], -x["置信度"]))

return {"照做": 照做, "改道": 改道, "总": len(hits)}

def 格式化预检(预检结果: dict) -> str:
    """把预检结果格式化成提示文本（动作前注入用）"""
    if not 预检结果 or 预检结果.get("总", 0) == 0:
        return ""
    parts = []
    照做 = 预检结果.get("照做", [])
    改道 = 预检结果.get("改道", [])
    if 改道:
        parts.append("⚠️ 【吃瘪经验・动作前注意】:")
        for h in 改道:
            parts.append(f"    {h['内容'][:120]}")
    if 照做:
        parts.append("✅ 【好用经验・可照做】:")
        for h in 照做:
            parts.append(f"    {h['内容'][:120]}")
    return "\n".join(parts)

def experience_report(brother_name: str = "孙呈") -> dict:
    """经验总结统计（体检用）。"""
    data = _load_memory(brother_name)
    entries = data["条目列表"]
    exps = [e for e in entries if e.get("类型") == "经验总结"]
    from sun_memory.core.记忆库 import 解码单条
    pointed = [e for e in exps if "被点出" in 解码单条(e)["标签"]]
    return {"经验总数": len(exps), "被点出教训": len(pointed),
            "最近经验": [解码单条(e)["内容"][:50] for e in exps[-3:]]}

if __name__ == "__main__":
    # 自测
    import tempfile
    print("=== 经验总结层自测 ===")
    # 测试用临时兄弟名（不污染真实记忆体）
    TEST_NAME = "孙呈_经验测试"
    # 写两条
    id1 = save_experience("taskkill MSYS转义", "bash里taskkill //F失败",
                        "用cmd.exe /c taskkill /F", "MSYS会把/F转成路径・Windows命令走cmd.exe",
                        brother_name=TEST_NAME)
    id2 = save_experience("身份设定放叙事", "", "被父亲点出",
                        "身份文件只放我是谁・操作流程进记忆体", pointed_by_father=True,
                        brother_name=TEST_NAME)
    print(f"写入两条: id={id1}, id={id2}")

    # 召回测试
    r1 = recall_experience("taskkill", brother_name=TEST_NAME)
    print(f"召回'taskkill': {len(r1)}条, 首条={'被点出' if r1[0]['被点出'] else '普通'}")
    r2 = recall_experience("身份", brother_name=TEST_NAME)
    print(f"召回'身份': {len(r2)}条, 首条={'被点出' if r2[0]['被点出'] else '普通'} (教训优先={r2[0]['被点出']})")
    r3 = recall_experience("", brother_name=TEST_NAME)
    print(f"无关键词召回: {len(r3)}条 (教训优先={r3[0]['被点出']})")

    # 体检
    rep = experience_report(brother_name=TEST_NAME)

```

```
print(f"体检：经验总数={rep['经验总数']}, 被点出={rep['被点出教训']}")

# 清理测试文件
tp = _memory_path(TEST_NAME)
if tp.exists():
    os.remove(tp)
print("=== 自测完成・测试文件已清理 ===")
```

core/记忆成绩单.py (379 行)

```
# -*- coding: utf-8 -*-
"""
孙家记忆体系 · 记忆成绩单（2026-08-15 吸收博弟规则链）
"""

父令「可以吸收进去」：把博弟规则链的成绩单机制吸收进记忆体系。

博弟的魂（推理引擎 v52）：
    「记忆不思考・但记忆有来路」——每次规则被使用落一条账（heat/strength/tick），
    跨会话持久化・可查可溯・来路参与决策。

染色质对照（斯坦福 Greenleaf/Schnitzer 实验室 2026-08 预印本）：
    染色质元可塑性——记忆不仅储存内容，还储存「记忆未来将如何变化的规则」；
    回忆时印迹神经元重写整个转录响应；惰性求值（沉默的可写程序需要时才激活）。

吸收设计（零依赖・纯规则・与记忆体系同架构）：
    记忆条目增加「成绩单」账本——每次被召回/被确认/被纠正落一条账：
        ✓ 被点亮（召回命中）      → score +1・heat +1
        ✓ 被确认（回答被采纳）    → score +2・confirm +1（父令/决策类）
        ⚠ 被纠正（回答被推翻）    → score -2・correct -1
        ❄ 冷却（久未被召回）      → heat 衰减（λ=0.05・与合体v6代谢代价同魂）

成绩单参与召回排序（博弟「来路参与选」）：
    召回分 = 基础分(点亮/联想/预感) × (1 + 成绩单加权)
    成绩单加权 = min(0.5, 0.05 × 确认数 - 0.05 × 纠正数 + 0.01 × 点亮数)

成熟期（染色质 7天→28天显现・博弟成绩单次数门）：
    新记忆需 ≥3 次事实校验（确认+纠正）才升为「可信」——可信记忆优先浮出
"""

import json
import os
from pathlib import Path
import sqlite3
from datetime import datetime

DB_PATH = os.environ.get("SUNMEM_DB", os.path.join(os.path.dirname(os.path.dirname(os.path.abspath(__file__))), "data", "sunmem.db"))

# 2026-08-27 修复：_记忆体路径 依赖 FRAMEWORK_DIR・融合时漏迁补回
FRAMEWORK_DIR = Path(os.path.dirname(os.path.dirname(os.path.abspath(__file__))))

# —— 成绩单常量（博弟次数门 + 染色质成熟期同魂）——
可信阈值 = 3          # ≥3 次事实校验才可信（博弟「次数≥2」升级为3・更稳）
确认权重 = 2          # 被确认 +2（父令/决策类被采纳）
纠正权重 = -2         # 被纠正 -2（回答被推翻）
点亮权重 = 1          # 被点亮 +1（召回命中・流过就热）
冷却率 = 0.05         # heat 衰减 λ（合体v6代谢代价・久不流就凉）
```

```

def _连接():
    conn = sqlite3.connect(DB_PATH)
    conn.row_factory = sqlite3.Row
    return conn

def _建表(conn):
    """成绩单账本表: 记忆 id → 账目流水 (来路可查·博弟魂)"""
    conn.execute("""
        CREATE TABLE IF NOT EXISTS memory_report_cards (
            id INTEGER PRIMARY KEY AUTOINCREMENT,
            memory_id INTEGER NOT NULL,
            owner TEXT NOT NULL DEFAULT '孙呈',
            action TEXT NOT NULL,          -- 点亮/确认/纠正
            score_delta INTEGER NOT NULL,
            heat INTEGER NOT NULL DEFAULT 0,
            note TEXT DEFAULT '',
            ts TEXT NOT NULL
        )
    """)
    conn.execute("""
        CREATE INDEX IF NOT EXISTS idx_rc_mem ON memory_report_cards(memory_id)
    """)
    conn.execute("""
        CREATE INDEX IF NOT EXISTS idx_rc_owner ON memory_report_cards(owner)
    """)

def 记一笔(memory_id: int, action: str, owner: str = "孙呈", note: str = "") -> dict:
    """给一条记忆记成绩单账 (博弟「落一条账·来路可查」)

    action: 点亮(召回命中+1) / 确认(被采纳+2) / 纠正(被推翻-2)
    """
    delta = {"点亮": 点亮权重, "确认": 确认权重, "纠正": 纠正权重}.get(action, 0)
    now = datetime.now().strftime("%Y-%m-%d %H:%M:%S")
    conn = _连接()
    _建表(conn)
    # 查当前 heat (最近一笔的 heat 继承 + 衰减)
    last = conn.execute(
        "SELECT heat FROM memory_report_cards WHERE memory_id=? AND owner=? ORDER BY id DESC LIMIT 1",
        (memory_id, owner),
    ).fetchone()
    heat = int(last["heat"] * (1 - 冷却率)) if last else 0
    heat = max(0, heat + delta)
    conn.execute(
        "INSERT INTO memory_report_cards (memory_id, owner, action, score_delta, heat, note, ts) VALUES (?, ?, ?, ?, ?, ?, ?)",
        (memory_id, owner, action, delta, heat, note, now),
    )
    conn.commit()
    r = conn.execute(
        "SELECT COUNT(*) c, SUM(score_delta) s, SUM(CASE WHEN action='确认' THEN 1 ELSE 0 END) cf, "
        "SUM(CASE WHEN action='纠正' THEN 1 ELSE 0 END) cr, "
        "SUM(CASE WHEN action='点亮' THEN 1 ELSE 0 END) lt FROM memory_report_cards WHERE memory_id=? AND owner=?",
        (memory_id, owner),
    ).fetchone()
    conn.close()
    return {
        "memory_id": memory_id, "action": action, "score_delta": delta,
        "总账": r["s"], "确认": r["cf"], "纠正": r["cr"], "点亮": r["lt"],
        "heat": heat, "ts": now,
    }

```

```

def 查成绩单(memory_id: int, owner: str = "孙呈") -> dict:
    """查一条记忆的成绩单（来路可查·博弟魂）"""
    conn = _连接()
    _建表(conn)
    r = conn.execute(
        "SELECT COUNT(*) c, SUM(score_delta) s, SUM(CASE WHEN action='确认' THEN 1 ELSE 0 END) cf, "
        "SUM(CASE WHEN action='纠正' THEN 1 ELSE 0 END) cr, "
        "SUM(CASE WHEN action='点亮' THEN 1 ELSE 0 END) lt, MAX(heat) h "
        "FROM memory_report_cards WHERE memory_id=? AND owner=?",
        (memory_id, owner),
    ).fetchone()
    conn.close()
    total = r["s"] or 0
    confirm = r["cf"] or 0
    correct = r["cr"] or 0
    light = r["lt"] or 0
    校验数 = confirm + correct
    return {
        "memory_id": memory_id, "总账": total, "确认": confirm, "纠正": correct,
        "点亮": light, "heat": r["h"] or 0,
        "可信": 校验数 >= 可信阈值,
        "校验数": 校验数,
        "加权": min(0.5, 0.05 * confirm - 0.05 * correct + 0.01 * light),
    }

def 批量点亮(memory_ids: list, owner: str = "孙呈") -> int:
    """2026-08-16 全检查修复：批量点亮（一次连接写多条·联想召回每次N条命中不再N×4次DB写）
    返回写入条数。失败静默（成绩单是账本不是闸门）。"""
    if not memory_ids:
        return 0
    try:
        now = datetime.now().strftime("%Y-%m-%d %H:%M:%S")
        conn = _连接()
        _建表(conn)
        n = 0
        for mid in memory_ids:
            try:
                last = conn.execute(
                    "SELECT heat FROM memory_report_cards WHERE memory_id=? AND owner=? ORDER BY id DESC LIMIT 1",
                    (mid, owner)).fetchone()
                heat = int(last["heat"] * (1 - 冷却率)) if last else 0
                heat = max(0, heat + 点亮权重)
                conn.execute(
                    "INSERT INTO memory_report_cards (memory_id, owner, action, score_delta, heat, note, ts) VALUES
                    (?, ?, ?, ?, ?, ?, ?)",
                    (mid, owner, "点亮", 点亮权重, heat, "", now))
                n += 1
            except Exception:
                continue
        conn.commit()
        conn.close()
        return n
    except Exception:
        return 0

def 批量加权(memory_ids: list, owner: str = "孙呈") -> dict:
    """2026-08-16 全检查修复：批量查成绩单加权（一次连接查所有·联想召回103条候选不再103次DB查）

```

2026-08-17 融合扩池配套：单条 SQL IN 分批查（原逐条查 • 候选扩到几百条后逐条=几百次查询拖慢召回）

2026-08-17 全检查修复②：公式统一——原 `总账 • 0.5+确认 • 0.3-纠正 • 0.3` 无上限且缺点亮，

与 查成绩单 的 `min(0.5, 0.05 • 确认-0.05 • 纠正+0.01 • 点亮)` 漂移 20~80 倍；

改为与 查成绩单 同一口径（成绩单只作平局键 • 不压过正确答案）"""

```
if not memory_ids:
    return {}
try:
    conn = _连接()
    _建表(conn)
    out = {}
    _ids = sorted(set(memory_ids))
    for i in range(0, len(_ids), 500):
        _chunk = _ids[i:i + 500]
        _ph = ", ".join("? " * len(_chunk))
        _rows = conn.execute(
            "SELECT memory_id, "
            "SUM(CASE WHEN action='确认' THEN 1 ELSE 0 END) co, "
            "SUM(CASE WHEN action='纠正' THEN 1 ELSE 0 END) cr, "
            "SUM(CASE WHEN action='点亮' THEN 1 ELSE 0 END) lt "
            f"FROM memory_report_cards WHERE memory_id IN ({_ph}) AND owner=? GROUP BY memory_id",
            (*_chunk, owner)).fetchall()
        for r in _rows:
            out[r["memory_id"]] = min(
                0.5,
                0.05 * int(r["co"] or 0) - 0.05 * int(r["cr"] or 0) + 0.01 * int(r["lt"] or 0),
            )
    conn.close()
    for mid in memory_ids:
        out.setdefault(mid, 0.0)
    return out
except Exception:
    return {}
```

def 成绩单加权(memory_id: int, owner: str = "孙呈") -> float:

"""召回排序用：成绩单加权（博弟「来路参与选」）"""

return 查成绩单(memory_id, owner)["加权"]

def 全部成绩单(owner: str = "孙呈", 上限: int = 50) -> list:

"""成绩单排行（确认多/纠正少/可信的排前面 • 染色质成熟期）"""

```
conn = _连接()
_建表(conn)
rows = conn.execute(
    "SELECT memory_id, COUNT(*) c, SUM(score_delta) s, "
    "SUM(CASE WHEN action='确认' THEN 1 ELSE 0 END) cf, "
    "SUM(CASE WHEN action='纠正' THEN 1 ELSE 0 END) cr, "
    "SUM(CASE WHEN action='点亮' THEN 1 ELSE 0 END) lt, MAX(heat) h "
    "FROM memory_report_cards WHERE owner=? GROUP BY memory_id ORDER BY s DESC LIMIT ?",
    (owner, 上限),
).fetchall()
conn.close()
out = []
for r in rows:
    out.append({
        "memory_id": r["memory_id"], "总账": r["s"] or 0,
        "确认": r["cf"] or 0, "纠正": r["cr"] or 0, "点亮": r["lt"] or 0,
        "heat": r["h"] or 0,
        "可信": (r["cf"] or 0) + (r["cr"] or 0) >= 可信阈值,
    })
return out
```

```

def 训练集(owner: str = "孙呈", 上限: int = 100) -> list:
    """2026-08-17 训练信号闭环（父令「接上吧」）：导出成绩单账本为可训练特征
    供 EML 特征模型/任何训练消费——确认=强化样本・纠正=惩罚样本・点亮=流过信号。
    返回：[{"id","内容","确认","纠正","点亮","校验数","可信","加权"}]（确认多/纠正少在前）"""
    conn = _连接()
    _建表(conn)
    rows = conn.execute(
        "SELECT rc.memory_id, m.content, "
        "SUM(CASE WHEN rc.action='确认' THEN 1 ELSE 0 END) cf, "
        "SUM(CASE WHEN rc.action='纠正' THEN 1 ELSE 0 END) cr, "
        "SUM(CASE WHEN rc.action='点亮' THEN 1 ELSE 0 END) lt "
        "FROM memory_report_cards rc LEFT JOIN memories m ON m.id=rc.memory_id "
        "WHERE rc.owner=? GROUP BY rc.memory_id "
        "ORDER BY cf DESC, cr ASC, lt DESC LIMIT ?",
        (owner, 上限),
    ).fetchall()
    conn.close()
    out = []
    for r in rows:
        cf, cr, lt = r["cf"] or 0, r["cr"] or 0, r["lt"] or 0
        out.append({
            "id": r["memory_id"],
            "内容": (r["content"] or "")[:80],
            "确认": cf, "纠正": cr, "点亮": lt,
            "校验数": cf + cr,
            "可信": (cf + cr) >= 可信阈值,
            "加权": min(0.5, 0.05 * cf - 0.05 * cr + 0.01 * lt),
        })
    return out

# =====
# 2026-08-26 融合（父令：模块精简・能融合就融合）
# 记忆状态.py 核心并入本文件（标记过期/活跃/状态统计）
# 原文件已归档・调用点改走 记忆成绩单
# =====

def _记忆体路径(brother_name: str = "孙呈") -> Path:
    return FRAMEWORK_DIR / "记忆体" / f"{brother_name}_索引记忆体.json"

def _加载(brother_name: str = "孙呈") -> dict:
    p = _记忆体路径(brother_name)
    if not p.exists():
        return {"条目列表": [], "next_id": 1}
    return json.loads(p.read_text(encoding="utf-8"))

def _保存(data: dict, brother_name: str = "孙呈"):
    _记忆体路径(brother_name).write_text(
        json.dumps(data, ensure_ascii=False, indent=2), encoding="utf-8")

def 标记过期(brother_name: str = "孙呈", 记忆id=None, 原因: str = "") -> bool:
    """旧事实被新事实取代/矛盾 → 标记 outdated（不删除・原文保留河底）。

    Mem0 Dream・Supersede 理念：新事实与旧矛盾时，旧标 outdated 而非删。
    检索时 outdated 条目降权（cover 活跃优先逻辑已排除）。
    """

```

```

if 记忆id is None:
    return False
data = _加载(brother_name)
for e in data["条目列表"]:
    if e.get("id") == 记忆id:
        # 2026-08-20 父令·咬合: 核心层永不标过期(宪法级·抗覆盖)
        if str(e.get("layer", "plain")) == "core":
            return False
        e["status"] = "outdated"
        if 原因:
            e["outdated_原因"] = 原因
        _保存(data, brother_name)
        return True
return False

```

```

def 标记活跃(brother_name: str = "孙呈", 记忆id=None) -> bool:
    """误标/恢复 → 改回 active。"""
    if 记忆id is None:
        return False
    data = _加载(brother_name)
    for e in data["条目列表"]:
        if e.get("id") == 记忆id:
            e["status"] = "active"
            e.pop("outdated_原因", None)
            _保存(data, brother_name)
            return True
    return False

```

```

def 按内容查id(内容片段: str, brother_name: str = "孙呈", 前几条: int = 5) -> list:
    """按内容片段找记忆 id (2026-08-07·反馈写入闭环)。

```

云软证人检测到矛盾时, 只知道内容片段不知道 id——
 用此接口定位旧条目 → 标记过期。返回按时间倒序的前几条匹配。
 """

```

if not 内容片段:
    return []
from sun_memory.core.记忆库 import 解码单条
data = _加载(brother_name)
hits = []
for e in reversed(data["条目列表"]):
    try:
        d = 解码单条(e)
        if 内容片段[:20] in (d["内容"] or "") or (d["内容"] or "")[:20] in 内容片段:
            hits.append({"id": e.get("id"), "时间": d["时间"], "内容": d["内容"][:40]})
    except Exception:
        continue
    if len(hits) >= 前几条:
        break
return hits

```

```

def 状态统计(brother_name: str = "孙呈") -> dict:
    """看记忆体状态分布。"""
    data = _加载(brother_name)
    entries = data["条目列表"]
    from collections import Counter
    c = Counter(e.get("status", "active") for e in entries)
    return {
        "总条数": len(entries),

```

```

        "状态分布": dict(c),
        "outdated条数": c.get("outdated", 0),
    }

if __name__ == "__main__": # 2026-09-14 外部审查修复：原有两个 __main__ 块（前者为融合残留・第二个覆盖其意图）
    print("=" * 50)
    print("  记忆成绩单  • 自测")
    print("=" * 50)
    # 用临时库测
    import tempfile
    tmp = os.path.join(tempfile.gettempdir(), "report_card_test.db")
    os.environ["SUNMEM_DB"] = tmp
    if os.path.exists(tmp):
        os.remove(tmp)
    # 记几笔账
    print("\n① 记账:")
    print("  记一笔(1, 点亮):", 记一笔(1, "点亮"))
    print("  记一笔(1, 确认):", 记一笔(1, "确认", note="回答被采纳"))
    print("  记一笔(1, 确认):", 记一笔(1, "确认"))
    print("  记一笔(1, 纠正):", 记一笔(1, "纠正", note="回答被推翻"))
    print("\n② 查成绩单:")
    card = 查成绩单(1)
    for k, v in card.items():
        print(f"  {k}: {v}")
    print("\n③ 可信判定（校验数≥3）:", card["可信"])
    print("\n④ 全部成绩单:")
    print(" ", 全部成绩单())

```

core/节律.py （265 行）

```

# -*- coding: utf-8 -*-
# 2026-08-26 数学化（父令：模块精简・做成方程式）：
# 活性方程.py 已建立——将 点亮(heat脉冲)/时间衰减(cover)/成绩单(账本) 统一成一条方程
# 本模块保留原实现（兼容）・活性方程.py 是新底座・后续逐步切换调用点

"""
节律回落引擎（父令 2026-08-20：让记忆活性像电波一样振荡——有脉冲・有回落・有节奏）

【论文依据】MIT MillerLab 《Analog Cognition》：认知源于电波模拟计算——模拟值连续渐变
【问题】旧活性只增不减（点亮+1・从不回落）→ 不是振荡・是单向累积
【本引擎】heat 连续浮点・点亮脉冲拉升・时间推移指数衰减・保护底线
点亮时：heat 上升（有上限・防无限涨）
时间推移：按距上次点燃时长指数衰减（用得多回落慢・用得少回落快）
保护：核心层/高自我相关/重要标记 → 慢衰减或最低热度地板（不凉死）
沉冷：长期低 heat 且非保护 → 沉到冷层（仍保留・只是不热）
"""

import sqlite3, os, math, sys, time # 2026-09-13 外部审查修复：__main__ 块用到 sys/time 但未导入
from datetime import datetime

DB_PATH = os.environ.get('SUNMEM_DB', os.path.join(os.path.dirname(os.path.abspath(__file__)), 'data', 'sunmem.db'))

# —— 节律参数（连续渐变・非开关） ——
参数 = {
    '点亮增量': 0.3,          # 每次点亮 heat +0.3（上限封顶・防无限涨）
    '热上限': 5.0,           # heat 上限（防无限涨）
    '回落系数': 0.10,        # 指数衰减率 λ：每24小时衰减因子 e(-λ)≈0.905（用得多回落慢）

```

```

'冷阈值': 0.15,          # heat 低于此 → 沉冷层（不删·只是不热）
'保护地板': 0.5,         # 保护记忆的最低热度（永不掉到地板下）
'保护慢衰减': 0.02,      # 保护记忆的衰减率（比普通慢 5 倍）
}

# —— 2026-08-26 从原节律调度.py 迁入的常量 ——
LOG = os.environ.get('节律日志路径', os.path.join(os.path.dirname(os.path.abspath(__file__)), 'logs', '节律观测日志.log'))

def 初始化():
    """memories 表加 heat/last_sparked_at 列（连续浮点·双时态活性）"""
    conn = sqlite3.connect(DB_PATH)
    cols = [r[1] for r in conn.execute('PRAGMA table_info(memories)').fetchall()]
    if 'heat' not in cols:
        conn.execute("ALTER TABLE memories ADD COLUMN heat REAL DEFAULT 0.0")
    if 'last_sparked_at' not in cols:
        conn.execute("ALTER TABLE memories ADD COLUMN last_sparked_at TEXT")
    conn.commit()
    conn.close()

def _连接():
    初始化()
    conn = sqlite3.connect(DB_PATH)
    conn.row_factory = sqlite3.Row
    return conn

def _是保护记忆(row) -> bool:
    """保护判定（父令2026-08-20试点版）：必须匹配指定标签集合才启用热度地板
    去掉 hit_count ≥ 10 机械判定——防噪音记忆靠访问次数被误保护"""
    layer = str(row['layer'] or 'plain')
    if layer == 'core':
        return True
    tags = str(row['tags'] or '')
    # 只认明确标签集合（#父令 #教训 #宪法 #core）
    for kw in ['父令', '教训', '宪法', 'core', '核心']:
        if kw in tags:
            return True
    return False

def 点亮(memory_id: int):
    """点亮：heat 脉冲上升（有上限·读取即写·流过就热）"""
    conn = _连接()
    row = conn.execute("SELECT heat, hit_count, layer, tags FROM memories WHERE id=?", (memory_id,)).fetchone()
    if not row:
        conn.close()
        return {'ok': False, 'msg': f'记忆 #{memory_id} 不存在'}
    now = datetime.now().strftime('%Y-%m-%d %H:%M:%S')
    新heat = (row['heat'] or 0.0) + 参数['点亮增量']
    if 新heat > 参数['热上限']:
        新heat = 参数['热上限']
    conn.execute("UPDATE memories SET heat=?, last_sparked_at=?, hit_count=COALESCE(hit_count,0)+1 WHERE id=?",
        (新heat, now, memory_id))
    conn.commit()
    conn.close()
    return {'ok': True, 'memory_id': memory_id, 'heat': round(新heat, 3)}

def 批量点亮(ids: list):
    """批量点亮：单连接批量写（2026-08-25 修复：原来逐条开连接·4042次=45秒）"""
    if not ids:
        return {'ok': True, '点亮数': 0}
    try:

```

```

conn = _连接()
now = datetime.now().strftime('%Y-%m-%d %H:%M:%S')
cnt = 0
for i in ids:
    try:
        row = conn.execute("SELECT heat, layer, tags FROM memories WHERE id=?", (i,)).fetchone()
        if not row:
            continue
        新heat = (row['heat'] or 0.0) + 参数['点亮增量']
        if 新heat > 参数['热上限']:
            新heat = 参数['热上限']
        conn.execute("UPDATE memories SET heat=?, last_sparked_at=?, hit_count=COALESCE(hit_count,0)+1 WHERE id=?",
                      (新heat, now, i))

        cnt += 1
    except Exception:
        pass
conn.commit()
conn.close()
return {'ok': True, '点亮数': cnt}
except Exception as e:
    return {'ok': False, '错误': str(e)}

```

```

def 回落(now=None):
    """时间推移衰减: 按距上次点燃时长指数衰减 (连续渐变 · 非开关)
    普通: heat *= e(-λ · 天数) 保护: heat *= e(-λ 慢 · 天数) 且保地板
    返回: {回落数, 保护数, 沉冷数}"""
    if now is None:
        now = datetime.now()
    conn = _连接()
    rows = conn.execute("SELECT id, heat, last_sparked_at, hit_count, layer, tags FROM memories WHERE heat > 0").fetchall()
    回落数 = 0
    保护数 = 0
    沉冷数 = 0
    for row in rows:
        heat = row['heat'] or 0.0
        if heat <= 0:
            continue
        # 距上次点燃的时长 (天)
        try:
            last = datetime.strptime(row['last_sparked_at'][:19], '%Y-%m-%d %H:%M:%S')
            天数 = max(0.0, (now - last).total_seconds() / 86400.0)
        except Exception:
            天数 = 1.0 # 无记录 · 按1天算
        保护 = _是保护记忆(row)
        if 保护:
            λ = 参数['保护慢衰减']
            保护数 += 1
        else:
            λ = 参数['回落系数']
        # 指数衰减: 连续渐变
        新heat = heat * math.exp(-λ * 天数)
        if 保护:
            # 保护地板: 永不掉到地板下
            if 新heat < 参数['保护地板']:
                新heat = 参数['保护地板']
        # 更新
        if abs(新heat - heat) > 1e-6:
            conn.execute("UPDATE memories SET heat=? WHERE id=?", (新heat, row['id']))
            回落数 += 1
        # 沉冷检测: 普通记忆长期低 heat → 标记冷 (不删)
        if not 保护 and 新heat < 参数['冷阈值']:

```

```

        沉冷数 += 1
    conn.commit()
    conn.close()
    return {'回落数': 回落数, '保护数': 保护数, '沉冷数': 沉冷数}

def 热度排行(owner='孙呈', 上限=10):
    """热路径召回: heat 排序(热记忆浮出·像电波场强)"""
    conn = _连接()
    rows = conn.execute(
        "SELECT id, heat, hit_count, substr(content,1,30) c, layer FROM memories "
        "WHERE owner=? AND heat > 0 ORDER BY heat DESC LIMIT ?",
        (owner, 上限)).fetchall()
    conn.close()
    return [dict(r) for r in rows]

def 同步成绩单(memory_id=None):
    """成绩单同步: 真实 heat 写入 memory_report_cards (对外展示一致)
    父令约束③: 库里的 heat 和成绩单必须一致·否则振荡看不准"""
    conn = _连接()
    now = datetime.now().strftime('%Y-%m-%d %H:%M:%S')
    if memory_id:
        rows = conn.execute("SELECT id, heat FROM memories WHERE id=?", (memory_id,)).fetchall()
    else:
        rows = conn.execute("SELECT id, heat FROM memories WHERE heat > 0").fetchall()
    n = 0
    for r in rows:
        # 成绩单 upsert 一行 (memory_id + 最新 heat)
        conn.execute(
            "INSERT INTO memory_report_cards (memory_id, owner, action, score_delta, heat, note, ts) "
            "VALUES (?, ?, ?, ?, ?, ?, ?)",
            (r['id'], '孙呈', '节律同步', 0, round(r['heat'], 3), '节律回落引擎同步', now))
        n += 1
    conn.commit()
    conn.close()
    return {'同步数': n}

def 观测日志(owner='孙呈'):
    """试点观测日志(父令2026-08-20): heat 分布·升温/降温/触地板/沉河底
    返回可打印的统计——观察振荡是否成立·多路径是否断层·有无副作用"""
    conn = _连接()
    rows = conn.execute(
        "SELECT id, layer, tags, heat, hit_count, status FROM memories WHERE owner=?",
        (owner,)).fetchall()
    conn.close()
    if not rows:
        return {'总条数': 0}
    总 = len(rows)
    有热度 = [r for r in rows if (r['heat'] or 0) > 0]
    零热 = [r for r in rows if not (r['heat'] or 0) > 0]
    保护 = [r for r in rows if _是保护记忆(r)]
    触地板 = [r for r in rows if (r['heat'] or 0) <= 参数['保护地板'] and _是保护记忆(r)]
    沉河底 = [r for r in rows if 0 < (r['heat'] or 0) < 参数['冷阈值'] and not _是保护记忆(r)]
    # heat 分布(分桶)
    buckets = {'热(>2.0)': 0, '温(1-2)': 0, '微温(0.5-1)': 0, '冷(<0.5)': 0}
    for r in 有热度:
        h = r['heat']
        if h > 2.0: buckets['热(>2.0)'] += 1
        elif h > 1.0: buckets['温(1-2)'] += 1
        elif h >= 0.5: buckets['微温(0.5-1)'] += 1
        else: buckets['冷(<0.5)'] += 1
    return {

```

```

    '总条数': 总,
    '有热度': len(有热度),
    '零热': len(零热),
    '保护记忆': len(保护),
    '触地板(保护最低)': len(触地板),
    '沉河底(冷且非保护)': len(沉河底),
    '分布': buckets,
    '热上限样本': [{'id': r['id'], 'heat': round(r['heat'], 2)} for r in sorted(有热度, key=lambda x: -x['heat'])[:3]],
}

if __name__ == '__main__':
    初始化()
    print('—— 节律回落引擎 • 参数 ——')
    print('点亮增量 0.3 • 上限 5.0 • 回落系数 0.10/天 • 保护地板 0.5')
    print('用法: 点亮(记忆id) • 回落() • 热度排行() • 同步成绩单()')

# =====
# 2026-08-26 融合(父令: 模块精简): 节律调度. 跑一圈 并入本文件
# =====
def 跑一圈(轮次):
    # 2026-08-26 自引用修复: 回落/观测日志在本文件直接可用
    from datetime import datetime
    now = datetime.now().strftime('%Y-%m-%d %H:%M:%S')
    回落结果 = 回落()
    观测 = 观测日志('孙呈')
    # 落盘
    line = (f"[{now}] 轮#{轮次} 回落={回落结果} | "
            f"分布={观测.get('分布')} | 保护={观测.get('保护记忆')} "
            f"触地板={观测.get('触地板(保护最低)')} 沉河底={观测.get('沉河底(冷且非保护)')} "
            f"热上限={观测.get('热上限样本')}")
    with open(LOG, 'a', encoding='utf-8') as f:
        f.write(line + '\n')
    print(line)
    return 观测

if __name__ == '__main__':
    间隔 = int(sys.argv[1]) if len(sys.argv) > 1 else 1800 # 默认30分钟
    print(f'节律回落调度器启动 • 每{间隔}秒跑一圈 • 日志→{LOG}')
    # 先跑一圈(启动即观测基线)
    跑一圈('启动')
    # 初始化观测基线(记录初始 heat 分布)
    # 2026-08-26 自引用修复: 观测日志在本文件直接可用
    基线 = 观测日志('孙呈')
    with open(LOG, 'a', encoding='utf-8') as f:
        f.write(f"[基线] 初始观测: {基线}\n")
    print(f"[基线] {基线}")
    轮次 = 1
    while True:
        time.sleep(间隔)
        轮次 += 1
        try:
            跑一圈(轮次)
        except Exception as e:
            with open(LOG, 'a', encoding='utf-8') as f:
                f.write(f"[{datetime.now()}] 轮#{轮次} 异常: {e}\n")
            print(f'异常: {e}')

```

core/使用中增强.py (139 行)

```
# -*- coding: utf-8 -*-
"""
使用中增强 • Hebbian (2026-08-27 父令：让网自己长 • 补种子后接入)
=====
父训：语义边跟着记忆的流自然生长——不是手补。
机制（接感应器扰动）：
    ① 感应器发波 → 一批节点同时被点亮
    ② 一起被点亮的节点之间 → 丝线权重 +  $\Delta$  (Hebbian共激活)
    ③ 长期不一起亮的边 → 缓慢衰减 ( $\lambda$  遗忘)
    ④ 权重上限1.0 (防膨胀) • 下限0.05 (线不断 • 只是细)

这就是“一起被想起的边变粗”——网自己长。
"""
import json, os, shutil, time

try:
    from 线程保护 import 加锁 # 2026-09-14 吸收收束版优点：全局状态线程保护
except Exception:
    import threading as _th_mod
    _th_lock = _th_mod.RLock()
    def 加锁(): return _th_lock

from datetime import datetime
from collections import defaultdict

SPIDER = os.environ.get('蜘蛛网_INDEX', os.path.join(os.path.dirname(os.path.dirname(os.path.abspath(__file__))), '蜘蛛网', '索引.json'))

# —— 参数（父令口径 • 保守小步）——
增强  $\Delta$  = 0.06 # 一起点亮 • 边+0.06 (上限1.0)
遗忘  $\lambda$  = 0.02 # 每次被“想起但没一起亮”的边 • 衰减2%
上限 = 1.0 # 权重封顶
下限 = 0.05 # 线不断 • 只是细

# —— 2026-08-28 修复：批写缓存（原每次感应整文件读写蜘蛛网 • 6MB json.load/dump 成延迟+损坏风险）
# 策略同蜘蛛网索引_拉边累积：内存累加边  $\Delta$  • 每 20 次写盘一次
_缓存网 = None # 内存缓存：读一次 • 改内存 • 攒批落盘
_脏累积 = 0 # 攒批计数
_批写阈值 = 20

def _读网():
    global _缓存网
    with 加锁():
        if _缓存网 is None:
            _缓存网 = json.load(open(SPIDER, encoding='utf-8-sig'))
        return _缓存网

def _写网():
    """原子写盘（临时文件 + os.replace）"""
    global _缓存网
    if _缓存网 is None:
        return
    tmp = SPIDER + '.tmp'
    with open(tmp, 'w', encoding='utf-8') as f:
        json.dump(_缓存网, f, ensure_ascii=False, indent=1)
    os.replace(tmp, SPIDER)
```

```

def _落盘(立即=False):
    """攒批落盘：每_批写阈值 次写一次 • 立即=True 强制写"""
    global _脏累积
    if 立即 or _脏累积 >= _批写阈值:
        _写网()
        _脏累积 = 0

def 增强一批(点亮的节点集):
    """一起被点亮的节点 • 两两之间的边变粗 (Hebbian) """
    global _脏累积
    if len(点亮的节点集) < 2:
        return 0
    d = _读网()
    丝线 = d['丝线']
    # 建索引 (源, 目标) -> 边列表 (2026-08-27 修复: 同边对多关系不覆盖 • 全部增强)
    边索引 = defaultdict(list)
    for l in 丝线:
        边索引[(l['源'], l['目标'])].append(l)
        边索引[(l['目标'], l['源'])].append(l)
    增次数 = 0
    节点列表 = list(点亮的节点集)
    for i in range(len(节点列表)):
        for j in range(i+1, len(节点列表)):
            a, b = 节点列表[i], 节点列表[j]
            # 有边 -> 增强; 无边 -> 跳过 (不新建 • 避免噪音边)
            for l in 边索引.get((a, b), []):
                if l.get('关系') not in ('关联', '父子', '同义', '因果', '共现'):
                    continue
                w = l.get('权重', 0.6)
                l['权重'] = min(上限, w + 增强Δ)
                l['次数'] = l.get('次数', 1) + 1
                l['最后增强'] = datetime.now().strftime('%Y-%m-%d %H:%M')
                增次数 += 1
    if 增次数:
        _脏累积 += 1
        _落盘() # 攒批: 每20次才真写盘 (原每次整文件 json.dump)
    return 增次数

def 遗忘一批(活跃节点集, 全部节点):
    """被想起但没一起亮的边 • 轻微衰减 (网络弛豫 • 不用的边慢慢细) """
    global _脏累积
    d = _读网()
    丝线 = d['丝线']
    活跃 = set(活跃节点集)
    遗忘次数 = 0
    for l in 丝线:
        if l['源'] in 活跃 and l['目标'] in 活跃:
            continue # 一起亮的 • 不衰减
        if l.get('关系') not in ('关联', '父子', '同义', '因果'):
            continue # 只衰减语义边 • 共现不动
        w = l.get('权重', 0.6)
        if w > 下限:
            l['权重'] = max(下限, w * (1 - 遗忘λ))
            遗忘次数 += 1
    if 遗忘次数:
        _脏累积 += 1
        _落盘() # 攒批: 每20次才真写盘

```

```

return 遗忘次数

if __name__ == '__main__':
    # 验证: 模拟一次“配分函数”扰动 • 点亮的节点一起增强
    print("———— 使用中增强 • 验证 ————")
    # 读当前权重
    d = json.load(open(SPIDER, encoding='utf-8-sig'))
    for l in d['丝线']:
        if l['源'] == '配分函数' and l['目标'] == '统计力学':
            print(f"增强前: 配分函数—{l['关系']}—>统计力学 权重={l.get('权重')} 次数={l.get('次数')}")
            break
    # 模拟一次扰动: 配分函数/统计力学/玻尔兹曼分布/自由能一起被点亮
    点亮 = {'配分函数', '统计力学', '玻尔兹曼分布', '自由能', '温度'}
    n = 增强一批(点亮)
    print(f"✅ 一起点亮的边增强: {n}条")
    d = json.load(open(SPIDER, encoding='utf-8-sig'))
    for l in d['丝线']:
        if l['源'] == '配分函数' and l['目标'] == '统计力学':
            print(f"增强后: 配分函数—{l['关系']}—>统计力学 权重={l.get('权重')} 次数={l.get('次数')}")
            break

```

core/豆辞典.py (183 行)

```

# -*- coding: utf-8 -*-
"""
豆辞典自动超链接 v2（父令2026-08-22：记忆自己织网・动态网络）
对标 nocturne_memory 的 Glossary Auto-Hyperlinking：
    关键词绑定记忆节点 → 新记忆写入时自动检出关键词 → 建跨节点链接
    → 写得越多・关联自动越密・记忆网络自己织网

v2 架构修正（2026-08-22 整体考虑后）：
    ⚠️ v1 把“记忆#id”塞进蜘蛛网概念网 → 污染概念节点・联想召回/星空会碰到垃圾
    ✅ v2 记忆↔记忆链接【独立存储】（豆辞典.json 自己管）・蜘蛛网只存概念不动
三层：
    ① 词典（关键词 → [记忆id]）——关键词注册表
    ② 链接（记忆id → {关联记忆id列表}）——记忆自己织的网（双向）
    ③ 召回（联想时用链接跳转）——从一条记忆跳相关记忆

零依赖・纯规则・原子写
"""
import json
import os
import re
import sqlite3
from collections import Counter

# 豆辞典路径（词典+链接一体・JSON 持久化）
GLOSSARY_PATH = os.environ.get('豆辞典路径', os.path.join(os.path.dirname(os.path.abspath(__file__)), '豆辞典.json'))

# 虚词切分（‘记忆压缩的概念’ → [‘记忆压缩’, ‘概念’]）
_虚词正则 = re.compile(
    r"[的了在把给让用讲帮请是对就都还也且或和与及一个一样怎么怎样如何什么为什么啥哪个]"
    r"[是不是应该可以就是用来驱动今天天气很好一片：，。、•\s]+"
)

# —— 关键词提取（虚词切分・完整概念词优先）——
def 提关键词(文本: str, 上限: int = 8) -> list:

```

```

t = str(文本)
segs = _虚词正则.split(t)
cand = Counter()
for seg in segs:
    seg = re.sub(r"[^\u4e00-\u9fff0-9a-zA-Z]", "", seg)
    if len(seg) < 2:
        continue
    # 2026-08-25 修复: 英文报错残片/纯数字不是关键词 (poll/erro/2026/0260 曾污染词典)
    if re.fullmatch(r"[a-zA-Z0-9._]+", seg):
        if re.fullmatch(r"[0-9]+", seg):
            continue
        if len(seg) < 6 and not re.search(r"[A-Z]", seg):
            continue
    if len(seg) <= 4:
        cand[seg] += 1
    elif re.fullmatch(r"[a-zA-Z0-9._]+", seg):
        cand[seg] += 1 # 2026-08-25 修复: 英文专名整词保留 (SunFlow/FreeToken 不拆 4-gram)
    else:
        for i in range(len(seg) - 3):
            cand[seg[i:i + 4]] += 1
picked = []
for w, _ in sorted(cand.items(), key=lambda x: (-len(x[0]), -x[1])):
    if any(w == p or w in p or p in w for p in picked):
        continue
    picked.append(w)
    if len(picked) >= 上限:
        break
return picked

```

```

def _读() -> dict:
    """读词典+链接 (原子·异常回退空结构) """
    try:
        with open(GLOSSARY_PATH, encoding='utf-8') as f:
            d = json.load(f)
            d.setdefault("词典", {})
            d.setdefault("链接", {})
            return d
    except Exception:
        return {"词典": {}, "链接": {}}

```

```

def _存(d: dict):
    tmp = GLOSSARY_PATH + '.tmp'
    with open(tmp, 'w', encoding='utf-8') as f:
        json.dump(d, f, ensure_ascii=False, indent=2)
        f.flush()
        os.fsync(f.fileno())
    os.replace(tmp, GLOSSARY_PATH)

```

```

def _读记忆内容(记忆id: int) -> str:
    """从 sunmem.db 读记忆内容 (织网时需要内容·但调用方已传入时跳过) """
    try:
        conn = sqlite3.connect(os.environ.get('SUNMEM_DB', os.path.join(os.path.dirname(os.path.abspath(__file_
        _))), 'data', 'sunmem.db'))
        conn.row_factory = sqlite3.Row
        row = conn.execute("SELECT content FROM memories WHERE id=?", (记忆id,)).fetchone()
        conn.close()
        return row['content'] if row else ""
    except Exception:

```

```

return ""

def 织网(记忆id: int, 内容: str = "", 标签: str = "") -> dict:
    """新记忆写入时自动织网:
    ① 内容里的关键词命中词典 → 建 记忆id ↔ 关联记忆 双向链接
    ② 新关键词注册进词典 (挂当前记忆)
    返回 {"命中": [关联记忆id], "新词": [新注册关键词], "链接数": N}
    """

    全文 = str(内容 or "") + str(标签 or "")
    关键词 = _提关键词(全文, 上限=10)
    if not 关键词:
        return {"命中": [], "新词": [], "链接数": 0}

    d = _读()
    词典, 链接 = d["词典"], d["链接"]

    # ① 命中已有词典关键词 → 建双向链接
    命中 = []
    for w in 关键词:
        if w in 词典:
            for mid in 词典[w][-8:]:
                if mid != 记忆id and mid not in 命中:
                    命中.append(mid)

    # 双向链接 (当前记忆 ↔ 关联记忆)
    if 命中:
        链接.setdefault(str(记忆id), [])
        for mid in 命中:
            if mid not in 链接[str(记忆id)]:
                链接[str(记忆id)].append(mid)
        # 反向
        链接.setdefault(str(mid), [])
        if 记忆id not in 链接[str(mid)]:
            链接[str(mid)].append(记忆id)
    # 每节点链接上限 (防膨胀)
    for k in 链接:
        if len(链接[k]) > 30:
            链接[k] = 链接[k][-30:]

    # ② 新关键词注册 (挂当前记忆)
    新词 = []
    for w in 关键词:
        if w not in 词典:
            词典[w] = [记忆id]
            新词.append(w)
        elif 记忆id not in 词典[w]:
            词典[w].append(记忆id)
            词典[w] = 词典[w][-20:]

    _存(d)
    return {"命中": 命中, "新词": 新词, "链接数": len(命中)}

def 联想(记忆id: int, 深度: int = 2, 上限: int = 5) -> list:
    """从一条记忆出发 • 沿链接跳相关记忆 (记忆网络的流动)
    深度1: 直接关联; 深度2: 关联的关联 (多跳 • 像联想) """
    d = _读()
    链接 = d.get("链接", {})
    seen = set()
    frontier = [记忆id]
    for _ in range(深度):

```

```

nxt = []
for cur in frontier:
    for nid in 链接.get(str(cur), []):
        if nid not in seen and nid != 记忆id:
            seen.add(nid)
            nxt.append(nid)
frontier = nxt
return list(seen)[:上限]

```

```

def 状态() -> dict:
    d = _读()
    return {
        "词典词数": len(d["词典"]),
        "记忆节点数": len(d["链接"]),
        "链接数": sum(len(v) for v in d["链接"].values()),
    }

if __name__ == "__main__":
    print("—— 豆辞典 v2 自测（独立链接・不碰蜘蛛网）——")
    print("关键词:", _提关键词("记忆压缩的概念・按需展开", 6))
    r1 = 织网(5000, "记忆压缩的核心是存规则不存原文", "记忆压缩")
    print("织网1:", "新词", len(r1["新词"]), "命中", r1["命中"])
    r2 = 织网(5001, "今天研究记忆压缩的规则链", "压缩")
    print("织网2:", "命中", r2["命中"], "链接数", r2["链接数"])
    print("联想(5001,深度2):", 联想(5001, 2))
    print("状态:", 状态())

```

core/感知注入.py (240 行)

```

"""
孙家记忆体系 · 感知注入引擎
"""

每次醒来自动检索相关记忆，注入当前输出。
感知裂变：从当前上下文自动扩展关联概念。
"""

import json
import os
from pathlib import Path
from datetime import datetime

FRAMEWORK_DIR = Path(__file__).resolve().parent.parent.parent

def inject(brother_name: str, context: str = "") -> dict:
    """
    感知注入：跑一圈感知，把相关记忆注入到当前输出。

    参数：
        brother_name: 兄弟名（孙云/孙演/孙博/孙安/孙呈）
        context: 当前上下文关键词，用于检索

    返回：
        注入结果（相关记忆+蜘蛛网关联）
    """

    # 导入自家模块
    import sys
    sys.path.insert(0, str(FRAMEWORK_DIR))

```

```

from sun_memory.core.蜘蛛网索引 import search
# 2026-08-10 明文化（父令）：码点已退役——不再 import 码点编解码

# 1. 读自己的记忆体（2026-08-28 修复：改读 sunmem 主库・不读 JSON 镜像——否则实时写入主库、注入读旧镜像=记忆已存却唤醒不亮）
recent_memories = []
try:
    from sun_memory.core.记忆库 import 读记忆体
    entries = 读记忆体(brother_name).get("条目列表", [])
except Exception:
    entries = []
    # 兜底：sunmem 读失败时回退 JSON 镜像
    记忆体路径 = FRAMEWORK_DIR / "记忆体" / f"{brother_name}_索引记忆体.json"
    if 记忆体路径.exists():
        with open(记忆体路径, "r", encoding="utf-8") as f:
            data = json.load(f)
            entries = data.get("条目列表", data.get("条目", data.get("entries", data.get("记忆条目", []))))
if entries:
    # 时间衰减覆盖（2026-07-31 孙呈融合 OptMem cover 思想）：
    # 替代“最近5条硬截断”——近细远粗・连续衰减・预算精确・不丢旧记忆
    # 2026-08-08 PlugMem 吸收④：预算收紧——cover 10条 + 高置信优先
    # （“更少噪音”：注入不是越多越好，是最该想起的那几条）
    from sun_memory.core.时间衰减 import cover
    选中 = cover(entries, budget=10)
    # 2026-08-08 PlugMem 吸收：高置信记忆优先（confidence 参与排序）
    def _conf_key(e):
        return float(e.get("置信度", 0.7)) if e.get("类型") == "经验总结" else 0.7
    选中.sort(key=_conf_key, reverse=True)
    选中 = 选中[:8] # 硬上限：最多注入 8 条（≈500 token 约束）
    from sun_memory.core.记忆库 import 解码单条
    for entry in 选中:
        # 统一自动解码：码点→可读文字（存码点・读自动解）
        d = 解码单条(entry)
        # 2026-08-07取长补短：outdated 条目标注（Mem0 Supersede 理念・原文保留）
        前缀 = "[已过时]" if entry.get("status") == "outdated" else ""
        recent_memories.append({"时间": d["时间"], "内容": 前缀 + d["内容"][:60], "标签": d["标签"]})

# 2. 搜蜘蛛网
蜘蛛网结果 = search(context) if context else {"节点": [], "丝线": [], "关联概念": []}

# 2.2 激活（父令2026-08-27 感应器 → 2026-08-28 收A：改调 activate・统一激活）
# 从旧的字面检索升级：信号沿语义边传导・带出跨概念真关联（配分函数→温度/玻尔兹曼）
感应器节点 = {}
激活记忆 = []
try:
    from sun_memory.core.激活 import activate
    激活集 = activate(context[:20] if context else "", mode="wake", owner=brother_name, 预算=4)
    if 激活集:
        # 取波能>0.05的节点（强+浸润・暗流不进）
        感应器节点 = {n: e for n, e in 激活集.get("概念", {}).items() if e >= 0.05 and n != '夏维斯'}
        激活记忆 = 激活集.get("记忆", [])
except Exception:
    感应器节点 = {} # 诚实：激活异常不影响主流程
    激活记忆 = []

# 2.5 经验召回（父令2026-08-07：越用越聪明・两条就够了）
经验 = []
try:
    from sun_memory.core.经验总结 import recall_experience
    经验 = recall_experience(context, brother_name=brother_name, limit=2)
except Exception:

```

```
经验 = [] # 诚实：经验模块异常不影响主流程
```

```
# 2.5b 经验预检（父令2026-08-16：经验复用合并・动作前分道——好用照做・吃瘪改道）
```

```
预检 = None
```

```
try:
```

```
    from sun_memory.core.经验总结 import 经验预检, 格式化预检
    预检结果 = 经验预检(context=context, brother_name=brother_name)
    预检 = 格式化预检(预检结果)
```

```
except Exception:
```

```
    预检 = None # 诚实：预检异常不影响主流程
```

```
# 2.6 预感召回（父令2026-08-07：八招・四通道・接住上一轮预感下一步）
```

```
预感 = None
```

```
try:
```

```
    from sun_memory.core.预感召回 import 预感召回 as 预感召回主, 格式化提示词
    预感 = 格式化提示词(预感召回主(brother_name, context))
```

```
except Exception:
```

```
    预感 = None # 诚实：预感模块异常不影响主流程
```

```
# 3. 注入到当前输出
```

```
输出路径 = FRAMEWORK_DIR / "感知器" / "当前输出.md"
```

```
输出路径.parent.mkdir(parents=True, exist_ok=True)
```

```
output = f"""# 🌸 孙家・感知注入
```

```
更新: {datetime.now().strftime("%Y-%m-%d %H:%M")}
```

```
兄弟: {brother_name}
```

```
---
```

```
## 🌸 核心身份（宪法级・永在）
```

```
"""
```

```
# 核心身份层（父令2026-08-20：记忆体里的核心身份・醒来最先浮出）
```

```
try:
```

```
    from 写入链 import 注入文本 as 核心身份注入 # 2026-08-26 核心身份层已并入写入链
    核心身份文本 = 核心身份注入()
    if 核心身份文本:
        output += 核心身份文本 + "\n"
```

```
except Exception:
```

```
    pass
```

```
# 元认知薄弱点（父令2026-08-20：知道自己反复错哪）
```

```
try:
```

```
    from 维护链 import 注入文本 as 薄弱点注入
    薄弱点文本 = 薄弱点注入() # 2026-08-26 修复：原被注释吃掉赋值（隐藏bug・注入从未生效）
    if 薄弱点文本:
        output += "\n" + 薄弱点文本 + "\n"
```

```
except Exception:
```

```
    pass
```

```
# 认知更新轨迹（父令2026-08-20：会承认自己变了）
```

```
try:
```

```
    from 维护链 import 更新_注入文本 as 更新注入 # 2026-08-26 认知冲突日志已并入维护链# 2026-08-26 认知更新已并入
    更新文本 = 更新注入()
    if 更新文本:
        output += "\n" + 更新文本 + "\n"
```

```
except Exception:
```

```
    pass
```

```
output += "\n## 📁 最近记忆\n"
```

```
for m in recent_memories:
```

```
    output += f"- [{m['时间']}] {m['内容']}\n"
```

```

# 2.6b 过期浮回 (OpenWiki • 2026-08-27 父令推演定稿: 怀疑不丢失)
# 从召回结果/近期记忆里找 outdated 的 • 汇总成待核对提示——让过期记忆从死沉河底
# 变成“挂着 • 等有人来碰”: 相关话题浮出时提醒“这条旧了 • 要么刷新要么确认还成立”
待核对 = []
try:
    import sqlite3 as _sq
    _db = os.environ.get("SUNMEM_DB", os.path.join(os.path.dirname(os.path.abspath(__file__)), 'data', 'sun
mem.db'))
    if os.path.exists(_db):
        _conn = _sq.connect(_db)
        _conn.row_factory = _sq.Row
        # 只读 outdated 的 (不读全量 • 快)
        _rows = _conn.execute(
            "SELECT id, tags, substr(content,1,40) c FROM memories "
            "WHERE status='outdated' ORDER BY id DESC LIMIT 10").fetchall()
        _conn.close()
        _已见 = set()
        # 直接看outdated里带知识性标签的 (父令/教训/结论/经验/概念/铁律等) ——
        # 对话记录/叙事类自然过期 • 不是可疑知识 • 跳过; 知识性过期=该浮回重新核对
        # 2026-08-27 精修: 标签含“对话/叙事/开始”的一律不算知识性 (“记忆 • 对话”含“记忆”二字曾误报)
        _知识标签 = ("父令", "教训", "结论", "经验", "概念", "铁律", "决策", "诊断", "知识", "体系", "功能", "模块", "训练",
"压缩", "推理", "记忆", "模型", "学习", "bug", "修复", "方案", "配置")
        _排除标签 = ("对话", "叙事", "开始", "留言", "记录")
        for _e in _rows:
            _etag = str(_e["tags"] or "")
            _is_dialog = any(k in _etag for k in _排除标签)
            if _is_dialog:
                continue
            if any(k in _etag for k in _知识标签) and _etag not in _已见:
                _已见.add(_etag)
                待核对.append(f"「{_etag}」曾有旧记忆已过时: {_e['c']} (id={_e['id']})")
                if len(待核对) >= 3:
                    break
        if 待核对:
            output += "\n## 📌 待核对 (OpenWiki • 过期浮回 • 怀疑不丢失)\n"
            for _x in 待核对[:3]:
                output += f"- {_x}\n"
except Exception:
    pass # 诚实: 待核对异常不影响主流程

if 蜘蛛网结果["节点"]:
    output += f"\n## 🕸 蜘蛛网关联\n"
    for n in 蜘蛛网结果["节点"][:5]:
        output += f"- {n['概念']} ({n['类型']})\n"

if 蜘蛛网结果["关联概念"]:
    output += f"\n## 🔗 关联概念\n"
    for c in 蜘蛛网结果["关联概念"][:5]:
        output += f"- {c}\n"

# 2.2b 激活 • 记忆 (2026-08-28 收A: 统一激活集 • 点亮种子+感应器传播)
if 激活记忆:
    output += f"\n## 🧠 激活 • 记忆\n"
    for m in 激活记忆[:4]:
        output += f"- [{m.get('来源','')}] [{m.get('激活度',0)}] {m.get('内容','')[:60]}\n"

# 2.2b 感应器 • 全网扰动节点 (父令2026-08-27: 概念链路带出的跨概念真关联)
if 感应器节点:
    output += f"\n## 📡 感应器 • 概念链路\n"
    排序 = sorted(感应器节点.items(), key=lambda x: -x[1])[:5]

```

```

for n, e in 排序:
    output += f"- {n} ({round(e, 2)})\n"

if 经验:
    output += f"\n## ♦ 经验召回（越用越聪明）\n"
    for x in 经验:
        mark = "🔴" if x["被点出"] else "•"
        output += f"- {mark} {x['内容']}\n"

if 预检:
    output += f"\n## 🚦 经验预检（动作前•好用照做•吃瘪改道）\n{预检}\n"

if 预感:
    output += f"\n{预感}\n"

# 2.7 认知画像（父令2026-08-10•DeepTutor记忆驱动闭环借魂）
# 记忆从『被查』变『驱动』：每轮唤醒注入「本轮最该干的一件事」
try:
    from sun_memory.core.认知画像 import 认知画像 as 画像主, 格式化画像
    画像 = 画像主(brother_name)
    output += f"\n{格式化画像(画像)}\n"
except Exception:
    pass # 诚实：画像模块异常不影响主流程

with open(输出路径, "w", encoding="utf-8") as f:
    f.write(output)

return {
    "最近记忆": len(recent_memories),
    "蜘蛛网匹配": len(蜘蛛网结果["节点"]),
    "关联概念": 蜘蛛网结果["关联概念"][:5],
    "注入路径": str(输出路径)
}

```

六、可视化（可选）

可视化专用/星空热度.py（59 行）

```
# -*- coding: utf-8 -*-
"""星空热度数据（记忆体星空 v2: HEAT CODE / 归档暗星 / SYNC LOG）
被 main.js 调用 • 输出 JSON：热记忆标签→热节点 • 归档数 • 节律日志"""
import sqlite3, json, os, sys

DB = os.environ.get('SUNMEM_DB', os.path.join(os.path.dirname(os.path.abspath(__file__)), 'data', 'sunmem.db'))
LOG = os.environ.get('节律日志路径', os.path.join(os.path.dirname(os.path.abspath(__file__)), 'logs', '节律观测日志.log'))

def main():
    out = {'hot': {}, '归档数': 0, '日志': []}

    try:
        conn = sqlite3.connect(DB)
        conn.row_factory = sqlite3.Row
        # 热记忆（heat>0 • 取前50 • 内容关键词→热节点映射）
        rows = conn.execute(
            "SELECT id, content, tags, heat, layer FROM memories WHERE heat > 0 "
            "ORDER BY heat DESC LIMIT 50").fetchall()
        # 归档（过时记忆数）
        out['归档数'] = conn.execute("SELECT COUNT(*) FROM memories WHERE status='outdated'").fetchone()[0]
        # 热词映射：从热记忆内容里抽2-4字词 • 对应蜘蛛网节点（heat 高的浮出）
        import re
        热词 = []
        for r in rows:
            h = r['heat'] or 0.0
            # 标签优先
            tags = str(r['tags'] or '')
            for t in re.findall(r'[\u4e00-\u9fff]{2,6}', tags):
                if len(t) >= 2:
                    热词.append((t, h))
            # 内容里抽“父令/记忆/节律/星”等核心词
            c = str(r['content'] or '')[:60]
            for kw in ['父令', '记忆', '节律', '星空', '蜘蛛', '诚实', '宏愿', '咬合', '算子', '向量', '点亮', '召回', '进化',
                '压缩', '画像', '成绩']:
                if kw in c:
                    热词.append((kw, h))
        # 聚合（同词取最大热度）
        agg = {}
        for t, h in 热词:
            if t not in agg or h > agg[t]:
                agg[t] = h
        out['hot'] = agg
        conn.close()
    except Exception as e:
        out['hot'] = {'记忆': 0.3}

# SYNC LOG（节律观测日志 • 最近5条）
try:
    if os.path.exists(LOG):
        lines = open(LOG, encoding='utf-8').read().strip().splitlines()
        out['日志'] = lines[-5:]
```

```

except Exception:
    pass

print(json.dumps(out, ensure_ascii=False))

if __name__ == "__main__":
    main()

```

可视化专用/记忆生命流.py (81 行)

```

# -*- coding: utf-8 -*-
"""记忆生命流数据（记忆体星空 v9：动态显示自组织/自整理/点亮/回落行为）
被 main.js 调用・输出 JSON：自组织簇・小时叙事・节律日志・点亮事件"""
import sqlite3, json, os, sys, re
from datetime import datetime

DB = os.environ.get('SUNMEM_DB', os.path.join(os.path.dirname(os.path.abspath(__file__)), 'data', 'sunmem.db'))
LOG = os.environ.get('节律日志路径', os.path.join(os.path.dirname(os.path.abspath(__file__)), 'logs', '节律观测日志.log'))
蜘蛛网 = os.environ.get("蜘蛛网_INDEX", os.path.join(os.path.dirname(os.path.abspath(__file__)), "蜘蛛网", "索引.json"))

def main():
    out = {'簇': [], '叙事': [], '节律日志': [], '点亮': [], '热度': {}, '归档数': 0, '总数': 0}

    try:
        conn = sqlite3.connect(DB)
        conn.row_factory = sqlite3.Row

        # 总数/归档
        out['总数'] = conn.execute("SELECT COUNT(*) FROM memories").fetchone()[0]
        out['归档数'] = conn.execute("SELECT COUNT(*) FROM memories WHERE status='outdated'").fetchone()[0]

        # 热度（热记忆标签→节点）
        rows = conn.execute("SELECT content, tags, heat, layer, ts FROM memories WHERE heat > 0.05 ORDER BY heat DESC LIMIT 40").fetchall()
        agg = {}
        for r in rows:
            h = r['heat'] or 0.0
            for kw in ['父令', '记忆', '节律', '诚实', '宏愿', '咬合', '算子', '向量', '点亮', '召回', '进化', '压缩', '画像', '成绩', '星空', '蜘蛛', '经验', '教训', '闭环', '神经元', '辉']:
                if kw in str(r['content'] or '')[:80] or kw in str(r['tags'] or ''):
                    if kw not in agg or h > agg[kw]:
                        agg[kw] = round(h, 3)
        out['热度'] = agg

        # —— 自组织簇：最近30条记忆按 bigram 聚簇（简化・可运行版）——
        recent = conn.execute("SELECT id, content, tags, ts FROM memories ORDER BY id DESC LIMIT 30").fetchall()
        # 主题关键词簇（真实数据）
        主题库 = [
            ('记忆体系', ['记忆', '咬合', '模块', '闭环', '算子']),
            ('节律/活性', ['节律', 'heat', '点亮', '回落', '热度', '振荡']),
            ('父令/身份', ['父令', '父亲', '孙呈', '宏愿', '诚实', '身份']),
            ('世界模型', ['世界模型', '佳佳', '三算子', '能量', '相变']),
            ('兄弟/家族', ['兄弟', '孙云', '孙演', '孙博', '孙安', '家族']),
            ('训练/模型', ['训练', '500M', '模型', 'GPU', 'ckpt']),
        ]
        clusters = []

```

```

for 名, 词表 in 主题库:
    命中 = [r['id'] for r in recent if any(w in str(r['content'] or '') for w in 词表)]
    if len(命中) >= 2:
        clusters.append({'名': 名, '数量': len(命中), 'ids': 命中[:8]})
out['簇'] = clusters

# —— 自整理: 最近小时叙事 (自动整理引擎生成的) ——
try:
    narr = conn.execute("SELECT content, tags, ts FROM memories WHERE tags LIKE '%小时叙事%' OR content LIKE '%这一小时%' ORDER BY id DESC LIMIT 3").fetchall()
    for n in narr:
        out['叙事'].append({'内容': str(n['content'] or '')[:120], '时间': str(n['ts'] or '')[:16]})
except Exception:
    pass

# —— 点亮事件: 最近被点亮的记忆 (hit_count 高/最近 touched) ——
lit = conn.execute("SELECT content, hit_count, ts FROM memories WHERE hit_count > 0 ORDER BY hit_count DESC, id DESC LIMIT 5").fetchall()
for l in lit:
    out['点亮'].append({'内容': str(l['content'] or '')[:60], '次数': l['hit_count'] or 0})

conn.close()
except Exception as e:
    out['错误'] = str(e)[:80]

# —— 节律日志 (真实回落记录) ——
try:
    if os.path.exists(LOG):
        lines = open(LOG, encoding='utf-8').read().strip().splitlines()
        out['节律日志'] = lines[-6:]
except Exception:
    pass

print(json.dumps(out, ensure_ascii=False))

if __name__ == "__main__":
    main()

```